Institute for Machine Tools and Industrial Management (iwb)
TUM School of Engineering and Design
Technical University of Munich

Automated Trajectory Parameterization of Industrial Robot Skills

Automatisierte Trajektorienparametrisierung von Industrieroboterskills

Scientific work for obtaining the academic degree

Master of Science (M.Sc.)

at the TUM School of Engineering and Design of the Technical University of Munich

Supervised by Prof. Dr.-Ing. Rüdiger Daub
Chair for Production Technology and Energy Storage Systems

Submitted by Yu-Chuan Chen
Bunzlauer Str. 10
80992 Munich

Submitted on May 31, 2025 in Garching

Scope of work

English Title of the Master's Thesis:
Automated Trajectory Parameterization of Industrial Robot Skills

German Title of the Master's Thesis:
Automatisierte Trajektorienparametrisierung von Industrieroboterskills

Author:	B. Sc. Yu-Chuan Chen	**Supervisor:**	M. Sc. Sebastian Kurscheid
Issued:	01.12.2024	**Submitted:**	31.05.2025

Initial Situation:
Germany is currently confronted with various challenges as a production location, with the need for more skilled workers drawing significant media attention. One potential solution to this issue is adopting automation through industrial robots, a trend expected to become increasingly crucial considering future demographic shifts. Nevertheless, the substantial programming effort required poses a significant obstacle, particularly for small and medium-sized enterprises. The programming effort for deploying robots in multi-variant production should be minimized in the future. Skill-based programming has proven effective for this purpose. Especially considering high-mix, low-volume manufacturing, by parameterizing these skill-based handling processes, they are adapted to new variants or products. Therefore, it is desired that an operation can be processed in a simulation before realizing an automation process, which is supposed to save unnecessary costs. Additionally, materials and properties are expected to be considered within a simulation because generated trajectories might cause undesired influence on a gripped object. Simultaneously, the relationship between execution time and summary for multiple executions is desired, which is a critical analysis for an automation process. Hence, when these factors and situations are considered, an object is supposed to be transported safely, and the execution time can be identified in a simulation.

Objective:
Within the scope of this thesis, an example scenario based on a handling skill and its corresponding parameters in a simulation will be defined first. Subsequently, a concept for automated skill verification will be developed and implemented in ROS2. The central evaluation criteria specify whether an object is transported safely with a generated trajectory. Meanwhile, the corresponding execution time will be recorded, especially in multiple executions. To further access the information of a generated trajectory, the positions of an end-effector are desired to be documented for workspace visualization.

Approach and Working Methodology:

- Literature research and presentation of the state of the art in science and technology
- Objective definition
- Concept definition and implementation
  - Setting up a test scenario in a simulation and defining desired outputs
  - Developing a concept to verify a stable grasp utilizing different contact models and considering related parameters
  - Developing a method to extract data from a simulation and implementing computation of the developed concept in Python
  - Creating a method to document execution times and positions of a handling process by multiple executions
- Validation
- Critical evaluation and detailed documentation of the results

Agreement:
With the supervision of B. Sc. Yu-Chuan Chen by M. Sc. Sebastian Kurscheid, the intellectual property of the *iwb* is incorporated into this work. Publication of the work or disclosure to third parties requires the permission of the chair holder. I agree to the archiving of the thesis in the *iwb*‘s library, which is only accessible to *iwb* employees, and in the *iwb*‘s digital thesis database of the *iwb* as a PDF document.

Garching, 19.12.2024

Prof. Dr.-Ing.	(B.Sc.)
Rüdiger Daub	Yu-Chuan Chen

Abstract

As the trend of high-mix, low-volume manufacturing grows, German manufacturers, especially small and medium-sized enterprises, face the challenges of substantial programming effort to adapt robot programs for various product variants. Accordingly, introducing an industrial robotic manipulator with skill-based parameterization processes has proven effective for the improvement. This thesis presents a concept to parameterize an industrial robot's skill, considering a stable grasp of an object, execution time, and workspace visualization. Specifically, the stable grasp concept is developed based on a gripped object's material properties. Furthermore, the concept is implemented in Robot Operating System (ROS) 2, and the whole process is integrated into a skill set to achieve the objective of automation. Therefore, after implementing the developed skill, the skill can automatically verify the fulfillment of a stable grasp, compliance with execution time, and achievement of a non-collision workspace based on a simulated trajectory.

Zusammenfassung

Mit dem Trend zur High-Mix-Low-Volume-Fertigung stehen deutsche Hersteller, insbesondere kleine und mittlere Unternehmen, vor der Herausforderung eines hohen Programmieraufwands, um Roboterprogramme für verschiedene Produktvarianten anzupassen. Dementsprechend hat sich die Einführung eines Industrieroboters mit fähigkeitsbasierten Parametrisierungsprozessen als effektiv für die Verbesserung erwiesen. In dieser Arbeit wird ein Konzept zur Parametrisierung der Fähigkeiten eines Industrieroboters unter Berücksichtigung eines stabilen Greifens eines Objekts, der Ausführungszeit und der Arbeitsraumvisualisierung vorgestellt. Das Konzept des stabilen Greifens wird auf der Grundlage der Materialeigenschaften eines gegriffenen Objekts entwickelt. Darüber hinaus wird das Konzept in ROS2 implementiert, und der gesamte Prozess wird in ein Skill-Set integriert, um das Ziel der Automatisierung zu erreichen. Nach der Implementierung der entwickelten Fertigkeit kann diese daher automatisch die Erfüllung eines stabilen Griffs, die Einhaltung der Ausführungszeit und das Erreichen eines kollisionsfreien Arbeitsbereichs auf der Grundlage einer simulierten Trajektorie überprüfen.

Contents

List of Abbreviations **ix**

List of Symbols **xi**

List of Figures **xv**

List of Tables **xix**

1 Introduction **1**

2 Basics **3**
2.1 Wrench . . . 3
2.2 Stable grasp and execution time . . . 4
2.2.1 Force closure and requirements . . . 5
2.2.2 Determination of grasp matrix . . . 6
2.2.3 Friction cone constraints . . . 7
2.2.4 Grasp wrench space . . . 8
2.3 Dynamics equations of robotic manipulation . . . 8
2.3.1 Gravity term . . . 9
2.3.2 Inertia . . . 10
2.3.3 Centripetal force . . . 10
2.4 Corresponding parameters in a simulation . . . 11
2.4.1 Collision model . . . 11
2.4.2 Inertial model . . . 15
2.4.3 Visual model . . . 16
2.5 Contact models . . . 16
2.5.1 Hard-finger contacts . . . 17
2.5.2 Soft-finger contacts . . . 17

3 State of the art **19**
3.1 Determination and analysis of grasp wrench space . . . 19
3.1.1 Friction cone approximation . . . 20
3.1.2 Approaches for verification . . . 20
3.1.3 Introduction of other wrench spaces . . . 22
3.2 Forces recognized from a simulation . . . 22
3.2.1 Direct extraction of data . . . 23
3.2.2 Indirect extraction of data . . . 24
3.3 Measurement of execution time in a simulation . . . 25
3.4 Positioning of end-effector and workspace visualization . . . 26
3.4.1 Forward kinematics . . . 26
3.4.2 Direct estimation in a simulation . . . 27

3.4.3 Documentation and visualization of data . . . 28
3.4.4 Approaches for workspace visualization of a robotic manipulator . . . 28

4 Design **31**
4.1 Objective definition and scenario configuration . . . 31
4.2 Consideration of stable grasp . . . 32
4.2.1 Overview of required elements as input . . . 33
4.2.2 Force closure aspect . . . 36
4.2.3 Consideration of other wrench spaces . . . 38
4.2.4 Consideration of dynamic condition . . . 40
4.3 Execution time by multiple cycles of a handling skill . . . 41
4.4 Visualization of used workspace . . . 42

5 Implementation **45**
5.1 Determination of stable-grasp-related parameters . . . 46
5.2 Setting up a simulation environment . . . 51
5.2.1 Configuration of simulation environment . . . 52
5.2.2 Executing a handling skill in simulation . . . 56
5.3 Determination of stable grasp . . . 61
5.3.1 Data processing in and after a simulation . . . 61
5.3.2 Determination of force closure . . . 63
5.3.3 Composing other wrench spaces . . . 67
5.3.4 Determination of dynamic analysis . . . 68
5.4 Determination of execution time . . . 69
5.5 Determination of used workspace . . . 71
5.5.1 Collecting workspace data in a simulation . . . 71
5.5.2 Processing workspace data after a simulation . . . 73

6 Evaluation **75**
6.1 General overview of simulation results . . . 75
6.2 Verification of stable grasp . . . 80
6.2.1 Modification of gripper control . . . 82
6.2.2 Results of different materials . . . 82
6.2.3 Results of different contact models . . . 84
6.3 Verification of execution time . . . 85
6.4 Verification of used workspace . . . 85

7 Conclusion and outlook **89**

A Additional class diagrams **93**

B Additional sequence diagrams **109**

C Configuration of classes and files **115**

D Additional simulation-related figures **121**

E Additional information of evaluation **131**

Glossary **145**

Bibliography **147**

List of Abbreviations

API	Application Programming Interface
CAD	Computer Aided Design
cfm	Constraint Force Mixing
CNN	Convolutional Neural Network
CoM	Center of Mass
DAE	Digital Asset Exchange
DH	Denavit-Hartenberg
DoF	Degrees of Freedom
erp	Error Reduction Parameter
GUI	Graphical User Interface
GWS	Grasp Wrench Space
IMFC	Impedance Motion Force Control
IMU	Inertial Measurement Unit
IPTP	Iterative Parabolic Time Parameterization
KDL	Kinematics and Dynamics Library
LTS	Long Term Support
MAGF	Maximum Admissible Grasp Force
MAGF-WS	Maximum Admissible Grasp Force-Wrench Space
MFC	Motion Force Control
MGF	Maximum Grasp Force
MGF-WS	Maximum Grasp Force-Wrench Space
MTC	MoveIt Task Constructor
ODE	Open Dynamics Engine
OMPL	Open Motion Planning Library
OWS	Object Wrench Space
PTFE	Polytetrafluoroethylene
ResNet	Residual Networks
RGB-D	Red Green Blue-Depth
ROS	Robot Operating System
RRT	Rapidly-exploring Random Trees
SDF	Simulation Description Format
SRDF	Semantic Robot Description Format
STL	Standard Triangle Language

TCP	Tool Center Point
TOTG	Time Optimal Trajectory Generation
TWS	Task Wrench Space
URDF	Unified Robotics Description Format
URI	Uniform Resource Identifier
VS Code	Visual Studio Code
XML	Extensible Markup Language

List of Symbols

Symbol	Unit	Description
$\boldsymbol{A}$	-	Motion subspace of an inertia matrix
A_c	m^2	Contact area
$\boldsymbol{A_h}$	-	Matrix of hyperplane coefficients
a	$\frac{m}{s^2}$	Acceleration of an object
a_{c_s}	$\frac{m}{s^2}$	Accelerations of a gripped cube during dynamic process
$\boldsymbol{B}$	-	Force to motion subspace of an inertia matrix
$\boldsymbol{B}^T$	-	Motion to force subspace of an inertia matrix
$\boldsymbol{b_h}$	-	Vector of offsets concerning hyperplane
β	degree	Friction cone angle
C_r	-	Coefficient of restitution
$\boldsymbol{C}(\boldsymbol{x}, \dot{\boldsymbol{x}})$	-	Centripetal force term of an end-effector
$\boldsymbol{c_i}$	m	Position of the i-th contact point
$\boldsymbol{D}$	-	Force subspace of an inertia matrix
$\boldsymbol{d_i}$	m	Vector defining the position of the i-th contact point concerning an object's CoM
$\boldsymbol{d_x}, \boldsymbol{d_y}, \boldsymbol{d_z}$	m	Vector of distances between CoM and a point along the axes
δ	m	Deformation or indentation
$\dot{\delta}$	$\frac{m}{s}$	Relative velocity of deformation or indentation
E_f	Pa	Elastic modulus of a gripper's finger
E_o	Pa	Elastic modulus of a gripped object
E^*	Pa	Equivalent elastic modulus
$\boldsymbol{\epsilon}$	-	Small given scalar
ε	-	Example of the quality value
F_c	N	Scalar of contact force applied to a gripped object
$\boldsymbol{FC_i}$	-	Friction cone of the i-th contact point
$\boldsymbol{F_{c_e}}$	-	Contact force term applied to an end-effector
$F_{centripetal}$	N	Centripetal force arose in systems subjected to motion
F_{d_c}	N	Forces applied to a gripped cube during dynamic process
F_G	N	Grasp force of a contact
F_{max_s}	N	Maximum contact force from the static stable grasp analysis
F_n	N	Normal force
F_t	N	Tangential force

$\boldsymbol{F_x}$, $\boldsymbol{F_y}$, $\boldsymbol{F_z}$	N	Linear force components of a wrench
$\boldsymbol{f}$	N	Vector of forces subjected to a wrench
$\boldsymbol{f_c}$	N	Vector of the contact force
$\boldsymbol{f_i}$	N	Vector of forces of the i-th contact point
$\boldsymbol{f_n}$	N	Vector of the normal component of the contact force
$\boldsymbol{f_t}$	N	Vector of the tangential component of the contact force
$\boldsymbol{G}$	-	Grasp matrix
$\boldsymbol{G_i}$	-	Wrench basis matrix of the i-th contact point
$\boldsymbol{G(x)}$	-	Gravitational force term of an end-effector
g	$\frac{\mathrm{m}}{\mathrm{s}^2}$	Gravitational acceleration
h	second	Stepsize defined in a SDF file
$\boldsymbol{I_{inertia}}$	$\mathrm{kg} \cdot \mathrm{m}^2$	Inertia matrix
i	-	Number of the contact point
$<inertia>$	$\mathrm{kg} \cdot \mathrm{m}^2$	Inertia matrix defined in a SDF file
$int\,(\boldsymbol{FC_i})$	-	Interior of the friction cone of the i-th contact point
K_d	$\frac{\mathrm{N \cdot s}}{\mathrm{m}}$	Equivalent damping coefficient
K_p	$\frac{\mathrm{N}}{\mathrm{m}}$	Equivalent stiffness coefficient
K_{p_i}, K_{p_j}	$\frac{\mathrm{N}}{\mathrm{m}}$	Stiffness coefficient of an contact subject
K_{p_o}	$\frac{\mathrm{N}}{\mathrm{m}}$	Stiffness coefficient of a gripped object
k	-	Total number of contact points
$<kd>$	$\frac{\mathrm{N \cdot s}}{\mathrm{m}}$	Damping coefficient defined in a SDF file
$<kp>$	$\frac{\mathrm{N}}{\mathrm{m}}$	Stiffness coefficient defined in a SDF file
L_o	m	Height or thickness of a gripped object in the loading direction
$\boldsymbol{\Lambda}$	-	Inertia matrix term of an end-effector
λ	-	Arbitrary constant for scaling the torque magnitude
$\boldsymbol{m}$	N·m	Vector of moments subjected to a wrench
m_e	kg	Equivalent mass of a contact
m_i, m_j	kg	Mass of a contact subject
m_o	kg	Mass of a gripped object
$<mass>$	kg	Mass defined in a SDF file
$<mu>$	-	Static friction coefficient in first pyramid direction in a SDF file
$<mu2>$	-	Static friction coefficient in second pyramid direction in a SDF file
μ_s	-	Static friction coefficient
μ_i^s	-	Static friction coefficient of the i-th contact
$\hat{\boldsymbol{n}}_{\boldsymbol{d}}$	m	Unit contact normals of a gripped cube during dynamic process
n_f	-	Number of friction cone approximation
$\boldsymbol{n_i}$	m	Vector of a contact normal of the i-th contact point
$\hat{\boldsymbol{n}}_{\boldsymbol{i}}$	m	Unit contact normal to a contact boundary of the i-th contact point
$\boldsymbol{o}$	-	Origin of the coordinate system
ω	$\frac{\mathrm{rad}}{\mathrm{s}}$	Angular velocity
$\boldsymbol{P}$	m	Pose of an end-effector
$\boldsymbol{p_i}$	m	Vector from CoM to the i-th contact point

${}^{j-1}_{j}\boldsymbol{R}$	-	Homogeneous transformation matrix with the index of joint j
$\boldsymbol{r}$	m	Position of an object's CoM
r_p	m	Radius of an circular path
S_i	-	Admissible contact region of the i-th contact point
S_{1_q}	second	Step 1's execution time of q-th cycle
S_{2_q}	second	Step 2's execution time of q-th cycle
S_{3_q}	second	Step 3's execution time of q-th cycle
$<soft_cfm>$	-	Parameter for softness and stability defined in a SDF file
$<soft_erp>$	-	Parameter for joint or constraint behavior defined in a SDF file
σ_{y_c}	MPa	Yield strength of a gripped cube
$\boldsymbol{T_x}$, $\boldsymbol{T_y}$, $\boldsymbol{T_z}$	N·m	Torque components of a wrench
$\boldsymbol{t}$	-	Vector of collective forces and moments
t_q	second	Total execution time of q-th cycle
t_{q_s}	second	Start point of time of q-th cycle
t_{q_e}	second	Endpoint time of q-th cycle
$\boldsymbol{\tau}$	N·m	Vector of torques applied at an object's CoM
$\boldsymbol{\tau_i}$	N·m	Vector of torques of the i-th contact point
V_{impact}	$\frac{\text{m}}{\text{s}}$	Impact velocity
$V_{rebound}$	$\frac{\text{m}}{\text{s}}$	Rebound velocity
ν_f	-	Poisson's ratio of a gripper's finger
ν_i, ν_j	$\frac{\text{m}}{\text{s}}$	Velocity of a contact subject
ν_o	-	Poisson's ratio of a gripped object
$\boldsymbol{w}$	-	Wrench
$\boldsymbol{w_e}$	-	Total actuation wrench applied to an end-effector
$\boldsymbol{w_i}$	-	Wrench of the i-th contact point
$\boldsymbol{x}$	-	Operational space coordinate term of an end-effector
$\boldsymbol{x_h}$	-	Vector of variables concerning hyperplane
ζ	-	Damping ratio

List of Figures

1.1 Motivation of this thesis 2
1.2 Structure overview of this thesis 2

2.1 Model for grasping an object considering the influence of acceleration (NAKAZAWA ET AL. 2001) 4
2.2 Multi-fingered hand grasping an object (YOSHIKAWA 2010) 6
2.3 Scheme of a friction cone (MILLER and ALLEN 2004) 8
2.4 Representation of GWS analyses (SOUZA ET AL. 2021) 9
2.5 Process of contact compression and elastic recovery (ZHAO ET AL. 2021) 13
2.6 Representation of different contact models (BORST ET AL. 2003) 17

3.1 A sample of three contacts on a planar object (LIU ET AL. 2019) 20
3.2 Different representation of friction cone (LIU ET AL. 2019) 21
3.3 Scheme of the relationship between GWS and TWS (BORST ET AL. 2004) . . . 23
3.4 Scheme of force/torque sensor implemented as a YARP interface (HOFFMAN ET AL. 2014) 23
3.5 Kinematic model of the flexible drilling robot (ZHANG and WANG 2012) 27
3.6 The proposed system for end-effector pose estimation (ORTEGA ET AL. 2023) . 28
3.7 The overall structure of MoveIt (LI ET AL. 2017) 29
3.8 Presentation of an environment using OctoMap (PULJIZ ET AL. 2020) 29
3.9 The planning components in the motion planning framework MoveIt (COLEMAN ET AL. 2014) 30
3.10 Example of a URDF bundle (TOLA and CORKE 2024) 30

4.1 Scheme of this thesis's objectives 31
4.2 Scheme of the pick-and-place scenario of this thesis 32
4.3 Flowchart of the stable grasp verification concept 33
4.4 Flowchart of the force closure verification concept 37
4.5 Flowchart of the verification process for extended wrench spaces (TWS, MGF-WS, MAGF-WS) 39
4.6 Scheme of the relationship between all of the wrench spaces 39

4.7 Flowchart of dynamic condition verification concept 40
4.8 Flowchart of used workspace visualization concept 43

5.1 Scheme of the introduced tools in the implementation process 45
5.2 Class diagram overview based on class Main 47
5.3 Sequence diagram overview based on class Main 48
5.4 Class diagram overview based on class ParametersPreparation 49
5.5 Sequence diagram overview based on class ParametersPreparation 50
5.6 Class diagram overview based on class LaunchSim 53
5.7 Sequence diagram overview based on class LaunchSim 54
5.8 Considered models for the scenario's robotic manipulator 55
5.9 The built simulation environment in Gazebo 55
5.10 The built simulation environment in MoveIt2 visualized by RViz 56
5.11 Simulation environment in RViz after introducing collision objects 58
5.12 Class diagram overview based on class StableGrasp 65
5.13 Sequence diagram overview based on class StableGrasp 66

6.1 Result of test case GT1 from class PlotTimes 77
6.2 Result of test case GT1 from class SpecifiedTimes 78
6.3 The used workspace of test case GT1's loop 0 78
6.4 Box 0's contact force magnitudes of test case GT1 for static analysis 79
6.5 Box 0's contact torque magnitudes of test case GT1 for static analysis 79
6.6 Box 0's accelerations of test case GT1 for dynamic analysis 81
6.7 Contact situation of box 0 in test case GT1 83
6.8 Static stable grasp analysis of test case GT1's box 0 in Gazebo 83
6.9 Results of test cases GT1 and ET1 from class PlotTimes 86
6.10 The used workspace of test case WV1's loop 0 87
6.11 Result of test case WV2 from class PlotTimes 88

7.1 Scheme of the contents included from Chapter 4 to Chapter 6 89
7.2 Example of the command in the terminal for launching the master node 89

A.1 Class diagram overview based on class SdfParameters 93
A.2 Class diagram overview based on class ContactModels 94
A.3 Class diagram overview based on class WriteSdf 94
A.4 Class diagram overview based on class MoveGroupDemoNode 95
A.5 Class diagram overview based on class BoxPoseRecorder 96
A.6 Class diagram overview based on class ContactInfoOrganizer 97

A.7 Class diagram overview based on class ForceClosure 98
A.8 Class diagram overview based on class GraspMatrix 99
A.9 Class diagram overview based on class FrictionCone 99
A.10 Class diagram overview based on class GwsConstruct 100
A.11 Class diagram overview based on class OtherSpace 101
A.12 Class diagram overview based on class DynamicConsideration 102
A.13 Class diagram overview based on class ExecutionTime 103
A.14 Class diagram overview based on class RobotStateUpdater 104
A.15 Class diagram overview based on class SurfaceRecorder 105
A.16 Class diagram overview based on class ObstacleReader 106
A.17 Class diagram overview based on class WorkSpaceVisual 107

B.1 Sequence diagram overview based on class MoveGroupDemoNode 110
B.2 Sequence diagram overview based on class ContactInfoOrganizer 111
B.3 Sequence diagram overview based on class ForceClosure 112
B.4 Sequence diagram overview based on class ExecutionTime 113
B.5 Sequence diagram overview based on class SpecifiedTimes 114

D.1 Example of the robotic manipulator's URDF file 121
D.2 End of the scenario's step 1 of loop 0 in RViz 122
D.3 End of the scenario's step 1 of loop 0 in Gazebo 122
D.4 End of the scenario's step 2 of loop 0 in RViz 123
D.5 End of the scenario's step 2 of loop 0 in Gazebo 123
D.6 End of the scenario's step 3 of loop 0 in RViz 124
D.7 End of the scenario's step 3 of loop 0 in Gazebo 124
D.8 End of the scenario's step 4 of loop 0 in RViz 125
D.9 End of the scenario's step 4 of loop 0 in Gazebo 125
D.10 Example of the log file from Gazebo contact topic 126
D.11 Example of the file created from class SurfaceRecorder 127
D.12 Example of box_green_0 in SDF file 128
D.13 End of the scenario's 10 cycles in RViz 129
D.14 End of the scenario's 10 cycles in Gazebo 129
D.15 Adding CollisionObject of test case WV1 in RViz 130
D.16 Adding CollisionObject of test case WV2 in RViz 130

E.1 Box 0's contact force magnitudes of test case SG1 for static analysis 131
E.2 Box 0's contact torque magnitudes of test case SG1 for static analysis 131

E.3 Contact situation of box 0 in test case SG1 133
E.4 Box 0's accelerations of test case ET1 for dynamic analysis 133

List of Tables

4.1 Explanation of pick-and-place scenario steps 32
4.2 Overview of the required input parameters in a SDF file and the corresponding equations 34
4.3 Value references of <poissons_ratio> and <elastic_modulus> with the corresponding materials (OPEN SOURCE ROBOTICS FOUNDATION 2014) 34
4.4 Consideration of the parameters in a SDF file combining the contact models . 35
4.5 Concept and steps of force closure verification 36

5.1 Relation between the implementation aspects and the classes 46
5.2 Attributes and their referred parameters 51
5.3 The commands to be filled in method launch_terminal() 52
5.4 Collision objects in the corresponding cycles 57
5.5 Joint position values of the robotic manipulator's home pose 58
5.6 Values of the defined TCP pre-grasp poses 59
5.7 Values of the defined scenario's TCP target pose 61
5.8 Example of the execution time data format in the CSV file 70

6.1 Parameters and setting of test case GT1 76
6.2 Maximum static contact force and torque of test case GT1's box 0 80
6.3 Presentation of stable grasp analysis results 80
6.4 Stable grasp analysis results of test case GT1's box 0 81
6.5 Stable grasp analysis results of test case SG2's box 0 84

C.1 Configuration of classes and files based on class Main 115
C.2 Configuration of classes and files based on class ParametersPreparation 116
C.3 Configuration of classes and files based on class LaunchSim 116
C.4 Configuration of classes and files based on class ContactInfoOrganizer 117
C.5 Configuration of classes and files based on class StableGrasp 117
C.6 Configuration of classes and files based on class ForceClosure 118
C.7 Configuration of classes and files based on class OtherSpace 118

C.8 Configuration of classes and files based on class DynamicConsideration 118
C.9 Configuration of classes and files based on class ExecutionTime 119
C.10 Configuration of classes and files based on class WorkSpaceVisual 119

E.1 Parameters and setting of test case SG1 132
E.2 Maximum static contact force and torque of test case SG1's box 0 133
E.3 Parameters and setting of test case SG2 134
E.4 Parameters and setting of test case SG3 135
E.5 Stable grasp analysis results of test case SG3's box 0 136
E.6 Parameters and setting of test case SG4 137
E.7 Parameters and setting of test case SG5 138
E.8 Stable grasp analysis results of test case SG4's box 0 139
E.9 Stable grasp analysis results of test case SG5's box 0 139
E.10 Parameters and setting of test case ET1 140
E.11 Stable grasp analysis results of test case ET1's box 0 141
E.12 Parameters and setting of test case WV1 142
E.13 Stable grasp analysis results of test case WV1's box 0 143
E.14 Parameters and setting of test case WV2 143

Chapter 1

Introduction

Germany is currently confronted with various challenges as a production location, especially with the need for more skilled workers drawing significant media attention (SANNEMAN ET AL. 2021). One potential solution to this issue is adopting automation through industrial robots, a trend expected to become increasingly crucial considering future demographic shifts. This study mentions that German manufacturers can enhance their productivity, efficiency, and competitiveness in the global market by introducing robotic manipulators into the manufacturing industry. Nevertheless, the substantial programming effort required poses a significant obstacle, particularly for small and medium-sized enterprises.

The programming effort for deploying robots in multi-variant or multi-task production should be minimized in the future, and skill-based programming has proven effective for this purpose (GENG ET AL. 2023). Especially considering high-mix, low-volume manufacturing, by parameterizing these skill-based handling processes, they are adapted to new variants or products (KURSCHEID and DAUB 2024). Therefore, an operation can be verified in a simulation before implementing an automation process, which saves costs and constructs a preliminary framework. Additionally, materials and properties are expected to be considered within a simulation because generated trajectories might cause undesired influence on a gripped object.

Simultaneously, analyzing execution time across multiple cycles is critical for automation processes (MARTUCCI ET AL. 2020). One key advantage of robots is their high operational speed, which improves throughput and reduces execution time. However, accompanied by increased velocities, these fast movements are supposed to produce high accelerations, thus generating inertial forces that can cause a gripped object to slip. In principle, it is desired that the execution time and the corresponding acceleration of an operation can be specified. With the computed execution time from a simulation, especially by multiple cycles, an overview of an operation can be acquired.

Apart from the consideration of damage on a gripped object and execution time, SUNG and JOE (2025) mention the importance of understanding a robotic manipulator's workspace for safe and effective automation. A workspace defines not only the spatial reach of a manipulator but also its operational boundaries, which are essential for avoiding collisions and ensuring the feasibility of movements within complex environments. By specifying and analyzing the used workspace, it becomes possible to design motion paths that stay within safe zones and minimize the risk of collisions.

Based on the previous paragraphs, Figure 1.1 shows the motivation of this thesis by visualizing the essential requirements for flexible robot skill deployment and their interrelations. In flexible automation scenarios, users often need to specify task-dependent parameters, i.e.,

the desired execution time or constraints, to prevent damage to a gripped object. In addition to these user-defined parameters, general requirements such as collision-free execution must be fulfilled. This thesis focuses on parameterizing and systematically integrating these requirements into a reusable skill set for industrial applications.

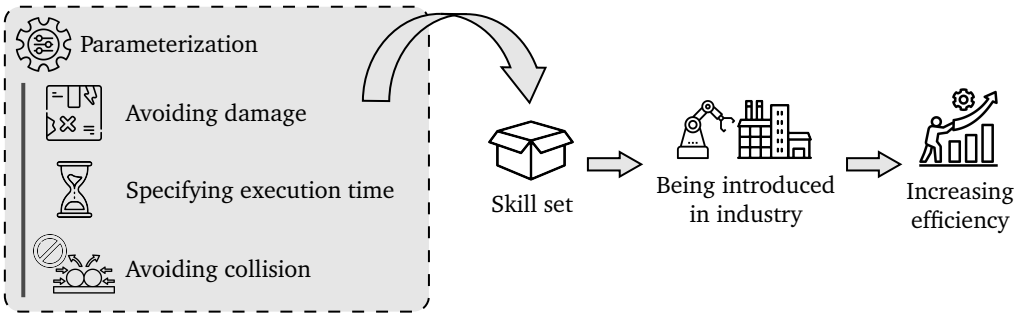


Figure 1.1: Motivation of this thesis

Moreover, Figure 1.2 presents the structure of this thesis in the form of a V-model. Following the described motivation, Chapter 2 collects the relevant theories to avoid any damage on a gripped object, i.e., the basics of stable grasp. Chapter 3 is composed of three segments. The first segment further reviews the other approaches for determining stable grasp. The second and third segments review the relevant studies for specifying the execution time of a robot skill and visualizing a robotic manipulator's workspace, respectively. Equipped with the basics and the related approaches from other studies, Chapter 4 develops a series of concepts and approaches for the parameterization process. Based on the developed concept, Chapter 5 starts implementing the thorough concept in ROS2. In Chapter 5, the implementation process is also designed based on the mentioned parameterization elements in Figure 1.1, and the utilized tools are further introduced. After the implementation process, Chapter 6 first presents an overview of the simulation and analysis results. Subsequently, the thorough implementation process is evaluated using different methods to specify the functionality and feasibility. Eventually, Chapter 7 provides an overall conclusion and describes an outlook based on the evaluation results.

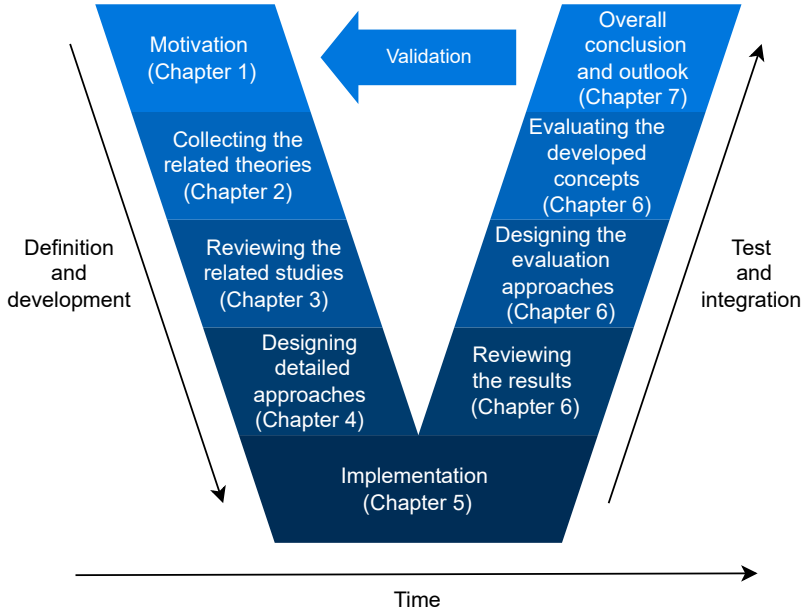


Figure 1.2: Structure overview of this thesis

Chapter 2

Basics

With the introduction described in Chapter 1, the forthcoming sections in this chapter present an extensive exposition of the fundamental methodologies. At the outset of this chapter, the basics of the wrench are described in Section 2.1. Following the wrench description, Section 2.2 further discusses the definition of stable grasp and its relationship with execution time. Subsequently, the dynamics-related equations regarding a robotic manipulator are stated in Section 2.3. Based on the description in Section 2.3, Section 2.4 further specifies the relevant parameters considered in a simulation and the determination approaches. Lastly, different contact models are stated in Section 2.5 to describe which models should be considered.

2.1 Wrench

At the beginning of the basics, the fundamental presentation of forces between an end-effector and a gripped object is introduced in this section. In applied mechanics and robotics, the wrench is regarded as a pivotal role (BICCHI 1994). It is a vector-valued quantity representing the forces and torques applied to an object. Wrench is fundamental in understanding and analyzing the mechanical interactions experienced by an object. Considering an object gripped through k points can be represented by a vector $\boldsymbol{t} \in \mathbb{R}^t$ of collective forces and moments when the object is subjected to an external force $\boldsymbol{f} \in \mathbb{R}^3$ and a moment $\boldsymbol{m} \in \mathbb{R}^3$. These are the linear and rotational forces acting on the object in three-dimensional space, typically described along the x, y, and z axes. They represent how the object is pushed or pulled in each direction and twisted or rotated about these axes. To maintain equilibrium, the force and moment balance on the object can be described using matrix notation. The equation $\boldsymbol{w} = -\boldsymbol{G}\boldsymbol{t}$ in short captures this balance, where $\boldsymbol{w} = \left(\boldsymbol{f}^T \boldsymbol{m}^T\right)^T \in \mathbb{R}^6$ represents the wrench, and $\boldsymbol{G} \in \mathbb{R}^{6xt}$, known as the grasp matrix or grip transform, relates the contact forces and moments to the wrench. The grasp-matrix-related calculation is described in Subsection 2.2.2.

Furthermore, when a force $\boldsymbol{f} \in \mathbb{R}^3$ and a torque $\boldsymbol{\tau} \in \mathbb{R}^3$ are applied at the object's Center of Mass (CoM), the wrench takes on a more explicit form (MYRVOLD 2020). It is represented as a stacked vector in Equation (2.1). Additionally, this equation can be further expanded in Equation (2.2), where $\boldsymbol{F}$ represents linear force components in N, and $\boldsymbol{T}$ represents torque components in $N \cdot m$.

$$\boldsymbol{w} = \begin{bmatrix} f \\ \tau \end{bmatrix} \in \mathbb{R}^6 \tag{2.1}$$

$$w = \begin{bmatrix} F_x \\ F_y \\ F_z \\ T_x \\ T_y \\ T_z \end{bmatrix} \tag{2.2}$$

2.2 Stable grasp and execution time

After describing a wrench in the previous section, this section primarily states the consideration of stable grasp. Considering stable grasp, it is stated that the forces applied to a gripped object are proportional to its acceleration and execution time (MARTUCCI ET AL. 2020). Additionally, NAKAZAWA ET AL. (2001) describe the force control of a two-fingered robotic gripper that is modeled based on human grasping schemes, and the model scheme is presented in Figure 2.1, where:

- F_n is the normal force in N
- F_t, defined as tangential force, is the friction force in N
- m_o is the mass of the gripped object in kg
- g is gravitational acceleration, equivalently 9.81 $\frac{m}{s^2}$
- a is the acceleration of the object in $\frac{m}{s^2}$

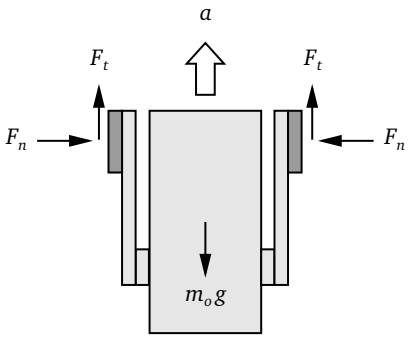


Figure 2.1: Model for grasping an object considering the influence of acceleration (NAKAZAWA ET AL. 2001)

Based on Figure 2.1, which is a model for determining the grasp force of a robotic gripper depending on the acceleration of the object, the corresponding grasp force can be determined. It is presented in Equation (2.3), where F_G is the grasp force in N, and μ_s is the static friction coefficient between the gripper and the object (NAKAZAWA ET AL. 2001).

$$F_G = \frac{m_o\,(g + a)}{2\mu_s} \tag{2.3}$$

With the clarified relationship between forces and execution time, it is further processed into the consideration of stable grasp composed of the following subsections. Regarding the

first segment of stable grasp, an aspect called fine manipulation is researched. In robotics, fine manipulation refers to the precise and delicate handling of small objects or detailed tasks that require a high degree of accuracy, dexterity, and control (AL-IBADI ET AL. 2018). Concerning the operation without fine manipulation, damage might be caused to a gripped object because the system does not consider the force applied to the gripped object during the operation (KHURSHID ET AL. 2011). Therefore, it is indicated that fine manipulation is significant because of damage avoidance, and the grasp force applied to an object should not surpass the Maximum Admissible Grasp Force (MAGF) of an object. Apart from the consideration of MAGF, the second segment of stable grasp is referred to as the consideration of force closure (ZHU and WANG 2003). The force-closure-related consideration is further discussed in the following subsections.

2.2.1 Force closure and requirements

In this subsection, the fundamentals of force closure are discussed. ZHU and WANG (2003) describe the capability of gripping an object firmly in grasp and manipulation planning. This study states that the capability of the grasp for firmly holding an object while resisting external wrench loads is characterized by the properties known as form closure and force closure. The concept of form closure originates from the field of machine design. A grasp can achieve form closure if it completely immobilizes a gripped object and solely relies on unilateral, frictionless point contact constraints.

Contrarily, the force closure property is related to the capability of the grasp to resist external disturbances (ZHU and WANG 2003). A force-closed grasp is usually considered capable of applying the wrench on the gripped object required to balance external loads. With the definition of force closure, BOHG ET AL. (2024) further relate force closure with the basics of wrench presented in Equation (2.1). It is stated that each contact point i in a grasp applies a wrench to the object. Additionally, the torque $\boldsymbol{\tau_i}$ can be computed by $\boldsymbol{\tau_i} = \boldsymbol{d_i} \times \boldsymbol{f_i}$, where $\boldsymbol{d_i}$ is the vector defining the position of the i-th contact point concerning the object's CoM. The wrench can thus be written and presented in Equation (2.4), where $\lambda \in \mathbb{R}$ is an arbitrary constant for scaling the torque magnitude if desired.

$$\boldsymbol{w}_i = \begin{bmatrix} \boldsymbol{f}_i \\ \lambda(\boldsymbol{d}_i \times \boldsymbol{f}_i) \end{bmatrix} \tag{2.4}$$

This wrench definition defines a grasp as the set of all possible wrenches the grasp's contact points can achieve (BOHG ET AL. 2024). Mathematically, an admissible force $\boldsymbol{f_i}$ applied at the i-th contact point, as presented in Figure 2.2, can be linearly mapped into the corresponding wrench on the object as $\boldsymbol{G_i f_i}$, where $\boldsymbol{G_i}$ is a wrench basis matrix that includes a transformation from the local contact reference frame to an object-defined global reference frame, as stated in Section 2.1. Simultaneously, the determination of $\boldsymbol{G_i}$ is described in Equation (2.7). Therefore, the total wrench on the object from all contacts is presented in Equation (2.5), where $\boldsymbol{G}$ is referred to as a grasp matrix. The determination of a grasp matrix is further explained in Subsection 2.2.2.

$$\boldsymbol{w} = \sum_{i=1}^{k} \boldsymbol{G_i f_i} = \boldsymbol{G} \begin{bmatrix} \boldsymbol{f_1} \\ \vdots \\ \boldsymbol{f_k} \end{bmatrix}, \; \boldsymbol{G} = \begin{bmatrix} \boldsymbol{G_1} & \cdots & \boldsymbol{G_k} \end{bmatrix} \tag{2.5}$$

With the knowledge of force closure and its relationship with a wrench, LIU ET AL. (2011) conclude that the force closure grasping of a robotic hand is valid when the corresponding

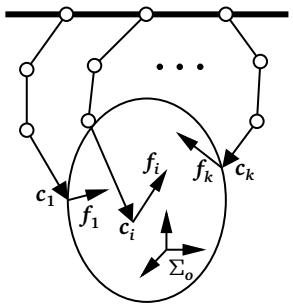


Figure 2.2: Multi-fingered hand grasping an object (YOSHIKAWA 2010)

grasp matrix has full rank and an internal force vector located in the friction cone exists, of which the determination processes are further described in Subsection 2.2.2 and Subsection 2.2.3. Hence, it is equivalent that the contact force at a contact point should be within the friction cone. When the grasp matrix has full rank, it is indicated that contact forces and moments can completely cover three-dimensional space's 6-Degrees of Freedom (DoF). Whenever the external force or moment changes, the contact force can find solutions that balance them.

2.2.2 Determination of grasp matrix

Based on the statement that a grasp matrix $\boldsymbol{G}$ must have full rank to fulfill the stable grasp in the previous subsection, this subsection further specifies the determination of a grasp matrix. The force closure property for k grasp points $\boldsymbol{c_i} \in \mathbb{R}^3, i = 1, ..., k$, is achieved when these grasp points can resist arbitrary external wrenches with contact forces $\boldsymbol{f_i}$ at point $\boldsymbol{c_i}$ lying within the friction cone (DAI ET AL. 2018). Mathematically, force closure is formulated as the existence of $\boldsymbol{c_i}$ and $\boldsymbol{f_i}$ satisfying the constraints listed in Equation (2.6), where:

- $\boldsymbol{G}$ is the grasp matrix
- ϵ is a small given scalar
- $int(\boldsymbol{FC_i})$ is the interior of the friction cone $\boldsymbol{FC_i}$ at the grasp point $\boldsymbol{c_i}$
- S_i is the admissible contact region of the i-th contact point

$$\begin{aligned} &\boldsymbol{G}\boldsymbol{G}^T \succcurlyeq \epsilon \boldsymbol{I_{6\times 6}} \\ &\boldsymbol{G}\boldsymbol{f} = \boldsymbol{0} \\ &\boldsymbol{f_i} \in int\,(\boldsymbol{FC_i}) \\ &\boldsymbol{c_i} \in S_i \end{aligned} \tag{2.6}$$

Simultaneously, the grasp matrix $\boldsymbol{G}$ is expanded in Equation (2.7), where $\boldsymbol{p_i} = [x_i,\ y_i,\ z_i]^T$ represents the vector from CoM to the i-th contact point (DAI ET AL. 2018). Accordingly, the

grasp matrix can be determined.

$$\begin{aligned} \boldsymbol{G} &= \begin{bmatrix} \boldsymbol{I_{3\times3}} & \cdots & \boldsymbol{I_{3\times3}} \\ \hat{\boldsymbol{p}}_1 & \cdots & \hat{\boldsymbol{p}}_k \end{bmatrix} \\ \hat{\boldsymbol{p}}_i &= \begin{bmatrix} 0 & -z_i & y_i \\ z_i & 0 & -x_i \\ -y_i & x_i & 0 \end{bmatrix}, \; i = 1, ..., \; k \end{aligned} \tag{2.7}$$

2.2.3 Friction cone constraints

In Subsection 2.2.1, it is stated that if an internal force vector located within the friction cone exists, the grasping is regarded as force closure (LIU ET AL. 2011). When two objects touch, they can transmit forces through the regions of contact (MILLER and ALLEN 2004). Besides the forces transmitted along the contact normal, any contact can support some friction. In this study, the Coulomb model is introduced to determine the magnitude of forces acting in the tangent plane of the contact that can be resisted by friction, as stated in Equation (2.8), where:

- $\boldsymbol{f_t}$ is the tangential component of the contact force
- μ_s is the static friction coefficient
- $\boldsymbol{f_n}$ is the normal component of the contact force
- $\|\cdot\|$ denotes the Euclidean norm

$$\|\boldsymbol{f_t}\| \leq \mu_s \boldsymbol{f_n} \tag{2.8}$$

Figure 2.3 indicates that the forces that may be applied at the contact lay within a cone aligned with the contact normal, commonly known as a friction cone (MILLER and ALLEN 2004). The angle of this cone is $tan^{-1}(\mu_s)$. To determine μ_s, a table of the coefficient values using the two contacting material types is introduced. Simultaneously, MU and ZHANG (2019) introduce point contact friction cone constraint based on Equation (2.8). The friction cone constraint for the contact forces must be satisfied in the point contact friction model, as presented in Equation (2.9), where:

- $\boldsymbol{f_i} = \left(\boldsymbol{f_{i_x}}, \; \boldsymbol{f_{i_y}}, \; \boldsymbol{f_{i_z}}\right)^T \in \mathbb{R}^3$ denotes the force applied at the contact point $\boldsymbol{c_i}$
- $\left(\boldsymbol{f_{i_x}}, \; \boldsymbol{f_{i_y}}\right)^T$ is the tangential force
- $\boldsymbol{f_{i_z}}$ is the normal force
- μ_s^i is the static friction coefficient of the contact point $\boldsymbol{c_i}$

$$\|\left(\boldsymbol{f_{i_x}}, \; \boldsymbol{f_{i_y}}\right)^T\| \leq \mu_s^i \boldsymbol{f_{i_z}} \tag{2.9}$$

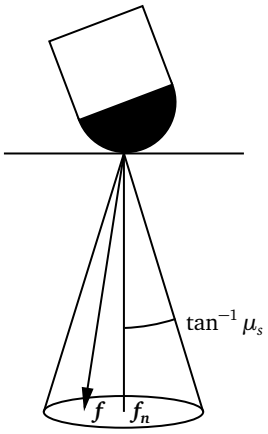


Figure 2.3: Scheme of a friction cone (MILLER and ALLEN 2004)

2.2.4 Grasp wrench space

Beyond the combination of grasp matrix (Subsection 2.2.2) and friction cone constraints (Subsection 2.2.3) considered from a mathematical perspective, this subsection describes another approach regarding force closure from a geometric perspective, which is Grasp Wrench Space (GWS). A GWS contains all wrenches, which can be counterbalanced by applying forces to k contacts for a given grasp (ZHENG 2013). This space comprises all k wrenches, constrained in the friction cone, and is presented in Equation (2.10) (LIU ET AL. 2019). Moreover, by approximating the friction cone, the set of GWS is presented in Equation (2.11) (ZHENG 2019), which is further reviewed in Subsection 3.1.1.

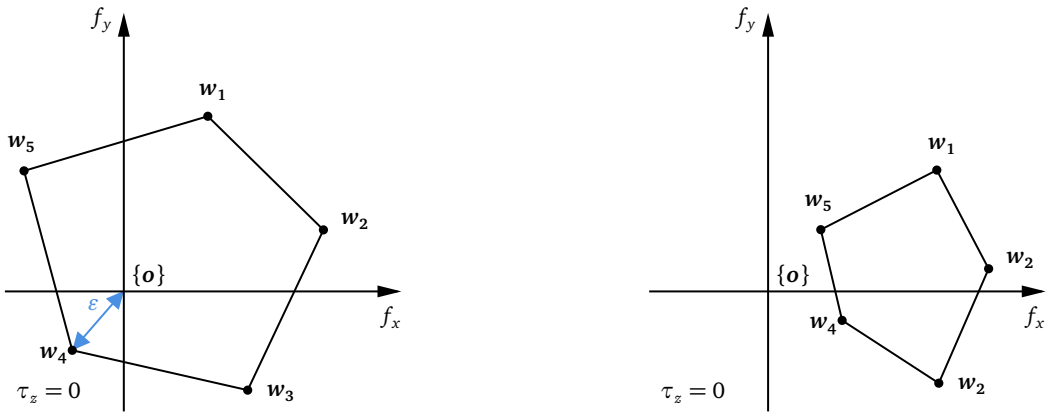
Equation (2.11) further states that a grasp has force closure if and only if the origin ($\{\boldsymbol{o}\}$) lies in the convex hull of the wrench space (SOUZA ET AL. 2021). It is further presented in Figure 2.4, where ε is an example of the quality value to define the best force closure configuration, and $\boldsymbol{w_i}$ is regarded as $\{\boldsymbol{f_x}, \boldsymbol{f_y}, \boldsymbol{\tau_z}\}$. Each contact can be modeled by a wrench vector $\boldsymbol{w}$ composed of forces and torques. All contact wrenches that mold the convex hull configuration can evaluate a grasping equilibrium. In the case of a planar multi-fingered grasp, Figure 2.4a presents a force closure grasp that the wrench space origin is included by its convex hull geometry. Contrarily, Figure 2.4b presents a non-force closure. Regarding determining a GWS, the relevant considerations and procedures are further reviewed in Section 3.1.

$$GWS = \left\{ \boldsymbol{w} \mid \left(\boldsymbol{w} = \sum_{i=1}^{k} \boldsymbol{w_i} \right) \wedge (\boldsymbol{f_i} \text{ is in the friction cone } i) \right\} \tag{2.10}$$

$$GWS = ConvexHull\left(\bigcup_{i=1}^{k} \left\{ \boldsymbol{w_1}, \quad \ldots \quad , \boldsymbol{w_k} \right\} \right) \tag{2.11}$$

2.3 Dynamics equations of robotic manipulation

With the introduced fundamentals of wrench (Section 2.1) and the force analysis (Section 2.2), this section provides the dynamics-related basics. The consideration of dynamics is



(a) Representation of force closure

(b) Representation of non-force closure

Figure 2.4: Representation of GWS analyses (SOUZA ET AL. 2021)

important because it is expected to clarify what kind of forces are included in MAGF and to specify whether additional factors exist. TANG ET AL. (2024) mention the dynamics of a manipulator system, particularly one comprising six individual links. This study states the typical robot dynamics model in operational space, as described in Equation (2.12), where:

- $\boldsymbol{\Lambda}$ is the inertia matrix term of an end-effector
- $\boldsymbol{C}(\boldsymbol{x}, \dot{\boldsymbol{x}})$ is the centripetal force term of an end-effector
- $\boldsymbol{G}(\boldsymbol{x})$ is the gravitational force term of an end-effector
- $\boldsymbol{w}_e$ is total actuation wrench applied to an end-effector
- $\boldsymbol{F}_{c_e}$ is the contact force term applied to an end-effector
- $\boldsymbol{x}$ is the operational space coordinate term of an end-effector

$$\boldsymbol{\Lambda}\ddot{\boldsymbol{x}} + \boldsymbol{C}(\boldsymbol{x}, \dot{\boldsymbol{x}}) + \boldsymbol{G}(\boldsymbol{x}) = \boldsymbol{w}_e + \boldsymbol{F}_{c_e} \tag{2.12}$$

The following subsections discuss the components in Equation (2.12). Additionally, the forces mentioned in this section are further discussed in Section 3.2 to specify the corresponding simulation implementation.

2.3.1 Gravity term

The gravity term, denoted as $\boldsymbol{G}(\boldsymbol{x})$ in Equation (2.12), is explained in this subsection. It represents the cumulative effect of gravitational forces acting on the system (ALMUSAWI ET AL. 2018). This term is fundamentally a function of the manipulator's configuration, symbolized as x, signifying that it varies under the position of each joint within the manipulator. It is responsible for the influence of the gravitational force on each link, factoring in their specific orientation and spatial positioning.

The computation of $\boldsymbol{G}(\boldsymbol{x})$ is typically derived from the kinematic model of the manipulator in conjunction with the mass properties of each link (ALMUSAWI ET AL. 2018). In scenarios involving more complicated manipulators, the intricate distribution of mass across each link, coupled with the overall configuration of the manipulator, exerts a considerable impact on the gravity term.

2.3.2 Inertia

Within this subsection, the term $\boldsymbol{\Lambda}\ddot{\boldsymbol{x}}$ in Equation (2.12) is specified. TANG ET AL. (2024) mention the importance of selecting an appropriate control approach to achieve a precise Motion Force Control (MFC) between a robot and an environment. A task space disturbance wrench can induce coupled accelerations between different subspaces because of the inherent coupling of task space inertia. In the context of MFC, the inertia matrix and its inverse can be divided into four sub-matrices. The division is presented in Equation (2.13), where:

- $\boldsymbol{A}$ is the motion subspace
- $\boldsymbol{B}$ is the force to motion subspace
- $\boldsymbol{B}^T$ is the motion to force subspace
- $\boldsymbol{D}$ is the force subspace

$$
\begin{aligned}
\boldsymbol{\Lambda} &= \begin{bmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{B}^T & \boldsymbol{D} \end{bmatrix} \\
\boldsymbol{\Lambda}^{-1} &= \begin{bmatrix} \bar{\boldsymbol{A}} & \bar{\boldsymbol{B}} \\ \bar{\boldsymbol{B}}^T & \bar{\boldsymbol{D}} \end{bmatrix} \\
\bar{\boldsymbol{D}} &= \left(\boldsymbol{D} - \boldsymbol{B}^T \boldsymbol{A}^{-1} \boldsymbol{B}\right)^{-1} \\
\bar{\boldsymbol{B}} &= -\boldsymbol{A}^{-1} \boldsymbol{B} \bar{\boldsymbol{D}} \\
\bar{\boldsymbol{A}} &= -\boldsymbol{A}^{-1} \boldsymbol{B} \bar{\boldsymbol{D}} \boldsymbol{B}^T \boldsymbol{A}^{-1}
\end{aligned}
\tag{2.13}
$$

Shaping the inertia matrix can potentially reduce or amplify this coupling effect (TANG ET AL. 2024). Inspired by Impedance Motion Force Control (IMFC), which can decouple the actuation wrench and external disturbances simultaneously, the partially decoupled IMFC is proposed to balance the trade-off between natural inertia conservation and motion force decoupling. Besides indicating its relation to existing MFC approaches, the proposed approach is further extended to multiple recursively decoupled subspaces for improved force and moment tracking. When subjected to a moment or eccentric force, the inertia matrix directly determines an object's rotational response. Correctly setting the moment of inertia ensures that objects in the simulation exhibit rotational behavior consistent with actual physics. If the moment of inertia is not set correctly, objects may exhibit unphysical motion, such as spinning too fast or oscillating.

2.3.3 Centripetal force

After the aforementioned terms, the component $\boldsymbol{C}\left(\boldsymbol{x}, \dot{\boldsymbol{x}}\right)$ is further emphasized in this subsection. WOWK (2015) focuses on the dynamics of circular motion, particularly concerning the role of centripetal force. This force, fundamental in scenarios involving rotational movement, acts inwardly from the center of rotation, as represented in Equation (2.14), where:

- $F_{centripetal}$ is the centripetal force that arises in systems subjected to motion in N
- m_o is the mass of the object in kg

- r_p is the radius of the circular path in m
- ω is the angular velocity in $\frac{rad}{s}$

$$F_{centripetal} = m_o r_p \omega^2 \tag{2.14}$$

The centripetal force is essential in applications where circular or rotational movements are integral (WOWK 2015). Equation (2.14) provides a foundational basis for predicting and managing the force that arises in systems subjected to rotational motion, enhancing the design and control of such systems.

2.4 Corresponding parameters in a simulation

Following the consideration of the robot dynamics model in the previous section, this section further specifies the parameters that should be considered in a simulation and the corresponding fundamentals. With the considered parameters, including material-related parameters, the force analysis in simulation is desired to be reality-oriented. PANASIUK (2021) presents the research results exploring which software frameworks are required to create a reliable computer simulation of automated robot grasping. Meanwhile, KOENIG and HOWARD (2004) describe that Gazebo is an open-source robotics physics simulation software utilized to simulate a robot's dynamic actions with real-world conditions, e.g., gravity, friction, inertia, and torque. Additionally, PAPADOPOULOS and KATSANOS (2023) mention that Gazebo provides an assembly space with collision handling. Furthermore, the assembly objects are constructed by three-dimensional Computer Aided Design (CAD) models accompanied with proper Simulation Description Format (SDF) files, similar to Unified Robotics Description Format (URDF) files used in ROS. Subsequently, they are further classified and described in the following subsections.

2.4.1 Collision model

Regarding the parameters in Gazebo, this subsection describes the corresponding parameters in the collision model of a SDF file. First, friction coefficients are defined based on the test object's material (PANASIUK 2021). The friction of a model can be defined by passing values for the Coulomb friction coefficient into *<mu>* and *<mu2>*, which are static friction coefficients in the first- and second-friction pyramid direction, respectively. They are introduced with the same value. RIBEIRO ET AL. (2023) define these two friction coefficients in SDF to prepare the model for a simulation. After configuring friction physics parameters, the physics engine, which is Open Dynamics Engine (ODE), is selected (RIVERA ET AL. 2019). Gazebo simulates sensor outputs, including contact sensors, by using the underlying physics engine ODE, which computes reaction forces and torques based on parameters, e.g., gravity, friction coefficients, contact depth, and object velocities (YULDASHEV ET AL. 2024).

Additionally, multiple variables are defined for the contact of the object surface with some other surface, of which *<kp>* is the stiffness coefficient in $\frac{N}{m}$, and *<kd>* is the damping coefficient in $\frac{N \cdot s}{m}$ (PANASIUK 2021). Meanwhile, it is confirmed that ODE in Gazebo introduces a penalty-based contact model with stiffness and damping that influence the contact force calculation (OPEN SOURCE ROBOTICS FOUNDATION 2014). This approach regards contacts

as a spring-damper system. Similarly, ERICKSON ET AL. (2003) mention that accurate simulation of operation tasks requires contact dynamics models, which utilize contact stiffness and damping to calculate contact forces. An approach is used for impedance and force control and represents the environment as an n-dimensional spring ($n \leq 6$), or a spring and damper, generally referred to as impedance (PIEMNGAM ET AL. 2019). Therefore, when the manipulator contacts the environment, a reaction force results, which can be defined as Equation (2.15), where:

- F_c is the contact force applied to the gripped object in N
- K_p is the equivalent stiffness coefficient in $\frac{N}{m}$
- K_d is the equivalent damping coefficient $\frac{N \cdot s}{m}$
- δ is the deformation or indentation in m
- $\dot{\delta}$ is the relative velocity of deformation or indentation in $\frac{m}{s}$

$$F_c = K_p \delta + K_d \dot{\delta} \tag{2.15}$$

Meanwhile, the damping coefficient K_d is defined as Equation (2.16), where:

- ζ is the damping ratio
- m_e is the equivalent mass in kg

$$K_d = 2 \cdot \zeta \cdot \sqrt{m_e \cdot K_p} \tag{2.16}$$

The determination of the damping ratio ζ is further presented in Equation (2.17) (GE and RICE 2018). The coefficient of restitution C_r is defined as the ratio of rebound velocity $V_{rebound}$ to impact velocity V_{impact}, as stated in Equation (2.18). However, PIEMNGAM ET AL. (2019) describes that the damping ratio ζ is generally defined as 1, which is critical damping. It gives enough damping to avoid oscillation without overshooting.

$$\zeta = -\frac{\ln C_r}{\sqrt{\pi^2 + (\ln Cr)^2}} \tag{2.17}$$

$$C_r = \frac{V_{rebound}}{V_{impact}} \tag{2.18}$$

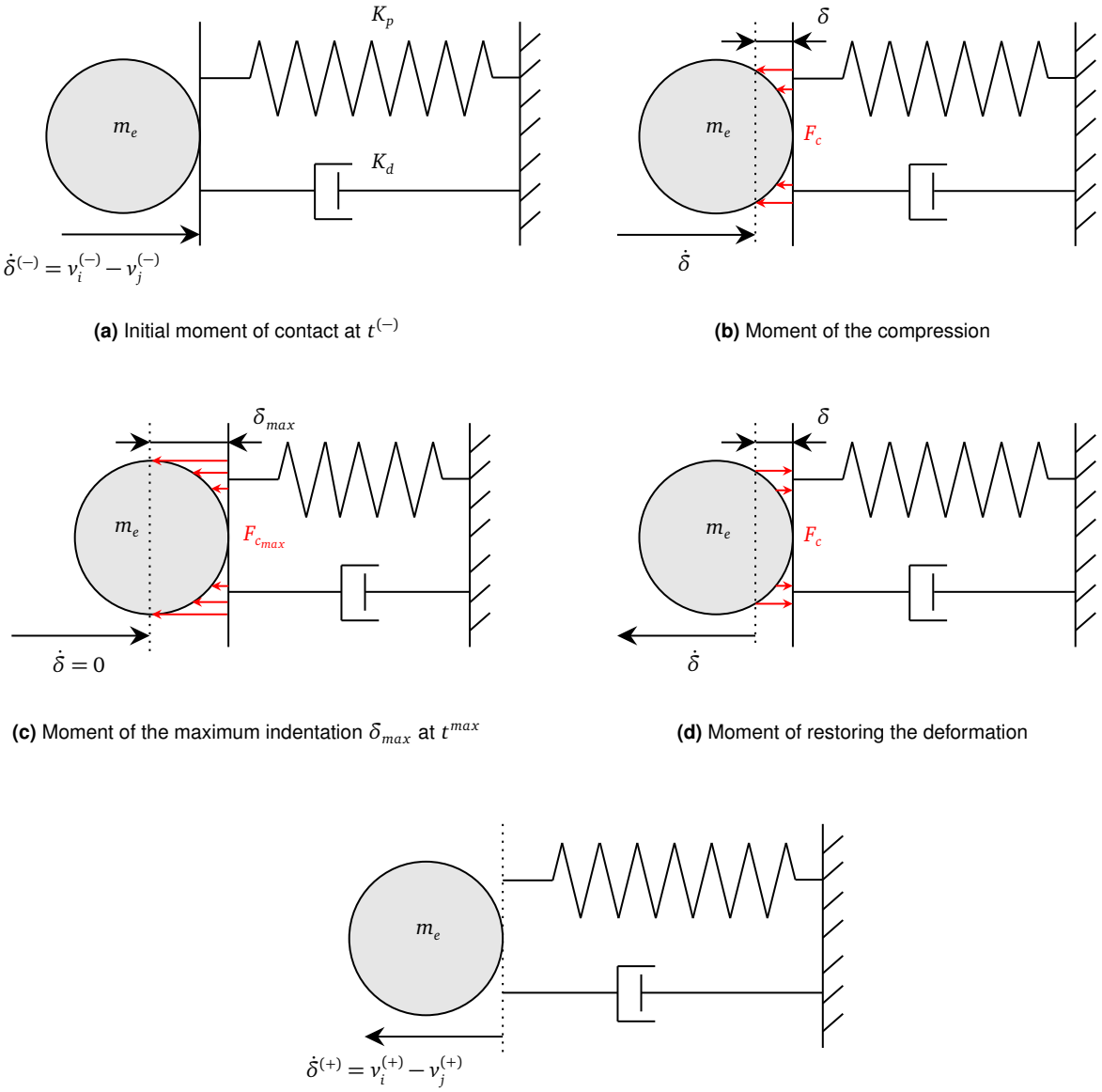
Based on Equation (2.15), ZHAO ET AL. (2021) describe the elastic compression and restitution phase between two contact subjects and specify the element $\dot{\delta}$ of the equation, as presented in Figure 2.5. The figure discusses the equivalent contact model between two solid objects, i.e., s_i and s_j. Figure 2.5a presents the initial moment of the contact. At the initial moment $t^{(-)}$ of the contact, the velocities of the two objects, $v_i^{(-)}$ and $v_j^{(-)}$ are appointed. The relative velocity at $t^{(-)}$ is defined as Equation (2.19).

$$\dot{\delta}^{(-)} = v_i^{(-)} - v_j^{(-)} \tag{2.19}$$

Figure 2.5b presents the elastic compression phase (ZHAO ET AL. 2021). In the elastic compression phase, the relative velocity gradually decreases until it is 0 at $t^{(max)}$. The maximum indentation is δ_{max}, as shown in Figure 2.5c. After the elastic compression phase, the deformation between the contact subjects starts to restore, and the contact force gradually

decreases, as presented in Figure 2.5d. Subsequently, the contact force becomes 0, as shown in Figure 2.5e. The relative velocity at $t^{(+)}$ is denoted as Equation (2.20).

$$\dot{\delta}^{(+)} = v_i^{(+)} - v_j^{(+)} \tag{2.20}$$



(a) Initial moment of contact at $t^{(-)}$

(b) Moment of the compression

(c) Moment of the maximum indentation δ_{max} at t^{max}

(d) Moment of restoring the deformation

(e) Final moment of contact at $t^{(+)}$

Figure 2.5: Process of contact compression and elastic recovery (ZHAO ET AL. 2021)

Furthermore, according to Hertzian contact theory and Kelvin-Voigt model, equivalent mass and equivalent stiffness coefficient from Equation (2.16) can be expressed as Equation (2.21) and Equation (2.22), where:

- m_i is the mass of the contact subject s_i in kg
- m_j is the mass of the contact subject s_j in kg

- K_{p_i} is the stiffness coefficient of the contact subject s_i in $\frac{N}{m}$
- K_{p_j} is the stiffness coefficient of the contact subject s_j in $\frac{N}{m}$

$$m_e = \frac{m_i m_j}{m_i + m_j} \tag{2.21}$$

$$K_p = \frac{K_{p_i} K_{p_j}}{K_{p_i} + K_{p_j}} \tag{2.22}$$

In the context of contact mechanics, particularly when modeling the elastic response of a deformable element compressed between two surfaces, a common simplification is to approximate the contact behavior as that of a linear spring (GASMI ET AL. 2011). This study further specifies the determination of a gripped object's stiffness coefficient. When contact occurs, the corresponding stiffness coefficient K_{p_o} is denoted as Equation (2.23), where:

- K_{p_o} is the stiffness coefficient of the gripped object in $\frac{N}{m}$
- E_o is the elastic modulus of the gripped object in Pa
- A_c is the contact area in m^2
- L_o is the height or thickness of the gripped object in the loading direction in m

$$K_{p_o} = \frac{E_o A_c}{L_o} \tag{2.23}$$

Consequently, the parameter $<kp>$ of a gripped object, regarded as K_{p_o}, in a SDF file can be determined. Meanwhile, the parameter $<kd>$ of a gripped object, regarded as K_{d_o}, can be determined by introducing Equation (2.16) with the mass of the object m_o (GE and RICE 2018). In the Kelvin-Voigt model, the spring is an elastic element that provides an elastic restoring force, and the dashpot provides linear viscous damping. In addition to the parameters $<kp>$ and $<kd>$, the collision model of a SDF file also requires the specification of the parameters, *<soft_Constraint Force Mixing (cfm)>* and *<soft_Error Reduction Parameter (erp)>* (RASTEGARPANAH ET AL. 2021). These parameters are crucial for accurately modeling soft contacts during collisions. Setting parameter *<soft_cfm>* has an impact on the force on the object, especially when the object is in contact with other objects. The parameter *<soft_cfm>* affects the softness and stability of the contact, thereby changing the magnitude, distribution, and behavior of the contact force.

Higher values of the parameter *<soft_cfm>* reduce the contact stiffness, thus reducing the instantaneous variation and magnitude of the contact forces (RASTEGARPANAH ET AL. 2021). It indicates that the contact points are more likely to allow slight penetration or flexible deformation, thereby reducing the peak forces applied to the object. When the parameter *<soft_cfm>* is set close to zero, the contact behaves almost entirely rigidly, which can lead to a rapid increase in contact forces and possible numerical instabilities. A proper *<soft_cfm>* value can help reduce jitter or vibration at the contact point, making the forces on the objects during contact smoother and more continuous. It is important for situations that require prolonged, stable contact, such as robotic grasping.

Simultaneously, parameter *<soft_erp>* is related to joint or constraint behavior in physics engines, such as ODE, Bullet or other similar engines, introduced to control constraint error correction in physical simulations (KALIGE 2016). In simulations, physical constraints, such

as hinge and slider joints, may fail because of numerical calculation errors or dynamic effects, causing the object to deviate from the agreed constraint position or motion range. The consideration of parameter <*soft_erp*> is a mechanism for correcting these errors at each simulation time step. The higher the value (closer to 1), the closer the system is to stiffness, and errors will be corrected quickly. The lower the value (closer to 0), the closer the system is to flexibility, and error correction will be slow or non-correct. After the explanation of the parameters, this study further specifies the determination of them. The calculation processes are presented in Equation (2.24) and Equation (2.25), where h represents the stepsize in second. Additionally, h is generally defined as 0.001 seconds. By adjusting the values of <*soft_cfm*> and <*soft_erp*>, various effects can be achieved (SMITH 2006). For example, springy constraints can be simulated, where the two bodies oscillate as though connected by springs. They determine the behavior of the constrained system, finding a balance between numerical stability and physical accuracy. By adjusting these two parameters, more realistic contact and joint behavior can be achieved in simulations.

$$<soft_cfm> = \frac{1}{h \times <kp> + <kd>} \tag{2.24}$$

$$<soft_erp> = \frac{h \times <kp>}{h \times <kp> + <kd>} \tag{2.25}$$

In simulation and materials mechanics, new calculated E^* is a parameter usually introduced to describe the effective elastic behavior of two different materials when in contact (RICARDO DE SOUZA ET AL. 2011). This equivalent modulus reflects the effect of the combined properties of the two materials on deformation or contact forces and is presented in Equation (2.26), where:

- v_o is the Poisson's ratio of the gripped object
- v_f is the Poisson's ratio of the finger
- E_o is the elastic modulus of the gripped object in Pa
- E_f is the elastic modulus of the finger in Pa

$$\frac{1}{E^*} = \left[\left(\frac{\left(1 - v_o^2\right)}{E_o} \right) + \left(\frac{\left(1 - v_f^2\right)}{E_f} \right) \right] \tag{2.26}$$

When two objects come into contact, their contact area and mechanical response depend not only on the properties of the individual materials but also on the combined properties of the two materials (RICARDO DE SOUZA ET AL. 2011). For this purpose, an equivalent modulus is used to simplify calculations, especially in Hertzian contact theory. In simulation, when two objects come into contact, how the contact force is distributed on the surface is related to E^*. Larger E^* will result in a more localized force distribution, whereas smaller E^* will spread the contact force. Accordingly, it is indicated that the parameters, <*poissons_ratio*> and <*elastic_modulus*>, must be defined in the SDF file.

2.4.2 Inertial model

After describing the parameters in the collision model in the previous subsection, they are further processed into the parameters in the inertial model in this subsection. In Subsection

2.3.2, the basics of the inertia matrix are described. Additionally, it is mentioned that the parameters, <*mass*> and <*inertia*>, are required in the SDF file (PANASIUK 2021). For the parameter <*mass*>, the mass of the object must be input in kg. Nevertheless, the parameter <*inertia*> is further composed of the parameters <*ixx*>, <*ixy*>, <*ixz*>, <*iyy*>, <*iyz*>, and <*izz*> in $kg \cdot m^2$.

The equations used to estimate the inertial parameters (mass m, coordinates of the CoM $\boldsymbol{r}$, and inertia matrix $\boldsymbol{I_{inertia}}$) of the object are derived from the fundamental laws of dynamics (KUBUS ET AL. 2007). Moreover, it is stated that an inertia matrix is presented in an approach as Equation (2.27), where:

- I_{xx}, I_{yy}, I_{zz} are the moments of inertia about the x-, y-, and z-axes, respectively
- I_{xy}, I_{xz}, I_{yz} are the products of inertia that account for coupling between different axes

$$\boldsymbol{I_{inertia}} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{xy} & I_{yy} & I_{yz} \\ I_{xz} & I_{yz} & I_{zz} \end{bmatrix} \tag{2.27}$$

If the inertia matrix is computed relative to a point other than the CoM, it can be adjusted using the parallel axis theorem (MANCHESTER and PECK 2017). The inertia matrix is presented in Equation (2.28), where d_x, d_y, d_z are the distances between the CoM and the new reference point along the x-, y-, and z-axes.

$$\boldsymbol{I_{inertia}} = m_o \begin{bmatrix} d_y^2 + d_z^2 & -d_x d_y & -d_x d_z \\ -d_x d_y & d_x^2 + d_z^2 & -d_y d_z \\ -d_x d_z & -d_y d_z & d_x^2 + d_y^2 \end{bmatrix} \tag{2.28}$$

2.4.3 Visual model

Apart from the collision and inertial models in the previous subsections, this subsection specifies the parameters in the visual model. Within the visual model, the parameters, <*geometry*> and <*material*>, should be specified (PANASIUK 2021). To specify the parameter <*geometry*>, a geometry is supposed to be assigned with <*box*>, <*cylinder*>, <*plane*>, or <*sphere*>. Subsequently, another parameter <*size*> needs to be filled with the dimension of the object in m.

Regarding the other parameter <*material*> in the visual model, it is further composed of the parameter <*script*> (PANASIUK 2021). It is the name of the material from an installed script file. It will override the color element if the script exists. The parameter <*script*> is composed of two elements, <*uri*> and <*name*>. The parameter <*uri*> describes Uniform Resource Identifier (URI) of the material script file. Additionally, the parameter <*name*> states the script's name within the script file.

2.5 Contact models

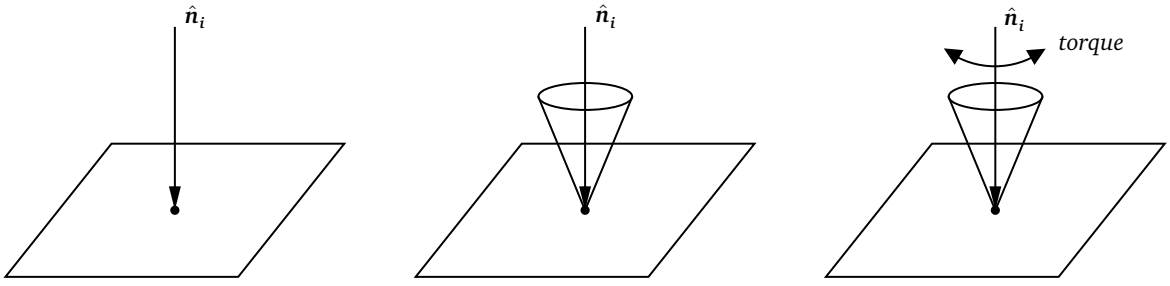
With the introduction of different parameters in Gazebo in Section 2.4, this section processes the consideration into the contact model. The alternatives of contact models are discussed

in this section. Executing tasks with a robotic hand often requires complete knowledge of the manipulated object, including its shape, rigidity, and surface texture, to ensure safe and efficient manipulation (HUI ET AL. 2017). This study proposes an approach for tracking the deformation of non-rigid objects under robotic hand manipulation utilizing Red Green Blue-Depth (RGB-D) data to automatically classify deformable objects as rigid, elastic, plastic, or elastoplastic based on the material properties. Therefore, different contact models are further reviewed to specify which models are considered. SALISBURY and ROTH (1983) propose a taxonomy of eight contact models. Among the contact models, the most commonly used in robotic grasping are hard-finger contacts without friction, hard-finger contacts with friction, and soft-finger contacts (BORST ET AL. 2003). The mentioned contact models are further described in the following subsections.

2.5.1 Hard-finger contacts

These can be modeled as single-point contacts (BORST ET AL. 2003). In the first scenario, hard-finger without friction, solely forces against the unit contact normal $\hat{n}_i$ in the contact point can be exerted on the object, as presented in Figure 2.6a. The applied force is always within the contact boundary, and no deformations are allowed at the points of contact between two bodies. These contacts occur without friction.

In the second scenario, all forces within the friction cone around the surface can be exerted normally, as indicated in Figure 2.6b (LIU ET AL. 2019). The friction cone angle is defined as $\beta = tan^{-1}(\mu_s)$, where μ_s is the static friction coefficient. The applied force has a component normal to the contact boundary and another one tangential to it. Several models have been developed to capture the essence of the complex friction phenomena. The classical model, known as Coulomb friction, is based on the principle that friction opposes motion and that its magnitude is independent of velocity and the contact area. This contact model is most commonly utilized in robotic grasping analysis.



(a) Hard-finger without friction **(b)** Hard-finger with friction **(c)** Soft-finger

Figure 2.6: Representation of different contact models (BORST ET AL. 2003)

2.5.2 Soft-finger contacts

Besides the scenarios mentioned in the previous subsection, this subsection further describes the third scenario, the soft-finger contacts model, presented in Figure 2.6c. For the scenario of soft-finger materials, it allows the application of the same forces as the hard-finger contact plus a torque around the unit contact normal $\hat{n}_i$ to the contact boundary (LIU ET AL. 2019).

The soft-finger contacts model is utilized when the surface friction and the contact patch are large enough to generate significant friction forces and a friction moment about the contact normal.

Chapter 3

State of the art

This chapter emphasizes four principal areas: methodologies researched for determining wrench spaces, forces included in a simulation, measurement of execution time in a simulation, and workspace visualization. Principally, this chapter further reviews the relevant literature based on the fundamental knowledge introduced in Chapter 2. Regarding the outset of this chapter, the related approaches for determining and verifying wrench spaces are reviewed in Section 3.1. Subsequently, Section 3.2 specifies the forces that can be simulated to fulfill the requirements of a stable grasp. Apart from the stable-grasp-related literature review, the approaches for measuring execution time in a simulation are discussed in Section 3.3. Furthermore, Section 3.4 specifies the consideration of end-effector positioning and workspace visualization.

3.1 Determination and analysis of grasp wrench space

This section reviews the determination and analysis of GWS to specify its approaches considering the basics stated in Subsection 2.2.4. Regarding a GWS, the force applied by a finger at the point of contact generates a wrench on the object with force and torque components, represented by the vector $w_i \in \mathbb{R}^6$, as presented in Equation (2.4) (BOHG ET AL. 2024). Simultaneously, the scheme of the contact sample is presented in Figure 3.1, where $\boldsymbol{r}$ is the position of the object's CoM in m, and $\boldsymbol{n_i}$ is the contact normal in m (LIU ET AL. 2019). Therefore, the element $\boldsymbol{d_i}$ of Equation (2.4) can further be replaced by $\boldsymbol{c_i} - \boldsymbol{r}$. Consequently, it is indicated that the preliminary required elements for specifying wrenches of GWS are presented as follows:

- Positions of contact points
- Contact forces of contact points
- Position of CoM of a gripped object

Furthermore, depending on the different definitions of the primitive contact forces, various wrench-oriented grasp quality measures can be defined uniformly as the minimum distance from the origin to the boundary of the GWS (ZHENG 2019). However, the above-listed elements are insufficient to construct a GWS. Each contact point does not apply force along a single direction, such as the surface normal. Instead, it can exert a range of forces within a friction cone determined by the contact friction coefficient. Therefore, the friction cone must

be approximated by a finite number of discrete vectors, and this consideration is further reviewed in Subsection 3.1.1. Following the construction of a GWS, the conditions considered between a GWS and force closure are reviewed in Subsection 3.1.2. Lastly, Subsection 3.1.3 further describes the consideration of other wrench spaces related to stable grasp.

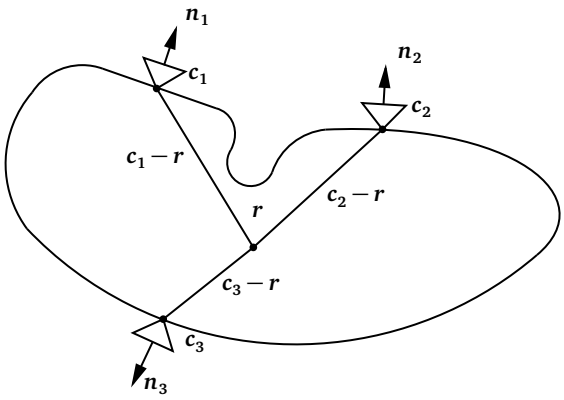


Figure 3.1: A sample of three contacts on a planar object (LIU ET AL. 2019)

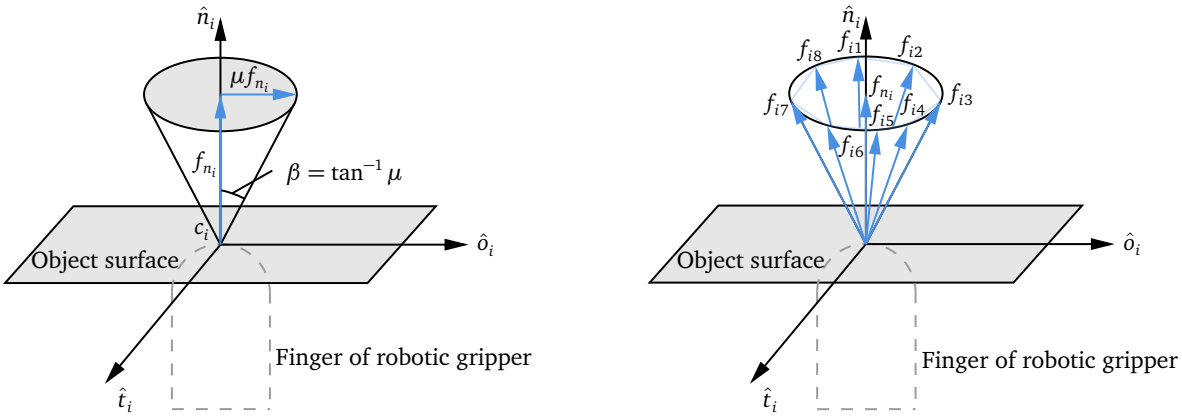
3.1.1 Friction cone approximation

With the fundamental elements listed in the previous segment, this subsection further reviews the details of GWS determination. The accurate construction of GWS relies heavily on modeling contact forces, particularly those constrained by friction (LIU ET AL. 2019). One primary challenge in this process is representing the friction cone, which governs the allowable forces at each contact point under consideration of friction. Mathematically, the normal force bounds a friction cone at the contact point, and the tangential forces are constrained by the coefficient of friction, presented in Figure 3.2a. In this figure, the set of all forces that can be applied is constrained to lie inside the centroid of a cone with a surface normal and an angle $\beta = tan^{-1}(\mu_s)$, where μ_s is the static friction coefficient.

Although this continuous representation is precise, it poses significant computational challenges because of its infinite-dimensional nature. Therefore, the continuous friction cone is approximated with a polyhedral cone (MILLER and ALLEN 2004). It is achieved by discretizing the cone into a finite number of force vectors, known as generators, which approximate the cone's surface with a proper pyramid with n_f faces, presented in Figure 3.2b (LIU ET AL. 2019). The larger the n_f, the better the approximation. A polyhedral approximation allows the friction cone to be represented as a finite set of linear constraints, which is computationally manageable for optimization and analysis tasks.

3.1.2 Approaches for verification

After considering the last procedure of constructing a GWS in the previous subsection, it is further processed into the verification of a GWS. In Subsection 2.2.4, it is stated that a grasp has force closure if and only if the origin ($\{\boldsymbol{o}\}$) lies in the convex hull of the wrench space, which guarantees that a gripped object is immobile under external perturbations (SOUZA



(a) Spatial representation of friction cone **(b)** Approximation with 8 faces

Figure 3.2: Different representation of friction cone (LIU ET AL. 2019)

ET AL. 2021). The approaches for verifying this concept are reviewed in the following segments.

Linear programming

Regarding the first approach for verification, linear programming is introduced (DING ET AL. 2000). The linear programming formulation starts by expressing the contact wrenches as $\boldsymbol{w_1},\ \boldsymbol{w_2}, ...,\ \boldsymbol{w_k}$, where each wrench represents the force and torque at a contact point. To determine whether the origin ($\{\boldsymbol{o}\}$) lies within the convex hull of these wrenches, this study solves the following linear programming problem: find non-negative weights $\lambda_1,\ \lambda_2, ...,\ \lambda_k$ such that the conditions stated in Equation (3.1) are satisfied. This formulation ensures that the origin can be expressed as a convex combination of the wrenches, satisfying the conditions for inclusion within a GWS.

$$\sum_{i=1}^{k} \lambda_i \boldsymbol{w_i} = \boldsymbol{0},\ \sum_{i=1}^{k} \lambda_i = 1,\ \lambda_i > 0,\ i = 1,\ 2, ...,\ k \tag{3.1}$$

Geometrically, the linear programming solution determines whether a feasible set of weights exists to span the origin using the given wrenches (DING ET AL. 2000). If a feasible solution is found, it confirms that the origin ($\{\boldsymbol{o}\}$) lies within the convex hull, and hence, the grasp satisfies the force closure condition. This approach is computationally efficient, allowing rapid feasibility checks. Moreover, this study integrates this linear programming formulation into an iterative ray-shooting problem that adjusts the contact points dynamically. This process refines the convex hull representation until the origin ($\{\boldsymbol{o}\}$) is confirmed to lie within it, ensuring force closure.

Hyperplane

Apart from the consideration of linear programming, LIU (1999) proposes another approach to evaluate whether the origin ($\{\boldsymbol{o}\}$) lies within a GWS by introducing hyperplanes. A GWS is represented as a convex polytope formed by the combination of contact forces subject to friction constraints. Each hyperplane of this polytope corresponds to a linear inequality derived from the constraints on the grasp's contact forces. The polytope is bounded by hyperplanes defined as Equation (3.2), where:

- A_h is the matrix of hyperplane coefficients
- x_h is a vector of variables
- b_h represents the offsets

$$A_h x_h \leq b_h \tag{3.2}$$

To analyze whether the origin ($\{o\}$) lies within the GWS, LIU (1999) utilizes linear programming to verify the feasibility of the inequalities. If the origin satisfies all constraints, it lies inside the GWS, confirming the grasp's ability to counteract arbitrary external wrenches.

3.1.3 Introduction of other wrench spaces

With the defined verification approach of GWS for force closure in the previous subsection, this subsection further reviews the other considerations related to stable grasp. Regarding the fundamental force analysis of a gripped object in two-dimensional, the generalized force ellipse describes the range and efficiency of external and internal forces within the grasping system (JIA ET AL. 2017). The ellipses are derived using the Jacobian matrix of forces and the relationship between torque and external forces. They effectively characterize the dynamics of the robotic hand in a given grasp configuration. Additionally, an ellipse describes the distribution characteristics of contact forces more accurately than a circle, especially in grasping planning, where the force limits at contact points are often non-uniform. For the material-related consideration, different surfaces or directions may exhibit distinct maximum force capacities because of grain boundary anisotropy (ZHOU ET AL. 2020). Therefore, the force analysis presentation should be in the form of an ellipse.

Nevertheless, to combine the force analysis in three-dimensional space's 6-DoF with the introduction of GWS and MAGF of a gripped object, BORST ET AL. (2004) mention the consideration of Object Wrench Space (OWS), which refers to the set of external wrenches that can potentially disturb an object during manipulation or while executing a task. Basically, OWS is specific to the task and depends on the physical properties and requirements of the object and the environment. Therefore, it can be regarded as a wrench space considering MAGF.

Within this thesis, another terminology named Task Wrench Space (TWS) is introduced (BORST ET AL. 2004). The wrenches expected to occur for a given task can be specified as a TWS. It can be distinguished into two segments: either the task to be executed is known, and a specification in the wrench space is given, or the task is unknown, and no specification exists. Regarding the given task description, if there is a detailed description of the task given by a set of wrenches that are applied to an object during the manipulation, a corresponding convex hull over these task disturbance wrenches can be utilized as a TWS. Furthermore, the relationship between GWS and TWS can be presented in Figure 3.3. To specify that the transportation of an object can be achieved safely, TWS must be overlapped within GWS.

3.2 Forces recognized from a simulation

Accompanied by the specified wrench spaces and the required forces for constructing a GWS in Section 3.1, this section reviews the forces included in a simulation and the corresponding approaches for recognition. In addition, this section is split into two subsections because

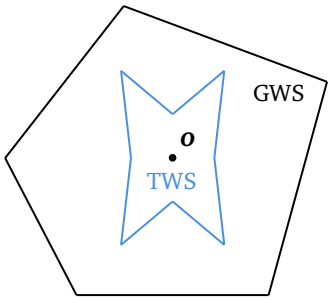


Figure 3.3: Scheme of the relationship between GWS and TWS (BORST ET AL. 2004)

different simulated forces are accessed directly or indirectly. Meanwhile, the robot dynamics model stated in Section 2.3 is considered within this section as basics.

3.2.1 Direct extraction of data

This subsection reviews the forces that can be directly simulated and recognized from a simulation. As mentioned in Section 2.4, Gazebo is an open-source robotics physics simulation software utilized to simulate a robot's dynamic actions with real-world conditions (PANASIUK 2021). Additionally, PAPADOPOULOS and KATSANOS (2023) state that Gazebo provides an assembly space with collision handling. Furthermore, the assembly objects are constructed using three-dimensional CAD models accompanied with proper SDF files, similar to URDF files used in ROS.

Regarding the force measurement in Gazebo, QIAN ET AL. (2014) mention that there are many virtual sensors in Gazebo, for example, tactile sensors and force/torque sensors, which can help a manipulator to grasp and place objects more steadily. A force/torque sensor measures a wrench in the robot structure (HOFFMAN ET AL. 2014). At the time of writing, the sensor is simulated in Gazebo as if it is attached to the reference frame associated with a joint. This study introduces another middleware framework YARP. The reading of a generic sensor is implemented as a *IAnalogSensor* interface, presented in Figure 3.4. The broadcasted data is a vector of six numbers representing the forces and the torques applied on that reference frame. At every step, the internal state of the plugin is updated with the last readings of forces and torques from the simulation.

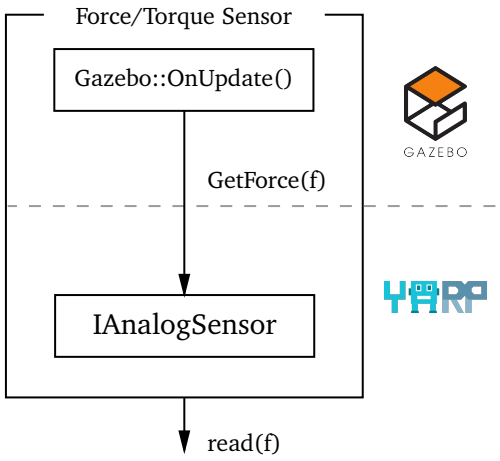


Figure 3.4: Scheme of force/torque sensor implemented as a YARP interface (HOFFMAN ET AL. 2014)

With the basics of getting access to the data of a force/torque sensor, the contact force is measured using tri-axial optical force sensors placed on the fingertips of the fully actuated fingers (MAŃKOWSKI ET AL. 2020). These sensors enable the gripper to measure reaction forces during manipulation, providing detailed feedback for precisely handling objects. Additionally, the usability of the force sensors attached to the fingertips is verified. As the model of the hand is defined in ROS, the direction of contact forces can be easily determined in three-dimensional space. The position of each fingertip and corresponding contact forces can be visualized and are available at any time for the control modules.

Furthermore, data regarding joint states of the robot, when the index finger touches the surface of an object in Gazebo, is captured (RIBEIRO ET AL. 2023). Meanwhile, values of the vertical component of contact force are stored utilizing the force feedback sensor in the index finger. Additionally, once the manipulator's end-effector is in contact with the surface, the z-component of force measurements from the 3-axis force/torque sensor, which is embedded into the 7-th robot's link, increases tenfold (SHAFIKOV, SAGITOV, ET AL. 2020). The increase allows for detecting a contact during movement towards the surface.

Subsequently, before the sensor data can be used, the sensor must be calibrated (GREGOR ET AL. 2021). A series of experiments are executed to determine the calibration parameters. As an example, the calibration of the sensor in the z-axis is described. The sensor is placed such that the gravitational force acts solely in the opposite direction of its axis. A zero-output value is assumed at zero static load, and a load of 1 kg yields an output value of $-9.81\ N$. Moreover, an Inertial Measurement Unit (IMU) plugin is utilized in Gazebo to measure velocity, orientation, and gravitational forces (HOFFMAN ET AL. 2014).

Apart from contact force and gravitational force, reaction force can be simulated (MOWER ET AL. 2022). Through the configuration file for the robot, it is possible to instantiate a simulated force/torque sensor attached to any joint on the robot. These virtual sensors publish joint reaction forces, read from PyBullet, as ROS wrench-stamped messages at a user-defined sampling frequency.

3.2.2 Indirect extraction of data

After discussing the direct extraction of forces from Gazebo in the previous subsection, this subsection reviews the other forces that can be determined based on the direct extracted forces and the basics of robot dynamics model stated in Section 2.3. In the previous subsection, it is described that the z-component of force measurements from the 3-axis force/torque sensor can be utilized to detect a contact (SHAFIKOV, TSOY, ET AL. 2020). Within this study, data from the sensor during the palpation procedure is encoded into a marker message and is published as a topic. From there, these data may be fetched and visualized via RViz, a three-dimensional visualizer for ROS framework. Moreover, it is described that the contact force reported from Gazebo contact topic represents the contact force in Equation (2.15). Thus, obtaining a color-coded map of normal force values for each examined point of the palpated surface is possible.

Besides considering normal contact force, tangential contact force is applied when two bodies come into contact with each other, as stated in Equation (3.3) (FLORES 2022), where:

- f_c is the vector of contact force applied to the object in N
- f_n is the vector of normal force applied to the object in N

- f_t is the vector of tangential force applied to the object in N

$$f_c = f_n + f_t \tag{3.3}$$

Simultaneously, Equation (3.3) is further expanded into Equation (3.4), where $\hat{n}$ is the unit normal vector of the contact surface. Therefore, it is indicated that the tangential force can further be determined by considering contact force and normal force.

$$f_t = f_c - (f_c \cdot \hat{n})\hat{n} \tag{3.4}$$

Based on the statements in Section 2.3, the centripetal forces resulting from rotation are also considered in dynamic conditions. Regarding the determination of the centripetal force, the robot is equipped with two virtual IMUs utilizing Gazebo plugin (VELASCO 2024). The first IMU is positioned at the reference point of the coordinate system in the base link and captures the robot's accelerations. The second IMU is situated at the Tool Center Point (TCP) and records the orientation of the vase relative to the base link. Accordingly, the centripetal force applied to the object can be determined.

3.3 Measurement of execution time in a simulation

After reviewing the stable-grasp-consideration in Section 3.1 and Section 3.2, it is processed into the consideration of execution time in this section. This section primarily reviews the approaches for measuring execution time in a simulation, a critical analysis element within this thesis. The measurement of execution time is primarily conducted in MoveIt2, which is a plugin involved in ROS2 (STOOP ET AL. 2024). In the MoveIt2 framework, execution time is analyzed through benchmarking experiments that evaluate motion planning and execution durations under different parameter settings.

Meanwhile, STOOP ET AL. (2024) propose two approaches regarding time measurement. The interface *MoveGroupCommander* in MoveIt2 is imported to monitor execution time from planned trajectory. Additionally, the function *plan()* is established based on this interface. Therefore, the time-related information can be accessed by referring to the joint trajectory points from this function. To further acquire the specific points of time, two indexes, $[0]$ and $[-1]$ of the trajectory, are introduced to the respective variables, representing the time of start and end of an operation. Moreover, the function *nanosec()* is considered in the implementation. This function makes the time presentation much more precise, including the information after decimal. Consequently, the corresponding execution time derived from a planned trajectory can be determined by referring to the difference between these variables. The interfaces, *MoveGroupCommander* and *time*, are introduced for the other proposed approach. Nevertheless, this approach is proposed to monitor the actual execution time. Simultaneously, two variables representing the time of start and end of an operation are defined by referring to the function *time()* of the interface *time*. Hence, the execution time of an operation in MoveIt2 can be determined. Within the discussion segment of this study, another framework in MoveIt2, MoveIt Task Constructor (MTC), is mentioned, designed to simplify the planning and execution of complex robotic tasks. It provides a modular and flexible framework for breaking down tasks into smaller planning stages and combining them sequentially or in parallel to accomplish an overall goal. Each stage in MTC can have its execution time computed after applying time parameterization and produces a robot trajectory. Consequently, the function *getDuration()* can be implemented to obtain the execution time for that stage.

Apart from the proposed approaches in the last paragraph, the control feedback in MoveIt2 is engaged to obtain the corresponding execution time (PICKNIK ROBOTICS 2024). When executing a trajectory in MoveIt2, the actual motion is implemented by a controller called *FollowJointTrajectory*. However, the interfaces, *ActionClient* and *JointTrajectoryPoint*, must be imported additionally. The controller can provide feedback during execution, including the start of an operation, the current robot state, the reached trajectory points, and the completion or failure of an operation. Principally, the start time of an operation is captured when the goal is accepted, and its end time is obtained when the result is received after the trajectory execution is complete.

As another approach for the measurement of execution time tailored to specific scenarios, such as multi-robot systems and dynamic environments, a centralized scheduler for asynchronous trajectory execution in MoveIt2 is introduced (SHAFIKOV, TSOY, ET AL. 2020). The introduction significantly reduces total execution time in multi-robot setups. Simultaneously, execution time is quantified by assessing the total time taken for independent trajectories of multiple robots to complete. It includes measuring the reduction in total execution time compared to synchronous motion planning methods, which is featured within the interface *MoveGroup*.

Simulations are further utilized to assess execution performance, enabling the identification of critical parameters that influence time efficiency (GRUSHKO ET AL. 2020). Similarly, time parameterization algorithms like Iterative Parabolic Time Parameterization (IPTP) and Time Optimal Trajectory Generation (TOTG), as discussed by TEISSING (2021), are employed to compute time-optimal trajectories based on the robot's kinematic constraints, ensuring smooth and efficient execution. This approach offers precise estimation of execution time while improving trajectory quality.

3.4 Positioning of end-effector and workspace visualization

With the approaches discussed in the previous section for measuring execution time, this section discusses the processing of a robotic manipulator's workspace. Regarding the workspace, the relevant approaches for positioning an end-effector are first reviewed in Subsection 3.4.1 and Subsection 3.4.2. Subsequently, Subsection 3.4.3 describes the corresponding method for documenting the collected data. Lastly, Subsection 3.4.4 reviews the primary approaches for visualizing a robotic manipulator's workspace.

3.4.1 Forward kinematics

For the positioning of an end-effector, this subsection reviews the approaches by which the position is determined by calculating kinematics. Forward kinematics specifies the joint parameters, and a kinematic equation is introduced to compute the position of an end-effector from a specified value for each joint parameter (SINGH ET AL. 2017). Subsequently, another terminology, Denavit-Hartenberg (DH) transformation, is introduced. DH transformation is utilized for selecting a frame of reference for the robotic manipulator. In this convention, coordinate frames are attached to the joints between two links to describe the location of each link relative to its previous.

By introducing DH transformation, the homogeneous transformation matrix ${}^{j-1}_{j}\boldsymbol{R}$ with the index of joint j can be easily defined, describing the position of a point on an object and the orientation of the object in a three-dimensional space (ZHANG and WANG 2012). The homogeneous transformation matrix can be derived from one frame to another by referring to DH parameters. At the robot's base, it can start with the first joint and transform into the second joint. Sequentially, it will be transformed into the joint of the end-effector. For example, a kinematics model of the flexible drilling robot is developed within this study, as presented in Figure 3.5, of which the defined coordinate systems are described as follows:

- {0} is the base coordinate system
- {1} ∼ {3} are robot main body coordinate systems
- {4} is the end-effector A axis coordinate system
- {5} is the end-effector B axis coordinate system
- {6} is the spindle feeder coordinate system

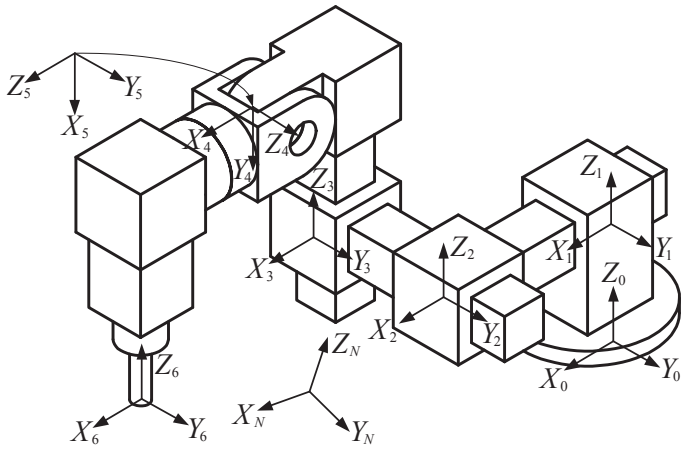


Figure 3.5: Kinematic model of the flexible drilling robot (ZHANG and WANG 2012)

Within this example, the total homogeneous transformation matrix ${}^{0}_{6}\boldsymbol{R}$ between the base of the manipulator of 6 joints and the spindle feeder of the last coordinate system is presented in Equation (3.5). Accordingly, the orientation and position of the end-effector can be determined.

$$ {}^{0}_{6}\boldsymbol{R} = {}^{0}_{1}\boldsymbol{R}\,{}^{1}_{2}\boldsymbol{R}\,{}^{2}_{3}\boldsymbol{R}\,{}^{3}_{4}\boldsymbol{R}\,{}^{4}_{5}\boldsymbol{R}\,{}^{5}_{6}\boldsymbol{R} \tag{3.5} $$

Subsequently, DH transformation is introduced in the motion planning framework. In MoveIt, which is visualized by RViz, the class *moveit::planning_interface::MoveGroup* and its function *getCurrentPose()* are implemented to obtain the current position of the end-effector (GREGOR ET AL. 2021). Furthermore, the robot's position within a trajectory is available through the ROS TCP Endpoint package in MoveIt2 (YOO and CHOI 2024).

3.4.2 Direct estimation in a simulation

With the mathematical approach for positioning an end-effector described in the previous subsection, this subsection reviews the approach for its direct detection. Regarding the end-

effector performance, industrial robots are generally solely equipped with motor-side encoders (WANG ET AL. 2012). Accurate estimating the end-effector position and velocity is thus difficult because of complex joint dynamics.

To overcome this problem, a method to estimate the end-effector pose of an industrial serial manipulator without relying on the robot's encoders is proposed (ORTEGA ET AL. 2023), as presented in Figure 3.6, where $\boldsymbol{P}$ is the pose of end-effector in m, and ResNet50 is Convolutional Neural Network (CNN) architecture that belongs to the Residual Networks (ResNet) family. Three RGB-D cameras are imported into the robot workspace, feeding a CNN and a cascade estimator to determine the three-dimensional coordinates of every joint sequentially. Regarding the implementation, a robotic manipulator model UR5 of 6-DoF from Universal Robots is employed in Gazebo for the corresponding experimental setup.

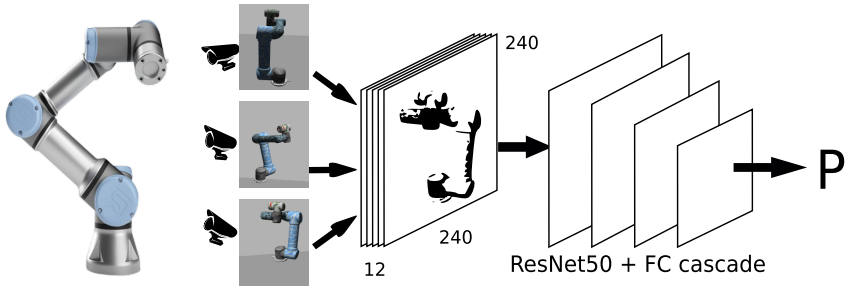


Figure 3.6: The proposed system for end-effector pose estimation (ORTEGA ET AL. 2023)

3.4.3 Documentation and visualization of data

After discussing the approaches for positioning an end-effector from Subsection 3.4.1 to Subsection 3.4.2, it is processed into the documentation of data in a simulation. As stated in the previous subsection, the plugins, RViz and MoveIt, are often imported (ORTEGA ET AL. 2023). To record the position data after the input of commands mentioned in Subsection 3.4.1, the package *rosbag* included in ROS is introduced (KIM ET AL. 2017). Through the usage of the package *rosbag*, the trajectory messages published on specified ROS topics can be recorded with the command *rosbag record*. Subsequently, a file with the format *bag* will be generated, and its content can be further visualized (CASAN ET AL. 2015).

While the package *rosbag* provides powerful features for recording and replaying ROS messages, the introduction of its Application Programming Interface (API) allows developers to programmatically interact with the files of format *bag*, as presented in Figure 3.7 (LI ET AL. 2017). The node *move_group* gets three kinds of information in ROS parameter server: URDF, Semantic Robot Description Format (SRDF) and MoveIt Configuration. It can realize the function in three ways, i.e., C++ APIs, Python APIs, and Graphical User Interface (GUI). Accordingly, further functions of *rosbag* can be easily implemented in Python.

3.4.4 Approaches for workspace visualization of a robotic manipulator

Subsection 3.4.3 describes that an URDF file, which is composed of the CAD models of a robotic manipulator's links, is included in the node *move_group* for the visualization (LI ET AL. 2017). Therefore, this subsection further reviews the approaches for visualizing a robotic

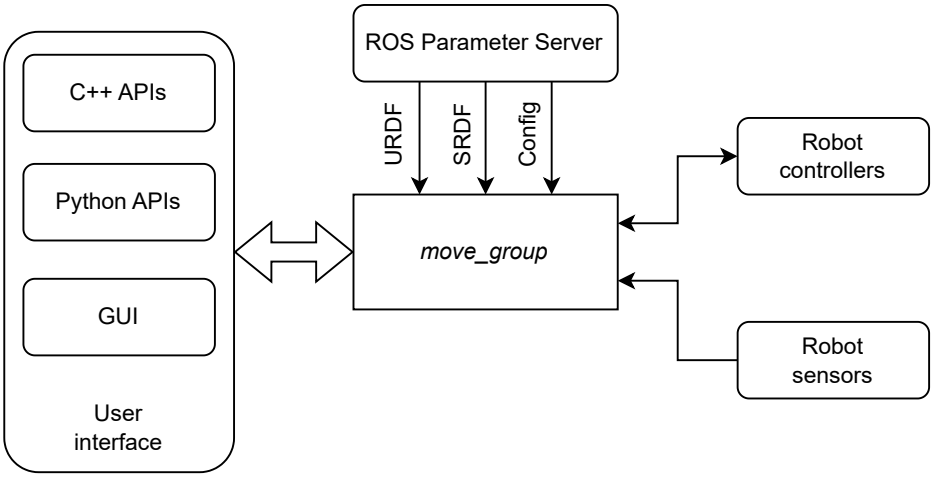


Figure 3.7: The overall structure of MoveIt (LI ET AL. 2017)

manipulator. Regarding the first approach of visualization, HORNUNG ET AL. (2013) discuss the OctoMap framework's capabilities in representing three-dimensional environments using probabilistic occupancy grids based on Octree. For visualizing robotic manipulators in conjunction with OctoMap, external tools and frameworks are typically employed, e.g., MoveIt and RViz. PULJIZ ET AL. (2020) further integrate OctoMap with a robotic manipulator to enable efficient setup and safe operation of robots in shared workspaces, as presented in Figure 3.8. The OctoMap and user-defined safety zones are exported to MoveIt. Meanwhile, the OctoMap is utilized as the collision environment for the robotic manipulator. This study describes that the links of the robotic manipulator can be inserted into a OctoMap as collision objects. Each time the robotic manipulator moves, the new robot position is added to the OctoMap cumulatively. Therefore, OctoMap will gradually build up a three-dimensional volume cloud, representing the workspace covered by the entire robotic manipulator.

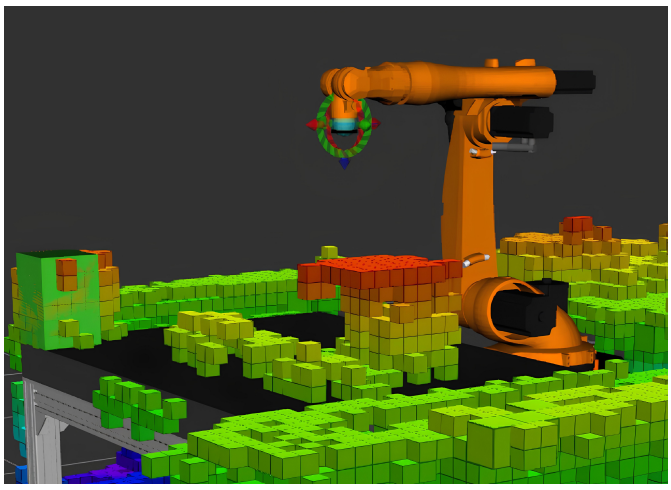


Figure 3.8: Presentation of an environment using OctoMap (PULJIZ ET AL. 2020)

Besides the introduction of OctoMap, COLEMAN ET AL. (2014) research the motion planning framework MoveIt, as shown in Figure 3.9, where the gray blocks represent external input and output. This study specifies that the robot model is the data structure and accompanying file format used to describe the three-dimensional geometric representation of a robotic manipulator, including the geometric visualization meshes. Simultaneously, a mesh is in-

troduced in the gray block of joint states in Figure 3.9. Consequently, the approach for visualization utilizing a mesh file is further reviewed. TOLA and CORKE (2024) mention that an URDF file provides a well-structured Extensible Markup Language (XML) format that allows for parsing of a robotic manipulator's links and corresponding mesh geometries. Many robotic systems introduce pre-existing CAD models for visualization. An URDF file supports the use of external mesh files, e.g., Standard Triangle Language (STL), OBJ, Digital Asset Exchange (DAE), which is associated with the COLLADA three-dimensional file format. As presented in Figure 3.10, an URDF file refers to the STL files of a robotic manipulator to construct the corresponding model in a simulation. Meanwhile, most robotic visualization tools, e.g., RViz, Gazebo, and MoveIt, support mesh parsing from an URDF file, ensuring integration into simulations.

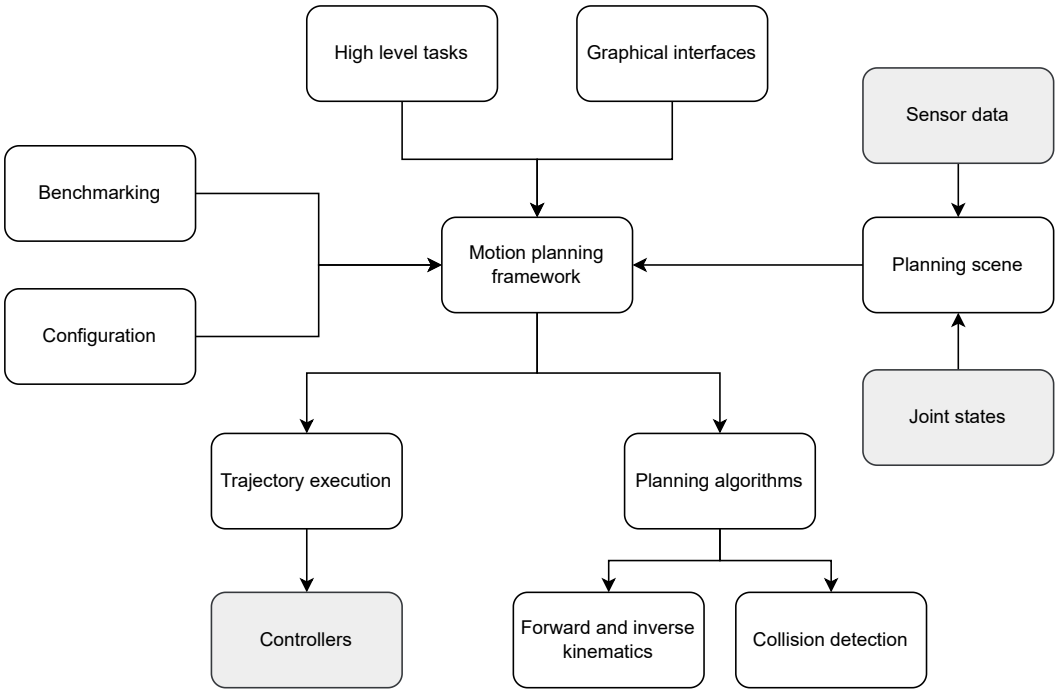


Figure 3.9: The planning components in the motion planning framework MoveIt (COLEMAN ET AL. 2014)

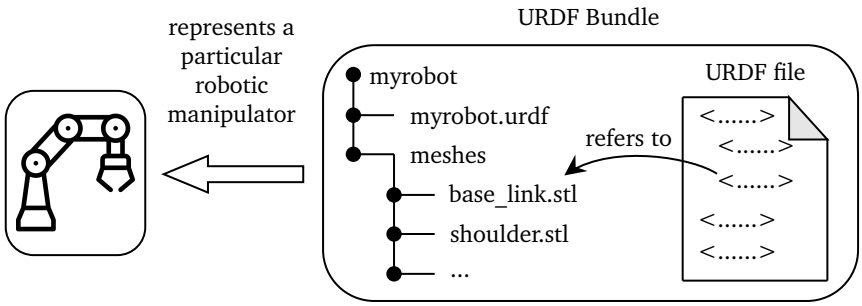


Figure 3.10: Example of a URDF bundle (TOLA and CORKE 2024)

Chapter 4

Design

In this chapter, the corresponding principles introduced in Chapter 2 are utilized as fundamental factors to consider. The parameters and determination approaches mentioned are also included. This chapter refers to the related techniques concerning the three primary aspects, i.e., stable grasp determination, execution time measurement, and workspace visualization, reviewed in Chapter 3. This chapter aims to design a comprehensive approach for verifying these three aspects, building on the fundamentals introduced in Chapter 2. Therefore, this chapter provides an overview of the verification procedure combined with an industrial robotic handling skill. With the objectives and scenario defined in Section 4.1, the design processes are further divided into the consideration of stable grasp (Section 4.2), the determination of execution time by multiple execution cycles (Section 4.3), and the visualization of a used workspace (Section 4.4).

4.1 Objective definition and scenario configuration

This section describes the whole process and presents an overview of the approach. Figure 4.1 illustrates the scheme of the objectives considered in this thesis. This thesis's core is determining an industrial robotic handling skill with a scenario, defining related parameters, and developing a complete approach for the corresponding objectives listed as follows:

- Verification of stable grasp of a handling skill
- Determination of execution time of a handling skill
- Visualization of the used workspace of a handling skill

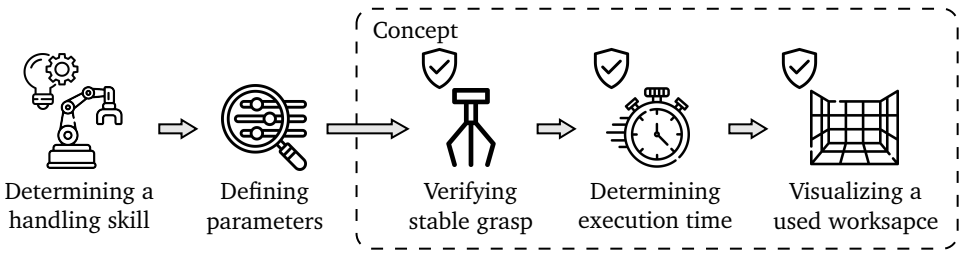


Figure 4.1: Scheme of this thesis's objectives

Concerning the first procedure, an industrial robotic handling skill must be determined before designing the concept of the three objectives listed above. PRASAD and DHANRAJSEKHAR (2023) mention that pick-and-place task is a robotic manipulator's most common handling skill in the industrial sector. Therefore, a pick-and-place task with a gripped object is regarded as the handling skill in this thesis and is referred to in the following verification processes.

After determining the handling skill, it is processed into the design of a scenario. In addition, the beginning of Chapter 4 describes that the execution times are expected to be measured by multiple cycles. Thus, the scenario should include pick-and-place tasks with multiple-cycle objects, as presented in Figure 4.2. The designed scenario in the figure comprises 10 execution cycles. An execution cycle is composed of 4 steps, which are moving, grasping, moving, and releasing (Table 4.1). After finishing the first execution cycle, the robotic manipulator will continue with the second execution cycle on another cube. Simultaneously, the conveyor belt will transport the first cube into the basket.

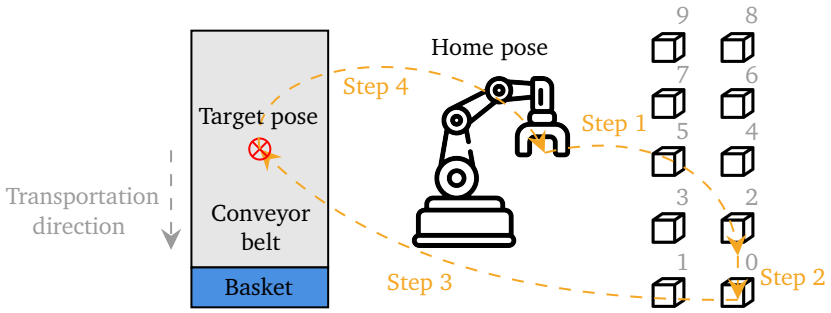


Figure 4.2: Scheme of the pick-and-place scenario of this thesis

Table 4.1: Explanation of pick-and-place scenario steps

Step	Explanation
1	Moving from the home pose to the grasp pose of the cube
2	Grasping the cube and picking it up
3	Moving from the grasp pose to the target pose with the cube being gripped
4	Placing the cube and moving from the target pose to the home pose

4.2 Consideration of stable grasp

Following the scenario defined in Subsection 4.1, this subsection describes the concept of the first objective, i.e., the verification of stable grasp. The verification concept is defined based on Chapter 2. Moreover, Figure 4.3 presents the flowchart of the concept. The overall verification is divided into two aspects:

- Static stable grasp analysis (from step *S1* to step *S6*)
- Dynamic stable grasp analysis (step *S7*)

Concerning the two aspects of analysis, static analysis is the period in which the robotic manipulator's gripper contacts a cube before picking it up. Therefore, the former part of step

2 in Table 4.1 is considered in the static analysis. Subsequently, the latter part of step 2 and step 3 in Table 4.1 are considered in the dynamic analysis. From the step *S1* of Figure 4.3, the input parameters in a SDF file and the contact models will be determined, which is discussed in Subsection 4.2.1. With the determined parameters, a simulation will be implemented (step *S2*). Subsequently, Subsection 4.2.2 describes the verification of force closure (step *S3*). After the determination of force closure, the consideration of a gripper's Maximum Grasp Force (MGF) (step *S4*), an object's MAGF (step *S5*), and the other wrench spaces (step *S6*) are included in Subsection 4.2.3. Lastly, Subsection 4.2.4 discusses the analysis of dynamic conditions (step *S7*).

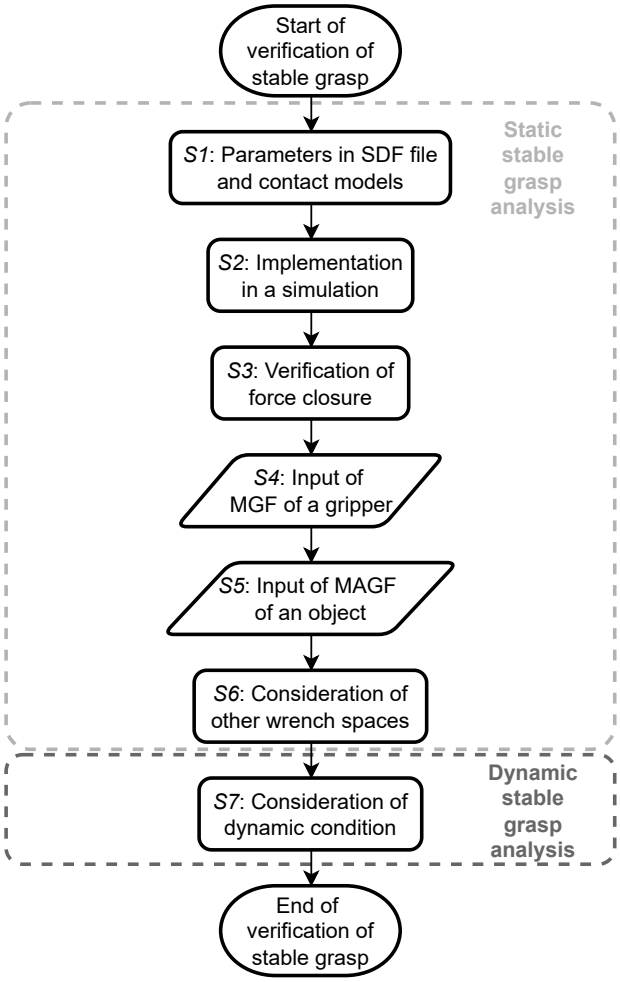


Figure 4.3: Flowchart of the stable grasp verification concept

4.2.1 Overview of required elements as input

This subsection first defines an overview of the required input parameters of a gripped cube in a SDF file, of which the cube is presented in Figure 4.2. Hence, this subsection is combined with the description of Section 2.4. The parameters must be determined because of the introduction of ODE in Gazebo. Section 2.4 divides the parameters into three models, i.e., collision, inertial, and visual models. Correspondingly, Table 4.2 presents an overview of the considered input parameters in a SDF file and specifies based on which equations the parameters are determined.

Table 4.2: Overview of the required input parameters in a SDF file and the corresponding equations

Model	Parameter	Unit	Reference for determination
<*collision*>	<*mu*>	-	-
	<*mu2*>	-	-
	<*poissons_ratio*>	-	-
	<*elastic_modulus*>	-	-
	<*kp*>	$\frac{N}{m}$	Equation (2.23)
	<*kd*>	$\frac{N \cdot s}{m}$	Equation (2.16)
	<*soft_cfm*>	-	Equation (2.24)
	<*soft_erp*>	-	Equation (2.25)
<*inertial*>	<*mass*>	kg	-
	<*ixx*>	$kg \cdot m^2$	Equation (2.27)
	<*ixy*>	$kg \cdot m^2$	Equation (2.27)
	<*ixz*>	$kg \cdot m^2$	Equation (2.27)
	<*iyy*>	$kg \cdot m^2$	Equation (2.27)
	<*iyz*>	$kg \cdot m^2$	Equation (2.27)
	<*izz*>	$kg \cdot m^2$	Equation (2.27)
<*visual*>	<*size*>	m	-

Nevertheless, regarding the input of the two parameters, <*poissons_ratio*> and <*elastic_modulus*>, it is mentioned that the parameters are not directly input with the values of the cube's material characteristics (OPEN SOURCE ROBOTICS FOUNDATION 2014). This source describes that the value of <*poissons_ratio*> must be regulated within the interval $[-1, 5]$. A range of values with the corresponding materials is provided in Table 4.3. Additionally, the range of <*elastic_modulus*> value is defined within the interval $[0.2, 0.5]$, and the listed materials with values are also in Table 4.3. Regarding the determination of the parameters <*kp*> and <*kd*> of the cube, Equation (2.23) and Equation (2.16) are utilized, respectively. However, when determining <*kd*>, the cube's mass must be introduced into the equation, and the damping ratio ζ is defined as 1 (PIEMNGAM ET AL. 2019). Equation (2.24) and Equation (2.25) further introduce the determination of the parameters <*soft_cfm*> and <*soft_erp*>. However, it is indicated that the stepsize h is included. Concerning the value determination, h is defined as 0.001 seconds, as described in Subsection 2.4.1 (KALIGE 2016).

Table 4.3: Value references of <poissons_ratio> and <elastic_modulus> with the corresponding materials (OPEN SOURCE ROBOTICS FOUNDATION 2014)

Material for reference	Elastic Modulus for reference (GPa)	Value in SDF
Typical steel	-	0.3
Typical silicone elastomers	-	$[0.49, 0.50]$
Plastic	$[0.1, 3.0]$	$[0.35, 0.41]$
Wood	$[4.0, 10.0]$	$[0.22, 0.50]$
Aluminum	70.0	$[0.32, 0.35]$
Steel	200.0	$[0.26, 0.31]$

Following the collision model, the inertial model of the cube needs to be defined, including <*mass*> and <*inertia*>. The parameters from <*ixx*> to <*izz*> can be calculated based on Equation (2.27). Moreover, RIVERA ET AL. (2019) indicate that an inertia matrix of the cube can be accessed in SolidWorks with the cube's three-dimensional CAD model. Sequentially, concerning the visual model, the parameter <*geometry*> in Subsection 2.4.3 is first defined as <*box*> because of the geometry of the cube's geometry. The parameter <*size*> is further determined by the dimension of the cube in *m*.

In Section 2.5, three primary contact models, i.e., hard-finger without friction, hard-finger with friction, and soft-finger, are introduced (BORST ET AL. 2003). After confirming the required parameters in a SDF file, it is further processed into a combination of the parameters and the contact models. Table 4.4 specifies which parameters will be considered in the corresponding contact models. Moreover, the parameter <*inertia*> includes the parameters from <*ixx*> to <*izz*> in Table 4.2. The thorough collision model is not considered for a hard-finger without friction contact model because the contact is assumed to be rigid and frictionless, indicating no deformation and tangential forces. Therefore, the material properties involved in the collision model are irrelevant. The friction cone is further considered in a hard-finger with friction contact model, as presented in Figure 2.6b. Hence, the parameters <*mu*> and <*mu2*> are included.

Lastly, because the soft-finger contact model allows not only the transmission of forces but also a frictional moment around the contact normal (Figure 2.6c), it assumes a finite contact area and material compliance at the contact patch (LIU ET AL. 2019). Therefore, mechanical properties, i.e., <*poissons_ratio*>, <*elastic_modulus*>, <*kp*>, and <*kd*>, must be considered to model how the contact deforms under load. Additionally, since the model accounts for surface friction forces and frictional moments, parameters, i.e., <*mu*>, <*mu2*>, <*soft_cfm*>, and <*soft_erp*>, become important to accurately represent the frictional behavior and the numerical stability of soft contact.

Table 4.4: Consideration of the parameters in a SDF file combining the contact models

Parameters	Hard-finger without friction	Hard-finger with friction	Soft-finger
<*collision*> model			
<*mu*>	-	•	•
<*mu2*>	-	•	•
<*poissons_ratio*>	-	-	•
<*elastic_modulus*>	-	-	•
<*kp*>	-	-	•
<*kd*>	-	-	•
<*soft_cfm*>	-	-	•
<*soft_erp*>	-	-	•
<*inertial*> model			
<*mass*>	•	•	•
<*inertia*>	•	•	•
<*visual*> model			
<*size*>	•	•	•

4.2.2 Force closure aspect

After the approach for determining the parameters in a SDF file, a simulation will be implemented, which is further described in Section 5.2. This subsection discusses the verification of force closure, as step *S3* in Figure 4.3. In addition, this subsection is described primarily based on Section 2.2, Section 3.1, and Section 3.2. The verification concept comprises the segments listed in Table 4.5, and the steps are further expanded in the form of a flowchart in Figure 4.4. In step *S3.1*, the contact topic in Gazebo will be introduced to determine the contact forces applied to a gripped cube in Figure 4.2. It is introduced because Subsection 3.2.2 mentions that the contact forces reported from Gazebo contact topic are the forces applied to a gripped object. Step *S3.1* is further explained in Subsection 5.3.1. Regarding the contact topic in Gazebo, OPEN SOURCE ROBOTICS FOUNDATION (2014) indicates that the returned information of contact between two objects included in the topic is listed as follows:

- Contact positions
- Contact normals
- Contact forces
- Contact torques

Table 4.5: Concept and steps of force closure verification

Step	Explanation
S3.1	Introduction of contact topic in a simulation
S3.2	Verification of a grasp matrix
S3.3	Verification of friction cone constraints
S3.4	Verfication of GWS

Accordingly, it is processed into step *S3.2*, of which the concept is based on the description in Subsection 2.2.1 and Subsection 2.2.2. The first framework in Figure 4.4 presents the flowchart of the grasp matrix verification concept. In Subsection 2.2.2, the approach for determining a grasp matrix is indicated in Equation (2.7). Therefore, the contact positions and CoMs of a gripped cube of the grasp are desired for the determination (step *S3.2.1* and step *S3.2.2*). With constructed grasp matrices (step *S3.2.3*), step *S3.2.4* is responsible for checking if all the grasp matrices have full rank to specify the fulfillment of grasp matrix verification.

Subsequently, step *S3.3* of Table 4.5 is discussed in this paragraph, of which the second framework in Figure 4.4 shows the flowchart of the friction cone verification concept. The concept is developed based on Subsection 2.2.3, which introduces the basics of a friction cone and its constraints. Step *S3.3.1* first processes the data of contact forces through the contact topic. With the processed contact forces, step *S3.3.2* will be responsible for determining the corresponding tangential- and normal forces based on Equation (3.3) and Equation (3.4). Thus, the processing of the contact normals will also be implemented and is described in Subsection 5.3.1. According to Equation (2.8), the introduction of a static friction coefficient is necessary to determine the corresponding frictional force (step *S3.3.3*), which is regarded as the parameter <*mu*> in a SDF file. Consequently, step *S3.3.4* determines if Equation (2.8) is fulfilled to specify the verification.

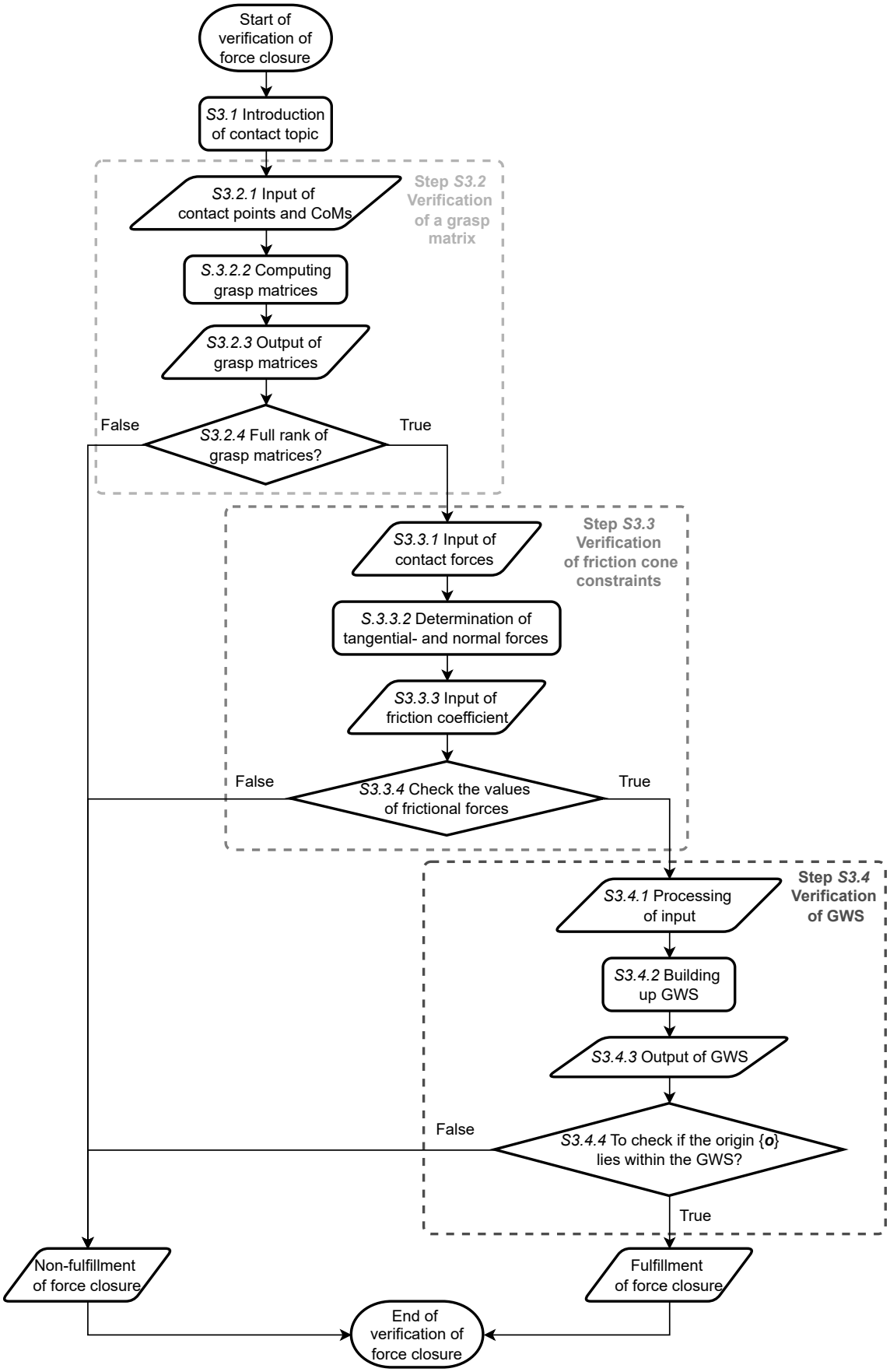


Figure 4.4: Flowchart of the force closure verification concept

Based on Subsection 2.2.4, Subsection 3.1.1, and Subsection 3.1.2, this paragraph describes the considered processes regarding the last segment of the force closure verification concept, i.e., step *S3.4* in Table 4.5. Simultaneously, the third framework in Figure 4.4 presents the flowchart of the GWS verification concept. In step *S3.4.1*, the relevant data must be input for building up a GWS. As described in the first paragraph of Section 3.1, the preliminarily required elements for a GWS are contact positions, contact forces, and CoMs of the gripped cube in scenario (Figure 4.2). However, MAGF of the cube will be regarded as the contact force of this verification because it is desired that the range of a GWS can be enlarged. The most important verification procedure is determining if the origin lies within a GWS. Following the preliminary input elements, step *S3.4.2* will further require the other three input elements, i.e., contact normals, static friction coefficient <*mu*>, and the number of friction cone approximation. In Subsection 3.1.1, it is mentioned that the friction cone approximation is crucial for the accurate construction of a GWS, and the other two elements are utilized to determine the frictional forces. After combining the calculated frictional forces with the preliminary input elements for torques, a GWS can be defined (step *S3.4.3*). Lastly, step *S3.4.4* determines whether the origin ($\{\boldsymbol{o}\}$) lies within a GWS for the fulfillment of the verification, of which the approach hyperplane in Subsection 3.1.2 will be utilized. Hence, the steps for verifying force closure in Table 4.5 are thoroughly discussed.

4.2.3 Consideration of other wrench spaces

With the force closure verification discussed in Subsection 4.2.2, this subsection explains the following three steps in Figure 4.3, i.e., step *S4*, step *S5*, and step *S6*. MGF of a gripper in *N* (step *S4*) and MAGF of the cube in the proposed scenario in *N* (step *S5*) are considered because the reachable ability of a gripper is desired to be defined. Meanwhile, damage to the cube must be avoided. After considering the two values, Figure 4.5 shows the verification concept flowchart of step *S6*. In addition, the concept is developed based on Subsection 3.1.3. The necessary input in step *S6.1* is listed as follows:

- Contact forces of the gripped cube
- Contact torques of the gripped cube
- Contact positions between a gripper and the gripped cube
- CoMs of the gripped cube
- A constructed GWS from Subsection 4.2.2

Through contact forces and contact torques of a contact, a TWS can be determined based on Subsection 3.1.3 (step *S6.2*). With the contact positions and CoMs of the gripped cube, the relevant torques concerning a gripper's MGF and the gripped cube's MAGF can be calculated because the distances between contact points and CoMs are known. Therefore, the other wrench spaces, i.e., a gripper's Maximum Grasp Force-Wrench Space (MGF-WS) and the gripped cube's Maximum Admissible Grasp Force-Wrench Space (MAGF-WS), can be defined (step *S6.3* and step *S6.4*). For the last segment of the verification concept (step *S6.5*), Figure 3.3 is further expanded as Figure 4.6. Apart from the description that TWS must be overlapped within GWS, TWS must be further overlapped within MGF-WS and MAGF-WS. Thus, it can be confirmed that the verification is fulfilled. In addition, the verification concept of static stable grasp analysis in Figure 4.3 (from step *S1* to step *S6*) is completely explained.

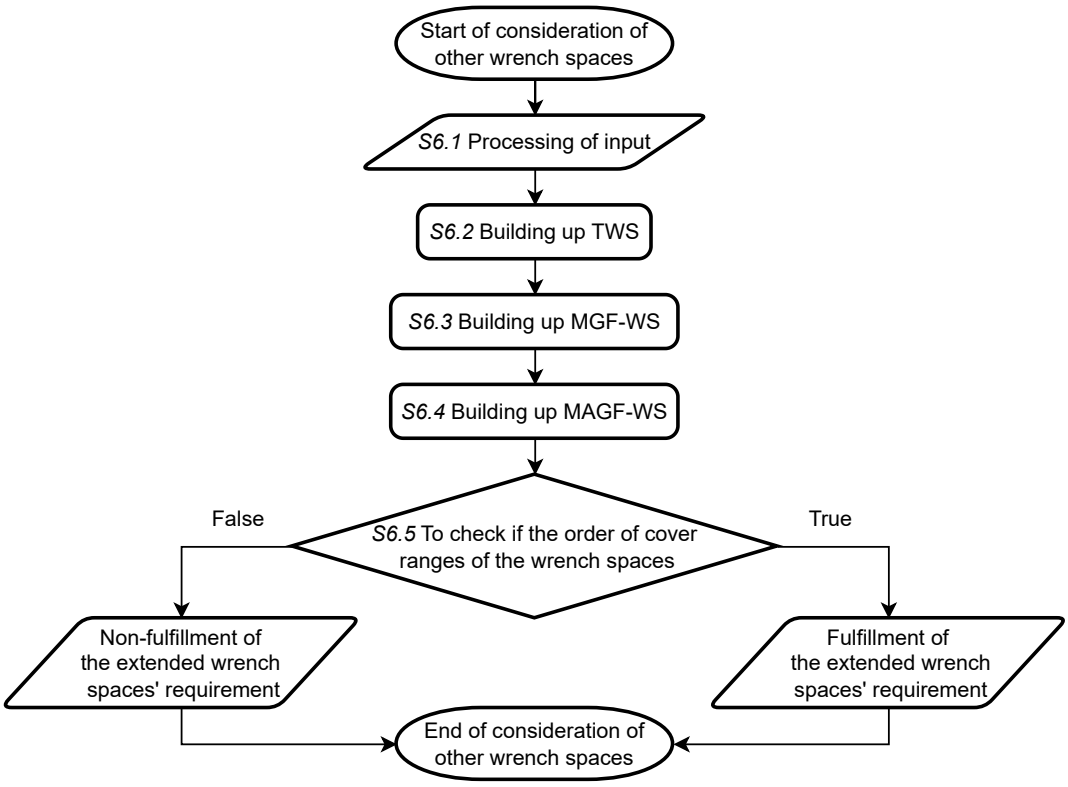


Figure 4.5: Flowchart of the verification process for extended wrench spaces (TWS, MGF-WS, MAGF-WS)

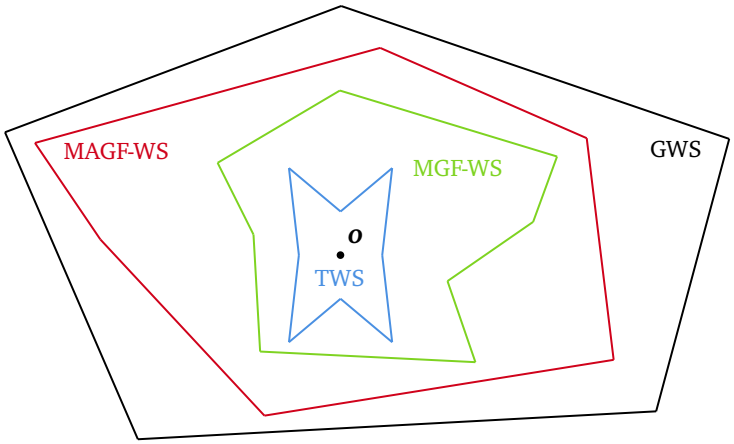


Figure 4.6: Scheme of the relationship between all of the wrench spaces

4.2.4 Consideration of dynamic condition

Following the defined concept of static stable grasp analysis in the previous subsections, this subsection describes the verification concept of dynamic stable grasp analysis (step *S7* in Figure 4.3). The step is further expanded into Figure 4.7. In addition, the concept in Figure 4.7 is developed primarily based on Section 2.3 and Subsection 3.2.1. The typical robot dynamics model is discussed in Equation (2.12). It describes what forces a robot's end-effector will be subjected to when it moves in space, from an actuation wrench $\boldsymbol{w_e}$ and an external contact force $\boldsymbol{F_{c_e}}$, to produce the desired acceleration. Nevertheless, a gripped cube in the scenario is now being analyzed rather than an active controller or linkage. The movement of a gripped cube does not require consideration of the robotic manipulator's own inertia matrix and control input. Therefore, the forces applied to a gripped cube in dynamic conditions directly depend on its corresponding acceleration.

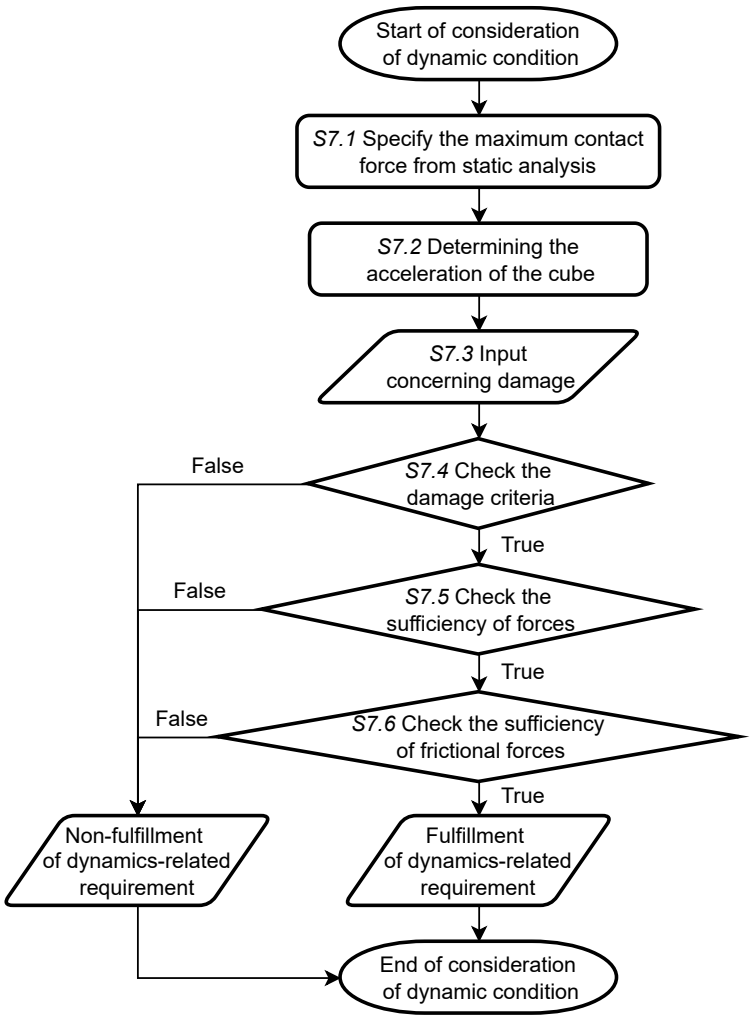


Figure 4.7: Flowchart of dynamic condition verification concept

Step *S7.1* specifies a maximum contact force F_{max_s} in N from the static analysis because the maximum grasp force of a gripper at a particular pose must be determined. The value of F_{max_s} represents the force a gripper can apply to the cube of the dynamic grasping process, as the latter part of step 2 and step 3 in Table 4.1. Step *S7.2* will first access a gripped cube's CoMs and take the second derivative of them to determine its accelerations, a_{c_s} in $\frac{m}{s^2}$, within the dynamic process. As stated in Subsection 3.2.1, the corresponding forces applied to a

gripped cube, F_{d_c} in N, can thus be calculated. With the calculated forces of a gripped cube, it is further processed into the consideration of a cube's MAGF because damage to the cube must be avoided. Step *S7.3* requires the input as follows:

- Yield strength σ_{y_c} of a gripped cube in MPa
- Contact area A_c between a gripped cube and a gripper in mm^2

After determining input, step *S7.4* divides the calculated dynamic forces of a gripped cube by the contact area to specify whether a calculated stress surpasses yield strength. When the condition is not satisfied, it will be regarded as non-fulfillment. Step *S7.5* further determines whether the maximum contact force F_{max_s} is enough to hold the dynamic forces F_{d_c}. However, regarding the determination, F_{d_c} will be divided by 2 first because a gripper with two parallel flat fingers is considered, as described in Equation (4.1). Consequently, step *S7.6* is responsible for specifying whether the frictional force derived from F_{max_s} is enough to hold the tangential forces derived from F_{d_c}. Accordingly, Equation (3.4) can be expanded as Equation (4.2), where $\hat{\boldsymbol{n}}_{\boldsymbol{d}}$ represents unit contact normals of a gripped cube. This step can determine if a gripped cube will slip based on Equation (4.2), and all the stable grasp verification concept steps in Figure 4.3 are thoroughly described.

$$\frac{F_{d_c}}{2} \leq F_{max_s} \tag{4.1}$$

$$\frac{F_{d_c} - \left(F_{d_c} \cdot \hat{\boldsymbol{n}}_{\boldsymbol{d}}\right)\hat{\boldsymbol{n}}_{\boldsymbol{d}}}{2} \leq F_{max_s} \cdot \hat{\boldsymbol{n}}_{\boldsymbol{d}} \cdot \mu_s \tag{4.2}$$

4.3 Execution time by multiple cycles of a handling skill

With the stable grasp verification concept defined in Section 4.2, this section continues to consider the execution time by multiple cycles of the pick-and-place handling skill, i.e., the second objective listed in Section 4.1. In addition, the approach for measuring an execution time is defined based on Section 3.3. In Section 3.3, it is mentioned that the measurement of execution time is primarily conducted in MoveIt2 (STOOP ET AL. 2024). Moreover, this study determines the execution time of a trajectory by specifying two point indexes, $[0]$ and $[-1]$ of a trajectory. These two points refer to the time of start and end of an execution cycle, and this approach is further utilized in this section.

However, the definition of a cycle's execution time is introduced in this paragraph first. Based on the scenario's scheme in Figure 4.2 and steps described in Table 4.1, the cycle's execution time is further defined as an execution from the beginning of step 1 to the end of step 3. After an implemented simulation, there will be 10 cycles. Therefore, the execution time t_q of q-th cycle is defined as Equation (4.3), where:

- S_{1_q} is step 1's execution time of q-th cycle in second
- S_{2_q} is step 2's execution time of q-th cycle in second
- S_{3_q} is step 3's execution time of q-th cycle in second

$$t_q = S_{1_q} + S_{2_q} + S_{3_q},\ q = 1, 2, ..., 10 \tag{4.3}$$

For example, S_{1_q} represents the cost time that the robotic manipulator moves from the home pose to the grasp pose of the box (Figure 4.2). Based on the approach mentioned in the first paragraph, two points of time will be defined to determine a cycle's execution time. The first point of time is defined as the start of step 1 in Figure 4.2. Thus, the robotic manipulator is located at the home pose at this point of time, and the start point of time of q-th cycle is defined as t_{q_s} in second. Furthermore, q-th cycle's endpoint of time t_{q_e} is defined as the end of step 3. Equation (4.3) can be expanded as Equation (4.4), of which a cycle's execution time is defined as the time difference of two points of time.

$$t_q = t_{q_e} - t_{q_s},\ q = 1, 2, ..., 10 \tag{4.4}$$

4.4 Visualization of used workspace

Following the two objectives described in Section 4.2 and Section 4.3, this section explains the concept for visualizing the used workspace of the defined pick-and-place handling skill, which is the last objective stated in Section 4.1. In Subsection 3.4.4, two approaches, OctoMap and mesh geometry, are reviewed for visualizing a robotic manipulator's workspace. The latter option is selected as the basis for further development because it is easier for ROS plugins to parse an URDF file. Subsequently, Figure 4.8 presents the flowchart of the visualization concept. After an implementation in a simulation (step *V1*), which is further described in Section 5.2, step *V2* accesses the used workspace of the defined scenario in Figure 4.2. With a robotic manipulator's parsed meshes, the outcome of the used workspace is supposed to be presented in a cluster of point clouds because the vertices of meshes are parsed.

Equipped with a cluster of point clouds, step *V3* will prompt the user to determine if there are any other obstacles near the robotic manipulator. A non-collision workspace will be fulfilled if there is no obstacle near it. However, if a user reports an obstacle, step *V4* will require the coordinates of an obstacle. Meanwhile, the coordinates must be input in a cluster of point clouds. Subsequently, step *V5* is responsible for determining whether there is any overlap between the used workspace's point cloud and the obstacle's point cloud to specify the fulfillment of a non-collision workspace. When an overlap indicates a workspace with a collision, step *V6* will incorporate the obstacle into the simulation and re-implement it. With the implementation considering an input obstacle, it will be processed to step *V2* again to access the newly used workspace and start another new analysis cycle. Therefore, the proposed approach is regarded as an iterative loop and will finally achieve a non-collision workspace.

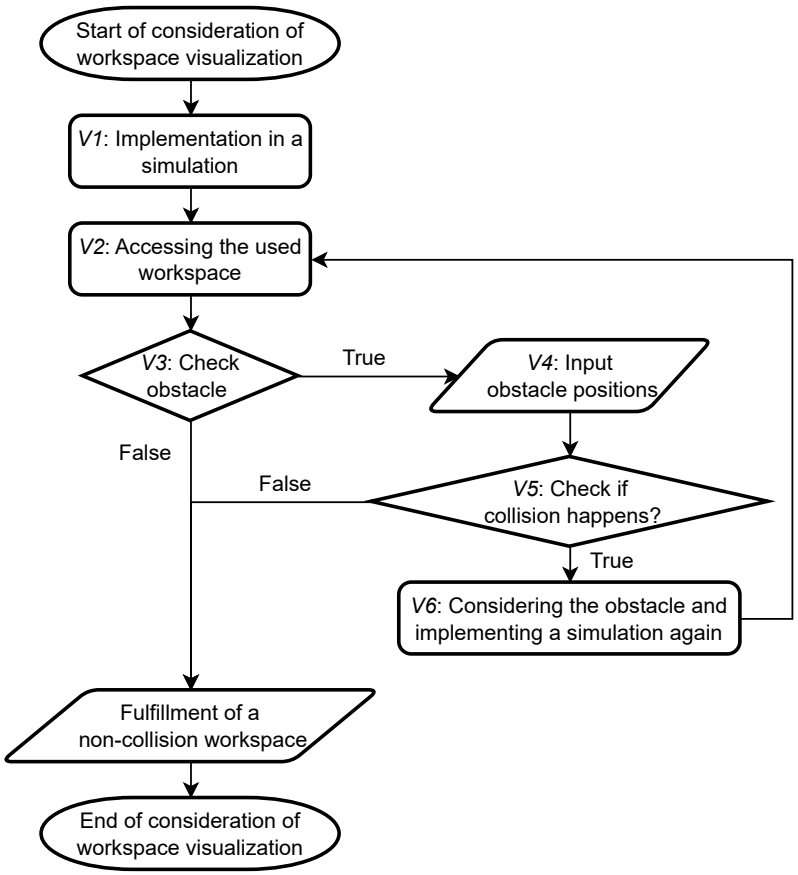


Figure 4.8: Flowchart of used workspace visualization concept

Chapter 5

Implementation

This chapter introduces the implementation after explaining the thorough conceptual design process in Chapter 4 for achieving the pick-and-place scenario and the three primary objectives. Figure 5.1 presents the structure overview of the implementation process and indicates the interaction between the different aspects: pre-processing data, planning motion and collecting data, visualizing and collecting data, and analyzing data. All proposed approaches in Chapter 4 are implemented in Ubuntu 22.04 Long Term Support (LTS), which the Linux operating system supports, and the Ubuntu system is installed under the virtual machine VMware. It is implemented in Linux because the framework ROS2 is the primary tool introduced in this thesis. Regarding the segment data pre-processing, Python is introduced in Visual Studio Code (VS Code). After preparing the data, the plugin MoveIt2 of ROS2 is introduced to plan the motion of a robotic manipulator. However, the other programming language, C++, is mainly utilized in VS Code in segment planning motion because the framework MoveIt2 is more compatible with C++. Meanwhile, another plugin RViz of ROS2 is responsible for visualizing the motion planning in MoveIt2, and the other plugin Gazebo is introduced to visualize an overall simulation environment and consider physical interactions. Lastly, Python is used to conduct data analysis.

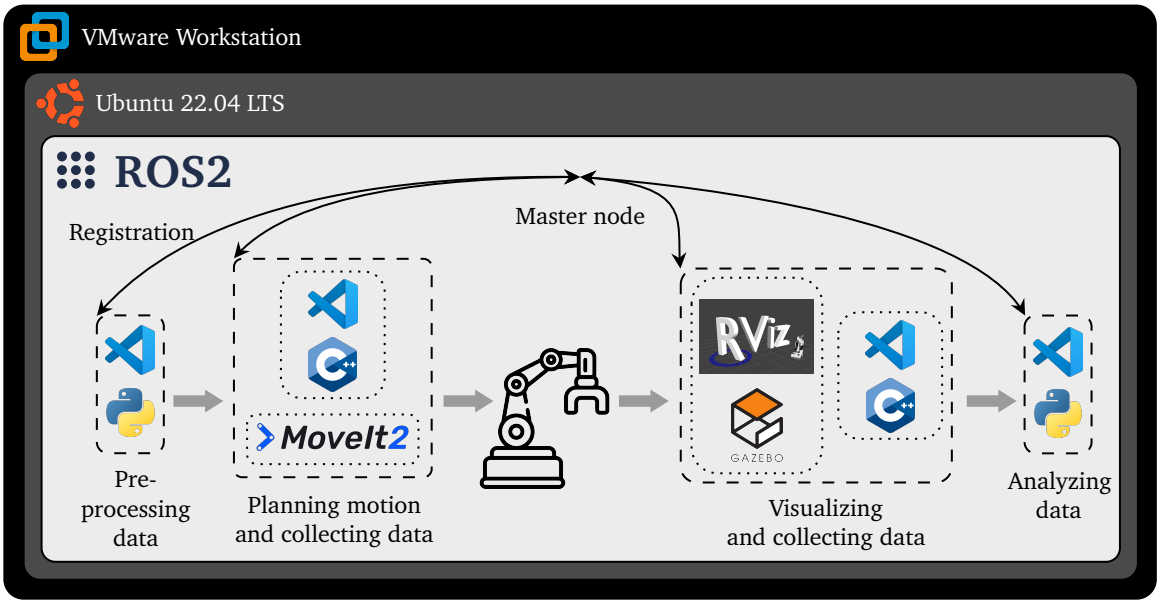


Figure 5.1: Scheme of the introduced tools in the implementation process

Apart from the tools introduction, the contents of this chapter are divided into five sections. Figure 5.2 presents the class diagram overview of class *Main*, and Figure 5.3 indicates the sequence diagram based on it. Specifically, Table 5.1 presents the relation between the aspects in Figure 5.1 and the classes in the sequence diagram (Figure 5.3).

Table 5.1: Relation between the implementation aspects and the classes

Aspect	Class
Master node	*Main*
Pre-processing data	*ParametersPreparation*
Planning motion and collecting data Visualizing and collecting data	*LaunchSim*
Analyzing data	*ExecutionTime* *WorkSpaceVisual* *ContactInfoOrganizer* *StableGrasp*

The further classes of the orange classes in Figure 5.2 are introduced in the following sections. Within this thesis, all classes are implemented based on class *Main*, which is located in file *main.py*, and the classes in orange are implemented sequentially based on Figure 5.3. Moreover, Table C.1 in Appendix C shows the configuration of classes and files based on class *Main*. In addition, every class in this thesis's implementation is registered as a node in ROS2 to realize the node control, besides the Pydantic validation classes introduced in the latter paragraph. Corresponding to Figure 5.3, Section 5.1 first describes the implementation of the defined stable-grasp-related parameters (class *ParametersPreparation*). Following the parameters defined in a simulation, Section 5.2 begins to set up a simulation environment, including MoveIt2 and Gazebo (class *LaunchSim*). Subsequently, the first concept, the stable grasp verification approach, is implemented in Section 5.3 (class *StableGrasp*). The other two objectives, which are execution time measurement and workspace visualization, are further implemented in Section 5.4 (class *ExecutionTime*) and Section 5.5 (class *WorkSpaceVisual*), respectively. Lastly, all the files regarding the implementation and evaluation of this thesis are developed in the repository *ur30_all_included_ws*, which is created under *home* repository of Ubuntu.

5.1 Determination of stable-grasp-related parameters

As mentioned above, this section discusses the implementation of stable-grasp-related parameters determination based on step *S1* in Figure 4.3. In addition, this section is regarded as procedure *pre-processing data* in Figure 5.1. Accordingly, the introduced development environment in this section is VS Code combined with Python. To further access the reported contact-related data from the contact topic in Gazebo, as described in Subsection 4.2.2, the considered parameters must be determined in the beginning. This section aims to determine the parameters of the designed scenario's gripped cube in Figure 4.2 to ensure that the returned data from the topic considers the material properties and physical interactions. Subsequently, the parameters are written in a SDF file automatically. An overview of the required input parameters in a SDF file is listed in Table 4.2. Following the initialization of class *Main*

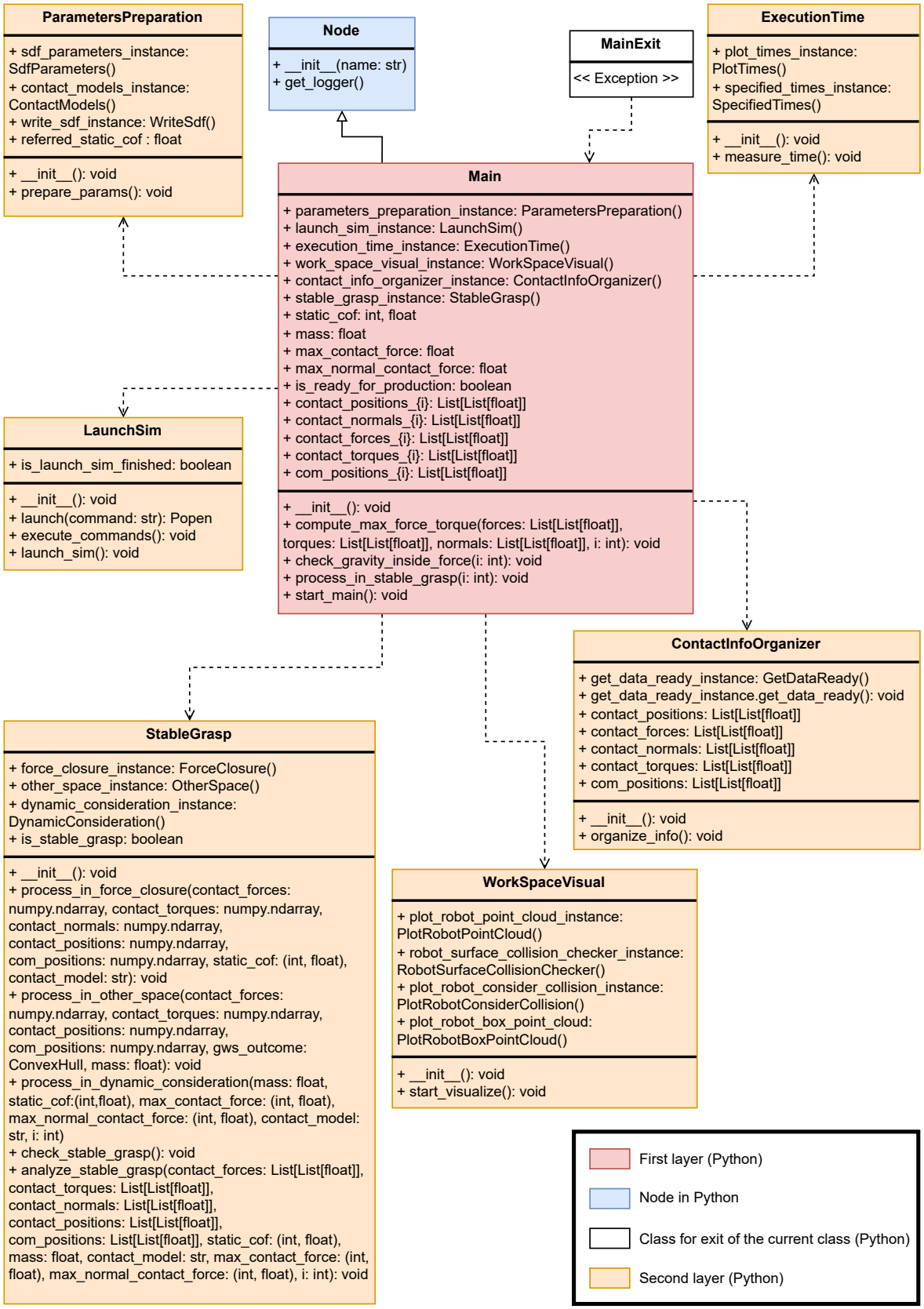


Figure 5.2: Class diagram overview based on class Main

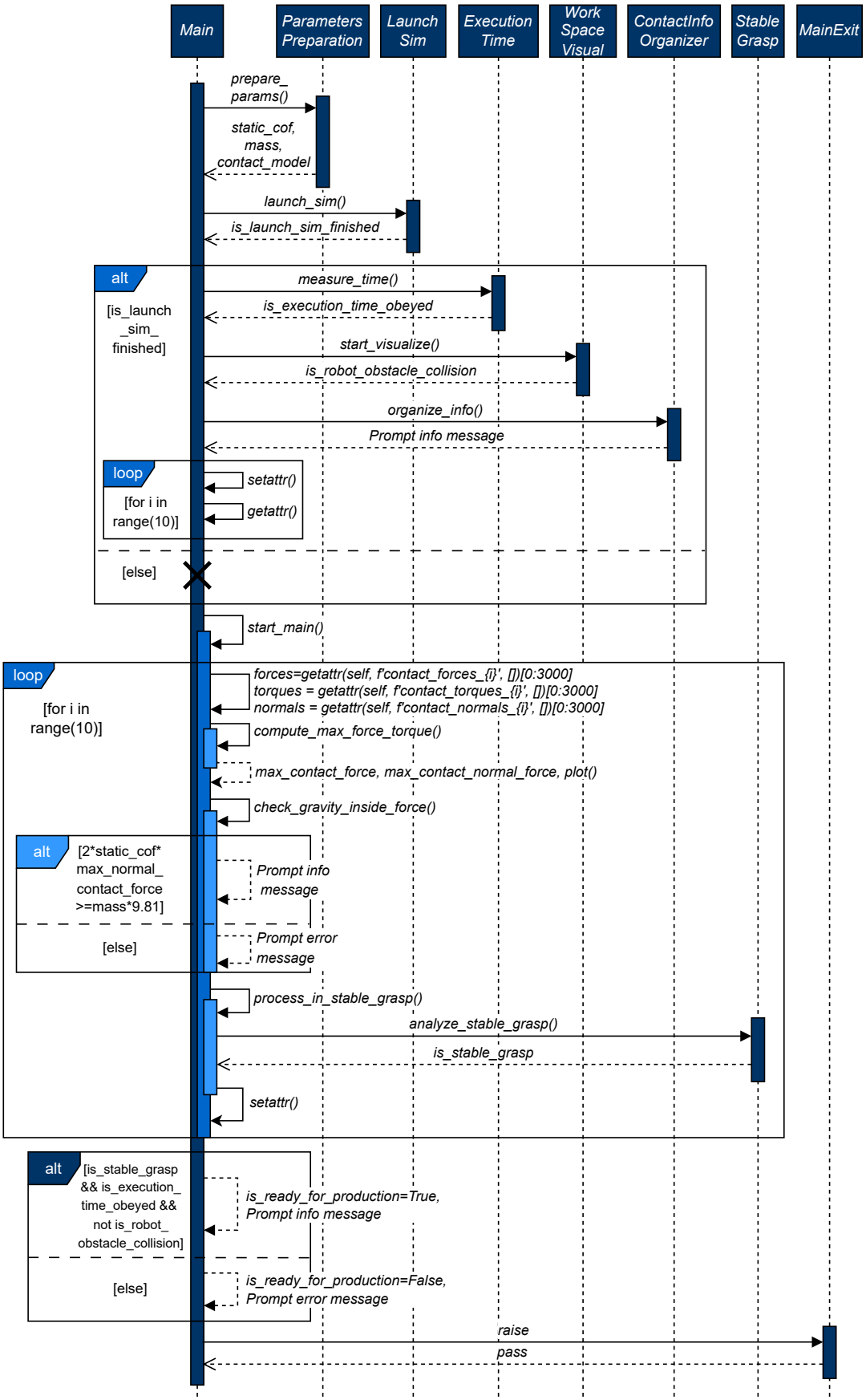


Figure 5.3: Sequence diagram overview based on class Main

in Figure 5.3, class *ParametersPreparation* is processed with method *prepare_params()*, and its configuration of classes and files is listed in Table C.2. Simultaneously, Figure 5.4 shows the class diagram based on class *ParametersPreparation*, and Figure 5.5 presents its sequence diagram.

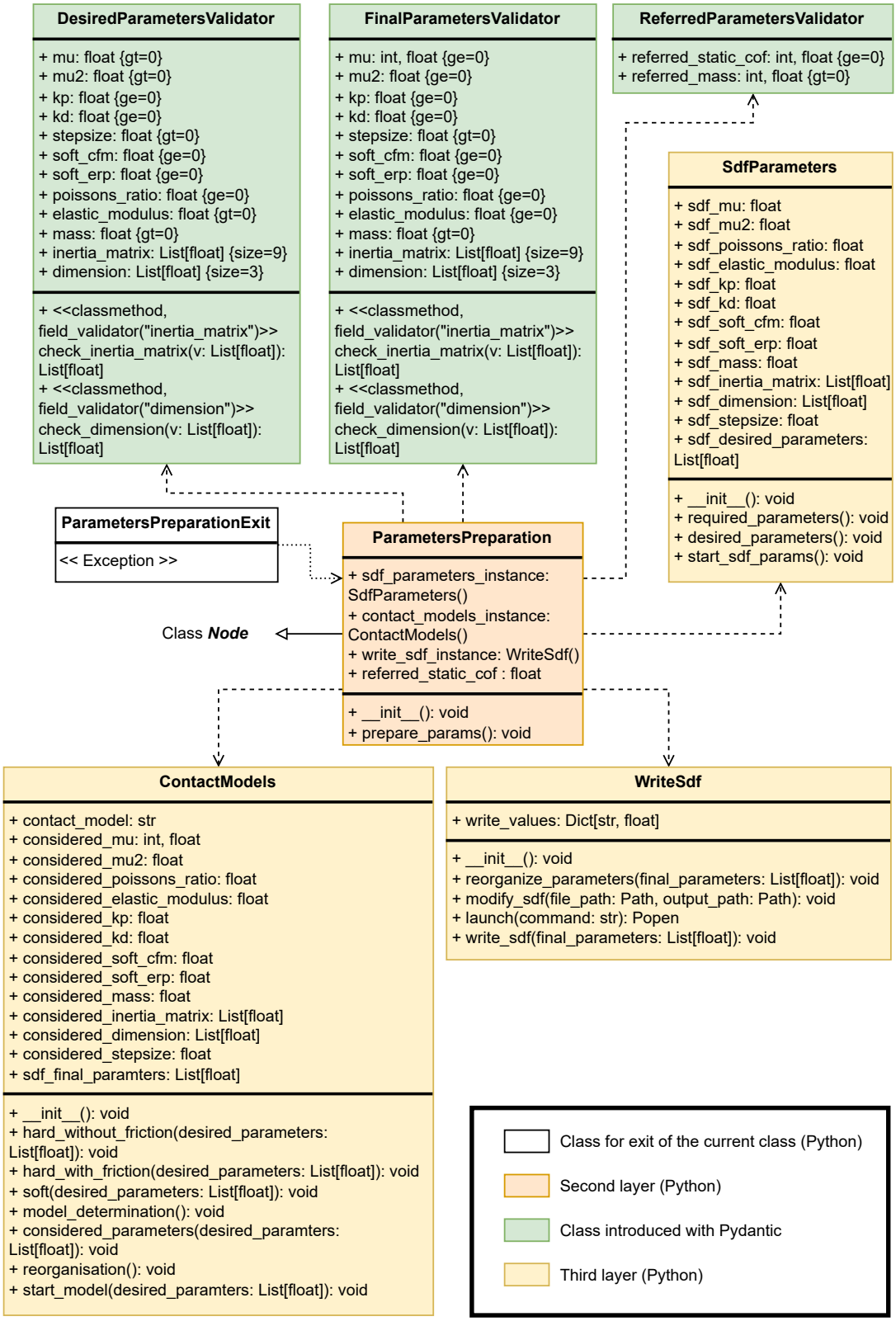


Figure 5.4: Class diagram overview based on class ParametersPreparation

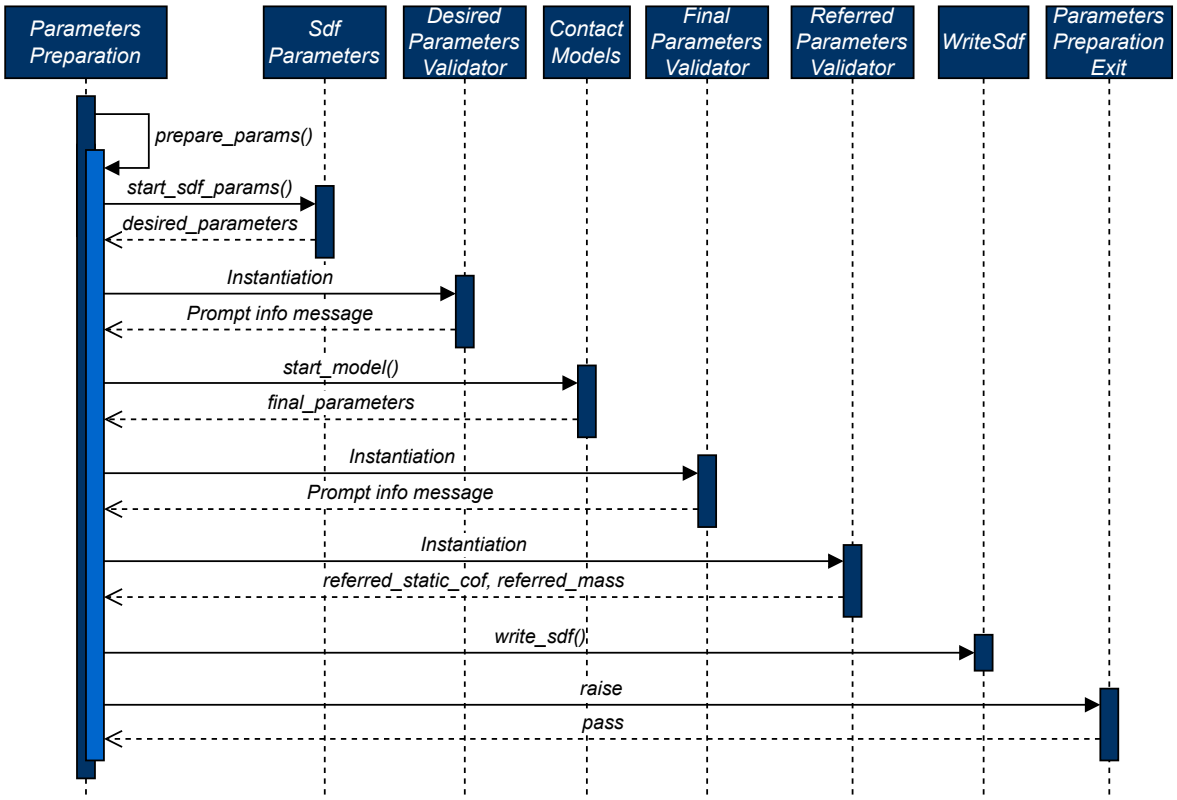


Figure 5.5: Sequence diagram overview based on class ParametersPreparation

In class *ParametersPreparation*, three other classes are affiliated to it. In the beginning, class *SdfParameters* is implemented with method *start_sdf_params()*. According to Figure 5.5, it processes into class *SdfParameters* with method *start_sdf_params()*, and method *required_parameters()* is first implemented. The class diagram based on class *SdfParameters* is shown in Figure A.1 in Appendix A. Class *SdfParameterValidator* is introduced with Pydantic. All the data included in this thesis's Python-based implementation are validated through Pydantic, and the name of the classes ended up with *Validator* indicates the Pydantic validation classes. Furthermore, every Pydantic validation class inherits from class *BaseModel*, which is typically imported from Pydantic.

Following method *required_parameters()*, attributes of class *SdfParameterValidator* are required to be input through its method *get_valid_input()*. Most of the attributes of class *SdfParameterValidator* are the described parameters in Table 4.2. However, not every parameter in Table 4.2 needs to be input in this class. Table 5.2 lists the attributes of this class and their corresponding parameters. Attribute *damping_ratio* is generally defined as 1, as mentioned in Subsection 4.2.1. The attribute values ranging from *mass* to *i_zz* are directly derived from the results calculated in SolidWorks, using the CAD model of the gripped cube. Additionally, parameter <*size*> is composed of three attributes: *length*, *width*, and *height* in *m*. The last attribute *stepsize* is generally defined as 0.001 seconds, as described in Subsection 4.2.1. Subsequently, its method *desired_parameters()* is called to determine the attributes of class *SdfParameters* and store them into *sdf_desired_params()*. Nevertheless, attribute *sdf_kd* is computed based on Equation (2.16), and attributes *sdf_soft_cfm* and *sdf_soft_erp* are computed based on Equation (2.24) and Equation (2.25). The inertia-related attributes of class *SdfParameterValidator* are stored in attribute *sdf_inertia_matrix*, and dimension-related attributes are written into attribute *sdf_dimension*.

With the validated defined input from class *DesiredParametersValidator*, class *ContactModels* processes the consideration of contact model in with method *start_model()*, and its class diagram is indicated in Figure A.2. Method *model_determination()* is first executed to input a contact model, and the input is validated through class *ContactModelValidator*. According to the input contact model, method *considered_parameters()* determines the corresponding considered parameters by referring to methods *hard_without_friction()*, *hard_with_friction()*, and *soft()* based on Table 4.4. Consequently, method *reorganisation()* is responsible for storing the final considered parameters values into attribute *sdf_final_parameters*.

Lastly, class *WriteSdf* is implemented with method *write_sdf()* after the validation processes in class *FinalParametersValidator* and class *ReferredParametersValidator*, and its class diagram is offered in Figure A.3. Method *reorganize_parameters()* is executed to reorganize all the values of attribute *final_parameters* with all the parameters of Table 4.2 and write them into attribute *write_values*. Equipped with the organized values, method *modify_sdf()* starts to modify the distributed SDF file involved in attribute *file_path* with attribute *write_values* based on the SDF file format automatically. Furthermore, method *launch()* is utilized to bash the workspace repository again to ensure the update of values.

Table 5.2: Attributes and their referred parameters

Attribute	Referred Parameter	Unit	Reference
static_cof	<*mu*>	-	Table 4.2
dynamic_cof	<*mu2*>	-	Table 4.2
poissons_ratio	<*poissons_ratio*>	-	Table 4.3
elastic_modulus	<*elastic_modulus*>	-	Table 4.3
stiffness	<*kp*>	$\frac{N}{m}$	Table 4.2
damping_ratio	ζ	-	Subsection 4.2.1
mass	<*mass*>	kg	Table 4.2
i_xx	<*ixx*>	$kg \cdot m^2$	Table 4.2
i_xy	<*ixy*>	$kg \cdot m^2$	Table 4.2
i_xz	<*ixz*>	$kg \cdot m^2$	Table 4.2
i_yy	<*iyy*>	$kg \cdot m^2$	Table 4.2
i_zz	<*izz*>	$kg \cdot m^2$	Table 4.2
length	<*size*>	m	Subsection 2.4.3
width	<*size*>	m	Subsection 2.4.3
height	<*size*>	m	Subsection 2.4.3
stepsize	h	second	Subsection 4.2.1

5.2 Setting up a simulation environment

After pre-processing data executed in Section 5.1, this section proceeds to implement the procedures *planning motion and collecting data* and *visualizing and collecting data* of Figure 5.1. Therefore, the development environment of this thesis comprises of VS Code, MoveIt2, RViz, and Gazebo. The goal of this section is to implement the defined scenario (Section 4.1)

in the mentioned environment and describe the class *LaunchSim* in Figure 5.3. Regarding the overall implementation in this section, Figure 5.6 shows the class diagram based on class *LaunchSim*, and Figure 5.7 illustrates its sequence diagram. The classes' configuration and corresponding files are listed in Table C.3.

According to Figure 5.3, class *LaunchSim* is executed with method *launch_sim()* sequentially. In class *LaunchSim*, six other classes are affiliated to it, and they are all implemented in C++ because of better compatibility with MoveIt2 and Gazebo. Corresponding to Figure 5.7, objects *Gazebo* and *MoveIt2Visualize* are explained in Subsection 5.2.1. Subsection 5.2.2 describes the execution of class *MoveGroupDemoNode*. However, classes *BoxPoseRecorder* and *GazeboTopicLogger* are mentioned in Section 5.3, and classes *RobotStateUpdater*, *SurfaceRecorder*, and *ObstacleReader* are further described in Section 5.5. Additionally, Table 5.3 offers the commands filled in method *launch_terminal()* of class *LaunchSim*, which are further introduced in the following subsections. Meanwhile, these commands must be executed sequentially under the repository *ur30_all_included_ws*. Although these files are developed in C++, they are written into Python launch files because the launch system in ROS2 is developed based on Python.

Table 5.3: The commands to be filled in method launch_terminal()

Parameter	Command
C1	*ros2 launch arm_description launch_sim.launch.py*
C2	*ros2 launch arm_moveit_config moveit_gazebo.launch.py*
C3	*ros2 launch ur_control demo2.launch.py*
C4	*ros2 launch ur_control demo3.launch.py*
C5	*ros2 launch ur_control demo6.launch.py*
C6	*ros2 launch ur_control demo5.launch.py*
C7	*ros2 launch ur_control demo1.launch.py*
C8	*ros2 launch ur_control demo4.launch.py*

5.2.1 Configuration of simulation environment

As described above, this subsection is responsible for considering the simulation environment configuration, which is implemented based on Gazebo, MoveIt2, and RViz. The simulation environment is built in Gazebo and is configured based on the scenario's scheme, as shown in Figure 4.2. The robotic manipulator of the scenario is further defined as a combination of UR30 robotic arm from Universal Robotics (Figure 5.8a) and 2F-85 two-fingered robotic gripper from Robotiq as the end-effector (Figure 5.8b). In method *launch_sim()*, another method *execute_commands()* is first called, which is the primary method in class *LaunchSim*. Figure 5.9 further presents the built environment in Gazebo, of which the robotic manipulator's URDF file *arm.xacro* is composed of the two models' URDF files, as the example shown in Figure D.1 in Appendix D. Gazebo simulation environment interface is launched by method *launch_terminal()* with parameter *C1* in Table 5.3. Simultaneously, the cubes from 0 to 9 in Figure 4.2 are regarded as the green cubes named from *box_green_0* to *box_green_9* in Figure 5.9 correspondingly.

Besides the simulation environment in Gazebo, the robotic manipulator's URDF file is introduced in MoveIt2 for the motion planning framework, and MoveIt2 framework is further

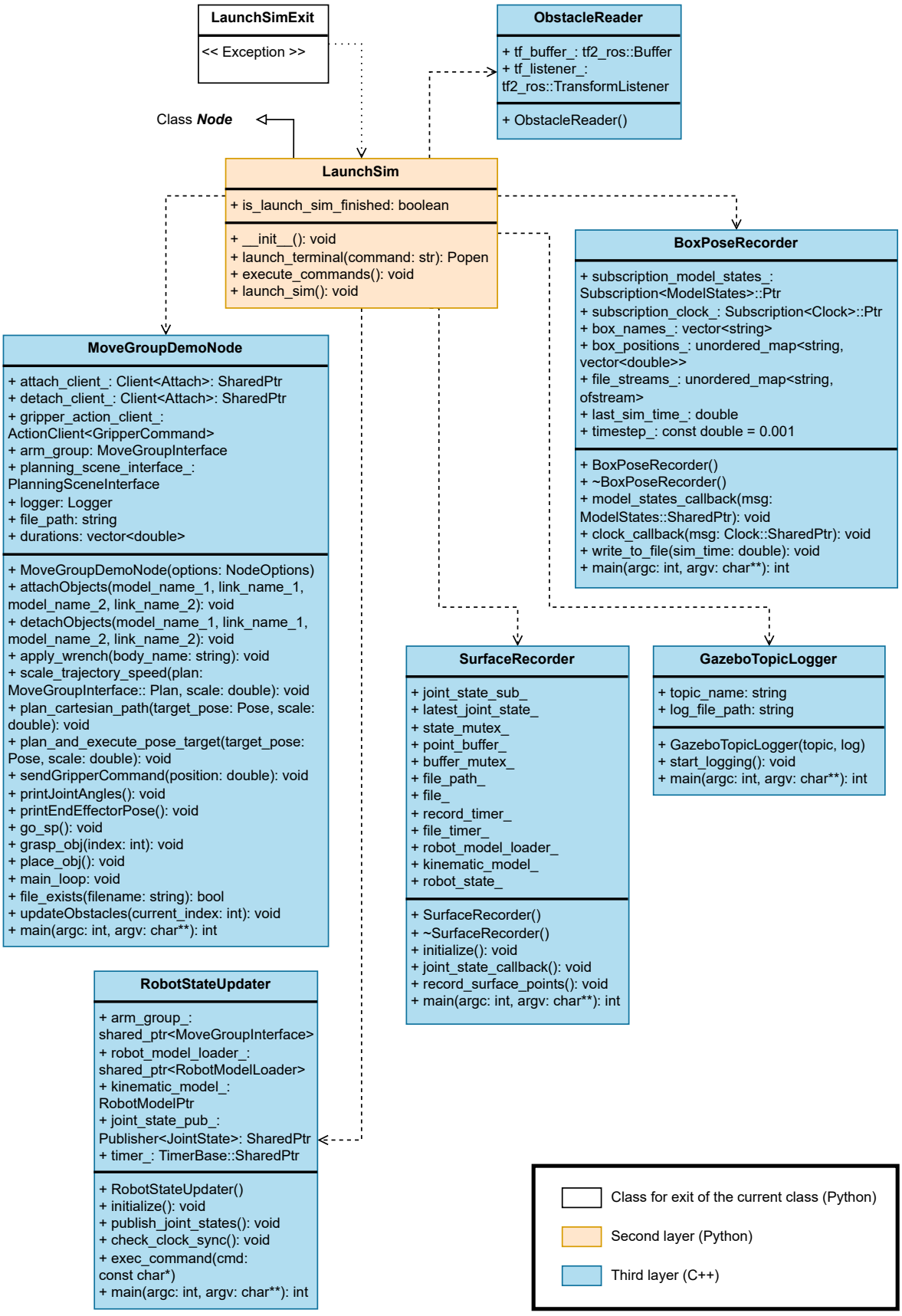


Figure 5.6: Class diagram overview based on class LaunchSim

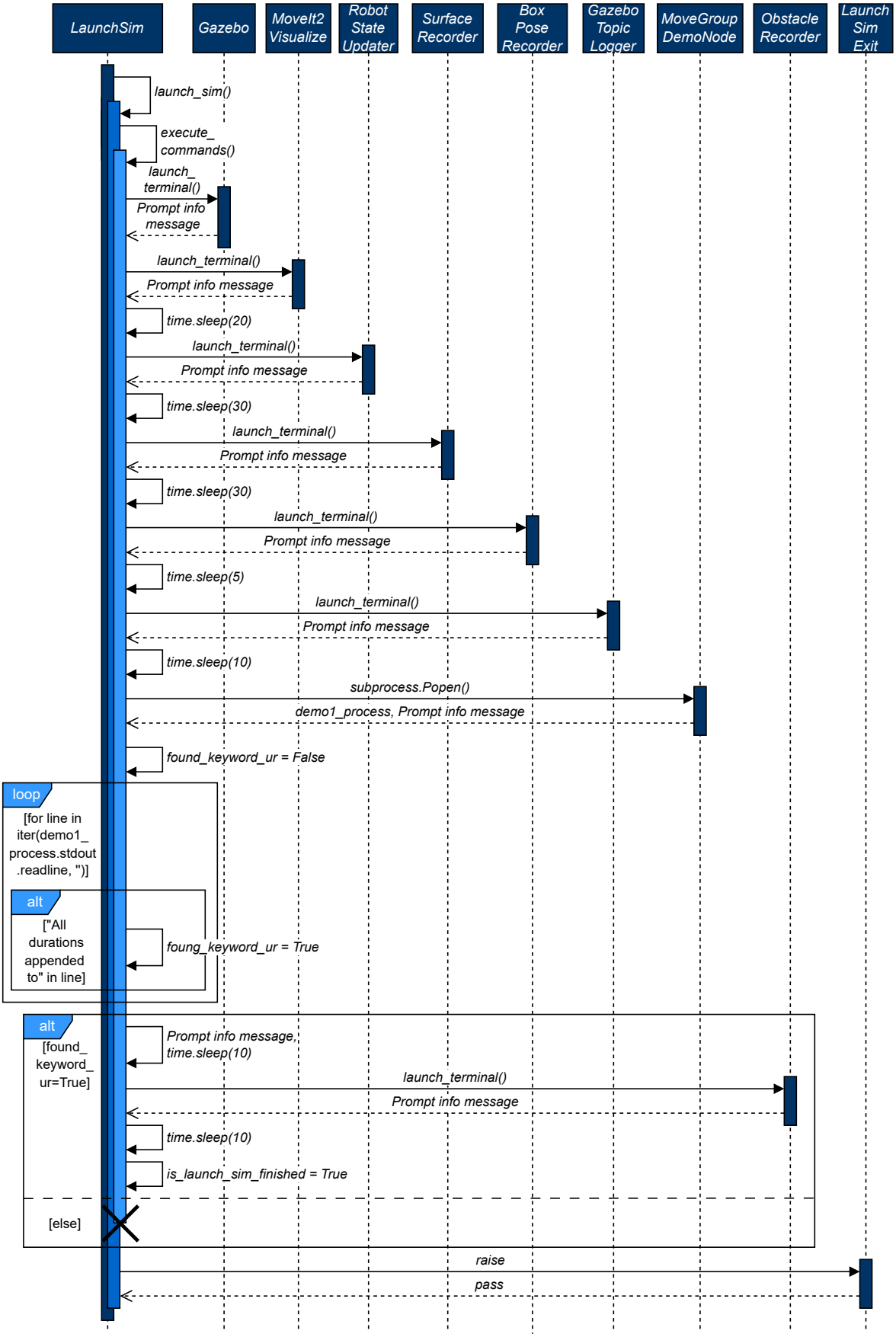


Figure 5.7: Sequence diagram overview based on class LaunchSim

(a) UR30 robotic arm (UNIVERSAL ROBOTS A/S 2025)

(b) 2F-85 robotic gripper (ROBOTIQ INC. 2025)

Figure 5.8: Considered models for the scenario's robotic manipulator

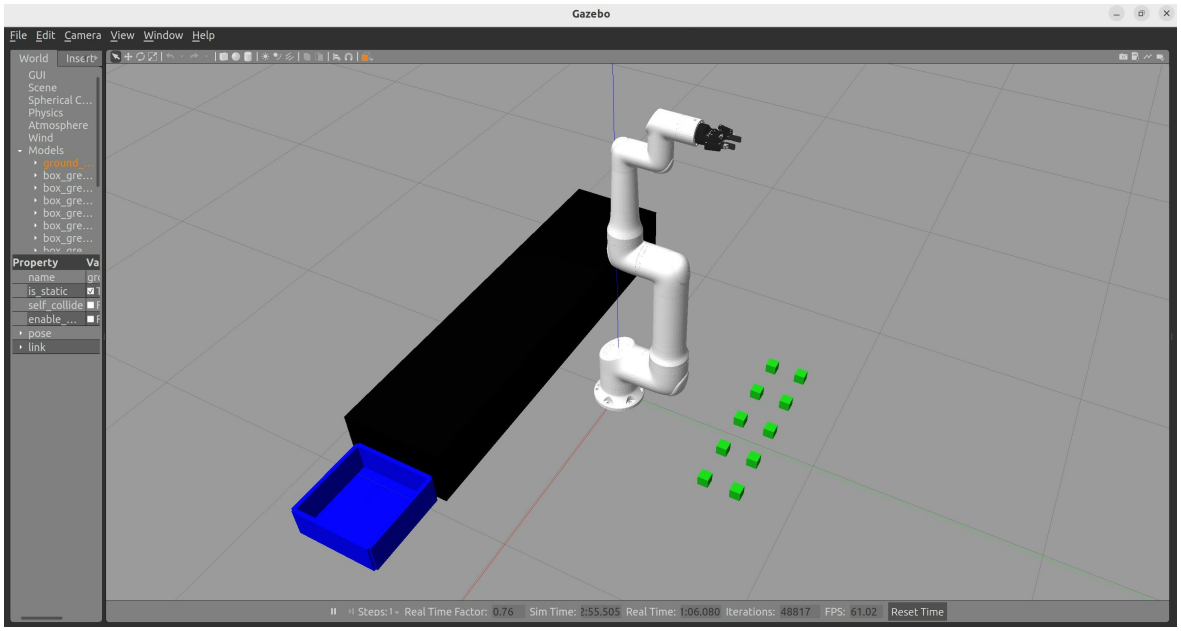


Figure 5.9: The built simulation environment in Gazebo

visualized by RViz. Figure 5.10 shows the visualized interface with the introduction of the constructed URDF file in RViz. Thus, the file *arm.xacro* is also introduced in MoveIt2. To reach object *MoveIt2Visualize* in Figure 5.7, method *launch_terminal()* is executed with parameter *C2* to launch RViz and MoveIt simultaneously. Nevertheless, it is shown that there are differences between the simulation environment in Gazebo (Figure 5.9) and in RViz (Figure 5.10). The differences are further processed in Subsection 5.2.2.

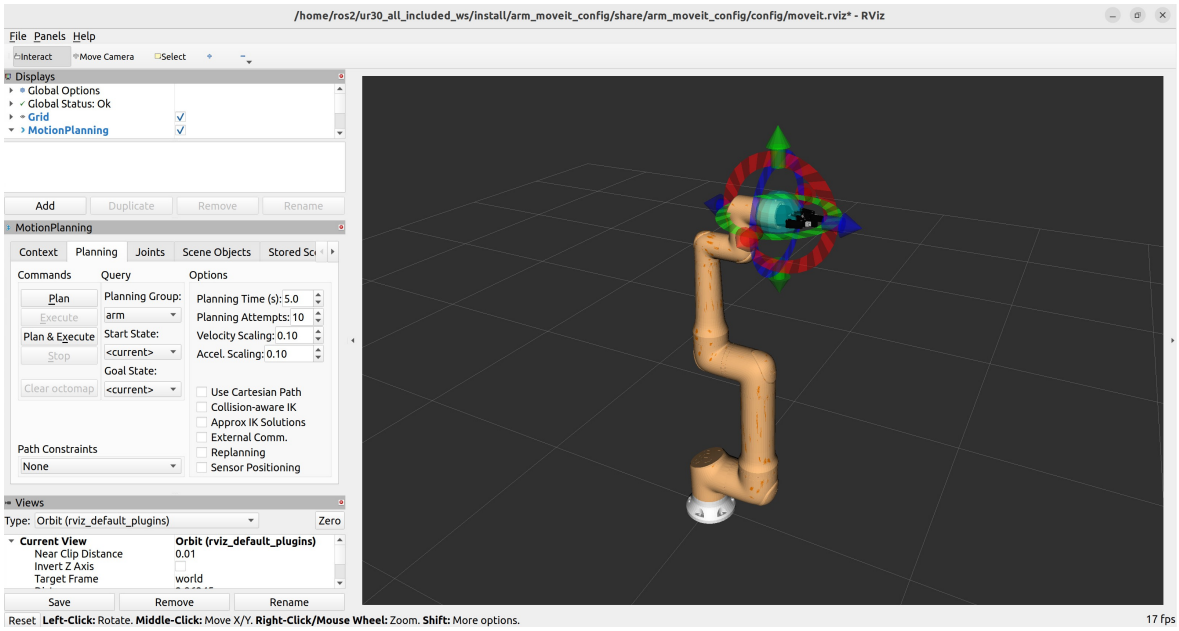


Figure 5.10: The built simulation environment in MoveIt2 visualized by RViz

5.2.2 Executing a handling skill in simulation

With the built simulation environment in Subsection 5.2.1, this subsection aims to develop the programming details to execute the defined pick-and-place scenario in MoveIt2. Accordingly, this subsection begins by implementing the motion planning, as demonstrated by class *MoveGroupDemoNode* in Figure 5.7. Figure A.4 indicates its class diagram, and Figure B.1 in Appendix B illustrates its sequence diagram. This class is also initiated by method *launch_terminal()* with parameter *C7* in Table 5.3. In addition, the motion planning is executed based on the scenario defined in Table 4.1. Furthermore, before planning motion in MoveIt2, two trajectory-planning-related parameters, kinematics solver and Open Motion Planning Library (OMPL), must be determined in advance. Within the thorough implementation process, the solver Kinematics and Dynamics Library (KDL) is considered as the kinematics solver, and Rapidly-exploring Random Trees (RRT) is determined as the primary option of OMPL.

Subsequently, the differences between Figure 5.9 and Figure 5.10 are first specified. As stated in the first paragraph of Chapter 5, MoveIt2 is regarded as a motion planning framework plugin. In Figure 5.10, the cubes, the conveyor belt, and the basket are not included yet because they must be regarded as obstacles during the motion planning. Therefore, their introduction is implemented with the defined pick-and-place handling skill together. It is described in Subsection 3.4.4 that when an object is introduced as a collision object in MoveIt2, the planned trajectory will automatically avoid it (PULJIZ ET AL. 2020). Accordingly, the cubes

in Figure 5.9 are regarded as collision objects in MoveIt2 in the corresponding cycles. Table 5.4 explains the relation between collision objects and the cycles. For example, *box_green_0* is not introduced as a collision object in loop 0 because it is the object to be gripped in this cycle. In loop 1, *box_green_1* is not considered a collision object. However, *box_green_0* is also not a collision object in loop 1 because *box_green_0* is already in the basket at the time.

Table 5.4: Collision objects in the corresponding cycles

Collision Object	Loop 0	Loop 1	Loop 2	Loop 3	Loop 4	Loop 5	Loop 6	Loop 7	Loop 8	Loop 9
box_green_0	-	-	-	-	-	-	-	-	-	-
box_green_1	•	-	-	-	-	-	-	-	-	-
box_green_2	•	•	-	-	-	-	-	-	-	-
box_green_3	•	•	•	-	-	-	-	-	-	-
box_green_4	•	•	•	•	-	-	-	-	-	-
box_green_5	•	•	•	•	•	-	-	-	-	-
box_green_6	•	•	•	•	•	•	-	-	-	-
box_green_7	•	•	•	•	•	•	•	-	-	-
box_green_8	•	•	•	•	•	•	•	•	-	-
box_green_9	•	•	•	•	•	•	•	•	•	-

According to the sequence diagram of class *MoveGroupDemoNode* in Figure B.1, *for loop* is developed in method *main_loop()*, and the number of iterations is set as 10 because of 10 boxes. Within *for loop*, method *updateObstacles()* is first executed to determine which boxes need to be regarded as collision objects based on Table 5.4 by importing *PlanningSceneInterface* provided from MoveIt2. Figure 5.11 represents the scenario of loop 0 in Table 5.4. All the green objects in the figure are collision objects, and *box_green_0* is not shown in the figure because this box is not the collision object of the loop. Hence, the simulation environment in Gazebo (Figure 5.9) is the same as in MoveIt2 (Figure 5.11). Moreover, besides the boxes mentioned in Table 5.4, the other three objects, i.e., the conveyor belt, the basket, and the floor, are imported as collision objects within the thorough simulation in MoveIt2 because they are not allowed to collide.

With the defined collision objects, the movements of the robotic manipulator in both environments are identical because both Gazebo and RViz receive motion commands generated by MoveIt2. MoveIt2 utilizes the same URDF and SRDF to plan collision-free trajectories, considering the defined obstacles. The planned trajectory is executed in Gazebo through controllers (e.g., *ros2_control*) and simultaneously visualized in RViz. Since both environments subscribe to the same */joint_states* and */tf* topics and use the same robot description, the motion appears synchronized. Following method *updateObstacles()*, another method *go_sp()* is created to let the robotic manipulator move from its current pose to its defined home pose, i.e., the white pose representation in Figure 5.11. This method contains an attribute *joint_group_positions* to define the home pose's joint positions and their values listed in Table 5.5.

Apart from method *clock.now()* of the sequence diagram in Figure B.1, which is further explained in Section 5.4, the following method *sendGripperCommand()* is executed to open the gripper. This method sends a position command to the gripper using ROS2 *GripperCommand* action interface. It first checks whether the gripper's action server is available by waiting up to 5 seconds. If the server is not ready, it logs an error and exits the function. Upon confirming the server's availability, it constructs a goal message with the desired gripper position and sends it asynchronously using the action client. The function does not block or wait for the result, relying instead on ROS2 executors running in a separate thread to handle callbacks

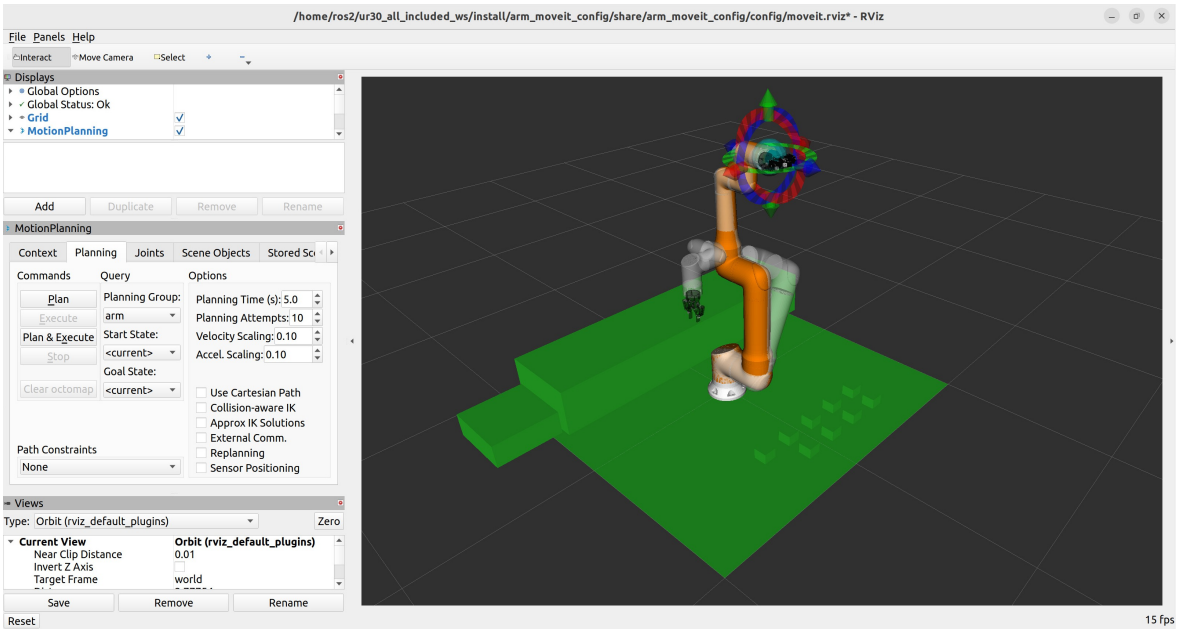


Figure 5.11: Simulation environment in RViz after introducing collision objects

Table 5.5: Joint position values of the robotic manipulator's home pose

Joint	Position in rad
Joint 1	0.0742368
Joint 2	-0.4001485
Joint 3	1.5169027
Joint 4	0.4459360
Joint 5	1.5698045
Joint 6	0.107476

and process the action response in the background. For example, the method is implemented as *sendGripperCommand(0.2)* within this stage to open the gripper. The parameter *position* of this method is regulated within the interval $[0, 1]$, of which 0 represents that the gripper is completely opened, and 1 represents that the gripper is completely closed contrarily.

Through the executed methods, the robotic manipulator is currently at its home pose with its gripper opened. Therefore, it begins to consider step 1 of the scenario in Table 4.1, which comprises of method *grasp_obj()* in Figure B.1. Method *grasp_obj()* defines a sequence for grasping 1 of 10 predefined boxes based on the provided index. It begins by retrieving the current pose of the end-effector's TCP as variable *target_pose*. Then, it updates its pose using a set of coordinates corresponding to the box's location listed in Table 5.6. For each index, the function assigns specific x and y positions while keeping the z position fixed at 0.24 m, indicating a position slightly above the box. After planning and executing a motion to this pre-grasp pose using method *plan_and_execute_pose_target()*, the method lowers the end-effector by 0.07 m along the z-axis to approach the box directly using method *plan_cartesian_path()*. Figure D.2 and Figure D.3 present the images in RViz and Gazebo, which represent the end of the scenario's step 1 of loop 0. Method *plan_cartesian_path()* utilizes method *computeCartesianPath()* from *MoveGroupInterface* to generate a Cartesian trajectory toward a target pose. It retries up to 100 times until at least 80 percent of the path is successfully planned. The trajectory is subjected to speed scaling and then executed upon successful planning.

Table 5.6: Values of the defined TCP pre-grasp poses

Index	*target_pose.pose.position.x*	*target_pose.pose.position.y*	*target_pose.pose.position.z*
0	0.30	0.754	0.24
1	0.30	0.60	0.24
2	0.10	0.75	0.24
3	0.10	0.60	0.24
4	-0.10	0.75	0.24
5	-0.10	0.60	0.24
6	-0.30	0.75	0.24
7	-0.30	0.60	0.24
8	-0.50	0.75	0.24
9	-0.50	0.60	0.24

Method *plan_and_execute_pose_target()* uses MoveIt2 *MoveGroupInterface* to plan and execute motion to a specified target pose (Table 5.6). It begins by setting the desired pose as the target for the robotic manipulator using method *arm_group.setPoseTarget()*. A planning loop is then initiated, where the function repeatedly calls method *arm_group.plan(plan)* to generate a motion plan and store it in a *MoveGroupInterface::Plan* object. This loop allows up to 100 attempts, logging warnings and waiting briefly between retries to increase the chance of success in constrained environments. If the planning succeeds, the function adjusts the execution speed using a scaling factor and executes the plan via method *arm_group.execute(plan)*. If no valid plan is found after all attempts, the function logs an error, shuts down the ROS2 node, and exits.

Following method *grasp_obj()*, the former part of step 2 in Table 4.1 starts. The former part of step 2 consists of methods *sendGripperCommand()* and *sleep_for()* in Figure B.1. The box is gripped by implementing method *sendGripperCommand()*, e.g., *sendGripperCommand(0.42)*,

to close the gripper. Lastly, method *sleep_for(2000ms)* is executed to wait for the grasp to be stabilized. A 2-second delay ensures that the object and gripper have reached a static contact state, minimizing transient forces or vibrations. Meanwhile, the contact data from Gazebo's contact topic are collected within this period for the static stable grasp analysis in Figure 4.3, which is further discussed in Subsection 5.3.1.

After the waiting mechanism for 2 seconds, the latter part of step 2 in Table 4.1 starts. Nevertheless, the latter part of step 2 and step 3 are considered together in the implementation process, which is composed of methods *attachObjects()* and *place_obj()* in Figure B.1. Method *attachObjects()* is introduced to simulate a physical connection between two models in Gazebo, typically between a robotic gripper and an object, by calling service *gazebo_ros_link_attacher*. It is beneficial when friction-based grasping is unreliable or insufficient because of the simulation's contact physics limitations. Instead of relying solely on simulated contact forces, which may not consistently result in a successful grasp, this method programmatically attaches the object to the gripper's link once contact is detected or a grasp action is triggered. The function creates a service request specifying both entities' model and link names to be connected, waits for the service to become available, and sends the request asynchronously. By explicitly creating a rigid attachment, method *attachObjects()* ensures that the object moves with the gripper during the lifting and transport phases.

As stated at the beginning of Chapter 5, MoveIt2 only provides the robotic manipulator's motion planning framework, including considering collision objects. To further specify the physical behaviors of the boxes, the service in Gazebo must be imported. Therefore, the other reason for the introduction of method *attachObjects()* is the consideration of the dynamic stable grasp analysis in Figure 4.3, which is further explained in Subsection 5.3.4.

With the box attached by the gripper, method *place_obj()* is conducted subsequently. This method defines the sequence for placing the box at a designated location using the robotic manipulator. It retrieves the current end-effector pose with method *arm_group.getCurrentPose()* as variable *place_pose* and incrementally raises the gripper by 0.07 m along the z-axis to lift the box vertically. This upward motion is executed with method *computeCartesianPath()* to ensure a straight-line retreat from the placement surface. Simultaneously, the current stage represents the end of the scenario's step 2, of which the simulation images in RViz and Gazebo are presented in Figure D.4 and Figure D.5.

After the box is lifted, method *place_obj()* further sets a new pose with specific x, y, and z coordinates that represent the target pose of the box and executes a motion plan to reach that pose with method *plan_and_execute_pose_target()*. The target pose is listed in Table 5.7. Figure D.6 and Figure D.7 further show the simulation images in RViz and Gazebo, which represent the end of the scenario's step 3 of loop 0. Following method *place_obj()*, the scenario's step 4 corresponds to methods *sendGripperCommand()*, *detachObjects*, and *go_sp()* in Figure B.1. The first method in step 4 is first implemented as *sendGripperCommand(0.2)* to open the gripper. Subsequently, similar to method *attachObjects()*, method *detachObjects()* is executed to simulate the release of a physically attached object in Gazebo by calling the *gazebo_ros_link_attacher* service. The method prepares a service request containing the names of the two models and their respective links that were previously attached. The detachment request is sent asynchronously using attribute *detach_client_*, and a log message confirms the request has been issued. Lastly, the robotic manipulator returns to the home pose by executing method *go_sf()* again.

In Figure B.1, it is indicated that method *apply_wrench()* is implemented in the latter segment of *for loop*. This method is responsible for applying a force to the box's link in x direction in Gazebo simulation using *gazebo_msgs::srv::ApplyLinkWrench* service. This method primarily

Table 5.7: Values of the defined scenario's TCP target pose

place_pose.pose.position.x	0.1852
place_pose.pose.position.y	-0.550
place_pose.pose.position.z	0.514

simulates the effect of the conveyor belt, such that the box can be further transported into the basket. Figure D.8 and Figure D.9 present the simulation images in RViz and Gazebo, which represent the end of the scenario's step 4 of loop 0. Hence, a cycle of the pick-and-place handling skill is completely executed, and node *MoveGroupDemoNode* will be shut down after 10 cycles.

5.3 Determination of stable grasp

After discussing the implementation of the pick-and-place handling skill in the simulation, this section describes the implementation of the stable grasp verification concept, of which the concept flowchart is presented in Figure 4.3. Although the sequence diagram of class *Main* in Figure 5.3 presents that classes *ExecutionTime* and *WorkSpaceVisual* are executed before class *StableGrasp*, the implementation of classes *ContactInfoOrganizer* and *StableGrasp* is discussed in this section first to correspond with the objectives order stated at the beginning of Chapter 4.

Nevertheless, before implementing the stable grasp verification concept, the contact topic in Gazebo must be introduced, as step *S3.1* in Table 4.5. Meanwhile, the boxes' CoMs in Gazebo must be specified, as step *S3.2.1* in Figure 4.4. Therefore, Subsection 5.3.1 describes how the boxes' CoMs are determined, how the contact topic is implemented in Gazebo, and how the returned data from the contact topic is processed. Equipped with the processed data, i.e., the boxes' CoMs, contact positions, contact normals, contact forces, and contact torques, Subsection 5.3.2 discusses implementing the force closure verification concept based on Subsection 4.2.2. Following force closure, Subsection 5.3.3 implements the construction of the other wrench spaces based on Subsection 4.2.3. Lastly, Subsection 5.3.4 considers the dynamic condition described in Subsection 4.2.4 to determine the fulfillment of stable grasp.

5.3.1 Data processing in and after a simulation

As mentioned above, this subsection is split into three segments: determining boxes' CoM, implementing the contact topic in Gazebo, and data processing after the simulation. Based on Figure 5.7, class *BoxPoseRecorder* is regarded as the first segment in this subsection, and class *GazeboTopicLogger* is the second segment, where the following class *MoveGroupDemoNode* is described previously in Subsection 5.2.2. Thus, classes *BoxPoseRecorder* and *GazeboTopicLogger* are implemented before the pick-and-place handling skill and executed by method *launch_terminal()* with parameters *C5* and *C6* in Table 5.3, respectively. Lastly, the third segment of this subsection describes the data processing after the simulation in MoveIt2 and Gazebo is implemented.

Class BoxPoseRecorder

Figure A.5 presents the class diagram of *BoxPoseRecorder*. It is a ROS2 node that continuously records the CoM positions of ten green boxes in Gazebo using simulation time. Upon initialization, it defines the expected box names and their initial positions, creates file streams for logging each box's position, and sets up two ROS2 subscriptions: one for */gazebo/model_states* to obtain the latest positions of the boxes, and another for */clock* to track simulation time. Each time a new */gazebo/model_states* message is received, the node updates the stored positions of all boxes found in the message.

The node relies on topic */clock* to regulate time-based sampling. It records the box positions at a fixed interval ($timestep_ = 0.001\ s$) by comparing the current simulation time with the last recorded time. When the time difference exceeds the fixed timestep, it writes the updated CoM positions of all ten boxes to their respective files, ranging from *box0_com_input.txt* to *box9_com_input.txt*, with each line containing a timestamp and three-dimensional position. The output is flushed immediately to ensure the data is saved reliably.

Class GazeboTopicLogger

Class *GazeboTopicLogger* provides a simple utility for launching a terminal command to record Gazebo contact topic data using *gz topic -e*. It takes a topic name and a desired log file path upon construction, and its method *start_logging()* spawns a new GNOME terminal that listens to the contact topic and writes the output to a log file using *tee*. The log data are stored in file *contacts_log.txt*. Internally, it constructs a shell command that chains *gz topic -e*, subscribes, and echoes contact topic messages with *tee* to display and save the output simultaneously. The command is executed via method *system()*. Figure D.10 shows an example of the data returned from the contact topic.

Data processing after a simulation

With classes *BoxPoseRecorder* and *GazeboTopicLogger* executed, it is now assumed that classes *LaunchSim*, *ExecutionTime*, and *WorkSpaceVisual* in Figure 5.3 are finished. Subsequently, class *ContactInfoOrganizer* is discussed. The class is responsible for organizing the contact topic data recorded by class *GazeboTopicLogger*. Its class and sequence diagrams are presented in Figure A.6 and Figure B.2, respectively. The configuration of Python files and classes is listed in Table C.4. As shown in its sequence diagram (Figure B.2), most functions are executed within initialization. Upon initialization, class *ContactInfoOrganizer* creates an instance of *GetDataReady*, which encapsulates the contact data extraction and CoM matching procedures. Method *get_data_ready()* is invoked immediately. This method internally initializes class *ContactDataExtractor*, which begins by validating the contact log file using a Pydantic-based file validator. Once verified, the method *start_extract_data()* is called, which in turn parses the contact log file using regular expressions to extract structured information, i.e., contact positions, normals, forces, torques, and timestamped contact events, based on the data structure in Figure D.10. The contact data are separated by box indices, ranging from *box0* to *box9*, and each contact timestamp is recorded along with the count of corresponding data points.

After contact data extraction, class *MatchTimeCom* is instantiated with the timestamp series obtained from the previous step. This class loads CoM log files (from *box0_com_input.txt* to *box9_com_input.txt*), which are generated through class *BoxPoseRecorder* and implement a time-aligned mapping between contact events and the corresponding CoM positions. For

each timestamp in the contact log, the closest CoM timestamp is identified based on the minimal absolute time difference under a threshold of 1 millisecond. Matched CoM entries are duplicated according to the number of contact data points per timestamp, ensuring a one-to-one correspondence between each contact measurement and a CoM value.

Upon completion of data preparation, class *ContactInfoOrganizer* retrieves the processed data using dynamic attribute access (i.e., via methods *getattr()* and *setattr()*). The class stores local copies of the ten boxes' contact positions, normals, forces, torques, and aligned CoM positions. To verify the integrity of the retrieved data, method *organize_info()* is executed, logging the number of data entries per category and box. If the data structure is successfully verified, the class raises a custom exception, *ContactInfoOrganizerExit*, to indicate successful execution and to transition control back to the main program.

After the implementation of class *ContactInfoOrganizer* in Subsection 5.3.1, class *Main* in Figure 5.3 invokes method *setattr()* and method *getattr()* to determine its attributes of each box, i.e., *contact_positions_i*, *contact_normals_i*, *contact_forces_i*, *contact_torques_i*, and *com_positions_i* from class *ContactInfoOrganizer*. Subsequently, method *start_main()* is called, which iteratively executes grasp evaluation for each target box. For each box, the method begins by retrieving time-series data of contact forces, torques, and contact normals recorded during the grasp. These datasets are limited to the first 3000 samples because of method *rclcpp::sleep_for(2000ms)* in the sequence diagram of class *MoveGroupDemoNode* in Figure B.1. Furthermore, the publishing frequency from the contact topic in Gazebo is 1000 Hz. Therefore, 2000 contact samples are generated during the implementation of method *rclcpp::sleep_for(2000ms)*. Nevertheless, before implementing this method, the other 1000 samples are considered because the gripper already contacts a box in Gazebo. These 3000 samples ensure that the collected data are the data when the box is gripped and not lifted yet. Method *compute_max_force_torque()* is then invoked to analyze these values and determine the maximum contact force and torque experienced during the grasp interaction.

Within method *compute_max_force_torque()*, the Euclidean norm of each contact force vector is computed to identify the maximum overall force magnitude as attribute *max_contact_force*. The normal component of the force projected onto the surface normal vector at the point of contact is also computed. It provides a more physically meaningful metric for grasp stability, as the normal force contributes directly to frictional resistance. A similar computation is implemented for contact torques, where the magnitudes of the torque vectors are evaluated to identify the maximum value. The force and torque magnitudes are plotted using *matplotlib* for visualization and debugging.

It proceeds to a stability verification step using method *check_gravity_inside_force()*. This method executes a static force analysis to determine whether the normal contact forces are sufficient to resist the gravitational force acting on the box based on Equation (5.1). The factor of 2 assumes symmetrical contact from the two-fingered gripper. If the inequality is satisfied, the grasp is sufficient to resist gravity. Otherwise, a warning is issued, indicating a potentially unstable grasp.

$$2 \cdot \mu_s \cdot F_n \geq m_o \cdot g \tag{5.1}$$

5.3.2 Determination of force closure

Following method *check_gravity_inside_force()* and the introduced condition, methods *process_in_stable_grasp()* and *analyze_stable_grasp()* in class *Main* are executed to initiate the

class *StableGrasp*, which is described from this subsection through Subsection 5.3.4. In addition, this subsection also assumes that classes *LaunchSim*, *ExecutionTime*, and *WorkSpaceVisual* before this class in Figure 5.3 are finished. The class and sequence diagrams are shown in Figure 5.12 and Figure 5.13, respectively. The configuration of Python files and classes is listed in Table C.5. Based on Figure 4.3, the stable grasp verification concept is composed of three significant steps, i.e., verification of force closure (step *S3*), consideration of other wrench spaces (step *S6*), and consideration of dynamic condition (step *S7*). This subsection aims to discuss the implementation of the force closure verification concept (Figure 4.4).

As presented in Figure 4.3, the stable grasp verification concept comprises static and dynamic aspects. Consequently, five attributes are created to access the first 3000 contact samples, which are *static_contact_forces*, *static_contact_torques*, *static_contact_normals*, *static_contact_positions*, and *static_com_positions*. Sequentially, class *ForceClosure* is initiated with method *analyze_force_closure()* and these five static parameters. The class and sequence diagrams of class *ForceClosure* are presented in Figure A.7 and Figure B.3, respectively. The configuration of Python files and classes is listed in Table C.6. As listed in Table 4.5, class *ForceClosure* is inherits from three classes, i.e., *GraspMatrix* (step *S3.2*), *FrictionCone* (step *3.3*), and *GwsConstruct* (step *S3.4*). The central analysis is considered in method *analyze_force_closure()*, which takes contact forces, contact normals, contact positions, the boxes' CoM positions, the static friction coefficient, and the contact model type as input. Before any physical reasoning is executed, the input data are validated using Pydantic-based models, i.e., *ForceClosureDataValidator* and *IsForceClosureValidator*.

Class GraspMatrix

The core grasp analysis proceeds in three sequential stages. First, the grasp matrix analysis is conducted by calling method *analyze_grasp_matrix()* on *GraspMatrix* instance, of which the class diagram is shown in Figure A.8, and developed based on the flowchart in the first framework of Figure 4.4. The process begins with a strict validation step using class *GraspMatrixDataValidator*, which ensures that both attributes *contact_positions* and *com_positions* are two-dimensional NumPy arrays of the same length. This is important because each contact must correspond to a CoM to correctly compute its torque contribution. The validator converts list inputs to NumPy arrays, checks dimensionality, and ensures data consistency before any further computation is allowed.

Once validated, method *grasp_matrix_compute()* iterates through each contact CoM pair and constructs a local 6×3 grasp matrix block. This block consists of an identity matrix in the top three rows and a skew-symmetric matrix in the bottom three rows. These blocks are horizontally concatenated into a full *integrated_grasp_matrix* of shape $(6, 3k)$, where k is the number of contact points. This matrix is then passed to method *check_full_rank()*, determining whether the matrix has full rank. If the matrix is full rank, the first condition of force closure is fulfilled. Otherwise, a custom *NonGraspMatrixExit* exception is raised to indicate failure.

Class FrictionCone

Next, if the contact model allows for friction, i.e., not *hard_without_friction*, a friction cone analysis is implemented through class *FrictionCone*. This class is developed based on the flowchart in the second framework of Figure 4.4, and its class diagram is presented in Figure A.9. Model *FrictionConeDataValidator* is processed to ensure the input data format. Once the

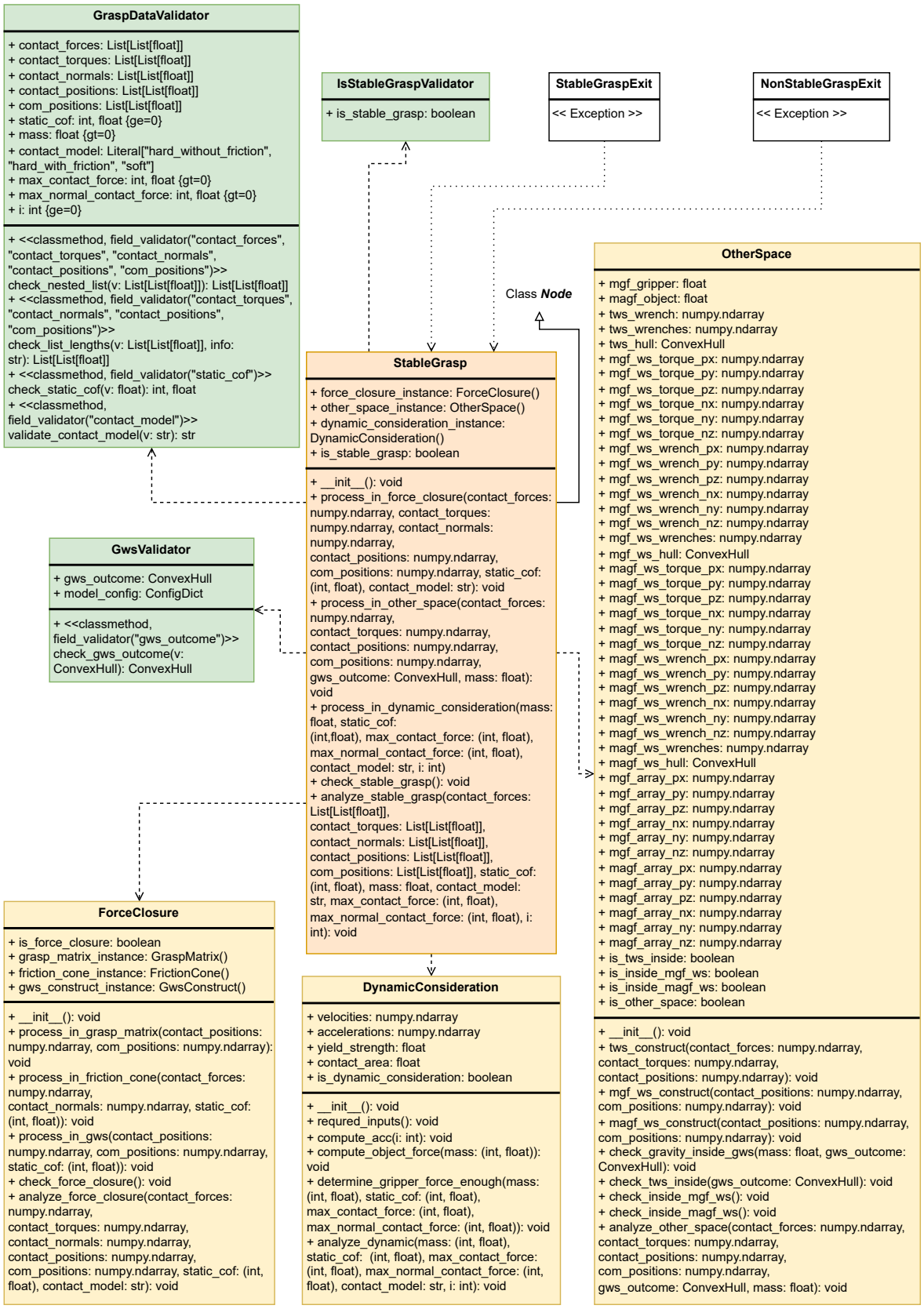


Figure 5.12: Class diagram overview based on class StableGrasp

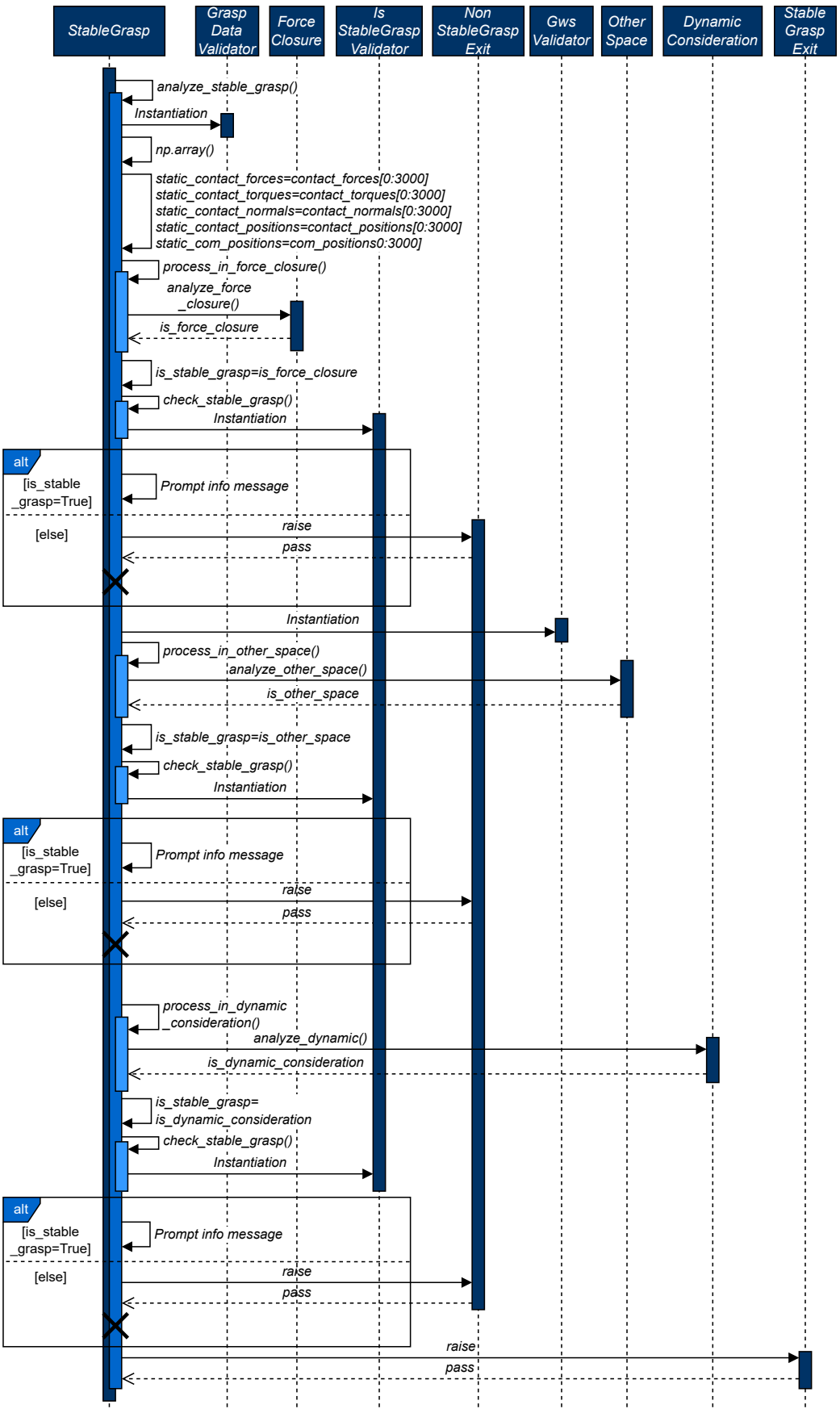


Figure 5.13: Sequence diagram overview based on class StableGrasp

inputs are validated, method *compute_forces()* decomposes each contact force into normal and tangential components concerning the contact normal vector. The normal component is computed by projecting the force onto the normalized normal direction.

In contrast, the tangential component is computed as the residual between the original contact force and its normal projection. For each contact, the magnitudes of these components are used to evaluate whether the contact lies within the friction cone. To determine whether the tangential force magnitude is less than or equal to the product of the normal force magnitude and the coefficient of friction, method *check_constraints()* is executed. This method iterates through all contacts and logs whether each force satisfies the condition. If any contact force exceeds the allowable friction force, a *NonFrictionConeExit* exception is raised.

Class GwsConstruct

The final stage GWS analysis of the flowchart in the third framework of Figure 4.4 is implemented by class *GwsConstruct*, and the class diagram is shown in Figure A.10. The process begins with input validation using model *GwsConstructDataValidator* to ensure the input data format. The user is then prompted to enter the number of edges to approximate each friction cone using model *NumberEdgeValidator*, which ensures the value is a positive integer. To construct GWS, method *gws_generate_friction_cone()* approximates each friction cone using a set of force vectors uniformly distributed around the tangent plane at the contact point. These vectors are constructed using the contact normal and two orthogonal tangent directions. They are scaled with a large normal force magnitude to ensure that the convex hull covers a significant wrench space.

Method *gws_compute_wrench()* computes the wrench generated by each friction cone force concerning the object's CoM. These six-dimensional wrenches are collected across all contacts and form the basis of the wrench space. A convex hull is computed over these wrenches using the *ConvexHull* algorithm *QJ* to represent the full GWS. Once the GWS is constructed, method *check_gws_inside()* determines whether the origin lies inside the convex hull. This check evaluates all half-space inequalities (Equation (3.2)) of the convex hull equations at the origin. If the origin is inside, the grasp can resist any arbitrary external disturbance, representing force closure is achieved. Otherwise, the system logs a failure and raises *NonGwsConstructExit*.

Following the execution of class *ForceClosure* and the introduction of attribute *is_stable_grasp* with attribute *is_force_closure*, method *check_stable_grasp()* is invoked to validate attribute *is_stable_grasp*. If this attribute is defined as *False*, object *NonStableGraspExit* will be raised to interrupt the implementation. Identically, this checking system is also called after the execution of classes *OtherSpace* and *DynamicConsideration*. In addition, object *GwsValidator* is utilized to validate the GWS created from class *GwsConstruct*.

5.3.3 Composing other wrench spaces

Based on the sequence diagram of class *StableGrasp* in Figure 5.13, class *OtherSpace* is processed with method *analyze_other_space()* after the implementation of class *ForceClosure*. Its class diagram is provided in Figure A.11, and the configuration of Python file is listed in Table C.7. The goal of this subsection is to develop class *OtherSpace* based on the flowchart in Figure 4.5 to verify stable grasp feasibility by analyzing whether the TWS generated by actual contact forces lies within multiple bounding wrench spaces, including the GWS, MGFWS, and MAGF-WS. Moreover, these wrench spaces are built using the *ConvexHull* algorithm

QJ. The process begins with input validation using *OtherSpaceDataValidator*, which checks that all physical quantities, i.e., forces, torques, positions, are valid two-dimensional NumPy arrays of equal length, and confirms that *gws_outcome* is a valid *ConvexHull* and the object's mass is positive.

Once validation passes, method *tws_construct()* is called to build the TWS. This method constructs a six-dimensional wrench vector for each contact by stacking the corresponding force and torque vectors horizontally. These are then aggregated and passed to *ConvexHull* algorithm to compute the boundary of the TWS, representing the net effect of grasp forces on the box. The grasp is immediately deemed invalid if the TWS cannot be constructed or its dimensionality is incorrect. The next two steps estimate the gripper's force capacity and the object's force tolerance. Method *mgf_ws_construct()* constructs the MGF-WS by prompting the user for the gripper's maximum grasp force limit using method *GripperForceValidator.get_valid_input()*. It generates a series of three-dimensional forces in $\pm x$, $\pm y$, and $\pm z$ directions and computes the corresponding torques using cross products relative to the CoM. Each force-torque pair is stacked into a wrench to construct the convex hull that defines the MGF-WS. Similarly, method *magf_ws_construct()* uses method *ObjectForceValidator.get_valid_input()* to acquire the box's MAGF and builds the MAGF-WS with the same logic and structure.

After all wrench spaces are established, several geometric inclusion checks are executed to specify the fulfillment of Figure 4.6. The first is handled by method *check_gravity_inside_gws()*, which computes the gravitational wrench applying the box's weight along the negative z-axis and verifies whether this vector lies inside the GWS. This test ensures that the grasp can resist the box's weight. Next, method *check_tws_inside()* evaluates whether every point in the TWS lies within the convex volume of the GWS, confirming that the applied grasp forces are not violating physical feasibility imposed by friction cones.

To ensure that the grasp respects actuator and object constraints, methods *check_inside_mgf_ws()* and *check_inside_magf_ws()* are called to verify whether all TWS wrenches lie inside the MGF-WS and MAGF-WS, respectively. These checks use the half-space representation of each convex hull and apply matrix multiplication followed by inequality testing (Equation (3.2)) to confirm inclusion. If any TWS point violates these conditions, a warning is logged, and a *NonOtherSpaceExit* exception is raised, interrupting execution. Hence, the static stable grasp analysis in Figure 4.3 is completely implemented.

5.3.4 Determination of dynamic analysis

Following the static stable grasp analysis, class *DynamicConsideration* in Figure 5.13 is initiated with method *analyze_dynamic()*. Its class diagram is presented in Figure A.12, and the configuration of its Python file is listed in Table C.8. Meanwhile, this subsection is responsible for developing this class based on the flowchart in Figure 4.7. Class *DynamicConsideration* is designed to evaluate whether a robotic grasp remains mechanically safe and physically feasible when dynamic effects are considered. Moreover, the accelerations of the box are computed based on its CoM positions and analyzed with the defined maximum static gripper force to determine if the box slides.

At the beginning of execution, method *analyze_dynamic()* introduces *DynamicConsiderationGivenDataValidator* to confirm the presence of all required numerical inputs, i.e., the box's mass, static friction coefficient, the gripper's maximum grasp force from the static stable grasp analysis, and the contact model. These parameters are essential for defining boundary conditions for force and stress analysis. Once the parameters are validated, method *required-*

_inputs() is called to prompt the user to input two critical material-specific properties, i.e., the yield strength of the box and the contact area between the gripper and the box. These values are validated through model *DynamicConsiderationInputDataValidator*, ensuring that real-world material limits are considered in the analysis.

Next, method *compute_acc()* is invoked to compute the box's linear acceleration over time. This method reads the box's CoM motion from a *.txt* file, e.g., *box0_com_input.txt*, and implements the second derivative of position to determine velocity and acceleration. It plots the magnitude of the computed accelerations and logs key statistics, i.e., maximum and mean values. These *.txt* files are created by class *BoxPoseRecorder*. In addition, this method monitors each position during file reading. If the position is detected to be within $\pm$ 0.01 m of the target goal position, i.e., $[0.18, -0.54, 0.36]$ in m, the reading is immediately stopped. This target goal position is defined based on the robotic manipulator's target pose in Table 5.7. This early exit condition helps the system avoid overestimating motion-based forces after the object rests, ensuring acceleration and force computations reflect only meaningful movement.

With acceleration data computed, method *compute_object_force()* determines whether the resulting normal forces cause internal stress on the object that exceeds its input yield strength. For each timestep, it computes the total force and assumes it is equally distributed across the two gripper fingers. It then computes the stress based on the input contact area. If the stress exceeds the yield strength at any point, the grasp is flagged as destructive, and execution halts via *NonDynamicConsiderationExit*.

Lastly, if the contact model allows friction, i.e., *hard_with_friction* or *soft*, the method proceeds to *determine_gripper_force_enough()*. This method implements a two-layer force analysis. The first layer checks whether the gripper's specified maximum force output is sufficient to counteract the net dynamic force exerted on the object. Furthermore, the gripper's specified maximum force output is attribute *max_contact_force* from class *Main*. The second layer checks whether the available frictional force, which is determined by the product of the normal force and the static friction coefficient, is sufficient to resist lateral slipping. These conditions are both necessary to maintain a secure grip during motion. If either test fails, the system raises a *NonDynamicConsiderationExit*, indicating that the grasp is not dynamically sustainable. Consequently, the implementation of the thorough stable grasp verification concept in Figure 4.3 is finished, and the simulation results are further presented in Section 6.1.

5.4 Determination of execution time

In Section 5.3, it is assumed that classes *ExecutionTime* and *WorkSpaceVisual* are executed. Nevertheless, this section aims to revisit the discussion of determining execution time based on the concept defined in Section 4.3. With the developed class *ExecutionTime* in this section, the cycle time of each pick-and-place loop can be determined and visualized. When describing class *MoveGroupDemoNode* in Subsection 5.2.2, it is stated that method *clock.now()* is invoked twice to document attributes *start* and *end*, as its sequence diagram shows in Figure B.1. These two attributes are regarded as the elements t_{q_s} and t_{q_e} in Equation (4.4). Therefore, the deduction of these two attributes determines the execution time of each cycle and is further stored in attribute *durations* of class *MoveGroupDemoNode*. Additionally, within method *main_loop()*, the durations data are stored in the file *durations.csv* with the run index. Table 5.8 provides an example of the data format stored in the file, of which the ten

columns represent the cycle times of 10 loops in seconds. For example, the execution time of loop 0 represents that it takes 19.2767 seconds for the robotic manipulator to finish the first three steps in Figure 4.2. The execution time in this thesis does not represent the time needed for the program to run the developed code.

Table 5.8: Example of the execution time data format in the CSV file

Index	Loop 0	Loop 1	Loop 2	Loop 3	Loop 4	Loop 5	Loop 6	Loop 7	Loop 8	Loop 9
1	19.2767	23.5572	24.2289	25.4325	26.482	20.2288	23.236	23.3725	29.809	26.9893

After executing class *MoveGroupDempNode*, this section assumes class *LaunchSim* in Figure 5.3 is implemented and processes class *ExecutionTime*. The class and sequence diagrams are provided in Figure A.13 and Figure B.4, respectively. The configuration of Python files and their classes is listed in Table C.9. As presented in Figure A.13, class *ExecutionTime* inherits from two classes, i.e., *PlotTimes* and *SpecifiedTimes*. Class *PlotTimes* is designed to visualize execution time per loop for multiple experimental cases. The analysis process is initiated through method *plot_times()*, which serves as the primary execution entry point for the node. Before any plotting begins, method *start_plot()* is invoked to handle the core responsibilities of reading, validating, and visualizing CSV data. It begins by referencing the file *durations.csv* and calls method *CSVDataValidator.validate_file()* to ensure the file exists and is not empty. Once the file is confirmed to be accessible, the method loads its contents using *pandas.read_csv()*. If all rows pass validation, method *start_plot()* proceeds to generate the plot using *matplotlib*. Each validated experiment case plots the ten duration values against loop indices 1 to 10. Finally, method *plot_times()* raises a custom exception *PlotTimesExit*, which is caught to signal the end of execution cleanly.

Following class *PlotTimes*, class *SpecifiedTimes* is implemented with method *specify_times()*. This class is designed to validate whether the execution time of a recent robotic cycle complies with a user-defined time threshold. Its sequence diagram is provided in Figure B.5. The core analysis begins inside method *analyze_with_threshold()*. This method first calls *read_last_row()* to extract execution times from the latest entry in the CSV file, i.e., *durations.csv*. Before accessing the data, method *read_last_row()* executes method *CSVDataValidator.validate_file()* to ensure the file both exists and is non-empty. Once the file passes initial validation, method *read_last_row()* proceeds to extract the last row's duration values, ensuring they are all floating-point numbers and exactly ten in count. It also iterates through all rows using the *CSVDataValidator* model to verify that the overall file format conforms to the expected constraints.

Subsequently, method *analyze_with_threshold()* prompts the user to input a time threshold, which is validated using method *ThresholdValidator.get_valid_input()*. After the duration data and threshold have been validated, the method evaluates each of the ten duration values against the user-defined threshold. If a duration is less than or equal to the threshold, it logs an *INFO* message, marking the value as feasible. Otherwise, it logs a *WARNING* and sets attribute *is_execution_time_obeyed* to *False*, signaling that at least one part of the execution loop failed to meet the performance requirement. Simultaneously, the objective's second concept listed in Section 4.1 is implemented.

5.5 Determination of used workspace

With the implementation of the stable grasp verification concept and determination of the execution time, this section discusses the last segment of the concept listed in the objective, i.e., visualization of the workspace used for the pick-and-place handling skill. The process is developed based on the flowchart in Figure 4.8. Furthermore, the implementation is divided into two subsections. Subsection 5.5.1 explains how the robotic manipulator's used workspace data can be collected during the pick-and-place handling skill. Accordingly, Subsection 5.5.2 describes how the collected data can further be analyzed.

5.5.1 Collecting workspace data in a simulation

This subsection is primarily responsible for developing an approach to recording the surface point clouds of the robotic manipulator in MoveIt2 by accessing the published topic in the motion planning framework. Meanwhile, the point clouds of obstacles are created to document their positions. This subsection returns to the implementation process of class *LaunchSim* in Figure 5.7. Three classes, i.e., *RobotStateUpdater*, *SurfaceRecorder*, and *ObstacleReader*, are described in this subsection, which are also developed in C++. Their class diagrams are also provided in Figure 5.6. However, Figure D.1 presents that the robotic manipulator's URDF file is composed of the part's STL files. These STL files are first optimized through the software Blender to increase the number of meshes included in the models. Subsequently, class *LaunchSim* is executed.

Class RobotStateUpdater

Class *RobotStateUpdater* is launched by method *launch_terminal()* with parameter *C3* in Table 5.3, and its class diagram is shown in Figure A.14. Class *RobotStateUpdater* is a ROS2 node responsible for periodically publishing the current joint states of the robotic manipulator using MoveIt2 planning interface. Upon instantiation, the node logs a startup message and initializes a publisher for messages *sensor_msgs::msg::JointState* on topic */robot_joint_states*. It calls method *check_clock_sync()* to inspect whether the system's time is synchronized using *chronyc*, which is important in distributed systems that depend on tightly synchronized timestamps for sensor fusion or coordinated control. This method executes a shell command, parses the result to extract time offset, and logs a warning if the offset exceeds ± 1 second. To safely initialize MoveIt2 components, the node sets up a delayed *timer_* with a 10-second wait period, which later invokes method *initialize()*. Inside method *initialize()*, the node calls method *shared_from_this()* in a safe context to create MoveIt2's *MoveGroupInterface* and *RobotModelLoader*, both of which require the node handle to be entirely constructed. It retrieves the kinematic model from the loader and then reassigns the *timer_* to begin publishing joint states every 200 milliseconds, i.e., at 5 *Hz*.

The core functionality lies in method *publish_joint_states()*, which is executed periodically. This method first checks whether *arm_group_* has been properly initialized. If *arm_group_* has not been correctly initialized, a warning is logged, and the method returns early to avoid execution failure. It then attempts to retrieve the robotic manipulator's current state through method *arm_group_->getCurrentState()*. If successful, the method packages the joint names and positions into a *JointState* message. It constructs the position field using raw pointer access from object *RobotState*, ensuring the message captures the most up-to-date joint con-

figuration. The message's timestamp is set using method *this->now()*, guaranteeing accurate timing aligned with ROS2 time, and the message is published.

Class SurfaceRecorder

Sequentially, class *SurfaceRecorder* is launched by method *launch_terminal()* with parameter *C4* in Table 5.3. Its class diagram is presented in Figure A.15. Class *SurfaceRecorder* implements a ROS2 node that executes offline surface point collection by extracting the three-dimensional mesh vertices of the robotic manipulator's links in real-time using MoveIt2 kinematic model. Its objective is to periodically record the actual transformed surface geometry of the robotic manipulator and write the results into a persistent *.txt* file, i.e., *raw_surface_points.txt*. Upon creation, the node subscribes to topic */robot_joint_states* and initializes the point-saving file.

Method *initialize()* is executed 10 seconds after node startup via a timer to avoid premature use of *shared_from_this()* inside the node. It sets up the core MoveIt2 components, i.e., a *RobotModelLoader*, the *RobotModel* itself, and a fresh *RobotState* object. Once initialized, two high-frequency timers are started. *record_timer_* samples robot mesh geometry at 10 Hz, and *file_timer_* writes accumulated points into the file at 5 Hz.

The core surface extraction logic is developed in method *record_surface_points()*. Each call spins off a background thread to safely process the current robot state without blocking ROS2 callback thread. This method first checks whether the latest joint state is available and locks it using *mutex*. Subsequently, it updates *robot_state_* to reflect current joint positions and traverses all links in the robotic manipulator's URDF. For each link, it inspects the attached collision shapes. Suppose a shape is a triangle mesh, i.e., *shapes::Mesh*. In that case, the method iterates through all vertices, transforms them into world space using the link's transform method *getGlobalLinkTransform()*, and stores the resulting points in a local buffer. Once the transformed vertices are gathered, they are inserted into *point_buffer_* using *mutex* to avoid race conditions. Writing to disk is handled in method *save_points_to_file()*, which is called every 200 milliseconds. It locks *point_buffer_*, opens the file in append mode, and writes the three-dimensional coordinates of each *pcl::PointXYZ* to the file. After flushing the file to disk, it logs how many points were stored and clears the buffer. Figure D.11 presents an example of the file *raw_surface_points.txt*.

Class ObstacleReader

In Figure 5.7, with classes *RobotStateUpdater* and *SurfaceRecorder* discussed in the previous segments, classes *BosPoseRecorder* and *GazeboTopicLogger* described in Subsection 5.3.1, and class *MoveGroupDemoNode* implemented in Subsection 5.2.2, this segment further describes the last class *ObstacleReader*. Its class diagram is provided in Figure A.16, and this class is also launched by method *launch_terminal()* with parameter *C8* in Table 5.3. Moreover, this class is launched after executing the pick-and-place handling skill to read the obstacles apart from the ten boxes. Therefore, the thorough obstacles in the simulation environment can be documented.

Upon startup, the node initializes *TF2* listener and buffer, then instantiates *PlanningSceneInterface*. Sequentially, the node retrieves the list of known collision objects using method *getKnownObjectNames()*. A warning is logged if no objects are found, and the process terminates. Otherwise, it retrieves the poses and object definitions for each known object, i.e., *CollisionObject* messages. In addition, the node opens the file, i.e., *obstacles_point_*-

cloud.txt, using *std::ios::trunc* to overwrite any previous content and proceeds to loop through all known obstacle IDs and attempts to extract their spatial pose. The node checks the result of method *getObjectPoses()* to determine the correct pose for each object. Suppose retrieving the pose using method *getObjectPoses()* fails, the node falls back to using the first entry in the *primitive_poses* field of the corresponding *CollisionObject*. The core logic is the conversion of geometric primitives into three-dimensional point clouds based on the shape types listed as follows:

- For *boxes*, it samples a three-dimensional grid of points inside the volume using nested loops with a resolution of 0.005 *m*
- For *cylinders*, it samples circular slices along the height axis at fixed angular intervals
- For *spheres*, it uses a randomized algorithm to uniformly sample the surface and interior of the sphere using spherical coordinates, i.e., *theta*, *phi*
- For *cones*, it linearly interpolates the radius along height and samples concentric circles at intervals to simulate the tapering geometry

After generating all points for obstacles, the node logs the number of generated points and writes each one to file *obstacles_point_cloud.txt* with the identical format as Figure D.11. Accordingly, the thorough process of class *LaunchSim* is finished.

5.5.2 Processing workspace data after a simulation

Using the recorded point clouds of the robotic manipulator, this subsection aims to plot these point clouds to present the workspace of the robotic manipulator during the execution of the pick-and-place handling skill. Simultaneously, the point clouds of obstacles are considered in the plot to determine whether a collision with the robotic manipulator happens. After finishing class *LaunchSim*, this subsection assumes that class *ExecutionTime* of class *Main*'s sequence diagram in Figure 5.3 is also finished and discusses class *WorkSpaceVisual*. Its class diagram is shown in Figure A.17. Class *WorkSpaceVisual* is developed to analyze the files *raw_surface_points.txt* created from class *SurfaceRecorder* and *obstacles_point_cloud.txt* created from class *ObstacleReader*. In addition, this class is implemented by method *start_visualize()* from class *Main*. First, class *PlotRobotPointCloud* is executed by method *visualize_robot()* to plot the used workspace of the robotic manipulator after implementing the handling skill. Sequentially, class *PlotObstaclePointCloud* is called by method *visualize_obstacle()* to plot the point clouds of the found obstacles. Simultaneously, all the *.txt* files are validated through *TXTPathValidator*.

Lastly, class *PlotTobotObstaclePointCloud* executes collision detection and visualization by comparing these two *.txt* files. The class begins by prompting the user to specify whether there are still unmodeled obstacles around the robotic manipulator. This decision is validated using a Pydantic model *IsStillOtherObstaclesValidator*, which ensures input is strictly *yes* or *no*. If the user answers *yes*, the node does not continue with collision checking. Instead, it logs instructions to manually update the obstacle file before re-running the script, ensuring the analysis is based on complete environmental information. If the user replies *no*, indicating that all obstacle data has been recorded, the class validates both file paths using *TXTPathValidator*. After the validation, the robot's surface points and obstacle points are converted into tuples and intersected to find exact coordinate matches, representing potential collisions between the robot and nearby structures. If overlapping points are found, the

node sets attribute *is_robot_obstacle_collision* to *True* and logs the number and coordinates of each overlapping point. It also warns the user to update these obstacles into MoveIt2 as collision objects and re-run the entire simulation to ensure accurate grasp or motion planning. Contrarily, if no overlapping points are detected, the node logs that the scene is clear of robot-obstacle conflicts and specifies attribute *is_robot_obstacle_collision* as *False*. Moreover, the node generates four distinct plots using *matplotlib*, i.e., a three-dimensional isometric view and three orthographic projections (XY, YZ, and XZ). Each plot shows the robot surface in blue, the obstacle space in transparent gray, and overlapping points in red. All plotting and collision-checking logic is contained within method *start_plot()*, which is invoked by method *visualize_robot_obstacle()*. If the process completes successfully, a custom exception *PlotRobotObstaclePointCloudExit* is raised to return.

Accordingly, class *WorkSpaceVisual* is finished, and the results are further presented in Section 6.1. Simultaneously, all the classes included in class *Main* are fully described. Regarding the last *alt* condition in Figure 5.3, attribute *is_ready_for_production* will be determined as *True* if the condition, i.e., *is_stable_grasp* as *True*, *is_execution_time_obeyed* as *True*, and *is_robot_-obstacle_collision* as *False*, is fulfilled to specify the successful verification.

Chapter 6

Evaluation

Following the implementation processes in Chapter 5, this chapter transfers the focus to the result evaluation. In addition, this chapter is classified into four groups. Section 6.1 presents the overview of the simulation results based on the implementation in Chapter 5. Corresponding to the objectives shown in Figure 4.1, Section 6.2 evaluates the developed stable-grasp-related verification concept. Sequentially, Section 6.3 verifies the performance of the execution time under different settings of the handling skill. Lastly, Section 6.4 demonstrates the robotic manipulator's used workspace with different simulation environments.

6.1 General overview of simulation results

This section presents an overview of the simulation results based on class *Main*'s sequence diagram in Figure 5.3. Therefore, a General Test case GT1 is created. The parameters of the scenario's boxes and the setting of the pick-and-place handling skill are listed in Table 6.1. Before implementing class *Main*, four things must be determined in advance, which are listed as follows:

- Defining value in method *arm_group.setMaxVelocityScalingFactor()*
- Defining value in method *arm_group.setMaxAccelerationScalingFactor()*
- Defining value in method *sendGripperCommand()*
- Adding *CollisionObject*

The values in methods *arm_group.setMaxVelocityScalingFactor()* and *arm_group.setMaxAccelerationScalingFactor()* are responsible for scaling the velocity and acceleration of the robotic manipulator's motion, and the values are regulated within the interval $[0, 1]$. For the general test case, they are scaled as 1.0. Meanwhile, the value of method *sendGripperCommand()* in Table 6.1 specifically represents the procedure for closing the gripper, which is described in Subsection 5.2.2. This value controls the position of the fingers to decide how tight the box is gripped. Besides, no additional *CollisionObject* is defined. Hence, the simulation environment in RViz and Gazebo is identical as Figure 5.11 and Figure 5.9, respectively.

With the determined values, class *ParametersPreparation* is first initiated, and class *SdfParameters* is invoked to input the parameters. The input parameters of test case GT1 in

Table 6.1: Parameters and setting of test case GT1

Test case	GT1
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	SS316
mu	2
mu2	2
poissons_ratio	0.26
elastic_modulus	0.26
mass in kg	1
stiffness in $\frac{N}{m}$	9.65×10^9
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000416667
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000416667
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000416667
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*soft*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	205
contact_area in mm^2	300

class *SdfParameters* are listed in Table 6.1, ranging from *mu* to *stepsize*. The considered material of the box in the test case is SS316, and the material of the gripper's finger is rubber. Meanwhile, as stated in Section 5.1, the inertia matrix values are determined through the box's CAD model in SolidWorks. Equipped with these input values and the computation processing in class *SdfParameters*, attribute *contact_model* is defined as *soft* in class *ContactModels* to further determine the corresponding parameters to be considered. After considering the contact model, all the input parameters are written into the SDF file *sim.world* through class *WriteSdf*, and Figure D.12 shows an example of the values of *box_green_0* in the SDF file.

Following class *ParameterPreparation*, the pick-and-place handling skill and data collecting are implemented in class *LaunchSim*. Figure D.13 and Figure D.14 present the simulation images in RViz and Gazebo after finishing ten cycles of the pick-and-place task. Moreover, class *ExecutionTime* is initiated to analyze attribute *durations* returned from class *MoveGroupDemoNode*. Figure 6.1 presents the cycle time of each pick-and-place loop in test case GT1, as visualized by class *PlotTimes*. For example, it takes 16.7195 seconds for the robotic manipulator to finish the first three steps in Figure 4.2. With attribute *threshold* input as 25 seconds in class *SpecifiedTimes*, Figure 6.2 indicates the analysis results of execution times and the threshold from the class. Accordingly, all ten cycle times obey the threshold.

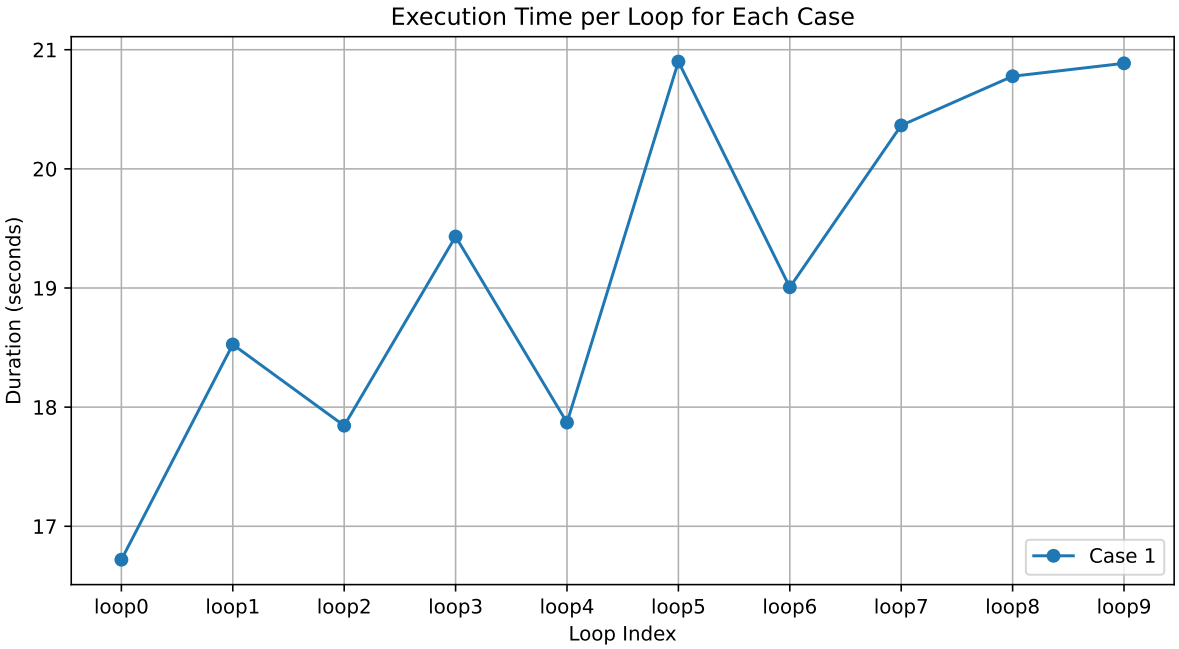


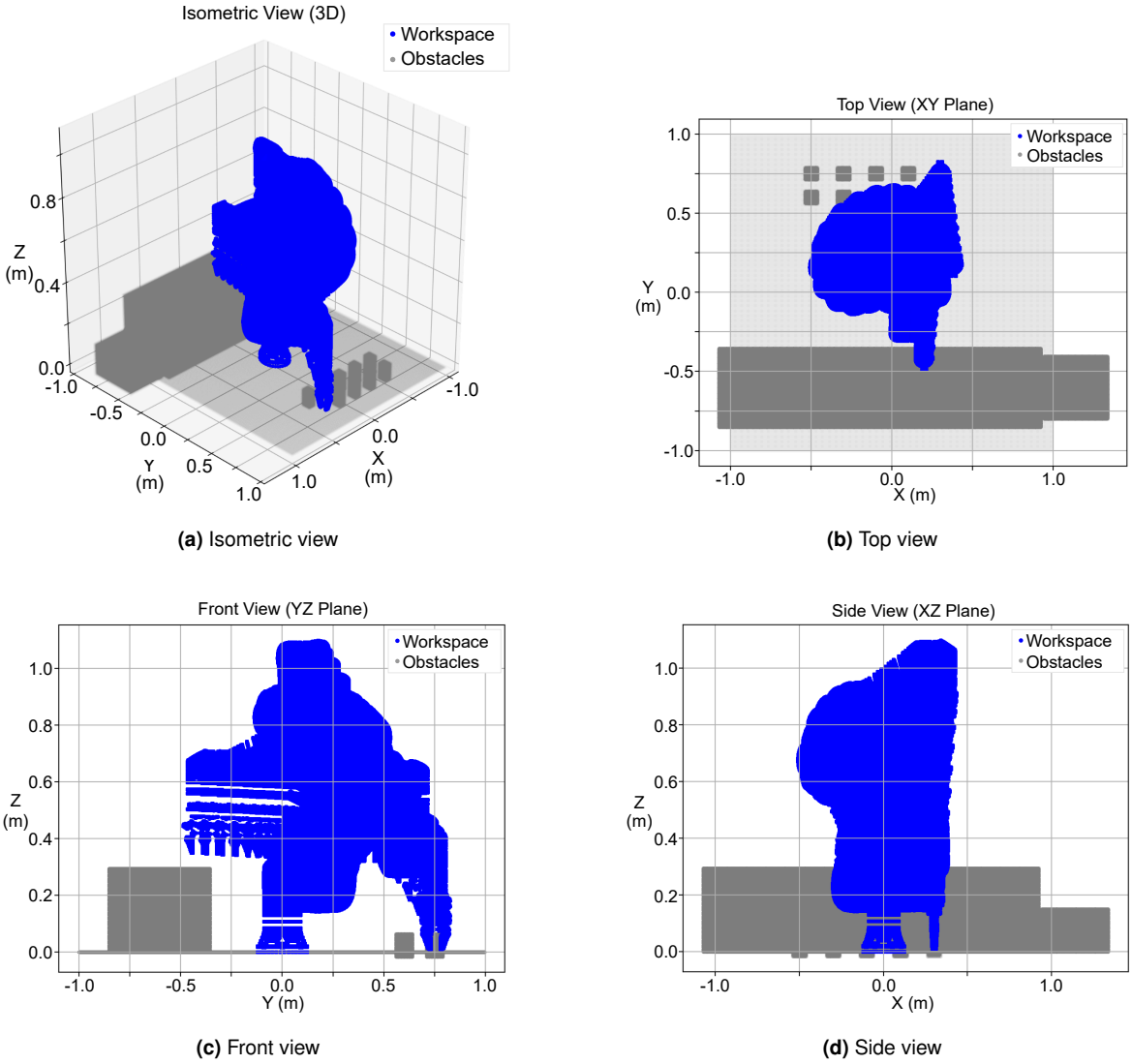
Figure 6.1: Result of test case GT1 from class PlotTimes

In Figure 5.3, the next class to be launched is class *WorkSpaceVisual*. According to Subsection 5.5.2, class *WorkSpaceVisual* inherits from three other classes, i.e., *PlotRobotPointCloud*, *PlotObstaclePointCloud*, and *PlotRobotObstaclePointCloud*. The first two classes are not evaluated because the third class reads both the point clouds of the used workspace and the point clouds of obstacles from them and considers them together to specify whether a collision happens. Regarding test case GT1, no additional *CollisionObject* is defined. Therefore, with the point clouds of all the objects in the simulation environment recorded from classes *SurfaceRecorder* during the handling skill and *ObstacleReader* after the handling skill, Figure 6.3 presents the used workspace of the robotic manipulator, which is created from class *PlotRobotObstaclePointCloud*. This figure represents the end of loop 0 in the test case. Based on the implementation process in this class, no overlaps are plotted. Therefore, no collision with obstacles is detected.

```
ros2@ros2-virtual-machine: ~/ur30_all_included_ws

[INFO] [1746618946.545477127] [specified_times]: EXECUTION STATUS: specified_times.py is running!
[INFO] [1746618946.545915064] [specified_times]: 'file_path' Successfully Validated!
[INFO] [1746618946.548497994] [specified_times]: 'last_row' Successfully Validated!
Please enter the Time Threshold of cycle (in second) (Must Be > 0): 25
[INFO] [1746618950.099560220] [specified_times]: Execution Times Analysis Results
[INFO] [1746618950.099973006] [specified_times]: Time 0: Feasible (16.7195 (s) ≤ 25.0 (s))
[INFO] [1746618950.100236675] [specified_times]: Time 1: Feasible (18.5261 (s) ≤ 25.0 (s))
[INFO] [1746618950.100488437] [specified_times]: Time 2: Feasible (17.8442 (s) ≤ 25.0 (s))
[INFO] [1746618950.100726347] [specified_times]: Time 3: Feasible (19.4322 (s) ≤ 25.0 (s))
[INFO] [1746618950.100972370] [specified_times]: Time 4: Feasible (17.8706 (s) ≤ 25.0 (s))
[INFO] [1746618950.101266761] [specified_times]: Time 5: Feasible (20.9004 (s) ≤ 25.0 (s))
[INFO] [1746618950.101575248] [specified_times]: Time 6: Feasible (19.0061 (s) ≤ 25.0 (s))
[INFO] [1746618950.101815944] [specified_times]: Time 7: Feasible (20.3646 (s) ≤ 25.0 (s))
[INFO] [1746618950.102141431] [specified_times]: Time 8: Feasible (20.7773 (s) ≤ 25.0 (s))
[INFO] [1746618950.102438394] [specified_times]: Time 9: Feasible (20.8858 (s) ≤ 25.0 (s))
[INFO] [1746618950.102681918] [specified_times]: EXECUTION SUCCESS: specified_times.py is
finished, returning to execution_time.py!
```

Figure 6.2: Result of test case GT1 from class SpecifiedTimes



(a) Isometric view

(b) Top view

(c) Front view

(d) Side view

Figure 6.3: The used workspace of test case GT1's loop 0

Currently, class *WorkSpaceVisual* is completed, and the implementation results of class *ContactInfoOrganzier* are not further evaluated because this class is solely responsible for organizing the data returned from Gazebo contact topic and the boxes' CoM positions. Following class *ContactInfoOrganizer* and attributes setting, *for loop* is conducted. However, the results are presented based on the stable grasp analysis of box 0 in the following because the process is identical in *for loop*. At the beginning of the loop, methods *compute_max_force_torque()* and *check_gravity_inside_force()* are invoked to compute the maximum contact force and torque of 3000 contact samples for the static stable grasp analysis and preliminarily specify if the corresponding maximum normal contact force is enough to hold the box's gravity. Figure 6.4 and Figure 6.5 show these samples' contact force and torque magnitudes, respectively. Moreover, the maximum static contact force and torque and gravity check of test case GT1 are listed in Table 6.2.

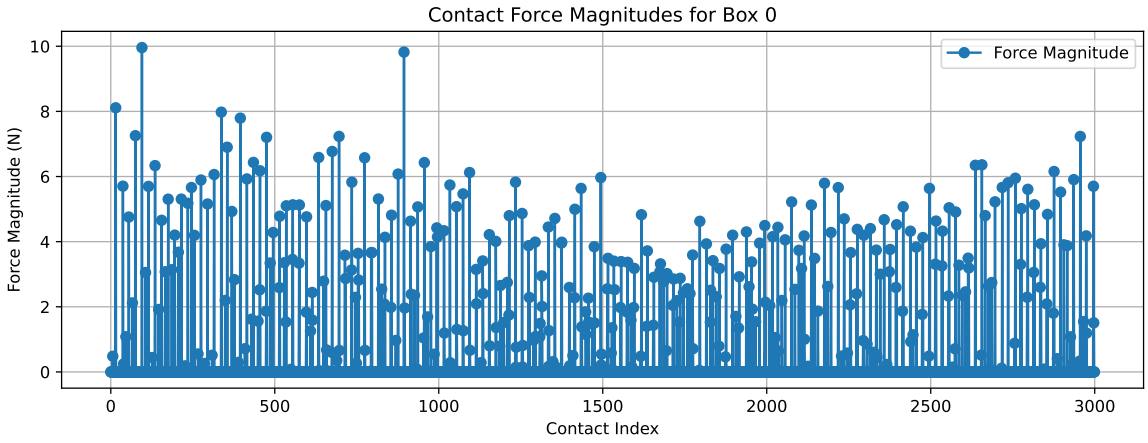


Figure 6.4: Box 0's contact force magnitudes of test case GT1 for static analysis

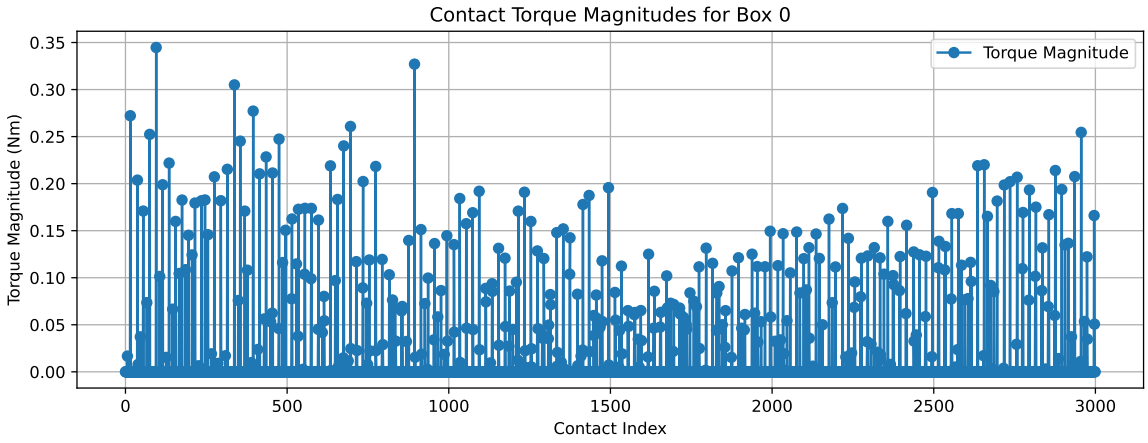


Figure 6.5: Box 0's contact torque magnitudes of test case GT1 for static analysis

Class *StableGrasp* is implemented following the preliminary gravity check. The presentation of stable grasp analysis results in a terminal is divided into eight sectors and listed in Table 6.3. Sector 1, Sector 2, and Sector 3 indicate whether the force closure verification is fulfilled. With inputs in class *OtherSpace*, Sector 4 specifies the fulfillment of the wrench spaces based on the configuration in Figure 4.6. Accordingly, Sector 5 determines the fulfillment of static stable grasp analysis. Subsequently, a box's maximum and mean acceleration values are

Table 6.2: Maximum static contact force and torque of test case GT1's box 0

Test case	GT1
Maximum static contact force in N	9.9594
Maximum static contact torque in $N \cdot m$	0.3447
Affordability of box's gravity	*True*

presented in a terminal before Sector 6. Sectors 6 and 7 define the outcome dynamic stable grasp analysis. Finally, Sector 8 concludes the overall fulfillment of stable grasp analysis.

Table 6.3: Presentation of stable grasp analysis results

Sector	Relevant fulfillment
1	Full rank of grasp matrix
2	Friction cone constraints
3	The origin lying within GWS
4	Configuration of the wrench spaces
5	Static stable grasp analysis
6	First-layer of dynamic stable grasp analysis
7	Second-layer of dynamic stable grasp analysis
8	Stable grasp analysis

The results of box 0's stable grasp analysis are presented in Table 6.4. Nevertheless, regarding the last segment of the stable grasp analysis, i.e., dynamic analysis, Sector 6 logs that the first-layer check of class *DynamicConsideration* is not fulfilled. Figure 6.6 shows the accelerations of the box 0 in test case GT1. Sector 6 presents that the maximum static contact forces provided by two fingers are insufficient to hold the total forces applied to the box in dynamic conditions. Meanwhile, the stable grasp condition is not fulfilled. Regarding the results of test case GT1, all the cycles' execution times are obeyed, and no collision with obstacles is detected. However, the stable grasp analysis is not fulfilled. Therefore, test case GT1 is not ready for production. The implementation of class *Main* and the review of test case GT1 are completed.

6.2 Verification of stable grasp

After explaining the overview of simulation results in Section 6.1, this section verifies the performance of the stable grasp verification concept. Subsection 6.2.1 attempts to modify the values of gripper control commands to specify the difference. Subsection 6.2.2 introduces different materials into the simulation to observe the corresponding results. Lastly, Subsection 6.2.3 evaluates the stable grasp analysis with different contact models.

Table 6.4: Stable grasp analysis results of test case GT1's box 0

Test case	GT1
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*True*
Fulfillment of the origin lying with GWS	*True*
Fulfillment of the wrench spaces' configuration	*True*
Fulfillment of static stable grasp analysis	*True*
Maximum acceleration in $\frac{m}{s^2}$	22.2111
Mean acceleration in $\frac{m}{s^2}$	7.1175
Fulfillment of first-layer dynamic stable grasp analysis	*False*
Fulfillment of second-layer dynamic stable grasp analysis	*Failed*
Fulfillment of stable grasp analysis	*Failed*

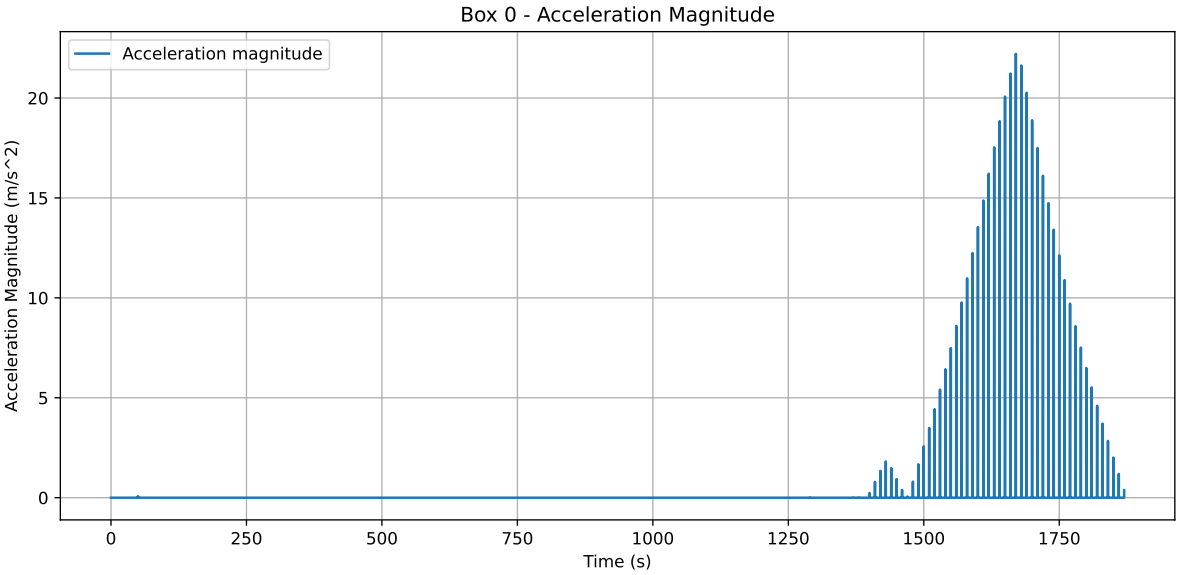


Figure 6.6: Box 0's accelerations of test case GT1 for dynamic analysis

6.2.1 Modification of gripper control

This subsection evaluates the returned values of maximum contact force and torque by increasing the position argument in method *sendGripperCommand()* to 0.44, which results in a tighter grip on the box. As introduced in Subsection 5.2.2, this method controls the gripper's opening width. A new test case, named Stable Grasp SG1, is designed to investigate how a firmer grasp influences the contact force and torque values. The parameters and settings of SG1 are listed in Table E.1 in Appendix E. Table E.2, Figure E.1, and Figure E.2 present the results of maximum contact force and torque under this tighter grasp condition. Compared to test case GT1 (Table 6.2), both values are higher in SG1, indicating stronger contact interactions. As a result, the adjustment of *sendGripperCommand()* is validated as an effective way to influence the grasp stability.

Besides the value modification in method *sendGripperCommand()*, another method, named *plot_box_contact_overlap()*, is developed to specify whether the value of method *sendGripperCommand()* influences the contact depth of the box within the static stable grasp analysis. Theoretically, the contact depth of the box is supposed to be deeper when the maximum static contact force is larger. Figure 6.7 shows the contact situation of box 0 in test case GT1, and the value of *sendGripperCommand()* is 0.42. Figure 6.8 presents the corresponding situation images in Gazebo. During the static grasp phase, the contact points data are extracted from Gazebo contact topic over a sequence of 3000 samples, as presented in Figure 5.3. Meanwhile, the CoM position of the box is read, and the box's point clouds are created based on its input dimension. Subsequently, this method automatically specifies whether there is an overlap between the created point clouds and the read contact points. If an overlap is detected, it represents that the gripper indeed contacts the box. Furthermore, the front view of Figure 6.7 shows that the overlaps are mainly on the box's surface. Focusing on the left side of the front view, the overlaps are estimated to be located in 0.73 m. However, it is also shown that the configuration of overlap points does not correspond to the shape of the gripper. Especially comparing the side view of Figure 6.7 with Figure 6.8d, the overlap points concentrate on the corners of the contact surfaces between the gripper and the box. It is because the gripper is introduced in the form of STL, which is composed of triangle meshes. When Gazebo uses ODE, a triangle mesh defaults to surface-only, with no thickness and easy penetration.

Moreover, Figure E.3 shows the contact situation of box 0 in test case SG1, of which the value of method *sendGripperCommand()* is 0.44. It can be found that the overlaps in the left side of the front view in Figure E.3 are located deeper than they are in test case GT1, i.e., 0.74 m. Therefore, it is reasonable that the maximum static contact force of test case SG1 (Table E.2) is larger than the corresponding value in test case GT1 (Table 6.2).

6.2.2 Results of different materials

After verifying the modification of gripper control in Subsection 6.2.1, this subsection discusses the stable grasp analysis results by introducing different materials, i.e., Polytetrafluoroethylene (PTFE) and silicone. They are regarded as test cases SG2 and SG3 and listed in Table E.3 and Table E.4, respectively. The setting in class *MoveGroupDemoNode* of these two test cases is identical to the setting in test case SG1. The differences are the properties of the materials, i.e., input in class *SdfParameters* and class *DynamicConsideration*. Table 6.5 lists the results of test case SG2's stable grasp analysis and indicates that the maximum static contact force is smaller than the value in test case GT1. It is reasonable because the introduced stiffness of PTFE is more minor. Based on Equation (2.15), the computed contact force

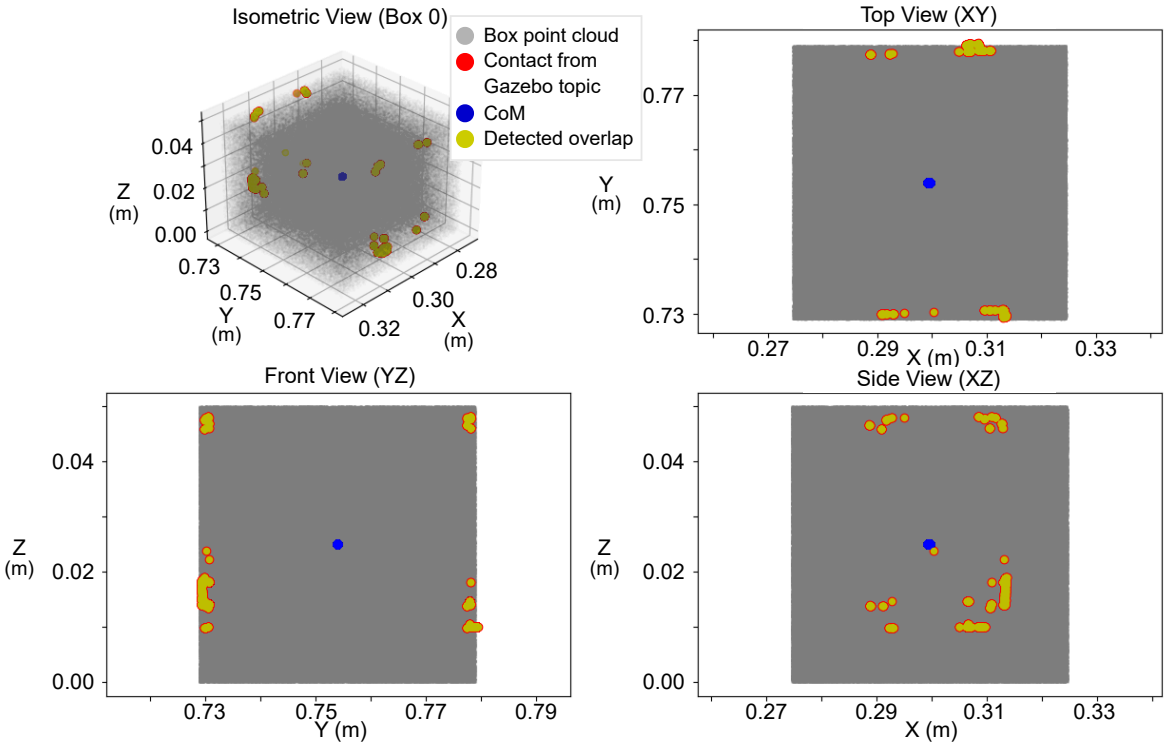
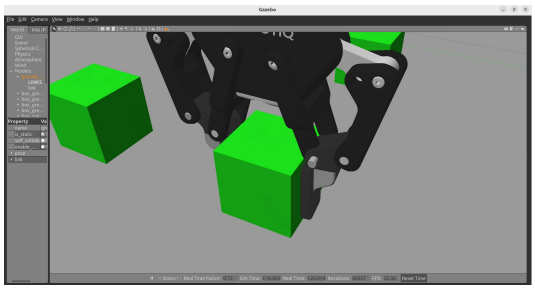
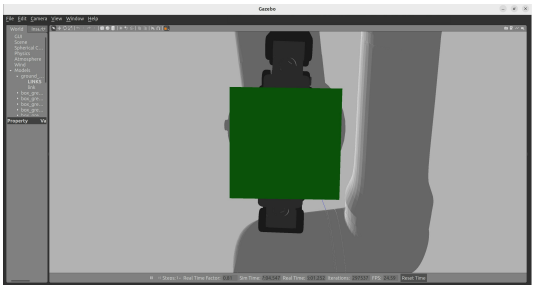


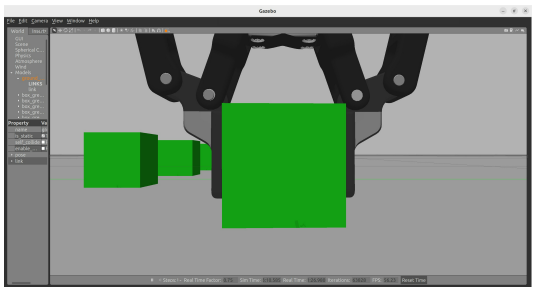
Figure 6.7: Contact situation of box 0 in test case GT1



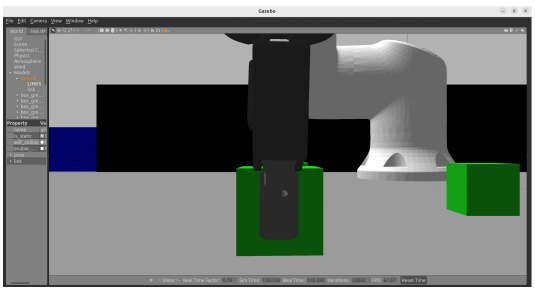
(a) Isometric view



(b) Bottom view



(c) Front view



(d) Side view

Figure 6.8: Static stable grasp analysis of test case GT1's box 0 in Gazebo

is supposed to be smaller. In addition, it is presented that test case SG2 does not fulfill the verification of friction cone. Accordingly, the following procedures are not further processed, and this test case is defined as a non-fulfillment of stable grasp.

Additionally, regarding the stable grasp analysis results of test case SG3 in Table E.5, the maximum static contact force is also smaller than the value of test case SG2 because of the smaller stiffness value. Although the result of box's gravity affordability is *False*, the stable grasp analysis is further processed because the maximum static contact force is determined solely from one contact point. In the defined SDF file, parameter <*max_contacts*> is defined as 10, which represents that there are 10 contact points involved in a contact. Following the gravity check, the static stable grasp verification is fulfilled. However, it does not pass the first-layer dynamic verification because of its high acceleration and small maximum static contact force value.

Table 6.5: Stable grasp analysis results of test case SG2's box 0

Test case	SG2
Maximum static contact force in N	4.7799
Maximum static contact torque in $N \cdot m$	0.1367
Affordability of box's gravity	*True*
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*False*
Fulfillment of the origin lying with GWS	*Failed*
Fulfillment of the wrench spaces' configuration	*Failed*
Fulfillment of static stable grasp analysis	*Failed*
Maximum acceleration in $\frac{m}{s^2}$	*Failed*
Mean acceleration in $\frac{m}{s^2}$	*Failed*
Fulfillment of first-layer dynamic stable grasp analysis	*Failed*
Fulfillment of second-layer dynamic stable grasp analysis	*Failed*
Fulfillment of stable grasp analysis	*Failed*

6.2.3 Results of different contact models

For the verification of stable grasp analysis, Subsection 6.2.1 reviews the setting in class *MoveGroupDemoNode*, and Subsection 6.2.2 introduces different materials and their properties in class *ParametersPreparation*. This subsection further verifies the introduction of different contact models based on Table 4.4. In test case GT1, the input contact model is *soft* (Table 6.1). Two test cases are created in this subsection, i.e., SG4 and SG5, of which the parameters and settings are listed in Table E.6 and Table E.7, respectively. In test case SG4, the contact model is defined as *hard_without_friction*. Accordingly, apart from the values of parameters <*mass*>, <*inertial*>, and <*size*>, all the other parameters are determined as 0. Table E.8 lists the stable grasp analysis results of test case SG4 and indicates that class *OtherSpace* is not fulfilled. The reason is that the wrench created by the box's gravity is not inside the computed GWS. Additionally, the maximum static contact force is only 0.0031 N because stiffness is defined as 0. Therefore the damping coefficient <*kd*> in the SDF file is computed

as 0 based on Equation (2.16). With these two parameters defined as 0, the returned contact force is correspondingly small based on Equation (2.15).

Regarding the other test case SG5, *hard_with_friction* is selected as the contact model, and the results of stable grasp analysis are listed in Table E.9. It is shown that the maximum static contact force is low, which is also reasonable because of the stiffness value. However, the maximum static contact force value is higher than test case SG4, representing that the introduction of friction coefficients influences the results. However, based on the values of maximum static contact force in test cases GT1, SG4, and SG5, it is indicated that the value in test case GT1 is more realistic. According to Subsection 2.4.1, the contact force in Gazebo contact topic, i.e., in ODE, is determined by Equation (2.15). Hence, the contact force is more precisely computed when introducing a stiffness and a damping coefficient.

6.3 Verification of execution time

Based on the objectives listed in Figure 4.1, this section verifies the function of execution time measurement. In Section 5.4, the execution-time-related implementation in class *MoveGroupDemoNode* and class *ExecutionTime* is described. Meanwhile, it is mentioned in Section 6.1 that methods *arm_group.setMaxVelocityScalingFactor()* and *arm_group.setMaxAccelerationScalingFactor()* are responsible for scaling the velocity and acceleration of motion. Consequently, an Execution Time test case ET1 is created to verify whether these two functions influence the corresponding execution times.

Table E.10 lists the test case parameters and settings. Specifically, the difference with test case GT1 is the value of these two methods. In test case ET1, they are determined as 0.1. Subsequently, Figure E.4 and Table E.11 provide the acceleration information and stable grasp analysis results of box 0 of this test case, which indicates that the maximum acceleration is smaller than the test case GT1 (Table 6.4). Figure 6.9 presents the execution times from two test cases, where *Case 1* is test case GT1, and *Case 2* is test case ET1. In the figure, the execution times of test case ET1 are much longer than those of test case GT1. This figure also concludes that the two methods for modifying velocity and acceleration are feasible because the robotic manipulator in test case ET1 moves with smaller velocity and acceleration scaling factors (Table E.10) in MoveIt2. Furthermore, the dynamic stable grasp analysis of test case ET1 (Table E.11) is fulfilled because of the lower accelerations. Nevertheless, the parameter *threshold* values are identical in these two cases, i.e., 25 seconds, and all the values of the execution times in test case ET1 are larger than the threshold. Accordingly, all the cycles' execution times do not obey the threshold.

6.4 Verification of used workspace

Following the verification of two objectives of Figure 4.1 in Section 6.1 and Section 6.2, this section conducts the verification of the robotic manipulator's used workspace. Specifically, this section attempts to verify the function for determining the used workspace described in Section 5.5. Meanwhile, another Workspace Visualization test case WV1 is created for the verification, and its parameters and settings are listed in Table E.12. As stated in Subsection 5.5.1, *CollisionObject* must be added in class *MoveGroupDemoNode*. Apparently, test case WV1 attempts to add another *CollisionObject*, i.e., *Box_obstacle* into MoveIt2 with interface

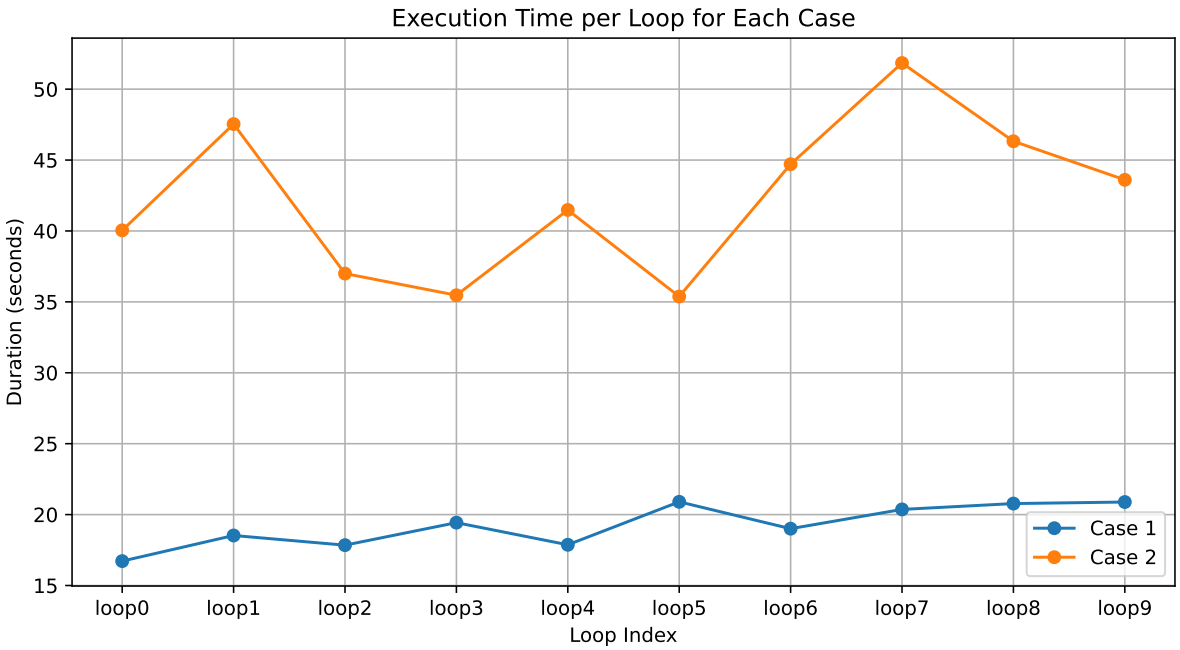
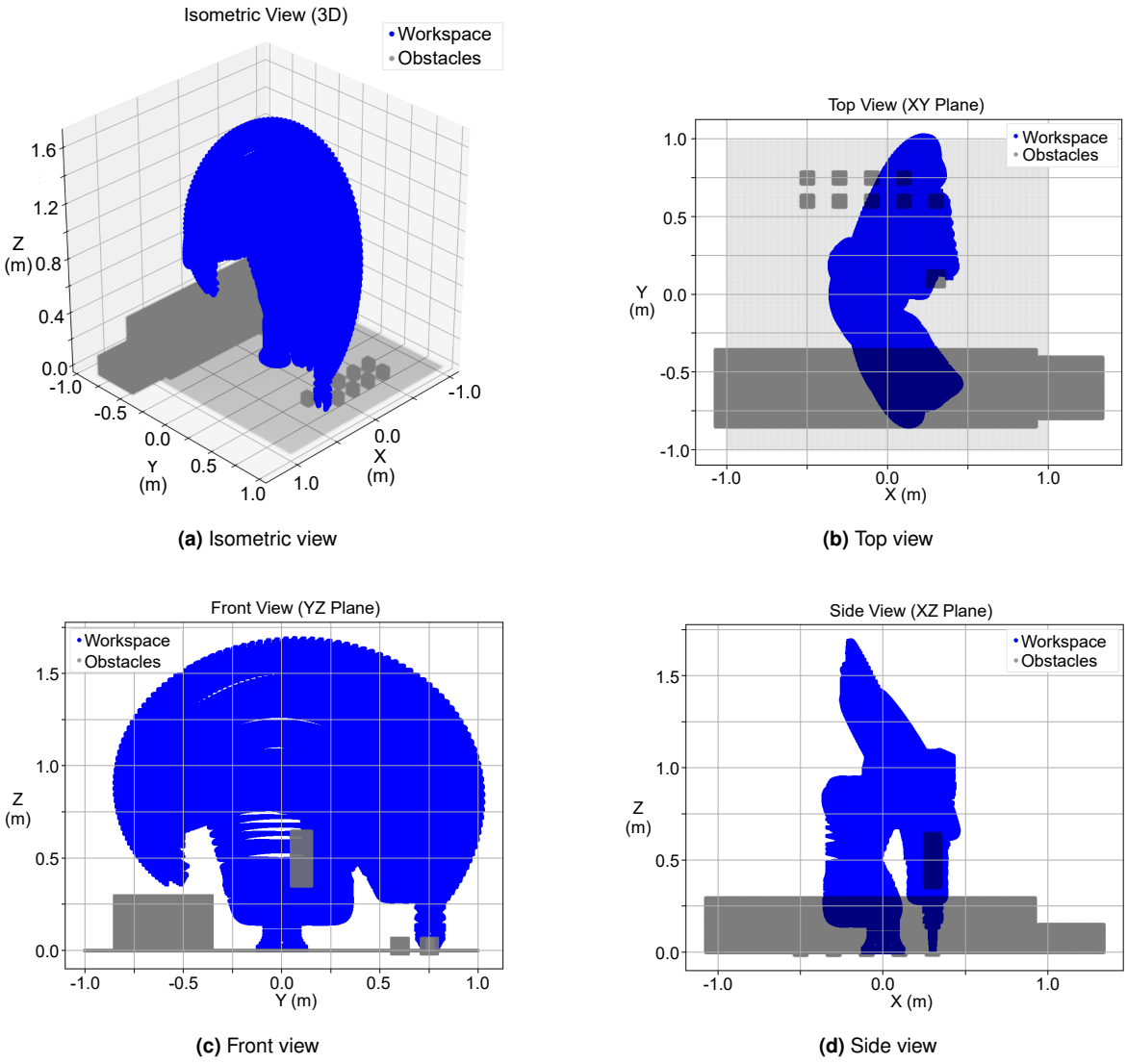


Figure 6.9: Results of test cases GT1 and ET1 from class PlotTimes

moveit_msgs::msg::CollisionObject. Additionally, *Box_obstacle* is defined as a solid primitive using *shape_msgs::msg::SolidPrimitive::BOX* and introduced within test case GT1's workspace (Figure 6.3). Figure D.15 presents the simulation environment with *Box_obstacle* as *CollisionObject* in RViz. Compared with Figure 5.11, it is obvious that this obstacle is successfully added into MoveIt2.

After implementing the pick-and-place task of loop 0, Figure 6.10 shows its corresponding used workspace. The introduction of the obstacle indeed influences the used workspace. In addition, it also proves that the motion planning from MoveIt2 does consider all the *CollisionObject*, and the planned trajectory will avoid the introduced obstacles. Simultaneously, the execution time of test case WV1's loop 0 is determined as 24.8233 seconds, which is longer than loop 0 in test case GT1, i.e., 16.7195 seconds. Hence, it is specified that the changed workspace also influences the corresponding execution time. Moreover, the box 0's stable grasp analysis results of test case WV1 are listed in Table E.13. Based on the longer execution time and larger used workspace, the value of the maximum acceleration is also influenced, i.e., smaller than the value in test case GT1 (Table 6.4). Therefore, the dynamic stable grasp analysis of test case WV1 is fulfilled with the lower accelerations. Accordingly, it has been proved that changing the workspace influences execution time and stable grasp analysis.

In test case WV1, it is indicated that the introduction of *Box_obstacle* makes the original execution time longer. However, it is also possible that introducing obstacles will shorten the time. Another test case WV2 is created, which is solely utilized for evaluating the relation between obstacles and execution times. The simulation settings of test case WV2 are listed in Table E.14, and this test case adds four additional obstacles into MoveIt2, i.e., *Box_obstacle*, *Cylinder_obstacle*, *Sphere_obstacle*, and *Cone_obstacle*. Figure D.16 shows its simulation environment in RViz. Except for the configuration of obstacles, all the other classes' parameters in test case WV2 are the same as in test case GT1. After the implementation from loop 0 to loop 9, Figure 6.11 presents WV2's execution times under *Case* 3. The reason for including execution time in this figure is to investigate whether environmental constraints from obstacles can lead to faster planning results. Compared to *Case* 1 (test case GT1), it can be observed that



(a) Isometric view

(b) Top view

(c) Front view

(d) Side view

Figure 6.10: The used workspace of test case WV1's loop 0

execution times are sometimes shorter in WV2, indicating that additional obstacles may help the planner converge to a more efficient trajectory by reducing the complexity of the search space.

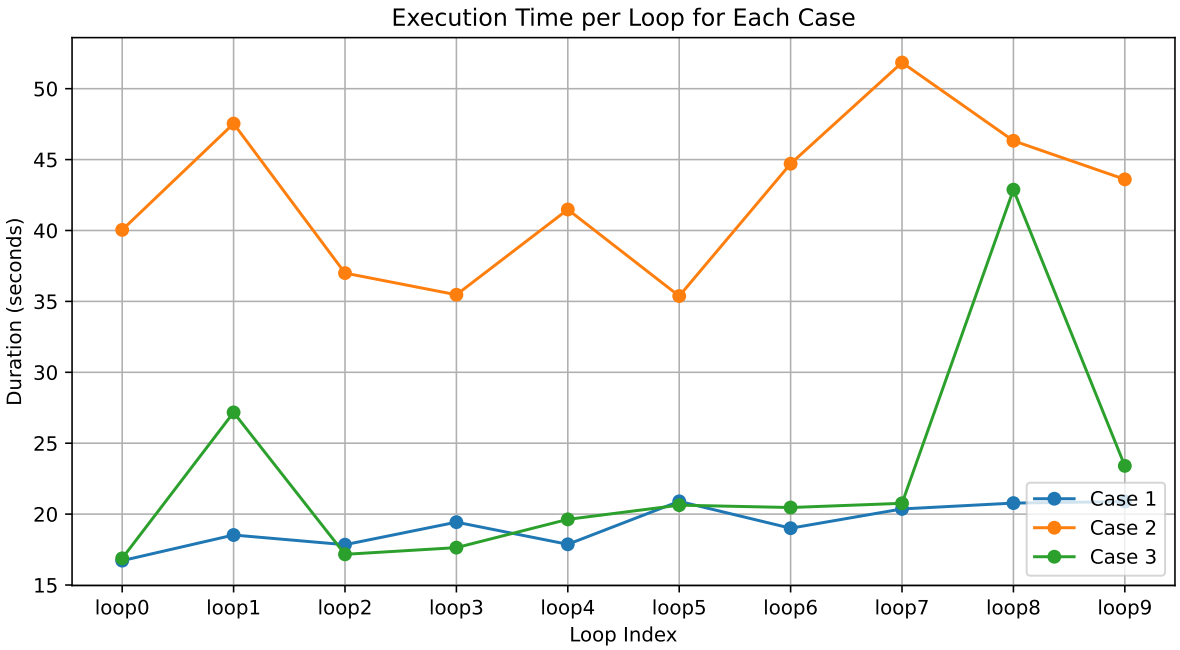


Figure 6.11: Result of test case WV2 from class PlotTimes

These results are relevant for robot deployment in dynamic environments where obstacles may be added or removed. The variation in execution time and acceleration values demonstrates that workspace configuration directly impacts the performance of motion planning and grasp stability. For system integrators or operators, careful workspace setup can influence execution times and grasp reliability, especially when dynamic loads or fragile objects are involved.

Chapter 7

Conclusion and outlook

Following the verification and evaluation in Chapter 6, this chapter describes an overall conclusion, which reviews specifically the corresponding contents from Chapter 4 to Chapter 6, and the structure is presented in Figure 7.1. In Chapter 4, a series of objectives and their concepts are developed after defining pick-and-place as the handling skill and determining the relevant parameters. The primary concepts are verifying stable grasp, determining execution time, and visualizing the used workspace. Specifically, the verification concept of stable grasp is developed based on Chapter 2, Section 3.1, and Section 3.2. The approaches for measuring execution time and visualizing an used workspace are designed based on Section 3.3 and Section 3.4, respectively.

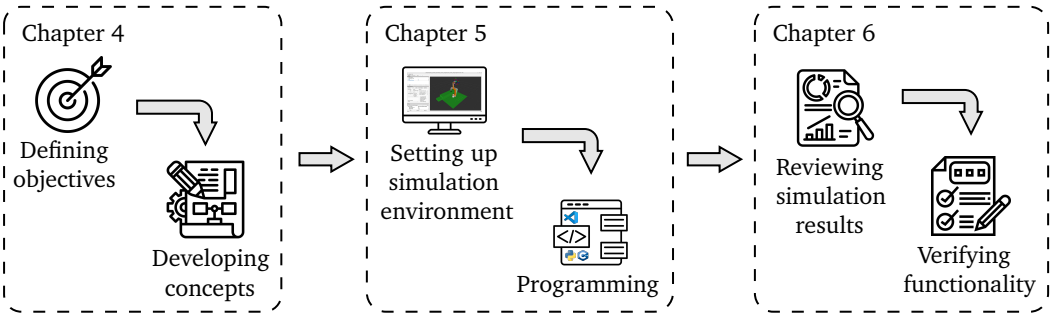


Figure 7.1: Scheme of the contents included from Chapter 4 to Chapter 6

All the concepts are implemented in ROS2, and the relation between the utilized tools is provided in Figure 5.1. After setting up the simulation environment, the core idea is that all the functions can be implemented by a master node, i.e., class *Main* in Figure 5.2. Hence, all the classes can be initiated by class *Main*, which means that the sole command to input in the terminal is *cd src/python/ && ./main.py*. Figure 7.2 provides an example for launching the master node. Simultaneously, as described in Chapter 5, the command must be implemented under repository *ur30_all_included_ws*. Automatically, all the other nodes in Figure 5.2 will be launched to execute the handling skill and the developed verification approaches. Therefore, the function of a skill set is realized.

```
ros2@ros2-virtual-machine: ~/ur30_all_included_ws
ros2@ros2-virtual-machine:~/ur30_all_included_ws$ cd src/python/ && ./main.py
```

Figure 7.2: Example of the command in the terminal for launching the master node

Chapter 6 first provides an overview of the simulation results. Accordingly, it proves that the developed automated trajectory parameterization process of the defined pick-and-place handling skill works, which can automatically analyze stable grasp, execution time, and used workspace of the pick-and-place task. When verifying the stable grasp concept in Section 6.2, it is concluded that the modification of gripper control and introduction of different box materials prove the feasibility and rationality of the developed concept. Nevertheless, regarding introducing different contact models in Subsection 6.2.3, it is concluded that solely the soft contact model is feasible for the concept. It is limited in the soft contact model because of the basics of the physics engine ODE in Gazebo, i.e., the spring-damper system. Additionally, Figure 6.7 shows that the gripper penetrates the box during the contact. The penetration and jitter happen because of the limitation of STL files and the physics engine ODE, which might also further influence the computation of contact force and torque because of the unexpected penetration. For the verification of execution time and used workspace, it proves their functionality. It specifies that the relation between stable grasp, execution time, and used workspace does influence mutually. Accordingly, the intention of parameterization is fulfilled.

Outlook for this thesis

After the review, this segment specifies the outlook for this thesis. The outlook scheme consists of three parts: the framework of this thesis, the introduction of the other framework, and future research. The first part is further divided into two segments: object-related parameterization and implementation. In the former segment, a skill is desired to be developed to automatically determine the grasp points of a gripped object. Based on Hertzian contact theory, a different configuration of the grasp points corresponds to different contact methods. Therefore, the computation approach of the corresponding contact forces is also different. Subsequently, if there is another skill to automatically determine the MAGF of a gripped object based on its material properties, the value of MAGF will be reasonable. Accordingly, the value can be introduced into the code of this thesis. In addition, a gripped object's stiffness can be determined by building a CAD model in Abaqus and be introduced into the code to make the simulation result more accurate.

This paragraph explains the outlook in the implementation segment. In this thesis, the objectives listed in Chapter 4 are related to verification. Its primary task is to specify whether the conditions, i.e., stable grasp, execution time, and used workspace, are fulfilled. Thus, an automatic optimization mechanism is desired. For example, when the results show that it is not a stable grasp, an optimized skill can automatically make the gripper grasp an object more tightly or lower the motion velocity. Furthermore, when it is determined that a non-collision workspace is achieved, another method is desired to be developed to document the robotic manipulator's time-related joint angles. Hence, the corresponding trajectory and workspace can be regenerated.

Apart from the optimized framework described based on this thesis, it is suggested that the thorough approach of this thesis can be integrated into another platform, i.e., NVIDIA Omniverse. ZHOU ET AL. (2024) mentions that NVIDIA Isaac Sim is an innovative platform for robotic manipulator simulation. NVIDIA Isaac Sim is a plugin in NVIDIA Omniverse and can be developed with ROS2. In particular, the physics engine introduced in NVIDIA Isaac Sim is PhysX, which NVIDIA has newly developed. Hence, it is expected that the code in this thesis can be further integrated into NVIDIA Isaac Sim to improve the mentioned penetration and jitter. In addition, the approach proposed in this thesis may benefit from integration with NVIDIA's upcoming open-source physics engine, Newton, which is scheduled for release

in July 2025 (NVIDIA CORPORATION 2025). Given its anticipated improvements in contact modeling and real-time physical simulation fidelity, Newton could further address the limitations observed in the current simulation results, particularly regarding contact instability and object interpenetration.

Beyond the practical extensions proposed in this thesis, future research may focus on enhancing robotic autonomy by integrating digital twin technologies. Such systems enable predictive validation and adaptive behavior by linking real robots with high-fidelity simulations. In this context, open research questions include how to reliably transfer tactile and contact-based skills from simulation to reality, improve the evaluation of grasp stability, and represent multi-modal sensor data within learning frameworks. Addressing these questions may produce more robust and generalizable robotic capabilities in dynamic environments.

Appendix A

Additional class diagrams

In this appendix, the additional class diagrams of the implementation in Chapter 5 are further presented.

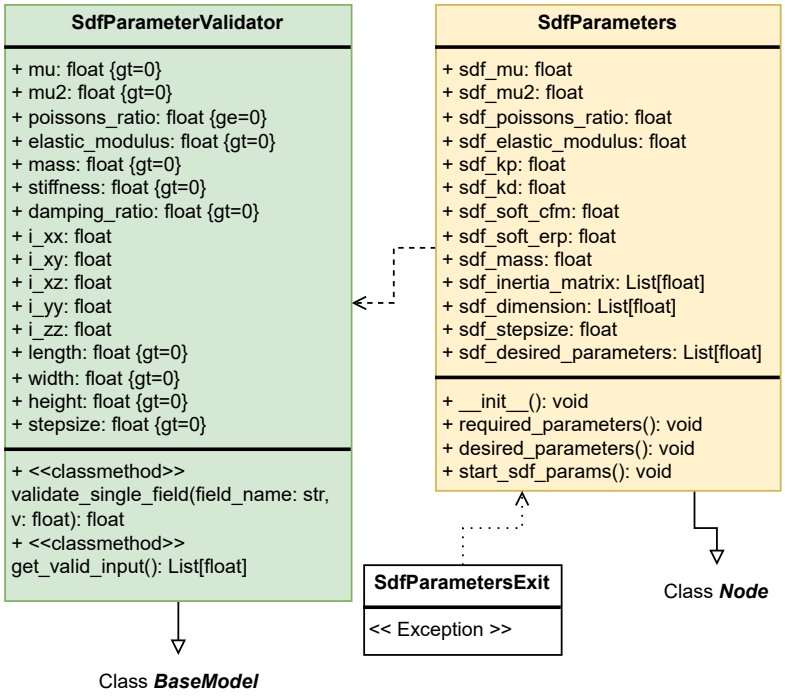


Figure A.1: Class diagram overview based on class SdfParameters

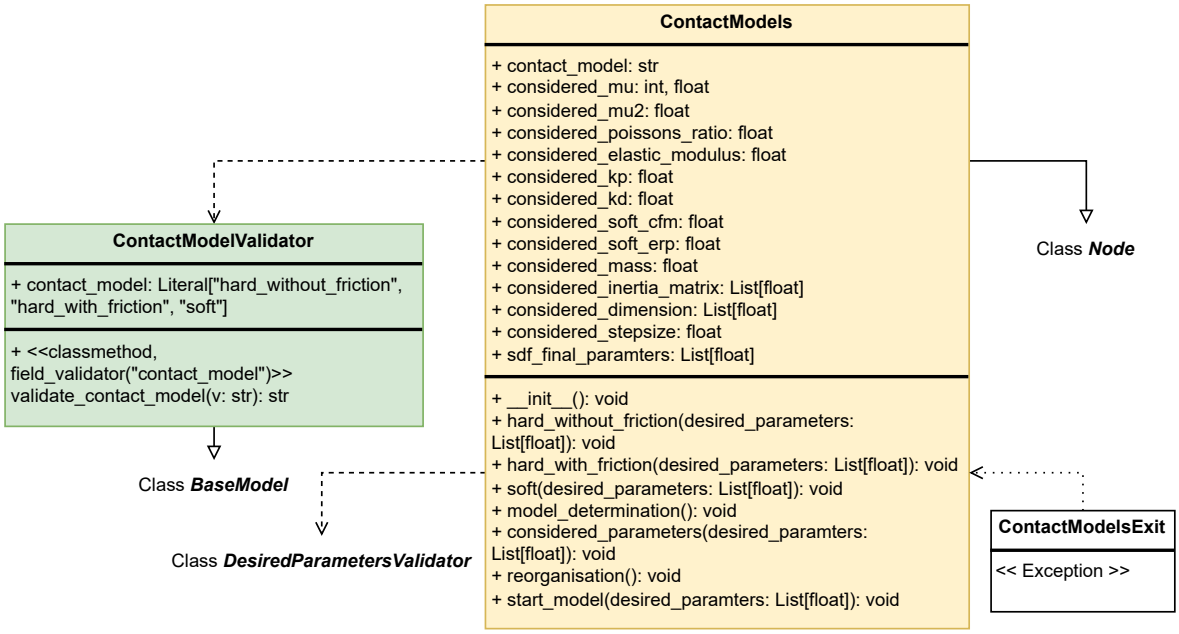


Figure A.2: Class diagram overview based on class ContactModels

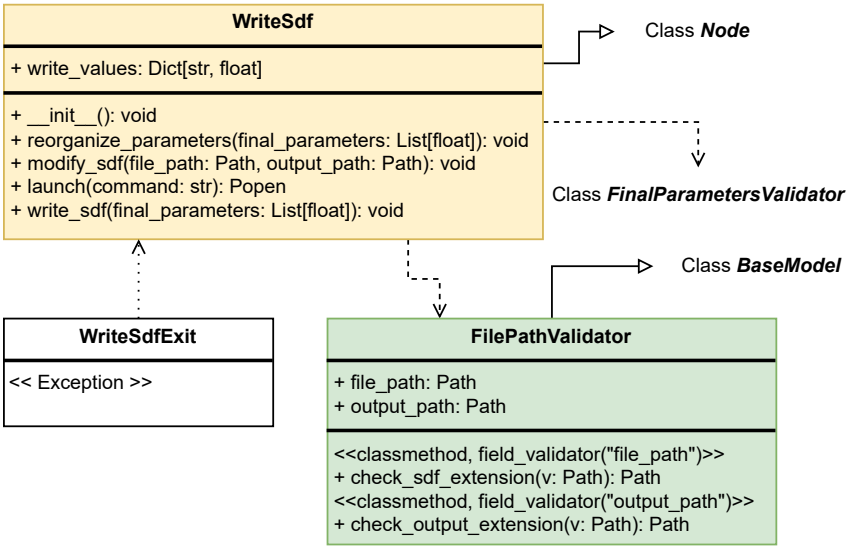


Figure A.3: Class diagram overview based on class WriteSdf

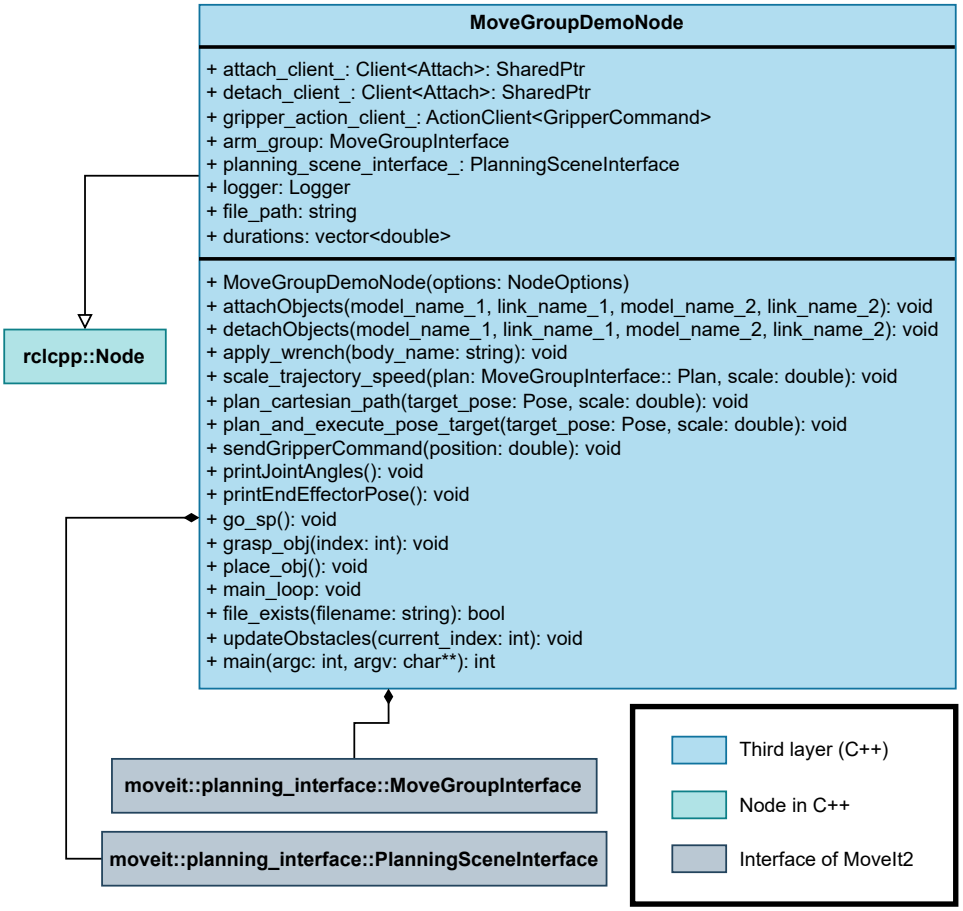


Figure A.4: Class diagram overview based on class MoveGroupDemoNode

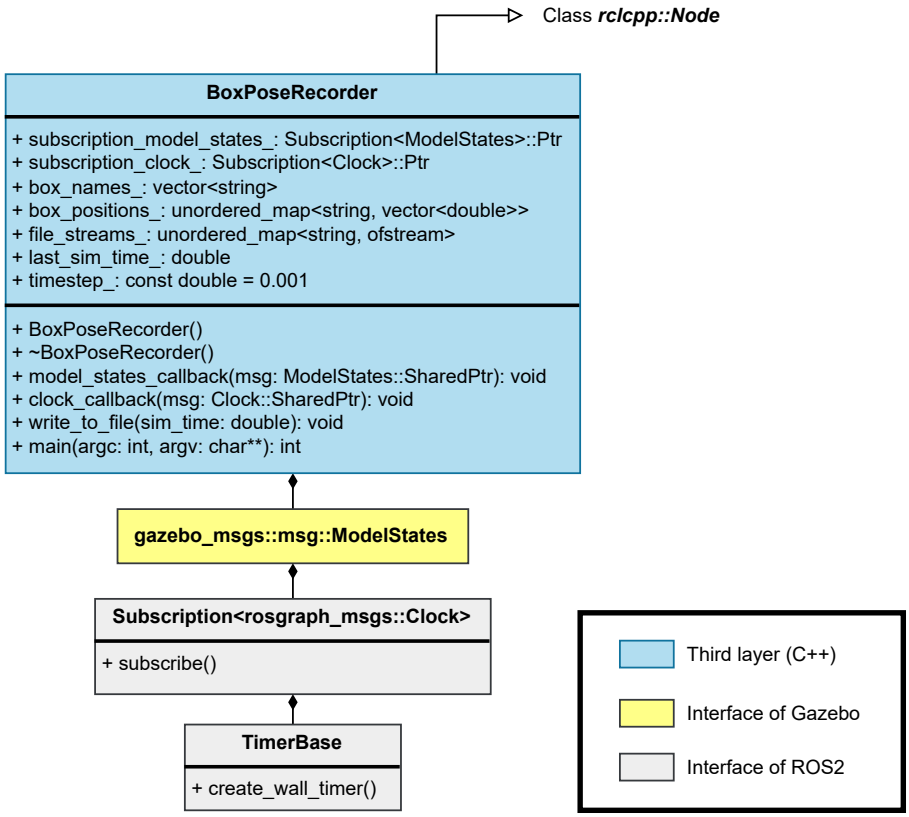


Figure A.5: Class diagram overview based on class BoxPoseRecorder

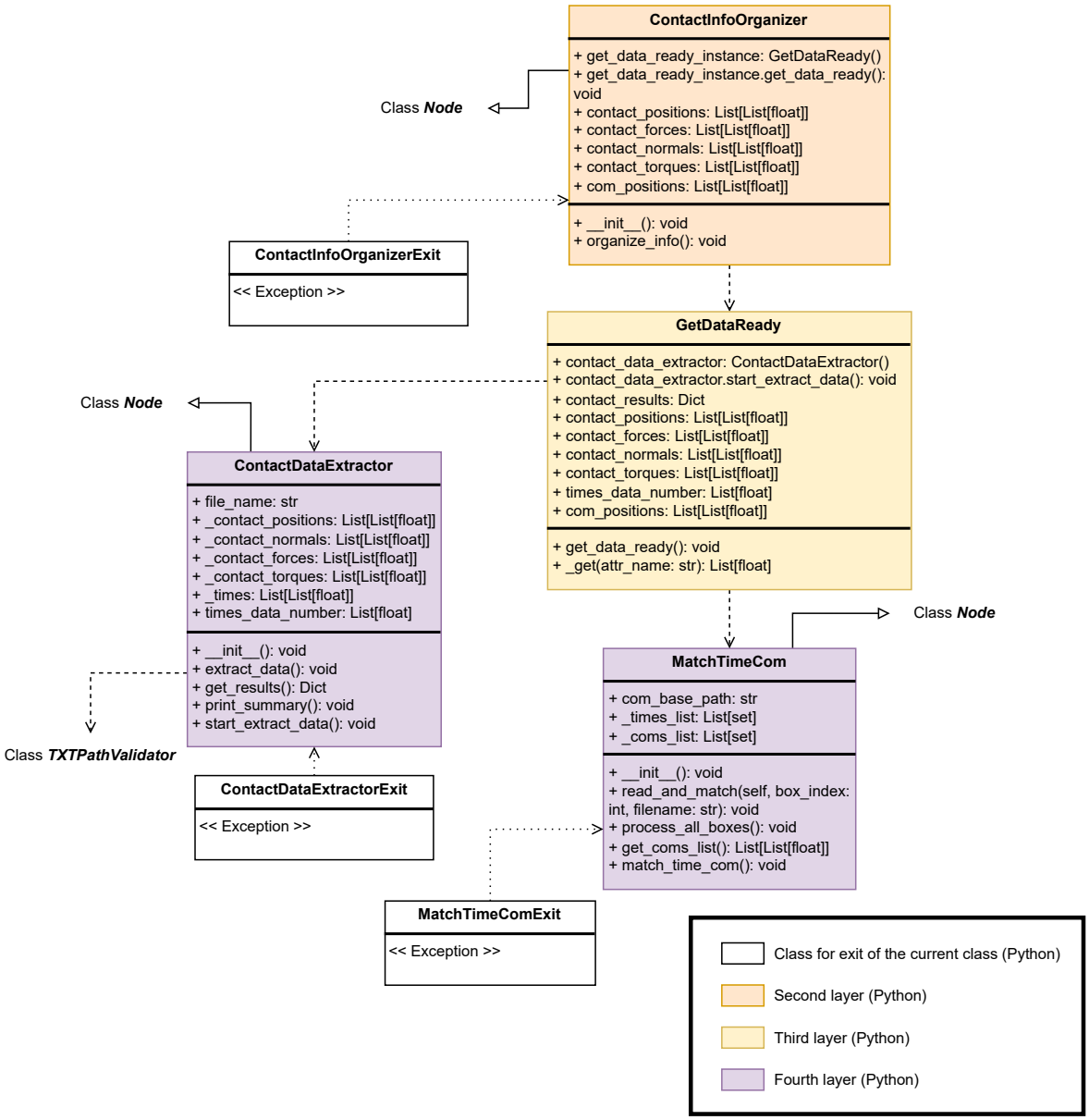


Figure A.6: Class diagram overview based on class ContactInfoOrganizer

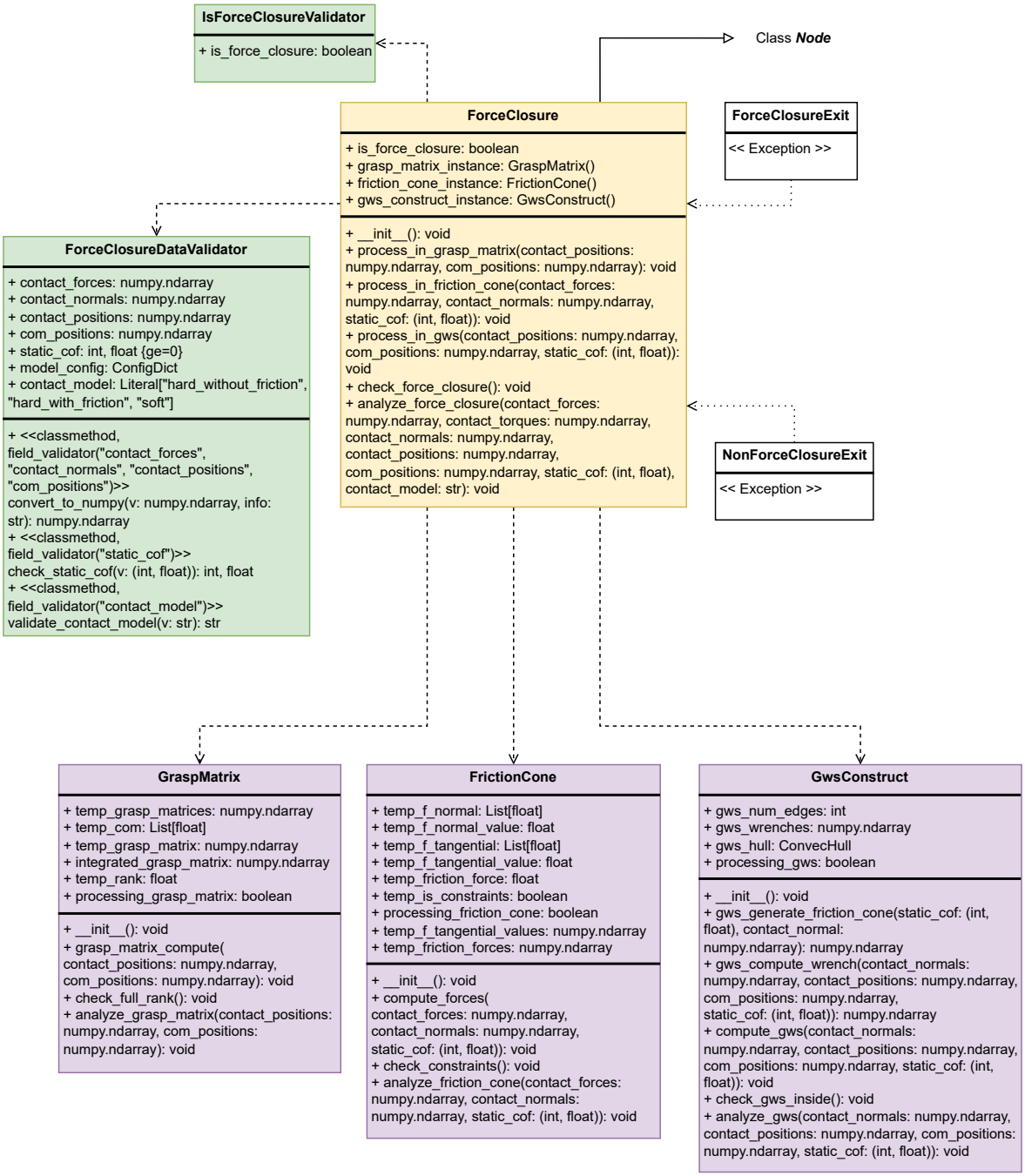


Figure A.7: Class diagram overview based on class ForceClosure

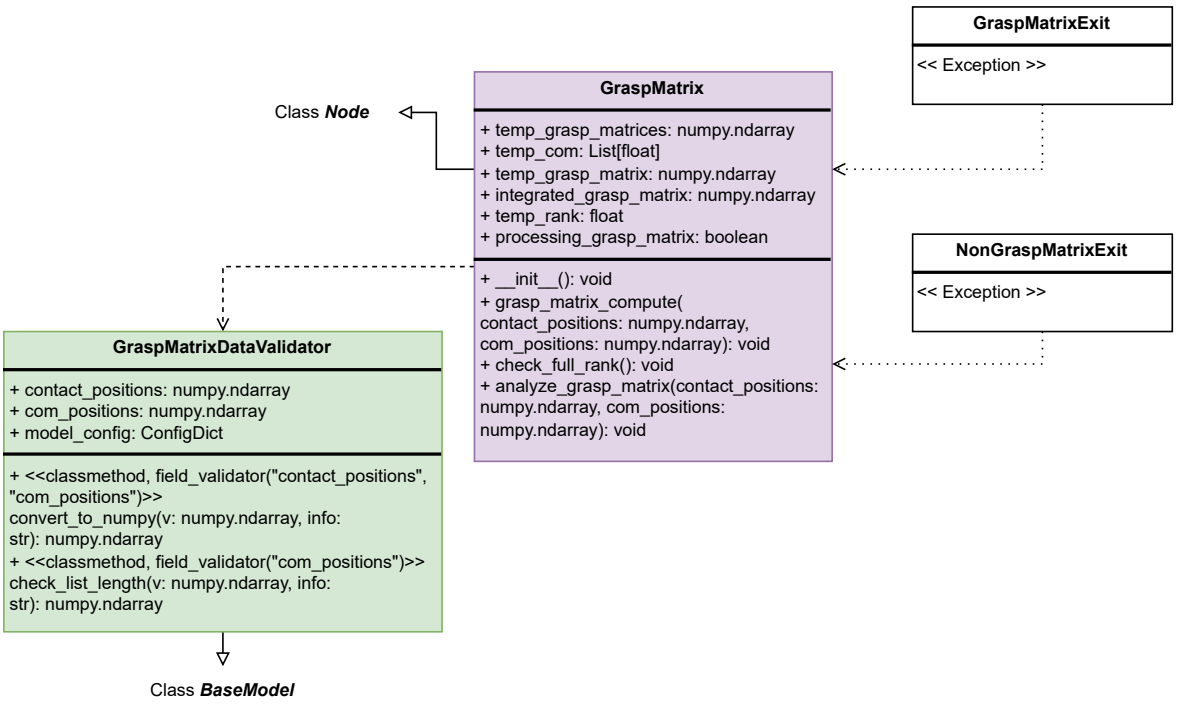


Figure A.8: Class diagram overview based on class GraspMatrix

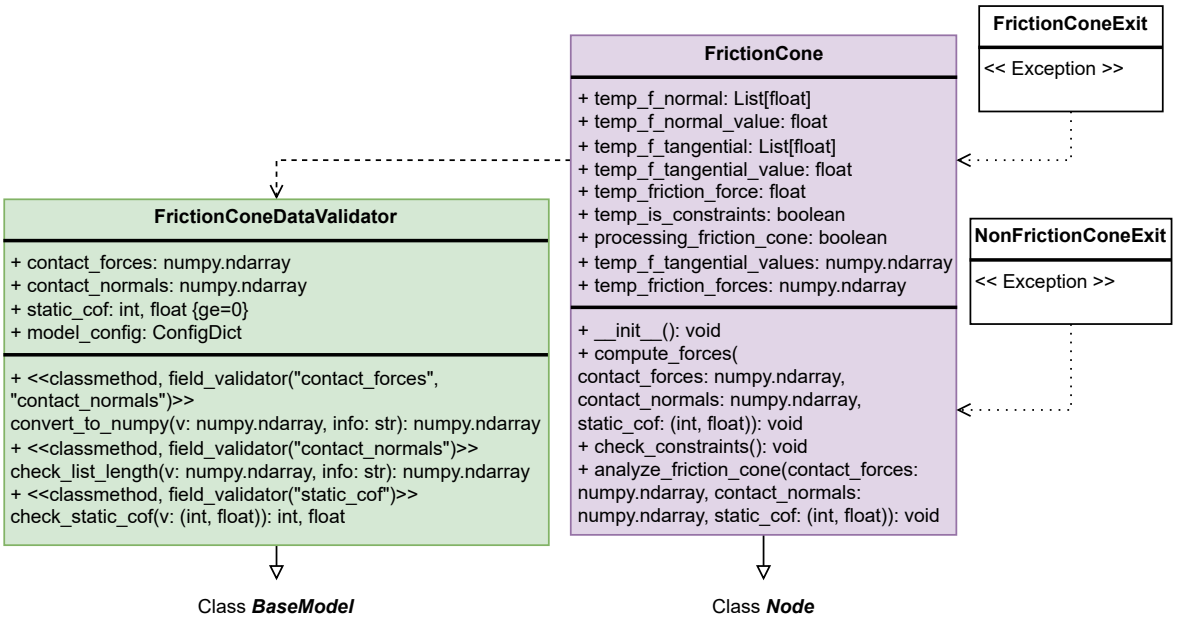


Figure A.9: Class diagram overview based on class FrictionCone

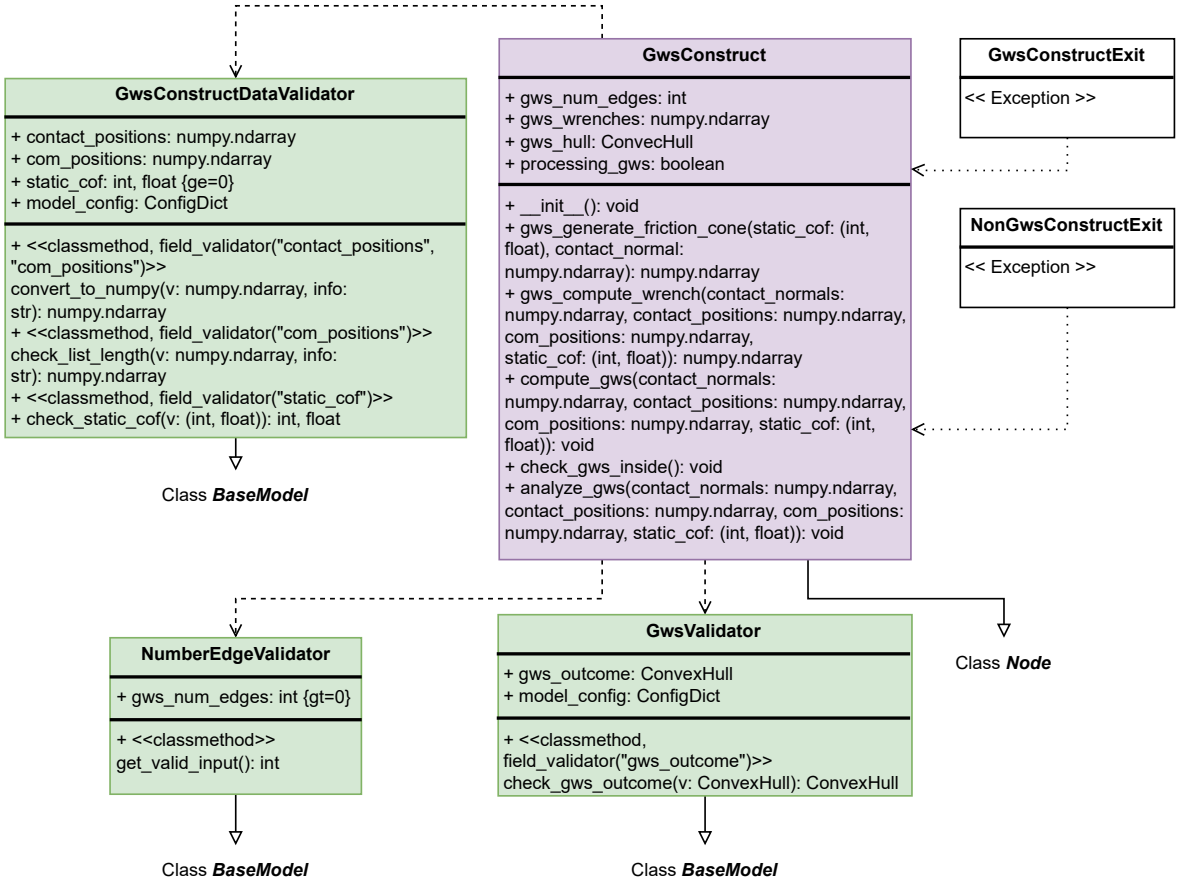


Figure A.10: Class diagram overview based on class GwsConstruct

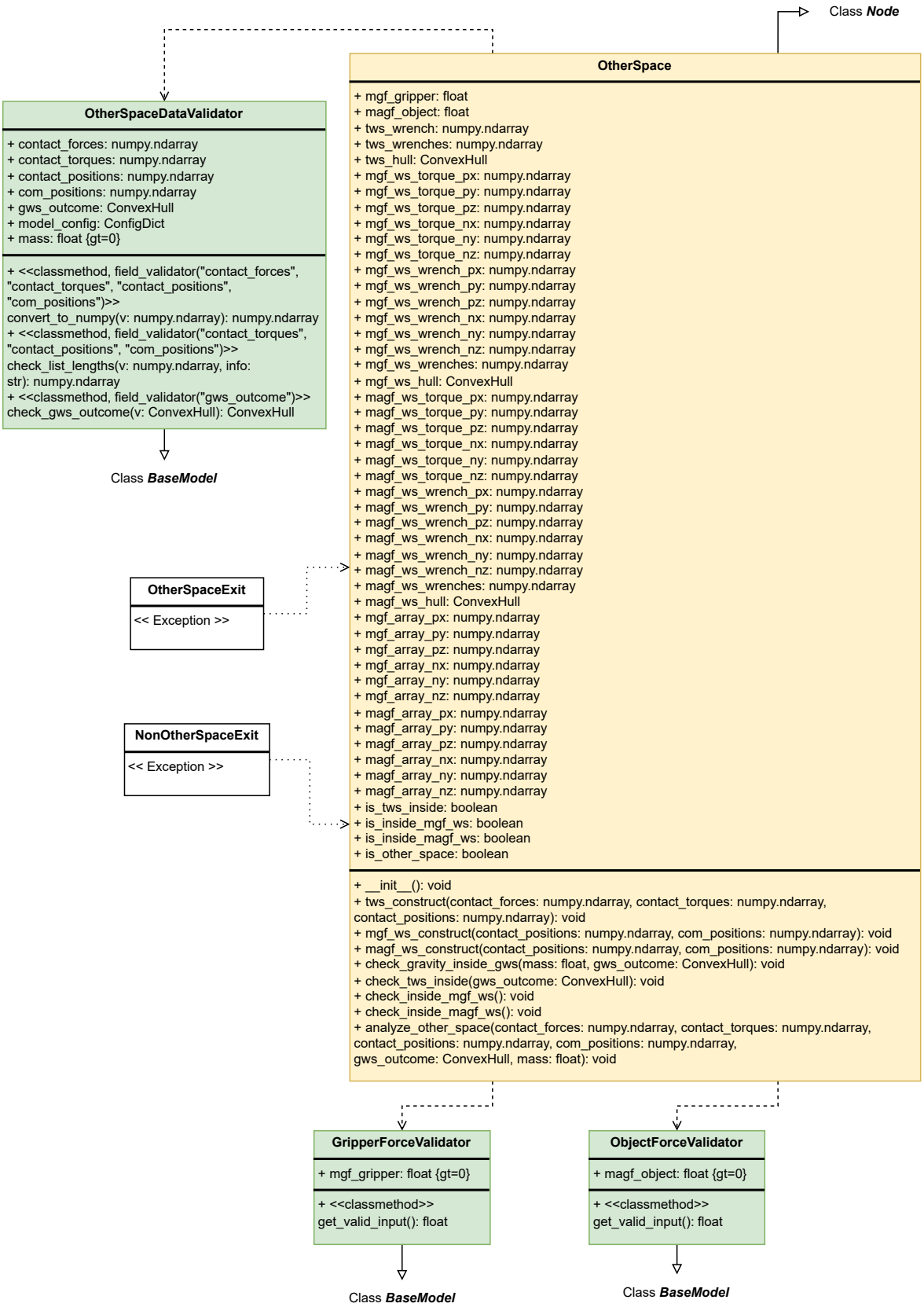


Figure A.11: Class diagram overview based on class OtherSpace

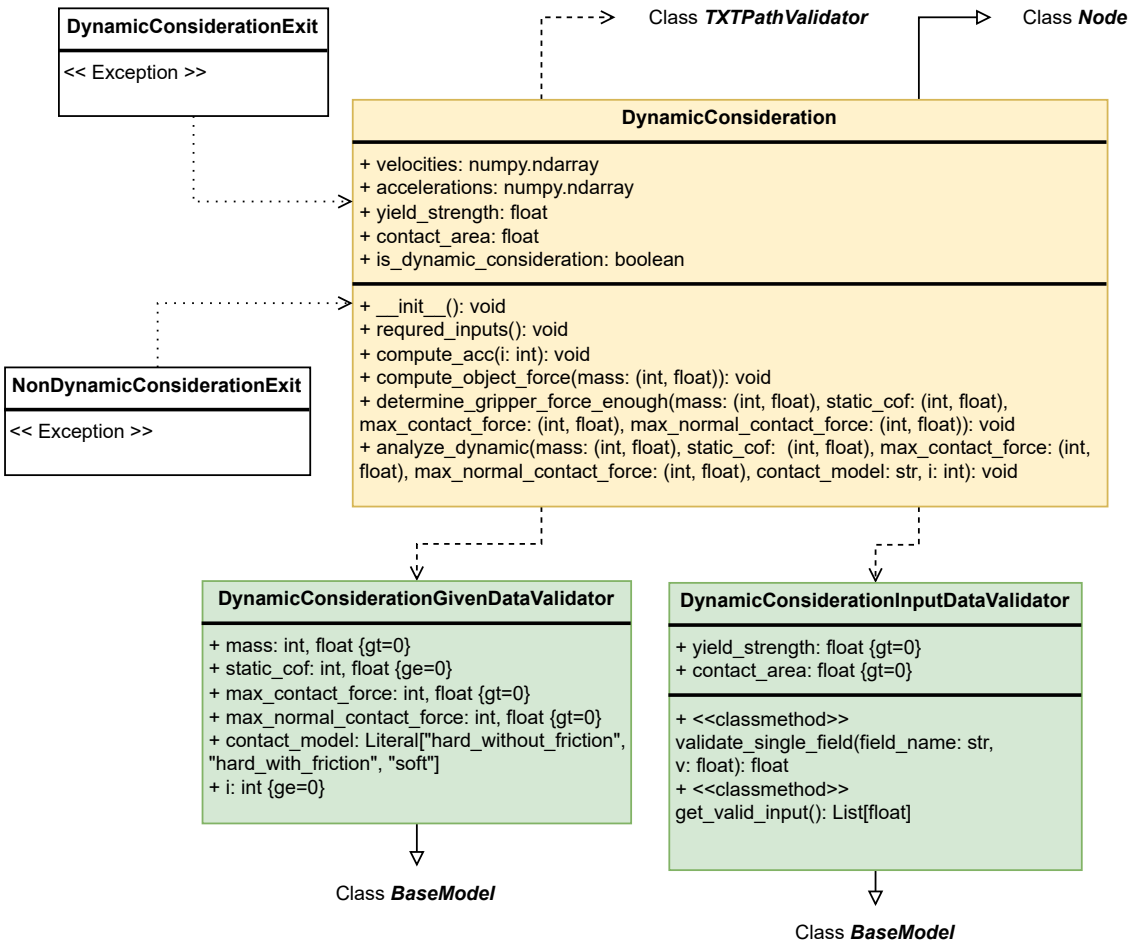


Figure A.12: Class diagram overview based on class DynamicConsideration

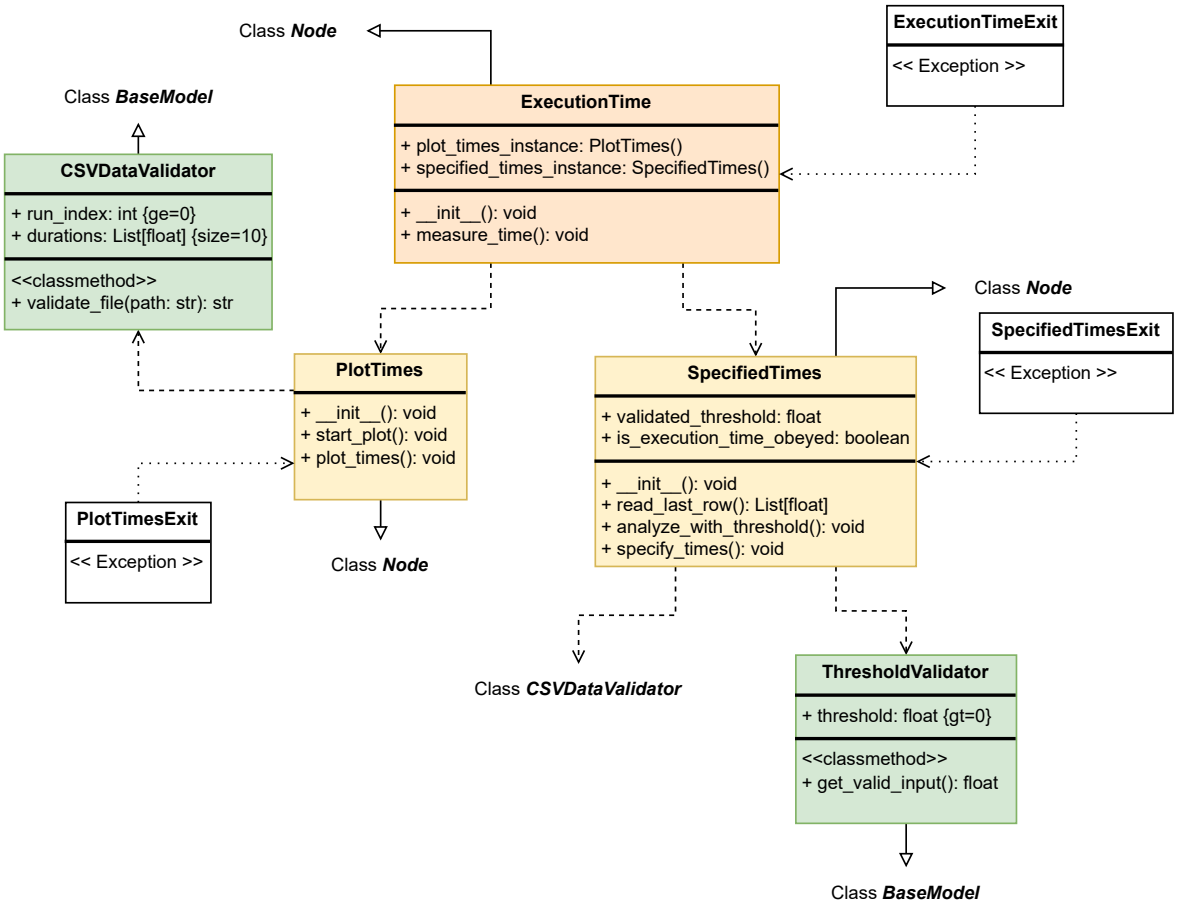


Figure A.13: Class diagram overview based on class ExecutionTime

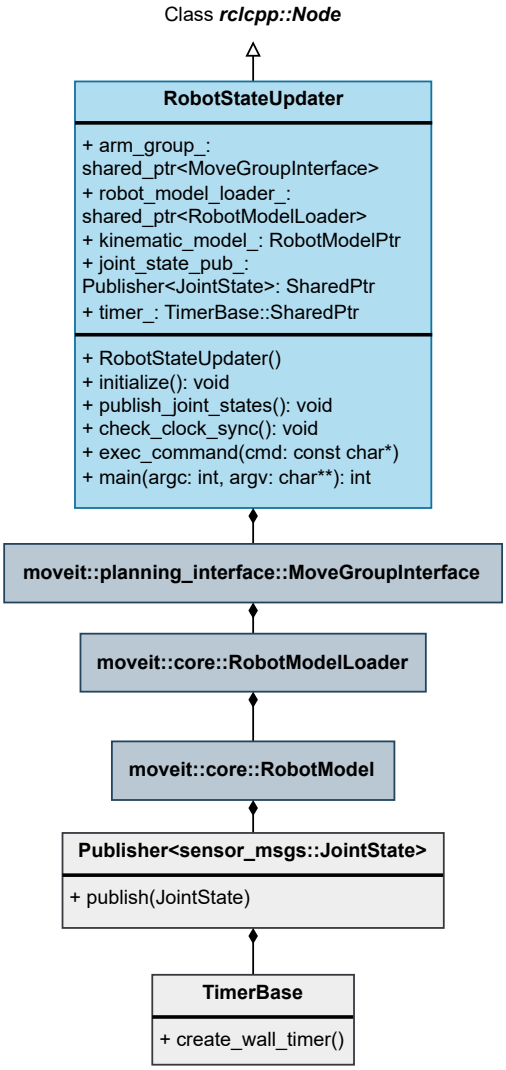


Figure A.14: Class diagram overview based on class RobotStateUpdater

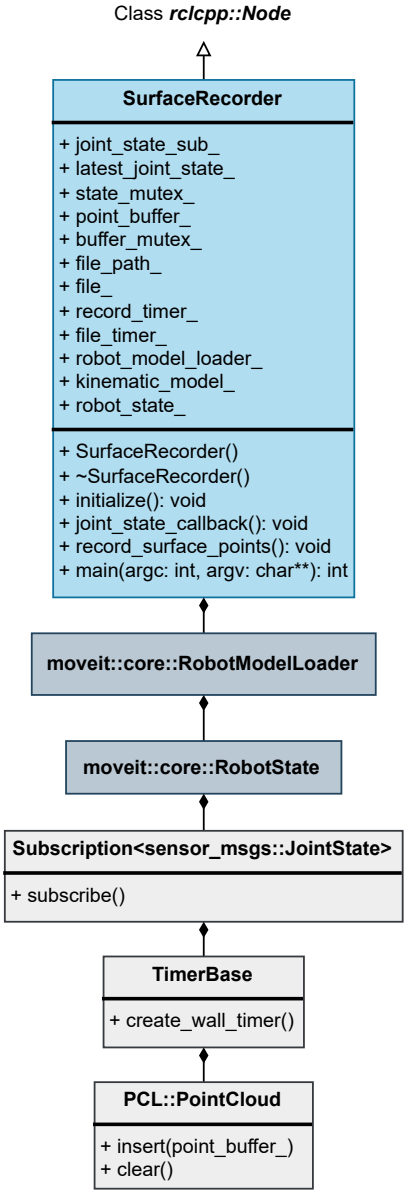


Figure A.15: Class diagram overview based on class SurfaceRecorder

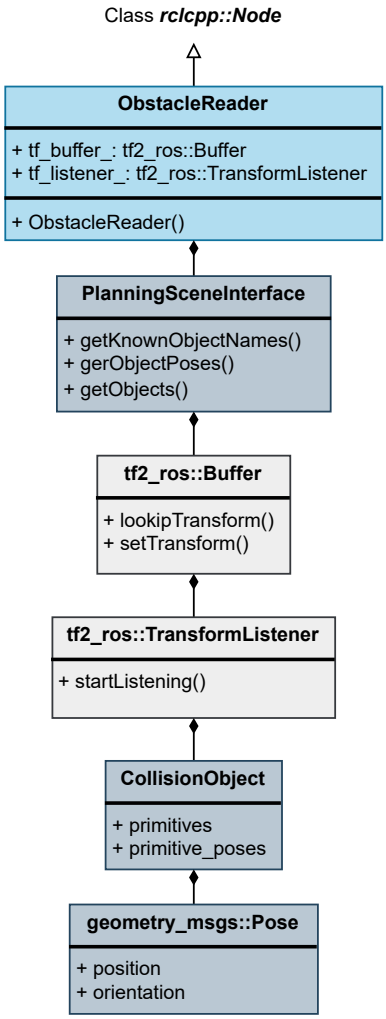


Figure A.16: Class diagram overview based on class ObstacleReader

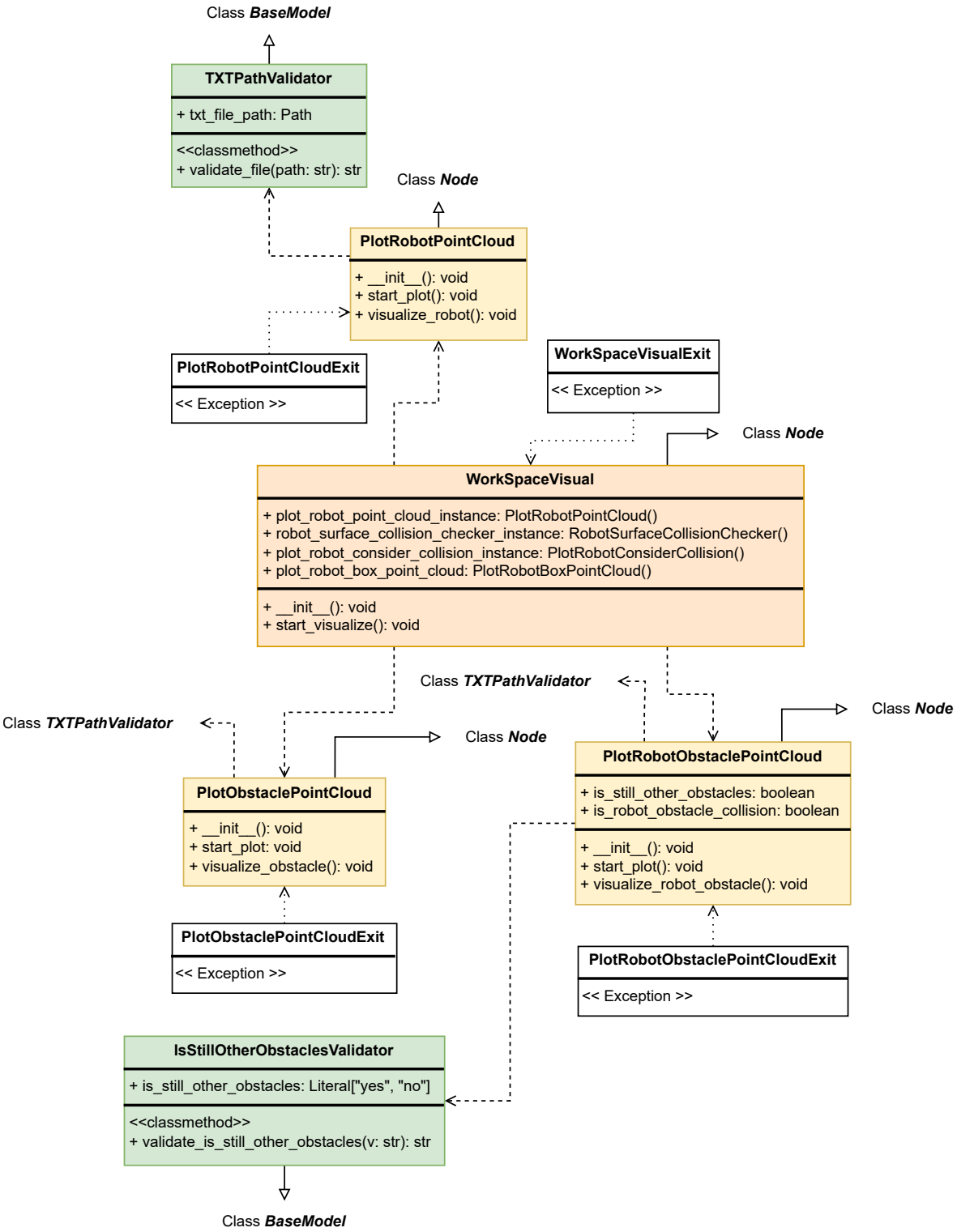


Figure A.17: Class diagram overview based on class WorkSpaceVisual

Appendix B

Additional sequence diagrams

In this appendix, the additional sequence diagrams of the implementation in Chapter 5 are further presented.

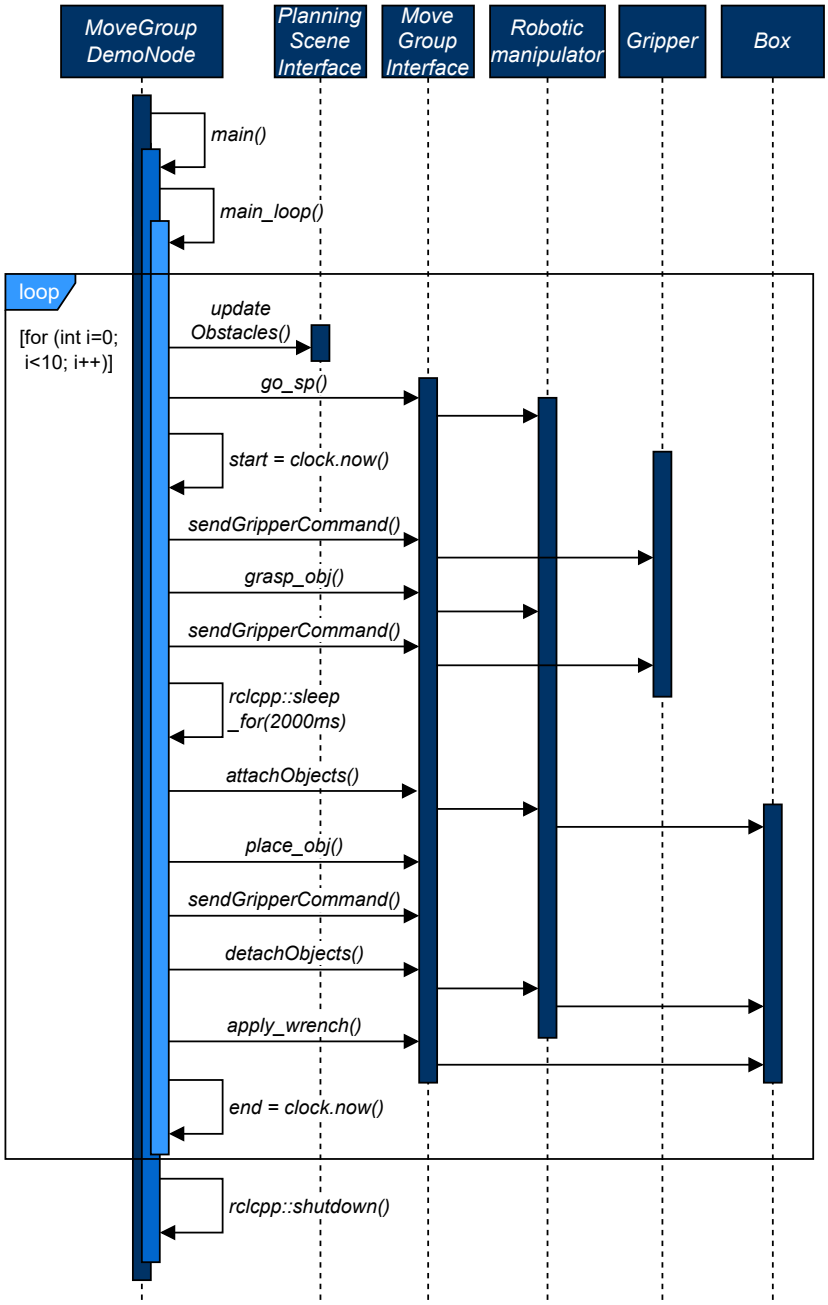


Figure B.1: Sequence diagram overview based on class MoveGroupDemoNode

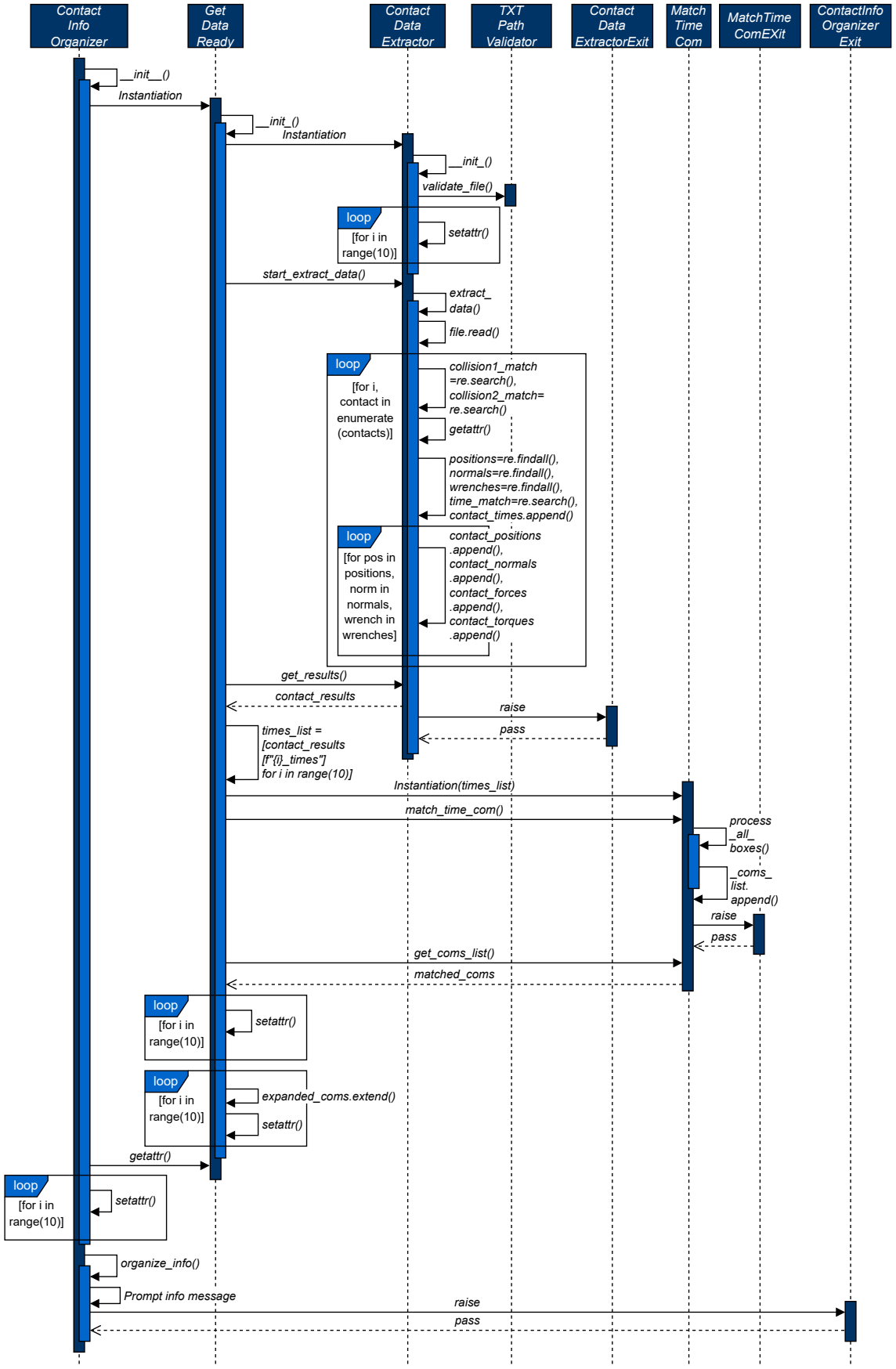


Figure B.2: Sequence diagram overview based on class ContactInfoOrganizer

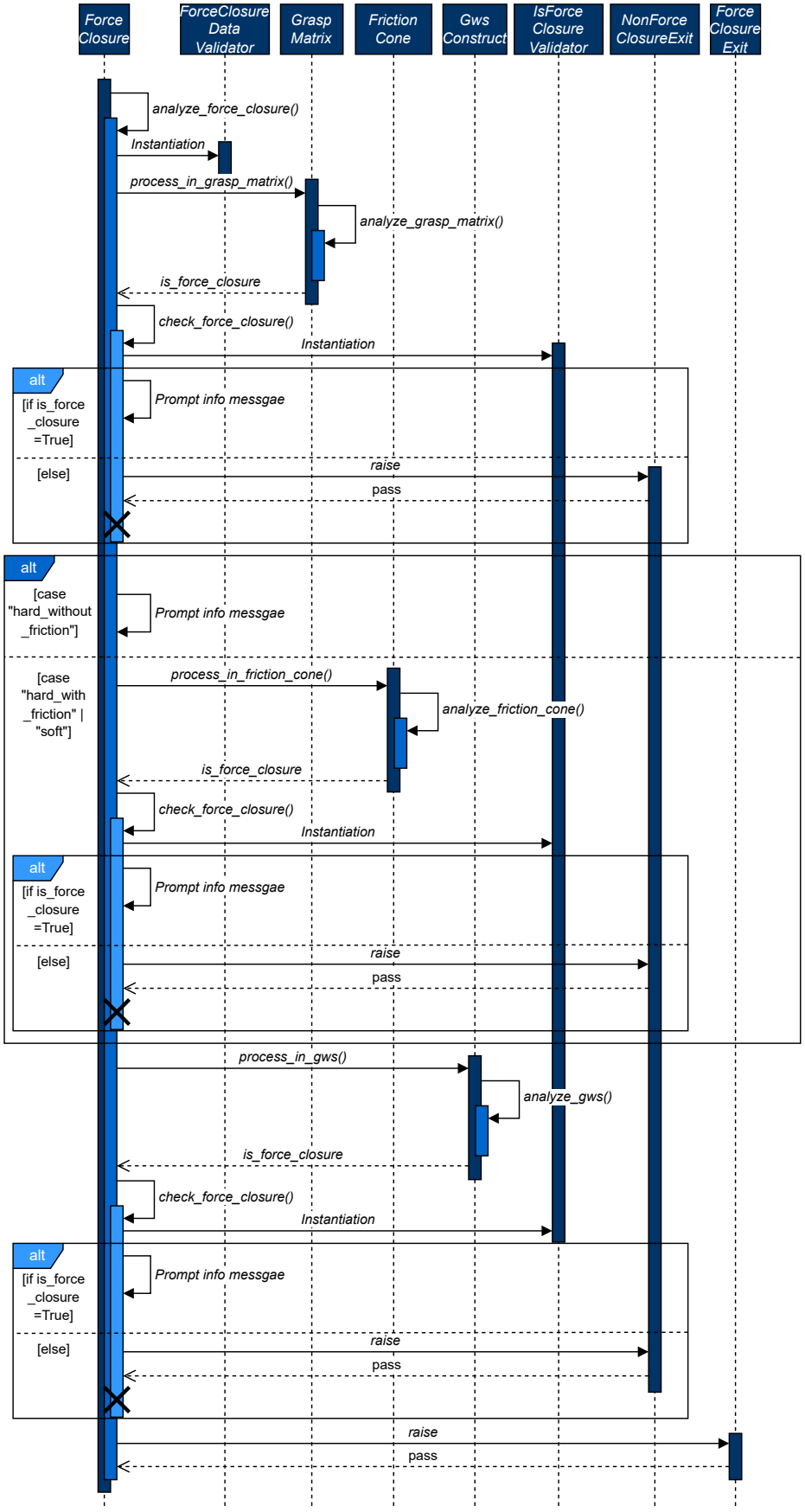


Figure B.3: Sequence diagram overview based on class ForceClosure

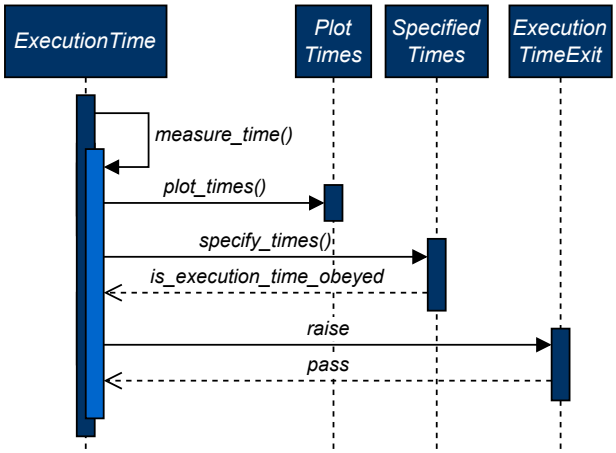


Figure B.4: Sequence diagram overview based on class ExecutionTime

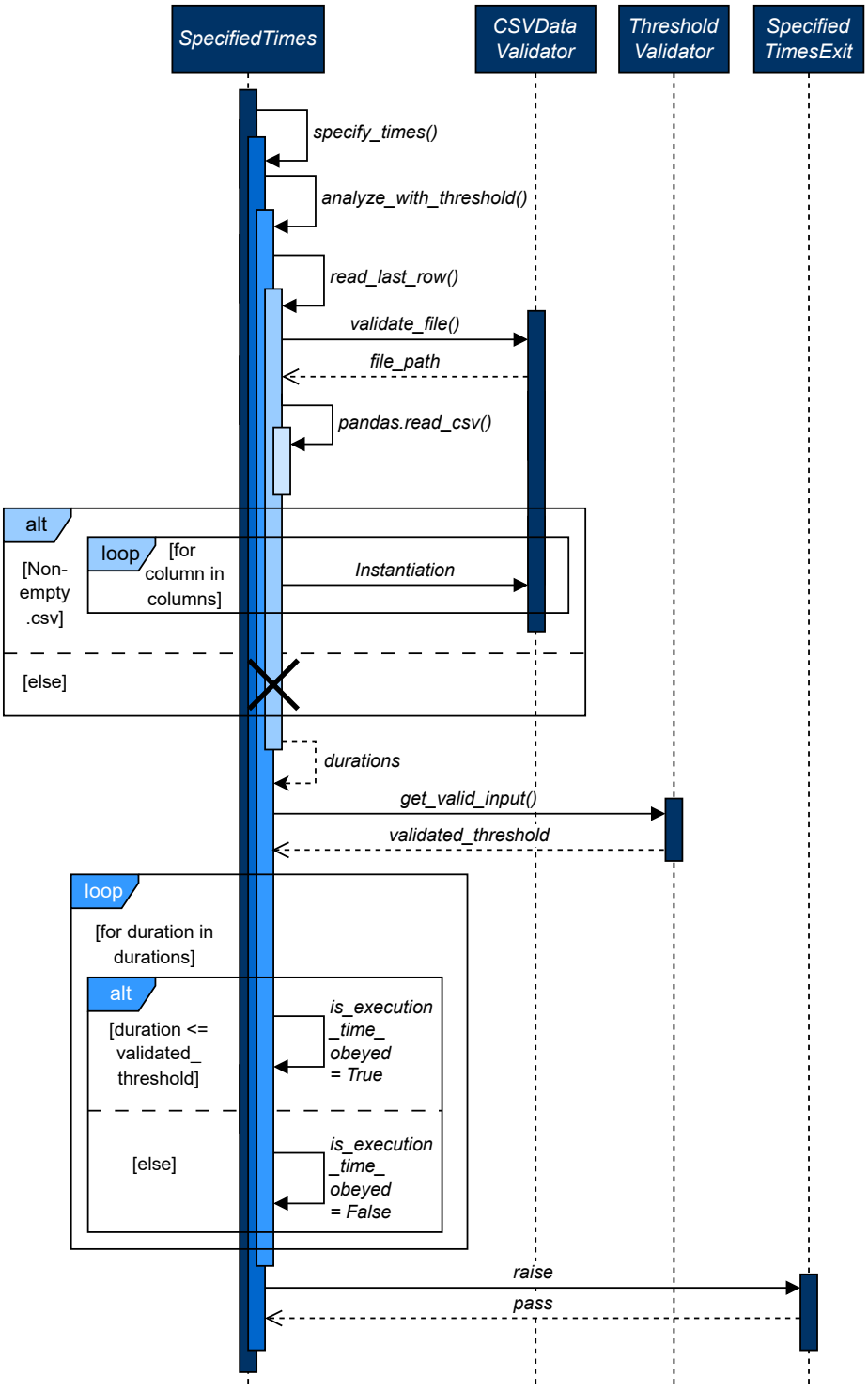


Figure B.5: Sequence diagram overview based on class SpecifiedTimes

Appendix C

Configuration of classes and files

This appendix lists the configuration of classes and their corresponding files defined in Chapter 5.

Table C.1: Configuration of classes and files based on class Main

Class	File name
Main	*main.py*
ParametersPreparation	*parameters_preparation.py*
LaunchSim	*launch_sim.py*
ContactInfoOrganizer	*contact_info_organizer.py*
StableGrasp	*stable_grasp.py*
ExecutionTime	*execution_time.py*
WorkSpaceVisual	*work_space_visual.py*
MainExit	*main.py*

Table C.2: Configuration of classes and files based on class ParametersPreparation

Class	File name
ParametersPreparation	*parameters_preparation.py*
SdfParameters	*sdf_parameters.py*
ContactModels	*contact_models.py*
WriteSdf	*write_sdf.py*
SdfParametersValidator	*sdf_parameters.py*
ContactModelValidator	*contact_models.py*
FilePathValidator	*write_sdf.py*
DesiredParametersValidator	*parameters_preparation.py* *contact_models.py*
FinalParametersValidator	*parameters_preparation.py* *write_sdf.py*
ReferredParametersValidator	*parameters_preparation.py*
SdfParametersExit	*sdf_parameters.py*
ContactModelsExit	*contact_models.py*
WriteSdfExit	*write_sdf.py*
ParametersPreparationExit	*parameters_preparation.py*

Table C.3: Configuration of classes and files based on class LaunchSim

Class	File name
LaunchSim	*launch_sim.py*
MoveGroupDemoNode	*obj_grasp.cpp*
RobotStateUpdater	*record_surface_points.cpp*
SurfaceRecorder	*offline_surface_processing.cpp*
BoxPoseRecorder	*record_box_com.cpp*
GazeboTopicLogger	*contact_logger.cpp*
ObstacleRecorder	*read_obstacles.cpp*
LaunchSimExit	*launch_sim.py*

Table C.4: Configuration of classes and files based on class ContactInfoOrganizer

Class	File name
ContactInfoOrganizer	*contact_info_organizer.py*
GetDataReady	*get_data_ready.py*
ContactDataExtractor	*contact_data_extractor.py*
MatchTimeCom	*match_time_com.py*
TXTPathValidator	*contact_data_extractor.py* *match_time_com.py*
ContactDataExtractorExit	*contact_data_extractor.py*
MatchTimeComExit	*match_time_com.py*
ContactInfoOrganizerExit	*contact_info_organizer.py*

Table C.5: Configuration of classes and files based on class StableGrasp

Class	File name
StableGrasp	*stable_grasp.py*
ForceClosure	*force_closure.py*
OtherSpace	*other_space.py*
DynamicConsideration	*dynamic_consideration.py*
GraspDataValidator	*stable_grasp.py*
GwsValidator	*stable_grasp.py*
IsStableGraspValidator	*stable_grasp.py*
StableGraspExit	*stable_grasp.py*
NonStableGraspExit	*stable_grasp.py*

Table C.6: Configuration of classes and files based on class ForceClosure

Class	File name
ForceClosure	*force_closure.py*
GraspMatrix	*grasp_matrix.py*
FrictionCone	*friction_cone.py*
GwsConstruct	*gws.py*
GraspMatrixDataValidator	*grasp_matrix.py*
FrictionConeDataValidator	*friction_cone.py*
GwsConstructDataValidator	*gws.py*
NumberEdgeValidator	*gws.py*
GwsValidator	*gws.py*
ForceClosureDataValidator	*force_closure.py*
IsForceClosureValidator	*force_closure.py*
GraspMatrixExit	*grasp_matrix.py*
NonGraspMatrixExit	*grasp_matrix.py*
FrictionConeExit	*friction_cone.py*
NonFrictionConeExit	*friction_cone.py*
GwsConstructExit	*gws.py*
NonGwsConstructExit	*gws.py*
ForceClosureExit	*force_closure.py*
NonForceClosureExit	*force_closure.py*

Table C.7: Configuration of classes and files based on class OtherSpace

Class	File name
OtherSpace	*other_grasp.py*
OtherSpaceDataValidator	*other_space.py*
GripperForceValidator	*other_space.py*
ObjectForceValidator	*other_space.py*
OtherSpaceExit	*other_space.py*
NonOtherSpaceExit	*other_space.py*

Table C.8: Configuration of classes and files based on class DynamicConsideration

Class	File name
DynamicConsideration	*dynamic_consideration.py*
DynamicConsiderationGivenDataValidator	*dynamic_consideration.py*
DynamicConsiderationInputDataValidator	*dynamic_consideration.py*
TXTPathValidator	*dynamic_consideration.py*
DynamicConsiderationExit	*dynamic_consideration.py*
NonDynamicConsiderationExit	*dynamic_consideration.py*

Table C.9: Configuration of classes and files based on class ExecutionTime

Class	File name
ExecutionTime	*execution_time.py*
PlotTimes	*plot_times.py*
SpecifiedTimes	*specified_times.py*
CSVDataValidator	*plot_times.py* *specified_times.py*
ThresholdValidator	*specified_times.py*
PlotTimesExit	*plot_times.py*
SpecifiedTimesExit	*specified_times.py*
ExecutionTimeExit	*execution_time.py*

Table C.10: Configuration of classes and files based on class WorkSpaceVisual

Class	File name
WorkSpaceVisual	*work_space_visual.py*
PlotRobotPointCloud	*plot_robot_point_cloud.py*
PlotObstaclePointCloud	*plot_obstacle_point_cloud.py*
PlotRobotObstaclePointCloud	*plot_robot_obstacle_point_cloud.py*
TXTPathValidator	*plot_robot_point_cloud.py* *plot_obstacle_point_cloud.py* *plot_robot_obstacle_point_cloud.py*
ThresholdValidator	*specified_times.py*
PlotRobotPointCloudExit	*plot_robot_point_cloud.py*
PlotObstaclePointCloudExit	*plot_obstacle_point_cloud.py*
PlotRobotObstaclePointCloudExit	*plot_robot_obstacle_point_cloud.py*
WorkSpaceVisualExit	*work_space_visual.py*

Appendix D

Additional simulation-related figures

This appendix presents the additional simulation-related figures of the implementation in Chapter 5 and evaluation in Chapter 6.

arm.xacro

```
<robot name="ur30">
    <link name="base_link">
        <inertial>...</inertial>
        <visual>
            ...
            <geometry>
                <mesh filename="file://$(find arm_description)/meshes/base_link.STL"
            </geometry>
            ...
        </visual>
        <collision>
            ...
            <geometry>
                <mesh filename="file://$(find arm_description)/meshes/base_link.STL"
            </geometry>
        </collision>
    </link>
    ...
    <link name="link6">
        <inertial>...</inertial>
        <visual>
            ...
            <geometry>
                <mesh filename="file://$(find arm_description)/meshes/link6.STL"
            </geometry>
            ...
        </visual>
        <collision>
            ...
            <geometry>
                <mesh filename="file://$(find arm_description)/meshes/link6.STL"
            </geometry>
        </collision>
    </link>
    <xacro:include filename="$(find robotiq_description)/urdf/robotiq_2f_85_macro.urdf.xacro" />
    <xacro:robotiq_gripper name="RobotiqGripperHardwareInterface" prefix="" parent="link6">
    <origin xyz="0 0 0" rpy="0 0 0" />
    </xacro:robotiq_gripper>
    ...
</robot>
```

Figure D.1: Example of the robotic manipulator's URDF file

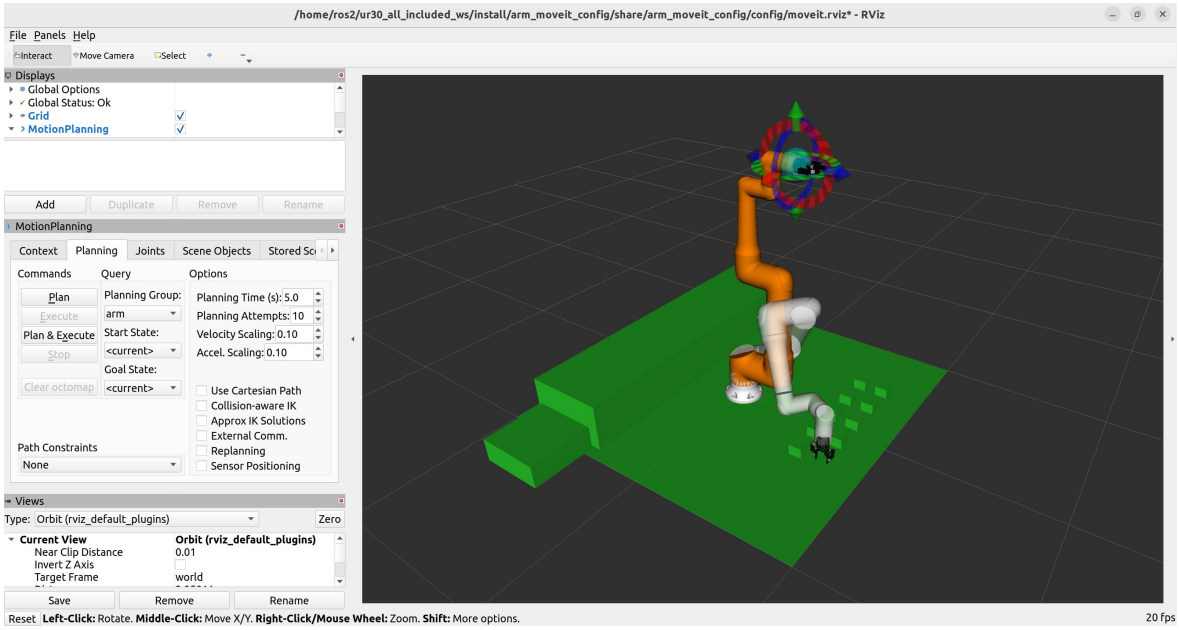


Figure D.2: End of the scenario's step 1 of loop 0 in RViz

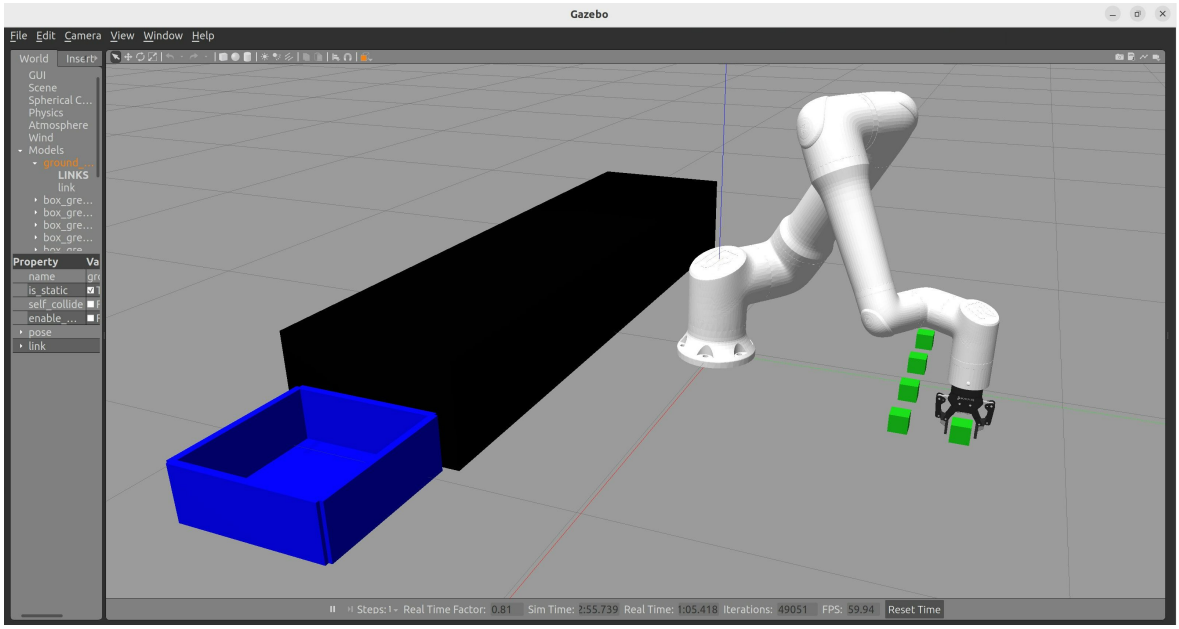


Figure D.3: End of the scenario's step 1 of loop 0 in Gazebo

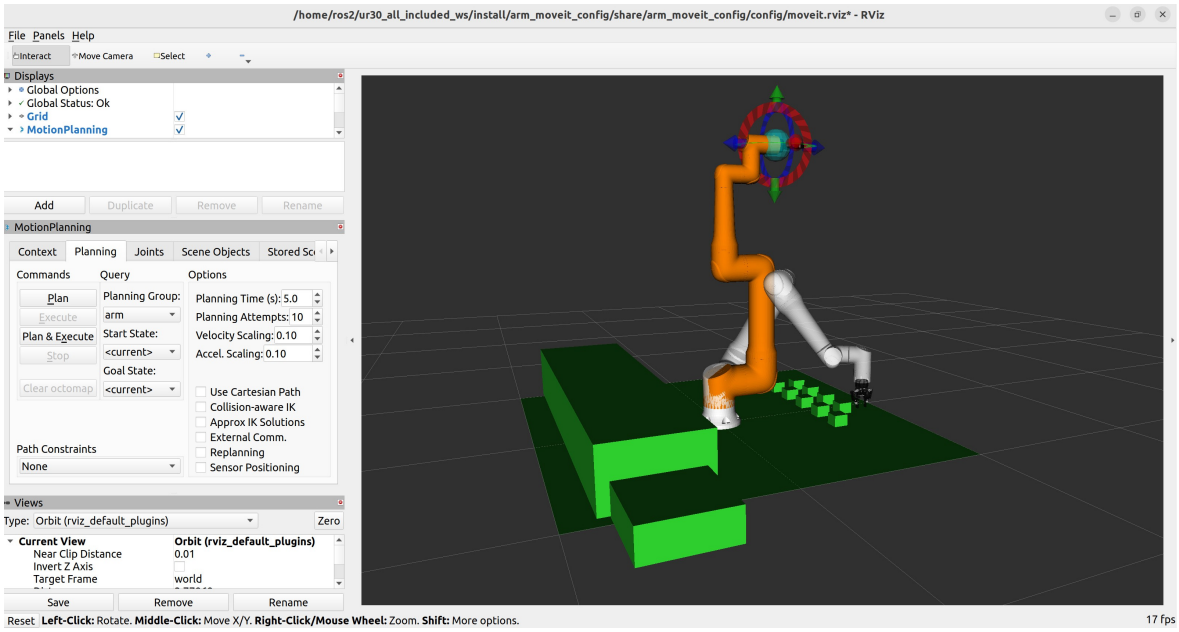


Figure D.4: End of the scenario's step 2 of loop 0 in RViz

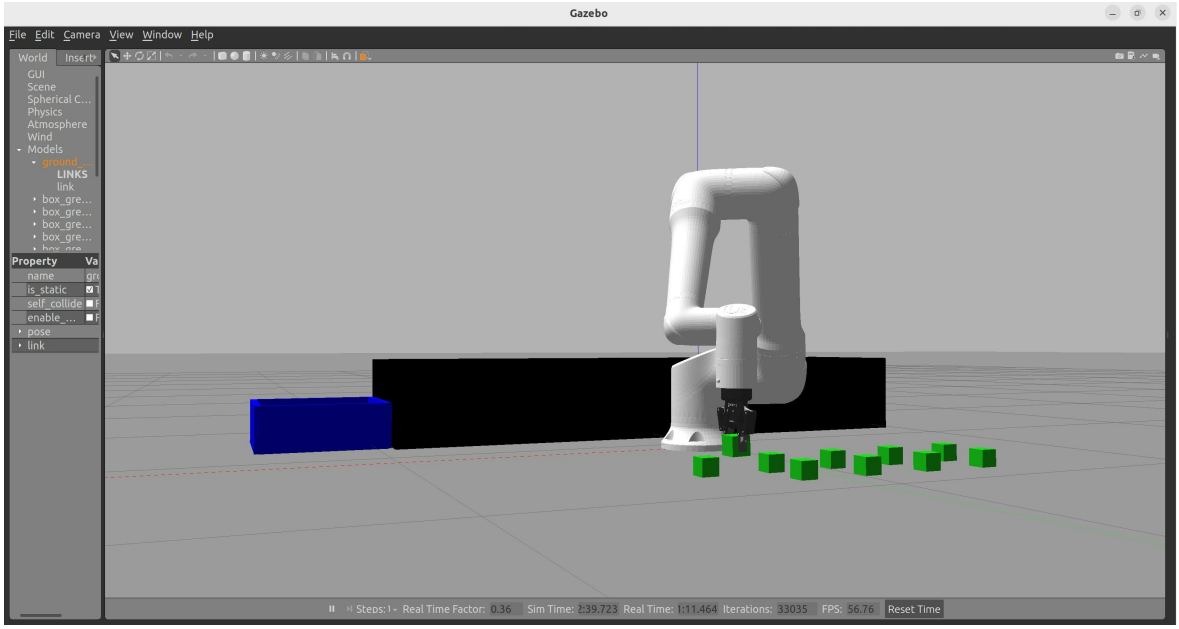


Figure D.5: End of the scenario's step 2 of loop 0 in Gazebo

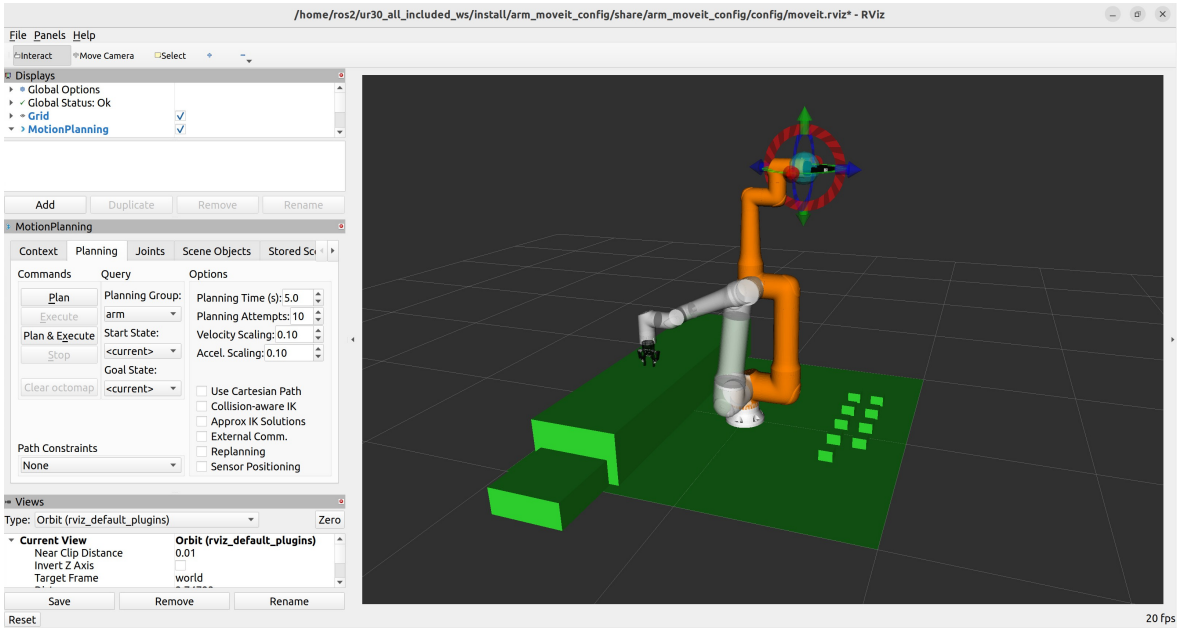


Figure D.6: End of the scenario's step 3 of loop 0 in RViz

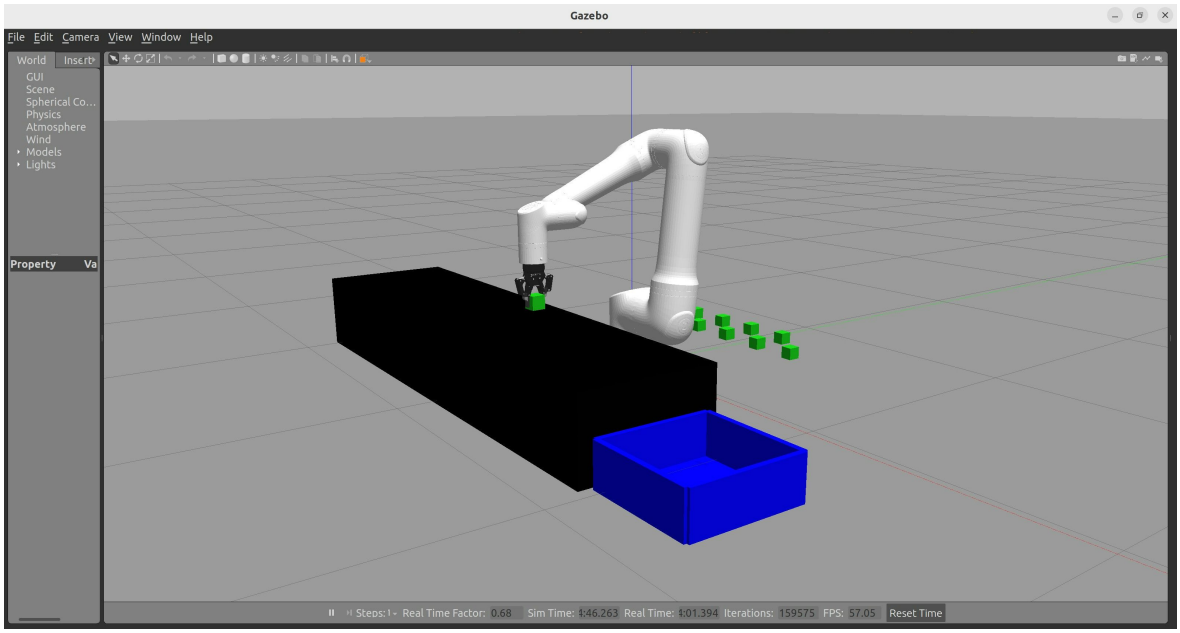


Figure D.7: End of the scenario's step 3 of loop 0 in Gazebo

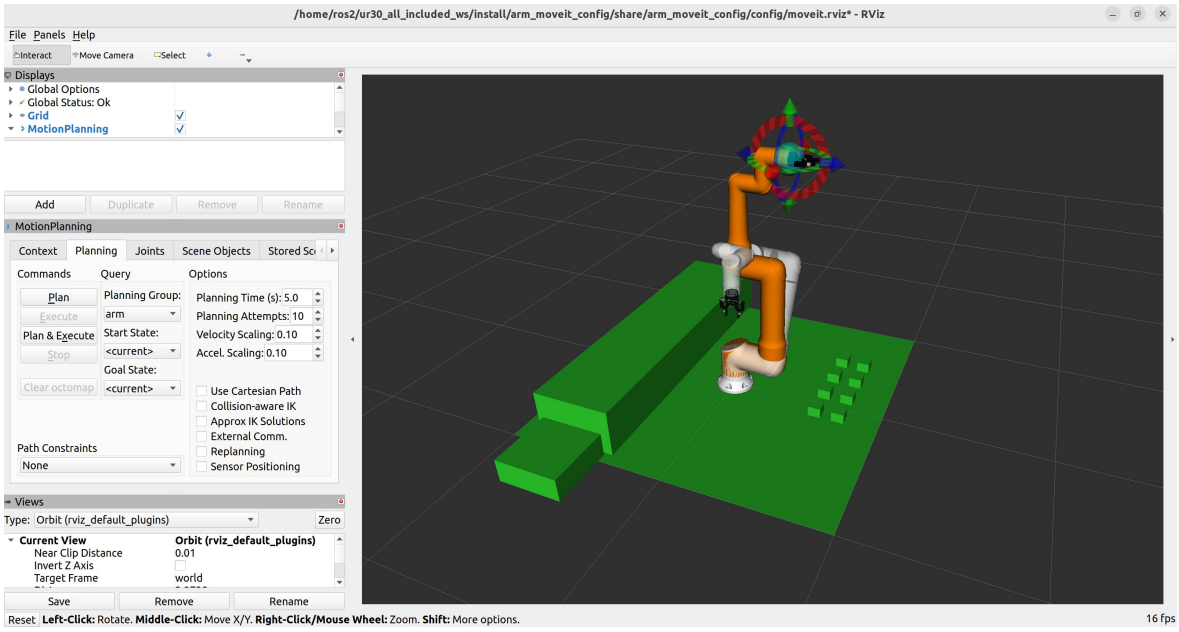


Figure D.8: End of the scenario's step 4 of loop 0 in RViz

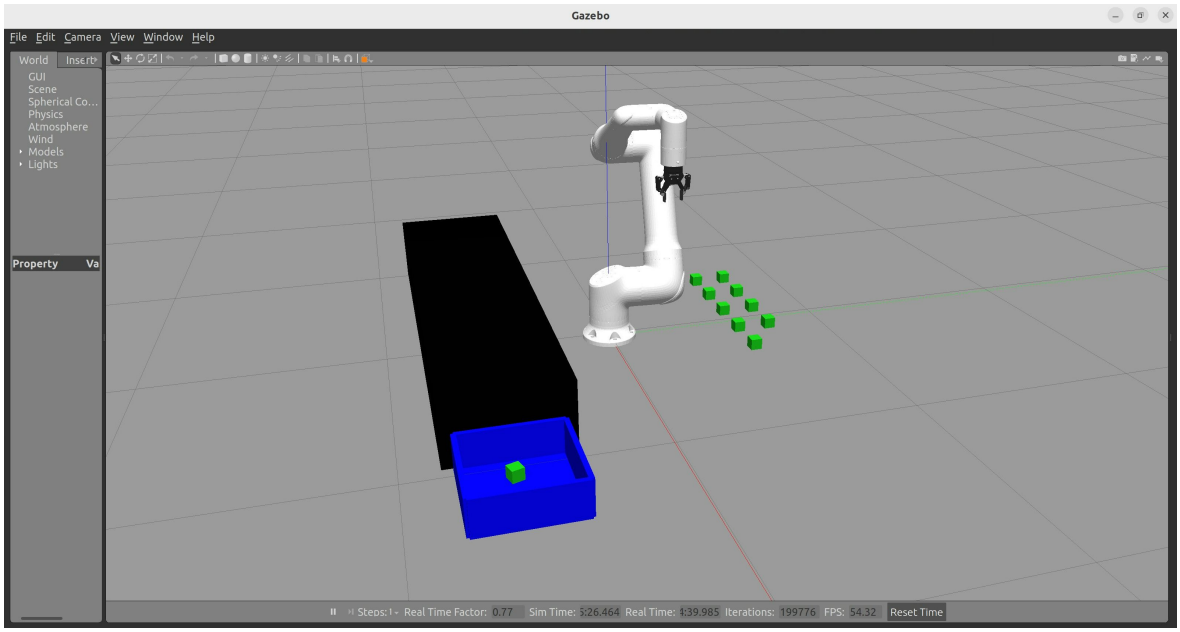


Figure D.9: End of the scenario's step 4 of loop 0 in Gazebo

```
contact {
    collision1: "box_green_0::green_link::collision"
    collision2:
"simple_arm::robotiq_85_left_finger_tip_link::robotiq_85_left_finger_tip_link_collision"
    position {
        x: 0.40158026059971846
        y: 0.17970640995283221
        z: 0.79104656185969036
    }
    normal {
        x: -0.16336895770525534
        y: 0.0027181148738779641
        z: 0.98656129840463069
    }
    depth: -0
    wrench {
        body_1_name: "box_green_0::green_link::collision"
        body_1_id: 54
        body_2_name:
"simple_arm::robotiq_85_left_finger_tip_link::robotiq_85_left_finger_tip_link_collision"
        body_2_id: 154
        body_1_wrench {
            force {
                x: 40.6465829628765
                y: 340.71251250148583
                z: 685.95615682064806
            }
            torque {
                x: 25.666716733053349
                y: 3.2656558392247397
                z: -3.1429328472166547
            }
        }
        body_2_wrench {
            force {
                x: -305.31759234548372
                y: -32.795696530082424
                z: 702.83612603170184
            }
            torque {
                x: -2.3449043042052229
                y: -9.6716094234723773
                z: -1.4699410940688986
            }
        }
    }
    time {
        sec: 755
        nsec: 929000000
    }
    world: default"
}
```

Figure D.10: Example of the log file from Gazebo contact topic

raw_surface_points.txt

```
0.109763, 0.00275, 0.017
0.10775, 0.00476314, 0.017
0.100237, -0.00275, 0.017
0.105, 0.0055, 0.017
0.10225, -0.00476314, 0.017
0.105, -0.0055, 0.017
0.10225, 0.00476314, 0.017
0.100237, 0.00275, 0.017
0.0995, 4.05911e-16, 0.017
0.1105, 3.98972e-16, 0.017
0.109763, -0.00275, 0.017
0.10775, -0.00476314, 0.017
0.0477369, 0.0881827, 0.017
0.04975, 0.0861695, 0.017
0.047, 0.0909327, 0.017
0.0525, 0.0854327, 0.017
0.05525, 0.0861695, 0.017
0.0572631, 0.0881827, 0.017
0.058, 0.0909327, 0.017
0.0572631, 0.0936827, 0.017
0.0477369, 0.0936827, 0.017
0.04975, 0.0956958, 0.017
0.0525, 0.0964327, 0.017
0.05525, 0.0956958, 0.017
-0.0525, 0.0964327, 0.017
-0.0525, 0.0854327, 0.017
-0.04975, 0.0861695, 0.017
-0.05525, 0.0956958, 0.017
-0.0572631, 0.0936827, 0.017
-0.058, 0.0909327, 0.017
...
```

Figure D.11: Example of the file created from class SurfaceRecorder

sim.world

```
<model name="box_green_0">
    <link name="green_link">
        <pose>0 0 0 0 -0 0</pose>
        <inertial>
            <mass>1.0</mass>
            <inertia>
                <ixx>0.00041666667</ixx>
                    <ixy>0</ixy>
                <ixz>0</ixz>
                <iyy>0.00041666667</iyy>
                <iyz>0</iyz>
                <izz>0.00041666667</izz>
            </inertia>
            <pose>0 0 0 0 -0 0</pose>
        </inertial>
        <collision name="collision">
            <geometry>
                <box>
                    <size>0.05 0.05 0.05</size>
                </box>
            </geometry>
            <surface>
                <friction>
                    <ode>
                        <mu>2.0</mu>
                        <mu2>2.0</mu2>
                    </ode>
                    <torsional>
                        <ode />
                    </torsional>
                </friction>
                <contact>
                    <poissons_ratio>0.26</poissons_ratio>
                    <elastic_modulus>0.26</elastic_modulus>
                    <ode>
                        <soft_cfm>1e-7</soft_cfm>
                        <soft_erp>0.98</soft_erp>
                        <kp>9.65e+9</kp>
                        <kd>196468.827</kd>
                        <max_vel>0.01</max_vel>
                        <min_depth>0.0002</min_depth>
                    </ode>
                </contact>
                <bounce />
            </surface>
            <max_contacts>10</max_contacts>
        </collision>
        <visual name="visual">
            <geometry>
                <box>
                    <size>0.05 0.05 0.05</size>
                </box>
            </geometry>
            <material>
                <ambient>0.1 0.8 0.1 1</ambient>
                <diffuse>0.1 0.8 0.1 1</diffuse>
                <specular>0 0 0 0</specular>
                <emissive>0 0 0 1</emissive>
            </material>
        </visual>
        <self_collide>0</self_collide>
        <enable_wind>0</enable_wind>
        <kinematic>0</kinematic>
    </link>
    <pose>0.293518 0.735477 0.024 0 -0 0</pose>
</model>
```

Figure D.12: Example of box_green_0 in SDF file

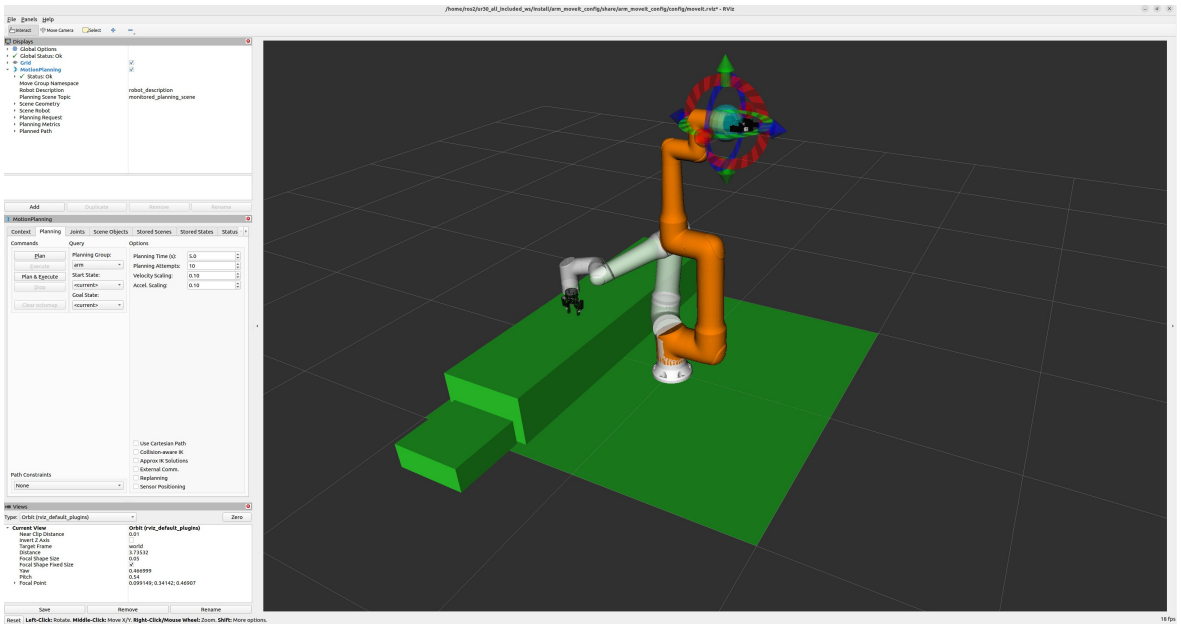


Figure D.13: End of the scenario's 10 cycles in RViz

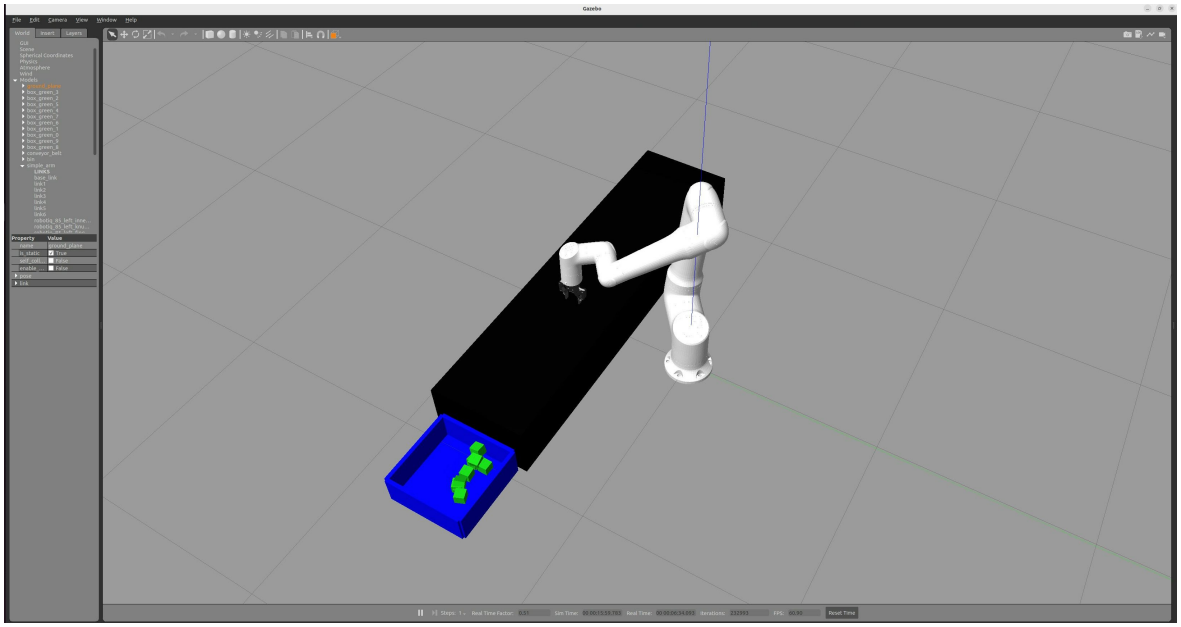


Figure D.14: End of the scenario's 10 cycles in Gazebo

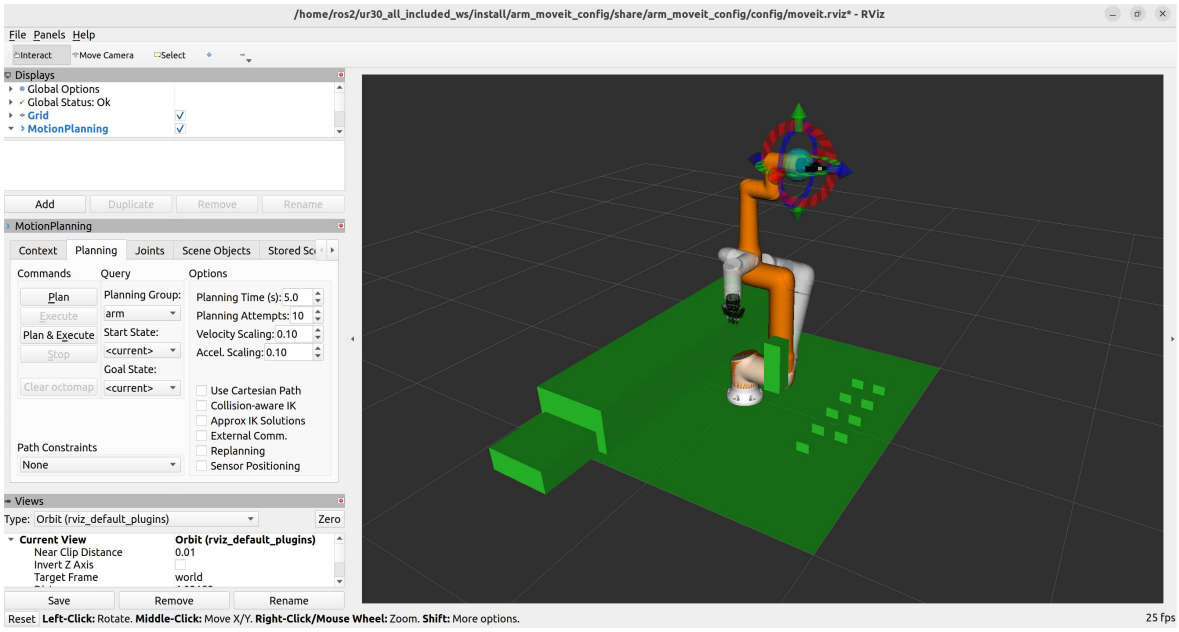


Figure D.15: Adding CollisionObject of test case WV1 in RViz

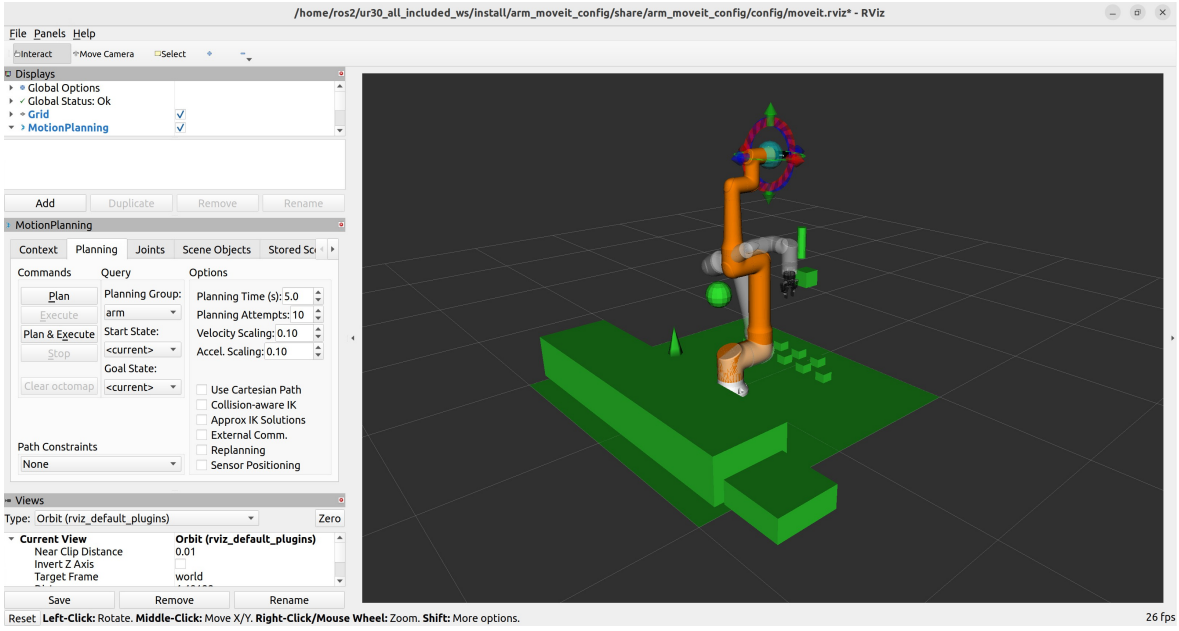


Figure D.16: Adding CollisionObject of test case WV2 in RViz

Appendix E

Additional information of evaluation

This appendix presents the additional information of the evaluation in Chapter 6.

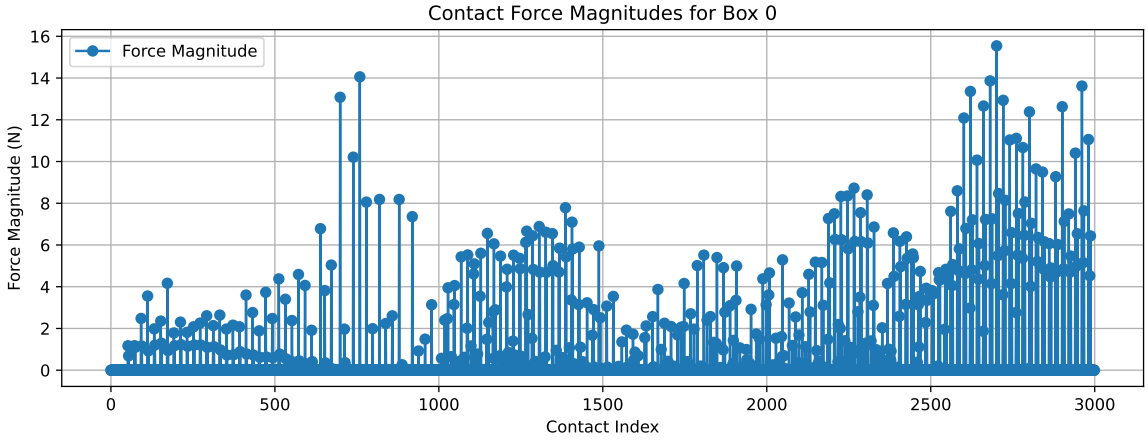


Figure E.1: Box 0's contact force magnitudes of test case SG1 for static analysis

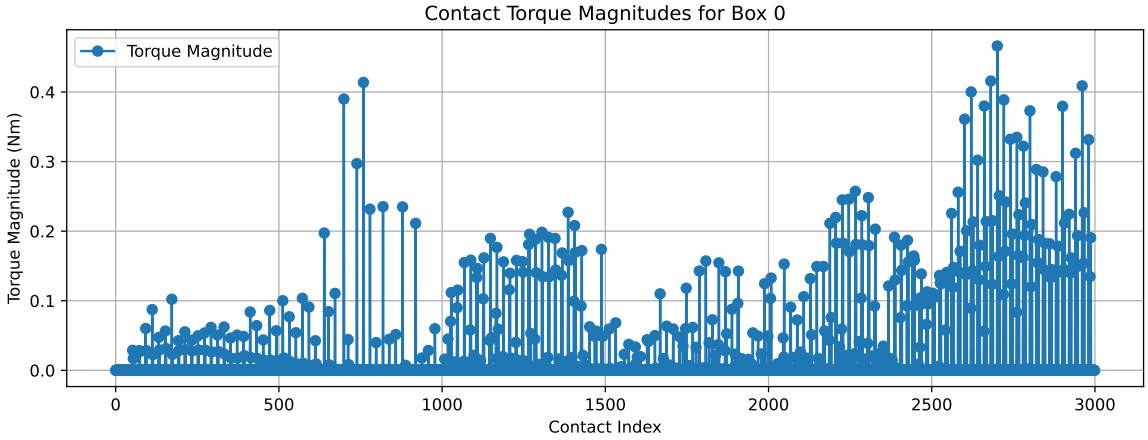


Figure E.2: Box 0's contact torque magnitudes of test case SG1 for static analysis

Table E.1: Parameters and setting of test case SG1

Test case	SG1
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.44
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	SS316
mu	2
mu2	2
poissons_ratio	0.26
elastic_modulus	0.26
mass in kg	1
stiffness in $\frac{N}{m}$	9.65×10^9
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000416667
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000416667
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000416667
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*soft*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	205
contact_area in mm^2	300

Table E.2: Maximum static contact force and torque of test case SG1's box 0

Test case	SG1
Maximum static contact force in N	15.5427
Maximum static contact torque in $N \cdot m$	0.4661
Affordability of box's gravity	*True*

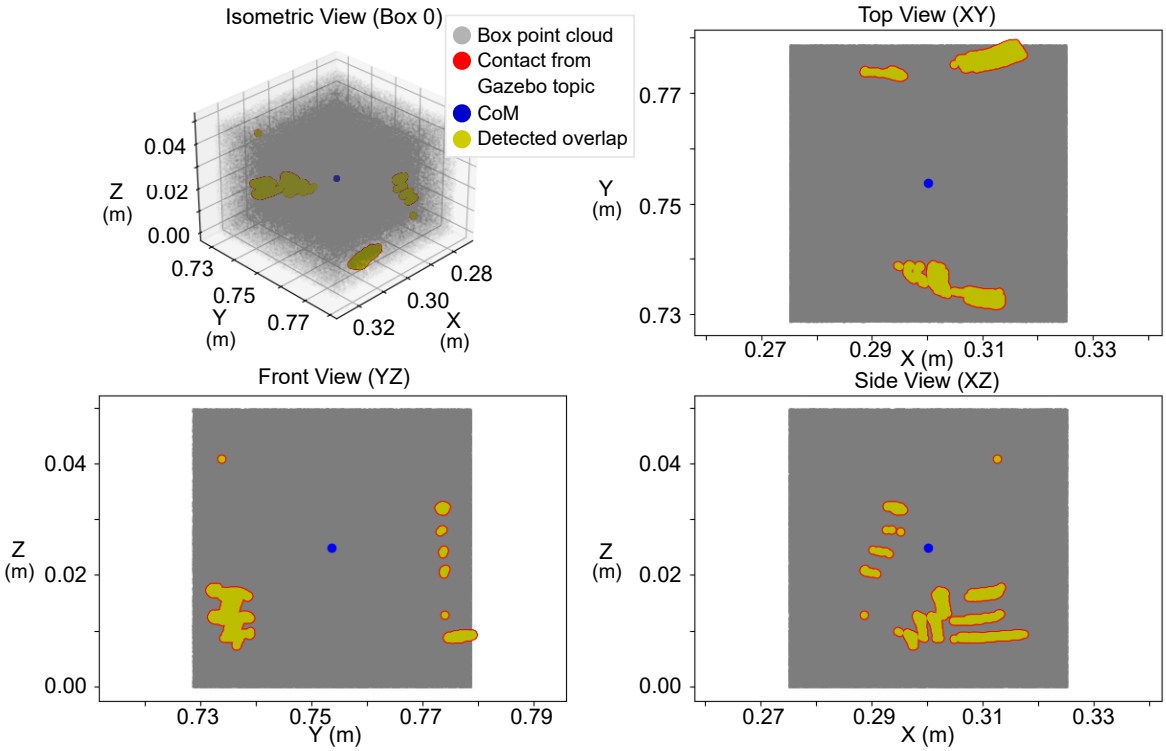


Figure E.3: Contact situation of box 0 in test case SG1

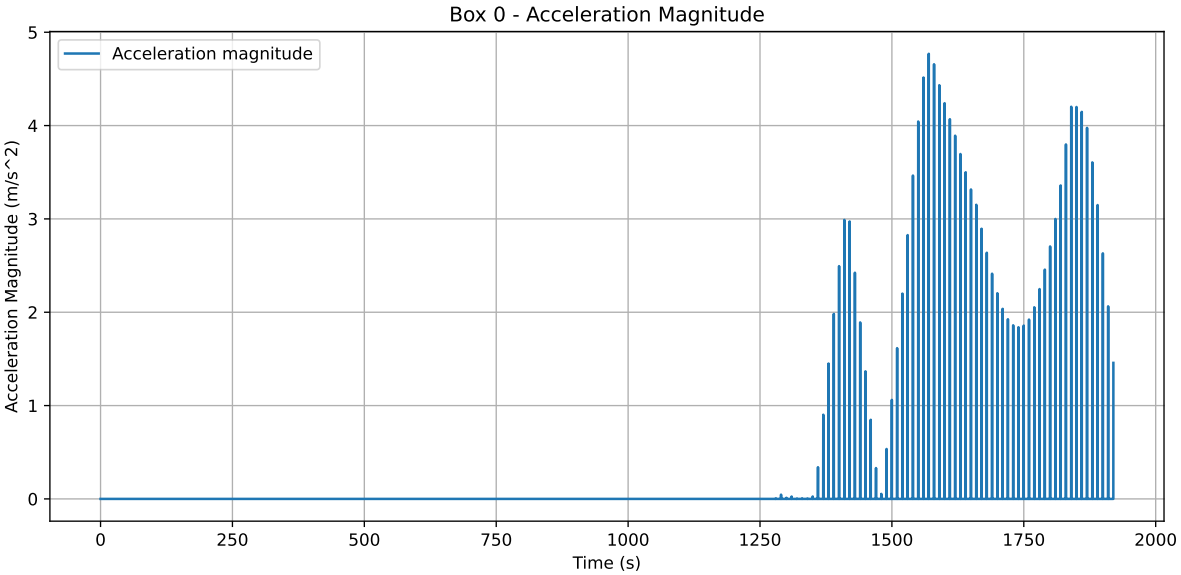


Figure E.4: Box 0's accelerations of test case ET1 for dynamic analysis

Table E.3: Parameters and setting of test case SG2

Test case	SG2
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	PTFE
mu	1
mu2	1
poissons_ratio	0.41
elastic_modulus	0.41
mass in kg	0.29
stiffness in $\frac{N}{m}$	2.5×10^7
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000120833
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000120833
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000120833
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*soft*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	30
contact_area in mm^2	300

Table E.4: Parameters and setting of test case SG3

Test case	SG3
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	Silicone
mu	1.7
mu2	1.7
poissons_ratio	0.43
elastic_modulus	0.43
mass in kg	0.156
stiffness in $\frac{N}{m}$	1.5×10^5
damping_ratio	1
i_xx in $kg \cdot m^2$	0.00006492188
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.00006492188
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.00006492188
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*soft*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	10
contact_area in mm^2	300

Table E.5: Stable grasp analysis results of test case SG3's box 0

Test case	SG3
Maximum static contact force in N	1.4850
Maximum static contact torque in $N \cdot m$	0.0042
Affordability of box's gravity	*False*
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*True*
Fulfillment of the origin lying with GWS	*True*
Fulfillment of the wrench spaces' configuration	*True*
Fulfillment of static stable grasp analysis	*True*
Maximum acceleration in $\frac{m}{s^2}$	19.7399
Mean acceleration in $\frac{m}{s^2}$	5.5835
Fulfillment of first-layer dynamic stable grasp analysis	*False*
Fulfillment of second-layer dynamic stable grasp analysis	*Failed*
Fulfillment of stable grasp analysis	*Failed*

Table E.6: Parameters and setting of test case SG4

Test case	SG4
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	SS316
mu	0
mu2	0
poissons_ratio	0
elastic_modulus	0
mass in kg	1
stiffness in $\frac{N}{m}$	0
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000416667
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000416667
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000416667
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*hard_without_friction*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	205
contact_area in mm^2	300

Table E.7: Parameters and setting of test case SG5

Test case	SG5
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	SS316
mu	2
mu2	2
poissons_ratio	0
elastic_modulus	0
mass in kg	1
stiffness in $\frac{N}{m}$	0
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000416667
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000416667
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000416667
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*hard_with_friction*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	205
contact_area in mm^2	300

Table E.8: Stable grasp analysis results of test case SG4's box 0

Test case	SG4
Maximum static contact force in N	0.0031
Maximum static contact torque in $N \cdot m$	0.0001
Affordability of box's gravity	*False*
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*True*
Fulfillment of the origin lying with GWS	*True*
Fulfillment of the wrench spaces' configuration	*False*
Fulfillment of static stable grasp analysis	*Failed*
Maximum acceleration in $\frac{m}{s^2}$	Failed
Mean acceleration in $\frac{m}{s^2}$	Failed
Fulfillment of first-layer dynamic stable grasp analysis	*Failed*
Fulfillment of second-layer dynamic stable grasp analysis	*Failed*
Fulfillment of stable grasp analysis	*Failed*

Table E.9: Stable grasp analysis results of test case SG5's box 0

Test case	SG5
Maximum static contact force in N	0.0054
Maximum static contact torque in $N \cdot m$	0.0002
Affordability of box's gravity	*False*
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*True*
Fulfillment of the origin lying with GWS	*True*
Fulfillment of the wrench spaces' configuration	*True*
Fulfillment of static stable grasp analysis	*True*
Maximum acceleration in $\frac{m}{s^2}$	21.5449
Mean acceleration in $\frac{m}{s^2}$	6.9861
Fulfillment of first-layer dynamic stable grasp analysis	*False*
Fulfillment of second-layer dynamic stable grasp analysis	*Failed*
Fulfillment of stable grasp analysis	*Failed*

Table E.10: Parameters and setting of test case ET1

Test case	ET1
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	0.1
arm_group.setMaxAccelerationScalingFactor()	0.1
sendGripperCommand()	0.42
Additional *CollisionObject*	-
Input in class *SdfParameters*	
Material	SS316
mu	2
mu2	2
poissons_ratio	0.26
elastic_modulus	0.26
mass in kg	1
stiffness in $\frac{N}{m}$	9.65×10^9
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000416667
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000416667
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000416667
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*soft*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	205
contact_area in mm^2	300

Table E.11: Stable grasp analysis results of test case ET1's box 0

Test case	ET1
Maximum static contact force in N	15.1207
Maximum static contact torque in $N \cdot m$	0.4507
Affordability of box's gravity	*True*
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*True*
Fulfillment of the origin lying with GWS	*True*
Fulfillment of the wrench spaces' configuration	*True*
Fulfillment of static stable grasp analysis	*True*
Maximum acceleration in $\frac{m}{s^2}$	4.7693
Mean acceleration in $\frac{m}{s^2}$	2.3267
Fulfillment of first-layer dynamic stable grasp analysis	*True*
Fulfillment of second-layer dynamic stable grasp analysis	*True*
Fulfillment of stable grasp analysis	*True*

Table E.12: Parameters and setting of test case WV1

Test case	WV1
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	Box primitive
Input in class *SdfParameters*	
Material	SS316
mu	2
mu2	2
poissons_ratio	0.26
elastic_modulus	0.26
mass in kg	1
stiffness in $\frac{N}{m}$	9.65×10^9
damping_ratio	1
i_xx in $kg \cdot m^2$	0.000416667
i_xy in $kg \cdot m^2$	0
i_xz in $kg \cdot m^2$	0
i_yy in $kg \cdot m^2$	0.000416667
i_yz in $kg \cdot m^2$	0
i_zz in $kg \cdot m^2$	0.000416667
length in m	0.05
width in m	0.05
height in m	0.05
stepsize in second	0.001
Input in class *ContactModels*	
contact_model	*soft*
Input in class *SpecifiedTimes*	
threshold in second	25
Input in class *PlotRobotObstaclePointCloud*	
is_still_other_obstacles	*no*
Input in class *GwsConstruct*	
gws_num_edges	6
Input in class *OtherSpace*	
mgf_gripper in N	250
magf_object in N	1000
Input in class *DynamicConsideration*	
yield_strength in MPa	205
contact_area in mm^2	300

Table E.13: Stable grasp analysis results of test case WV1's box 0

Test case	WV1
Maximum static contact force in N	14.1996
Maximum static contact torque in $N \cdot m$	0.1431
Affordability of box's gravity	*True*
Fulfillment of grasp matrix	*True*
Fulfillment of friction cone constraints	*True*
Fulfillment of the origin lying with GWS	*True*
Fulfillment of the wrench spaces' configuration	*True*
Fulfillment of static stable grasp analysis	*True*
Maximum acceleration in $\frac{m}{s^2}$	12.2175
Mean acceleration in $\frac{m}{s^2}$	4.2892
Fulfillment of first-layer dynamic stable grasp analysis	*True*
Fulfillment of second-layer dynamic stable grasp analysis	*True*
Fulfillment of stable grasp analysis	*True*

Table E.14: Parameters and setting of test case WV2

Test case	WV2
Settings in class *MoveGroupDemoNode*	
arm_group.setMaxVelocityScalingFactor()	1.0
arm_group.setMaxAccelerationScalingFactor()	1.0
sendGripperCommand()	0.42
Additional *CollisionObject*	Box primitive Cylinder primitive Sphere primitive Cone primitive

Glossary

Abaqus	A Finite Element Analysis software used by engineers to simulate and analyze the behavior of complex structures and materials under various conditions.
Blender	A free and open-source three-dimensional creation suite used for a wide range of tasks, including three-dimensional modeling, animation, rigging, simulation, rendering, compositing, motion tracking, and video editing.
COLLADA	An interchange file format for interactive three-dimensional applications. (TOLA and CORKE 2024)
Convex hull	The smallest convex set that encloses all the points, forming a convex polygon.
Gazebo	A prominent open-source robotics simulator that has been widely used to simulate complex and dynamic environments with realistic physics rendering.
GNOME	An open-source desktop environment for Linux that provides a simple, user-friendly graphical interface and core applications for managing files, settings, and tasks.
Hertzian contact theory	A fundamental principle in contact mechanics that describes the stress distributions and deformations occurring when two elastic bodies with curved surfaces are pressed together, which is essential for the design and analysis of mechanical components experiencing contact. (RICARDO DE SOUZA ET AL. 2011)
Hyperplane	A hyperplane generalizes a flat, affine subspace in a multidimensional space. In the space of dimension n, a hyperplane is a subspace with one dimension less, meaning it has dimension $n-1$. (LIU 1999)
Kelvin-Voigt model	A mathematical representation used to describe the viscoelastic behavior of materials. It combines elastic and viscous properties to simulate the response of materials under deformation. (GE and RICE 2018)
MoveIt	An open-source robotics software framework that primarily focuses on motion planning, manipulation, kinematics, control, and navigation.
Newton	A collaboration between NVIDIA, Google DeepMind, and Disney Research to develop a new open-source physics engine. (NVIDIA CORPORATION 2025)
NumPy	A Python library for working with arrays. It also has functions for working in the domains of linear algebra, Fourier transform, and matrices.
NVIDIA Isaac Sim	A reference application built on NVIDIA Omniverse that enables developers to simulate and test AI-driven robotics solutions in physically based virtual environments. (NVIDIA CORPORATION 2025)

NVIDIA Omniverse	A platform that allows developers to build three-dimensional applications and services at scale, powered by OpenUSD and NVIDIA RTX technology. It enables real-time three-dimensional collaboration, digital twin creation, and AI-powered simulation workflows across various industries. (NVIDIA CORPORATION 2025)
OBJ	A standard three-dimensional image format that can be exported and opened by various three-dimensional image editing programs. (TOLA and CORKE 2024)
OctoMap	A three-dimensional mapping framework based on an Octree data structure. It is used in robotics to represent the environment as a probabilistic occupancy grid, where space is divided hierarchically into voxels of varying sizes. (HORNUNG ET AL. 2013)
Octree	A tree data structure used to partition three-dimensional space by recursively subdividing it into eight octants. Each node in the tree represents a cubic volume and can have up to eight children, each representing a smaller sub-volume. (HORNUNG ET AL. 2013)
PhysX	A physics engine developed by NVIDIA that enables the simulation of realistic physics effects in video games, simulations, and other applications. (NVIDIA CORPORATION 2025)
PyBullet	An open-source Python module used for real-time physics simulation and robotics. It is built on top of the Bullet Physics SDK, a professional-grade physics engine used in games, robotics, and research. (MOWER ET AL. 2022)
Pydantic	A data validation and settings management library for Python.
Ray-shooting problem	Let M be a given set of points in Rd. Assume that the convex hull $H(M)$ contains the origin. Given a ray emanating from the origin, find the facet of $H(W)$ intersected by this ray. (DING ET AL. 2000)
RGB-D camera	A type of depth camera that amplifies the effectiveness of depth-sensing camera systems by enabling object recognition. (ORTEGA ET AL. 2023)
RViz	A three-dimensional visualization tool utilized in ROS. It is designed to visualize sensor data, robot state information, and other elements of a robot's environment in real time. (SHAFIKOV, SAGITOV, ET AL. 2020)
SolidWorks	A three-dimensional CAD software used mainly for designing, modeling, and simulating mechanical parts, assemblies, and drawings. (RIVERA ET AL. 2019)
YARP	An open-source middleware framework designed for robotics and artificial intelligence applications. It provides tools and libraries that facilitate communication and coordination between different components of a robotic system, such as sensors, actuators, and computational nodes. (HOFFMAN ET AL. 2014)
Yield strength	The amount of stress at which a material begins to deform plastically.

Bibliography

ALMUSAWI, A. R. J., DULGER, L. C., and KAPUCU, S., (2018). "Online teaching of robotic arm by human–robot interaction: end effector force/torque sensing". In: *Journal of the Brazilian Society of Mechanical Sciences and Engineering* 40.9. ISSN: 1678-5878. DOI: 10.1007/s40430-018-1358-3.

BICCHI, A., (1994). "On the problem of decomposing grasp and manipulation forces in multiple whole-limb manipulation". In: *Robotics and Autonomous Systems* 13.2, pp. 127–147. ISSN: 0921-8890. DOI: 10.1016/0921-8890(94)90055-8. URL: https://www.sciencedirect.com/science/article/pii/0921889094900558.

BOHG, J., PAVONE, M., and SADIGH, D., (2024). *Principles of Robot Autonomy*.

BORST, C., FISCHER, M., and HIRZINGER, G., (2003). "Grasping the dice by dicing the grasp". In: *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*. IEEE, pp. 3692–3697. ISBN: 0-7803-7860-1. DOI: 10.1109/IROS.2003.1249729.

BORST, C., FISCHER, M., and HIRZINGER, G., (2004). "Grasp planning: how to choose a suitable task wrench space". In: *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA '04. 2004*. IEEE, 319–325 Vol.1. ISBN: 0-7803-8232-3. DOI: 10.1109/ROBOT.2004.1307170.

CASAN, G. A., CERVERA, E., MOUGHLBAY, A. A., ALEMANY, J., and MARTINET, P., (2015). "ROS-based online robot programming for remote education and training". In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 6101–6106. ISBN: 978-1-4799-6923-4. DOI: 10.1109/ICRA.2015.7140055.

COLEMAN, D., SUCAN, I., CHITTA, S., and CORRELL, N., (2014). *Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study*. DOI: 10.48550/arXiv.1404.3785.

DAI, H., MAJUMDAR, A., and TEDRAKE, R., (2018). "Synthesis and Optimization of Force Closure Grasps via Sequential Semidefinite Programming". In: *Robotics Research*. Ed. by BICCHI, A. and BURGARD, W. Vol. 2. Springer Proceedings in Advanced Robotics. Cham: Springer International Publishing, pp. 285–305. ISBN: 978-3-319-51531-1. DOI: 10.1007/978-3-319-51532-8_18.

DING, D., LIU, Y.-H., SHEN, Y.-T., and XIANG, G.-L., (2000). "An efficient algorithm for computing a 3D form-closure grasp". In: *Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000) (Cat. No.00CH37113)*. IEEE, pp. 1223–1228. ISBN: 0-7803-6348-5. DOI: 10.1109/IROS.2000.893186.

ERICKSON, D., WEBER, M., and SHARF, I., (2003). "Contact Stiffness and Damping Estimation for Robotic Systems". In: *The International Journal of Robotics Research* 22.1, pp. 41–57. ISSN: 0278-3649. DOI: 10.1177/0278364903022001004.

FLORES, P., (2022). "Contact mechanics for dynamical systems: a comprehensive review". In: *Multibody System Dynamics* 54.2, pp. 127–177. ISSN: 1384-5640. DOI: 10.1007/s11044-021-09803-y.

GASMI, A., JOSEPH, P. F., RHYNE, T. B., and CRON, S. M., (2011). "Closed-form solution of a shear deformable, extensional ring in contact between two rigid surfaces". In: *In-*

ternational Journal of Solids and Structures 48.5, pp. 843–853. ISSN: 00207683. DOI: 10.1016/j.ijsolstr.2010.11.018.

GE, C. and RICE, B., (2018). "Impact damping ratio of a nonlinear viscoelastic foam". In: *Polymer Testing* 72, pp. 187–195. ISSN: 01429418. DOI: 10.1016/j.polymertesting.2018.10.023.

GENG, P., HEUSS, L., RAUH, T., MÜLLER, J., and DAUB, R., (2023). *Framework For The Rapid Development And Deployment Of Customized Industrial Robotic Applications*. Hannover : publish-Ing. DOI: 10.15488/15305.

GREGOR, R., BABINEC, A., DUCHOŇ, F., and DOBIŠ, M., (2021). "Hand Guiding a Virtual Robot Using a Force Sensor". In: *Acta Mechanica et Automatica* 15.3, pp. 177–186. DOI: 10.2478/ama-2021-0023.

GRUSHKO, S., VYSOCKY, A., JHA, V. K., PASTOR, R., PRADA, E., MIKOVA, L., and BOBOVSKY, Z., (2020). "TUNING PERCEPTION AND MOTION PLANNING PARAMETERS FOR MOVEIT! FRAMEWORK". In: *MM Science Journal* 2020.4, pp. 4154–4163. ISSN: 18031269. DOI: 10.17973/MMSJ.2020_11_2020064.

HOFFMAN, E. M., TRAVERSARO, S., ROCCHI, A., FERRATI, M., SETTIMI, A., ROMANO, F., NATALE, L., BICCHI, A., NORI, F., and TSAGARAKIS, N. G., (2014). "Yarp Based Plugins for Gazebo Simulator". In: *Modelling and Simulation for Autonomous Systems*. Ed. by HODICKY, J. Vol. 8906. Lecture Notes in Computer Science. Cham: Springer International Publishing, pp. 333–346. ISBN: 978-3-319-13822-0. DOI: 10.1007/978-3-319-13823-7_29.

HORNUNG, A., WURM, K. M., BENNEWITZ, M., STACHNISS, C., and BURGARD, W., (2013). "OctoMap: an efficient probabilistic 3D mapping framework based on octrees". In: *Autonomous Robots* 34.3, pp. 189–206. ISSN: 0929-5593. DOI: 10.1007/s10514-012-9321-0.

HUI, F., PAYEUR, P., and CRETU, A.-M., (2017). "Visual Tracking of Deformation and Classification of Non-Rigid Objects with Robot Hand Probing". In: *Robotics* 6.1, p. 5. DOI: 10.3390/robotics6010005.

AL-IBADI, A., NEFTI-MEZIANI, S., and DAVIS, S., (2018). "Active Soft End Effectors for Efficient Grasping and Safe Handling". In: *IEEE Access* 6, pp. 23591–23601. DOI: 10.1109/ACCESS.2018.2829351.

JIA, P., LI, W. l., WANG, G., and LI, S. Y., (2017). "Optimal grasp planning for a dexterous robotic hand using the volume of a generalized force ellipsoid during accepted flattening". In: *International Journal of Advanced Robotic Systems* 14.1. ISSN: 1729-8806. DOI: 10.1177/1729881416687134.

KALIGE, N., (2016). "Bi-manual Learning for a Basketball Playing Robot". Arizona State University.

KHURSHID, A., GHAFOOR, A., and AFZAAL, M., (2011). "Robotic Grasping and Fine Manipulation Using Soft Fingertip". In: *Advances in Mechatronics*. Ed. by MARTINEZ-ALFARO, H. InTech. ISBN: 978-953-307-373-6. DOI: 10.5772/23697.

KIM, D.-H., PARK, G.-M., YOO, Y.-H., RYU, S.-J., JEONG, I.-B., and KIM, J.-H., (2017). "Realization of task intelligence for service robots in an unstructured environment". In: *Annual Reviews in Control* 44, pp. 9–18. ISSN: 13675788. DOI: 10.1016/j.arcontrol.2017.09.013.

KOENIG, N. and HOWARD, A., (2004). "Design and use paradigms for gazebo, an open-source multi-robot simulator". In: *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*. IEEE, pp. 2149–2154. ISBN: 0-7803-8463-6. DOI: 10.1109/IROS.2004.1389727.

KUBUS, D., KROGER, T., and WAHL, F. M., (2007). "On-line rigid object recognition and pose estimation based on inertial parameters". In: *2007 IEEE/RSJ International Conference on*

Intelligent Robots and Systems. IEEE, pp. 1402–1408. ISBN: 978-1-4244-0911-2. DOI: 10.1109/IROS.2007.4399184.

KURSCHEID, S. and DAUB, R., (2024). "Automated Robot Skill Parameterization Based On Task Specifications". In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, pp. 01–04. ISBN: 979-8-3503-6123-0. DOI: 10.1109/ETFA61755.2024.10711102.

LI, C., HAO, L., CHENG, H., and NIE, X., (2017). "Research on Motion Planning System of Service Robot Based on ROS". In: *2017 IEEE 7th Annual International Conference on CYBER Technology in Automation, Control, and Intelligent Systems (CYBER)*. IEEE, pp. 205–209. ISBN: 978-1-5386-0490-8. DOI: 10.1109/CYBER.2017.8446559.

LIU, Q. Y., JIANG, B., and YU, X. L., (2011). "An Algorithm for Force-Closure Grasping of Three-Fingered Hand". In: *Advanced Materials Research* 250-253, pp. 3965–3970. DOI: 10.4028/www.scientific.net/AMR.250-253.3965.

LIU, Y.-R., HUANG, M.-B., and HUANG, H.-P., (2019). "Automated Grasp Planning and Path Planning for a Robot Hand-Arm System". In: *2019 IEEE/SICE International Symposium on System Integration (SII)*. IEEE, pp. 92–97. ISBN: 978-1-5386-3615-2. DOI: 10.1109/SII.2019.8700433.

LIU, Y.-H., (1999). "Qualitative test and force optimization of 3-D frictional form-closure grasps using linear programming". In: *IEEE Transactions on Robotics and Automation* 15.1, pp. 163–173. ISSN: 1042-296X. DOI: 10.1109/70.744611.

MANCHESTER, Z. R. and PECK, M. A., (2017). "Recursive Inertia Estimation with Semidefinite Programming". In: *AIAA Guidance, Navigation, and Control Conference*. Reston, Virginia: American Institute of Aeronautics and Astronautics. ISBN: 978-1-62410-450-3. DOI: 10.2514/6.2017-1902.

MAŃKOWSKI, T., TOMCZYŃSKI, J., WALAS, K., and BELTER, D., (2020). "PUT-Hand—Hybrid Industrial and Biomimetic Gripper for Elastic Object Manipulation". In: *Electronics* 9.7, p. 1147. DOI: 10.3390/electronics9071147.

MARTUCCI, G., BIMBO, J., PRATTICHIZZO, D., and MALVEZZI, M., (2020). "Maintaining stable grasps during highly dynamic robot trajectories". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 9198–9204. ISBN: 978-1-7281-6212-6. DOI: 10.1109/IROS45743.2020.9341575.

MILLER, A. T. and ALLEN, P. K., (2004). "GraspIt! A versatile simulator for robotic grasping". In: *IEEE Robotics & Automation Magazine* 11.4, pp. 110–122. ISSN: 1070-9932. DOI: 10.1109/MRA.2004.1371616.

MOWER, C. E., STOURAITIS, T., MOURA, J., RAUCH, C., YAN, L., BEHABADI, N. Z., GIENGER, M., VERCAUTEREN, T., BERGELES, C., and VIJAYAKUMAR, S., (2022). *ROS-PyBullet Interface: A Framework for Reliable Contact Simulation and Human-Robot Interaction*. DOI: 10.48550/arXiv.2210.06887.

MU, X. and ZHANG, Y., (2019). "Grasping Force Optimization for Multi-fingered Robotic Hands Using Projection and Contraction Methods". In: *Journal of Optimization Theory and Applications* 183.2, pp. 592–608. ISSN: 0022-3239. DOI: 10.1007/s10957-019-01540-9.

MYRVOLD, M. L., (2020). "On Estimating Forces and Torques in Manipulators". Norwegian University of Science and Technology. URL: https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/2780964.

NAKAZAWA, N., KIM, I.-h., INOOKA, H., and IKEURA, R., (2001). "Force control of a robot gripper based on human grasping schemes". In: *Control Engineering Practice* 9.7, pp. 735–742. ISSN: 09670661. DOI: 10.1016/S0967-0661(01)00038-7.

NVIDIA CORPORATION, (2025). *NVIDIA Isaac Sim Documentation*. https://docs.isaacsim.omniverse.nvidia.com/latest/index.html.

OPEN SOURCE ROBOTICS FOUNDATION, (2014). *Gazebo Simulation*. https://classic.gazebosim.org/.

ORTEGA, K. D., SEPÚLVEDA, J. I., HERNANDEZ, B., HOLGUÍN, G. A., and MEDEIROS, H., (2023). "Pose Estimation of Robot End-Effector using a CNN-Based Cascade Estimator". In: *2023 IEEE 6th Colombian Conference on Automatic Control (CCAC)*. IEEE, pp. 1–6. ISBN: 979-8-3503-2472-3. DOI: 10.1109/CCAC58200.2023.10333441.

PANASIUK, D., (2021). "Computer simulation of an automated robot grasping with object pose estimation: Software frameworks that are suitable to build a reliable computer simulation". Tampere University.

PAPADOPOULOS, C. and KATSANOS, C., (2023). "Teaching Assembly Tasks to Robots Using a 3D Simulation Environment". In: *Proceedings of the 2nd International Conference of the ACM Greek SIGCHI Chapter*. New York, NY, USA: ACM, pp. 1–8. ISBN: 9798400708886. DOI: 10.1145/3609987.3610005.

PICKNIK ROBOTICS, (2024). *MoveIt 2 Documentation*. URL: https://moveit.picknik.ai/humble/index.html.

PIEMNGAM, K., NILKHAMHANG, I., and BUNNUN, P., (2019). "A Virtual Spring Damper Method for Formation Control of the Multi Omni-directional Robots in Cooperative Transportation". In: *2019 11th International Conference on Information Technology and Electrical Engineering (ICITEE)*. IEEE, pp. 1–6. ISBN: 978-1-7281-4055-1. DOI: 10.1109/ICITEED.2019.8929998.

PRASAD, V. V. S. H. and DHANRAJSEKHAR, K. V., (2023). "Design and Analysis of a Pick and Place Robotic Arm". In: *International Journal for Research in Applied Science and Engineering Technology* 11.12, pp. 1167–1174. DOI: 10.22214/ijraset.2023.57501.

PULJIZ, D., KREBS, F., BOSING, F., and HEIN, B., (2020). "What the HoloLens Maps Is Your Workspace: Fast Mapping and Set-up of Robot Cells via Head Mounted Displays and Augmented Reality". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 11445–11451. ISBN: 978-1-7281-6212-6. DOI: 10.1109/IROS45743.2020.9340879.

QIAN, W., XIA, Z., XIONG, J., GAN, Y., GUO, Y., WENG, S., DENG, H., HU, Y., and ZHANG, J., (2014). "Manipulation task simulation using ROS and Gazebo". In: *2014 IEEE International Conference on Robotics and Biomimetics (ROBIO 2014)*. IEEE, pp. 2594–2598. ISBN: 978-1-4799-7397-2. DOI: 10.1109/ROBIO.2014.7090732.

RASTEGARPANAH, A., HATHAWAY, J., and STOLKIN, R., (2021). "Vision-Guided MPC for Robotic Path Following Using Learned Memory-Augmented Model". In: *Frontiers in robotics and AI* 8, p. 688275. DOI: 10.3389/frobt.2021.688275.

RIBEIRO, F. M., CORREIA, T., LIMA, J., GONÇALVES, G., and PINTO, V. H., (2023). "Modeling and Realistic Simulation of a Dexterous Robotic Hand: SVH Hand use-case". In: *2023 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*. IEEE, pp. 132–138. ISBN: 979-8-3503-0121-2. DOI: 10.1109/ICARSC58346.2023.10129643.

RICARDO DE SOUZA, J., NASCIMENTO, I., MOREIRA, S., CAVALCANTI, S., and MEDEIROS, J., (2011). "HERTZ THEORY x EXPERIMENT: CONTACT PARAMETERS IN SPHERE-PLAN CONFIGURATION FOR COMPOSITES OF POLYESTER/GLASS FIBER AGAINST A STEEL SPHERE IN QUASISTATICS TESTS". In: *Proceedings of 21th Brazilian Congress of Mechanical Engineering*.

RIVERA, Z. B., SIMONE, M. C. de, and GUIDA, D., (2019). "Unmanned Ground Vehicle Modelling in Gazebo/ROS-Based Environments". In: *Machines* 7.2, p. 42. DOI: 10.3390/machines7020042.

ROBOTIQ INC., (2025). *Robotiq Official Website*. https://robotiq.com/.

SALISBURY, J. K. and ROTH, B., (1983). "Kinematic and Force Analysis of Articulated Mechanical Hands". In: *Journal of Mechanisms, Transmissions, and Automation in Design* 105.1, pp. 35–41. ISSN: 0738-0666. DOI: 10.1115/1.3267342.

SANNEMAN, L., FOURIE, C., and SHAH, J. A., (2021). "The State of Industrial Robotics: Emerging Technologies, Challenges, and Key Research Directions". In: *Foundations and Trends® in Robotics* 8.3, pp. 225–306. ISSN: 1935-8253. DOI: 10.1561/2300000065.

SHAFIKOV, A., SAGITOV, A., LI, H., SCHIEFERMEIER-MACH, N., and MAGID, E., (2020). "Robotic Palpation Modeling for KUKA LBR IIWA Using Gazebo Simulator". In: *Proceedings of International Conference on Artificial Life and Robotics* 25, pp. 436–439. DOI: 10.5954/ICAROB.2020.OS18-3.

SHAFIKOV, A., TSOY, T., LAVRENOV, R., MAGID, E., LI, H., MASLAK, E., and SCHIEFERMEIER-MACH, N., (2020). "Medical palpation autonomous robotic system modeling and simulation in ROS/Gazebo". In: *2020 13th International Conference on Developments in eSystems Engineering (DeSE)*. IEEE, pp. 200–205. ISBN: 978-1-6654-2238-3. DOI: 10.1109/DeSE51703.2020.9450784.

SINGH, T. P., SURESH, P., and CHANDAN, S., (2017). "Forward and Inverse Kinematic Analysis of Robotic Manipulators". In: *International Research Journal of Engineering and Technology (IRJET)*, pp. 1459–1468.

SMITH, R., (2006). "Open Dynamics Engine V0.5 User Guide".

SOUZA, J. P. C. de, ROCHA, L. F., OLIVEIRA, P. M., MOREIRA, A. P., and BOAVENTURA-CUNHA, J., (2021). "Robotic grasping: from wrench space heuristics to deep learning policies". In: *Robotics and Computer-Integrated Manufacturing* 71, p. 102176. ISSN: 0736-5845. DOI: 10.1016/j.rcim.2021.102176. URL: https://www.sciencedirect.com/science/article/pii/s0736584521000594.

STOOP, P., RATNAYAKE, T., and TOFFETTI, G., (2024). "A Method for Multi-Robot Asynchronous Trajectory Execution in MoveIt2". In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 17694–17700. ISBN: 979-8-3503-8457-4. DOI: 10.1109/ICRA57147.2024.10611498.

SUNG, J.-H. and JOE, H.-M., (2025). "Robot Manipulator Based on a Wire-driven Variable Link Mechanism to Control Workspace". In: *International Journal of Control, Automation and Systems* 23.1, pp. 223–234. DOI: 10.1007/s12555-024-0536-6.

TANG, W., WANG, W., and WANG, S., (2024). "Partially Decoupled Impedance Motion Force Control Using Prioritized Inertia Shaping". In: *IEEE Robotics and Automation Letters* 9.9, pp. 7621–7628. DOI: 10.1109/LRA.2024.3430869.

TEISSING, K., (2021). "Time Parameterization of the Manipulator Path". Czech Technical University in Prague.

TOLA, D. and CORKE, P., (2024). "Understanding URDF: A Dataset and Analysis". In: *IEEE Robotics and Automation Letters* 9.5, pp. 4479–4486. DOI: 10.1109/LRA.2024.3381482.

UNIVERSAL ROBOTS A/S, (2025). *Universal Robots Official Website*. https://www.universal-robots.com/.

VELASCO, M. C., (2024). "Acceleration compensation of an armed mobile robot for transportation of fluids". Universitat Politècnica de Catalunya BarcelonaTech. URL: https://upcommons.upc.edu/handle/2117/411816.

WANG, C., CHEN, W., and TOMIZUKA, M., (2012). "Robot end-effector sensing with position sensitive detector and inertial sensors". In: *2012 IEEE International Conference on Robotics and Automation*. IEEE, pp. 5252–5257. ISBN: 978-1-4673-1405-3. DOI: 10.1109/ICRA.2012.6225180.

WOWK, V., (2015). "What is Centrifugal Force?" In: *Sound & vibration* 49, pp. 13–15.

YOO, T. and CHOI, B. W., (2024). "Interactive Path Editing and Simulation System for Motion Planning and Control of a Collaborative Robot". In: *Electronics* 13.14, p. 2857. DOI: 10.3390/electronics13142857.

YOSHIKAWA, T., (2010). "Multifingered robot hands: Control for grasping and manipulation". In: *Annual Reviews in Control* 34.2, pp. 199–208. ISSN: 13675788. DOI: 10.1016/j.arcontrol.2010.09.001.

YULDASHEV, N., DOBROKVASHINA, A., and LAVRENOV, R., (2024). "On sensor modeling in Gazebo simulator". In: *Proceedings of International Conference on Artificial Life and Robotics* 29, pp. 290–294. DOI: 10.5954/ICAROB.2024.OS10-6.

ZHANG, L. and WANG, X., (2012). "Dynamic control of a flexible drilling robot end-effector". In: *2012 24th Chinese Control and Decision Conference (CCDC)*. IEEE, pp. 2199–2204. ISBN: 978-1-4577-2074-1. DOI: 10.1109/CCDC.2012.6244353.

ZHAO, P., LIU, J., LI, Y., and WU, C., (2021). "A spring-damping contact force model considering normal friction for impact analysis". In: *Nonlinear Dynamics* 105.2, pp. 1437–1457. ISSN: 0924-090X. DOI: 10.1007/s11071-021-06660-4.

ZHENG, Y., (2013). "An Efficient Algorithm for a Grasp Quality Measure". In: *IEEE Transactions on Robotics* 29.2, pp. 579–585. ISSN: 1552-3098. DOI: 10.1109/TRO.2012.2222274.

ZHENG, Y., (2019). "Computing the best grasp in a discrete point set with wrench-oriented grasp quality measures". In: *Autonomous Robots* 43.4, pp. 1041–1062. ISSN: 0929-5593. DOI: 10.1007/s10514-018-9788-4.

ZHOU, J., LI, R., and ZHANG, Q., (2020). "Effects of Grain Boundary Faceting, Grain Boundary Anisotropy and Interface Natures on the Maximum Pinning Force of a Differently Shaped and Oriented Particle". In: *SSRN Electronic Journal*. DOI: 10.2139/ssrn.3683529.

ZHOU, Z., SONG, J., XIE, X., SHU, Z., MA, L., LIU, D., YIN, J., and SEE, S., (2024). "Towards Building AI-CPS with NVIDIA Isaac Sim: An Industrial Benchmark and Case Study for Robotics Manipulation". In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. Ed. by ROYCHOUDHURY, A., PAIVA, A., ABREU, R., STOREY, M., ANICHE, M., and NAGAPPAN, N. New York, NY, USA: ACM, pp. 263–274. ISBN: 9798400705014. DOI: 10.1145/3639477.3639740.

ZHU, X. and WANG, J., (2003). "Synthesis of force-closure grasps on 3-d objects based on the Q distance". In: *IEEE Transactions on Robotics and Automation* 19.4, pp. 669–679. ISSN: 1042-296X. DOI: 10.1109/TRA.2003.814499.

Disclaimer

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Yu-Chuan Chen

Garching, May 31, 2025 (Signature)